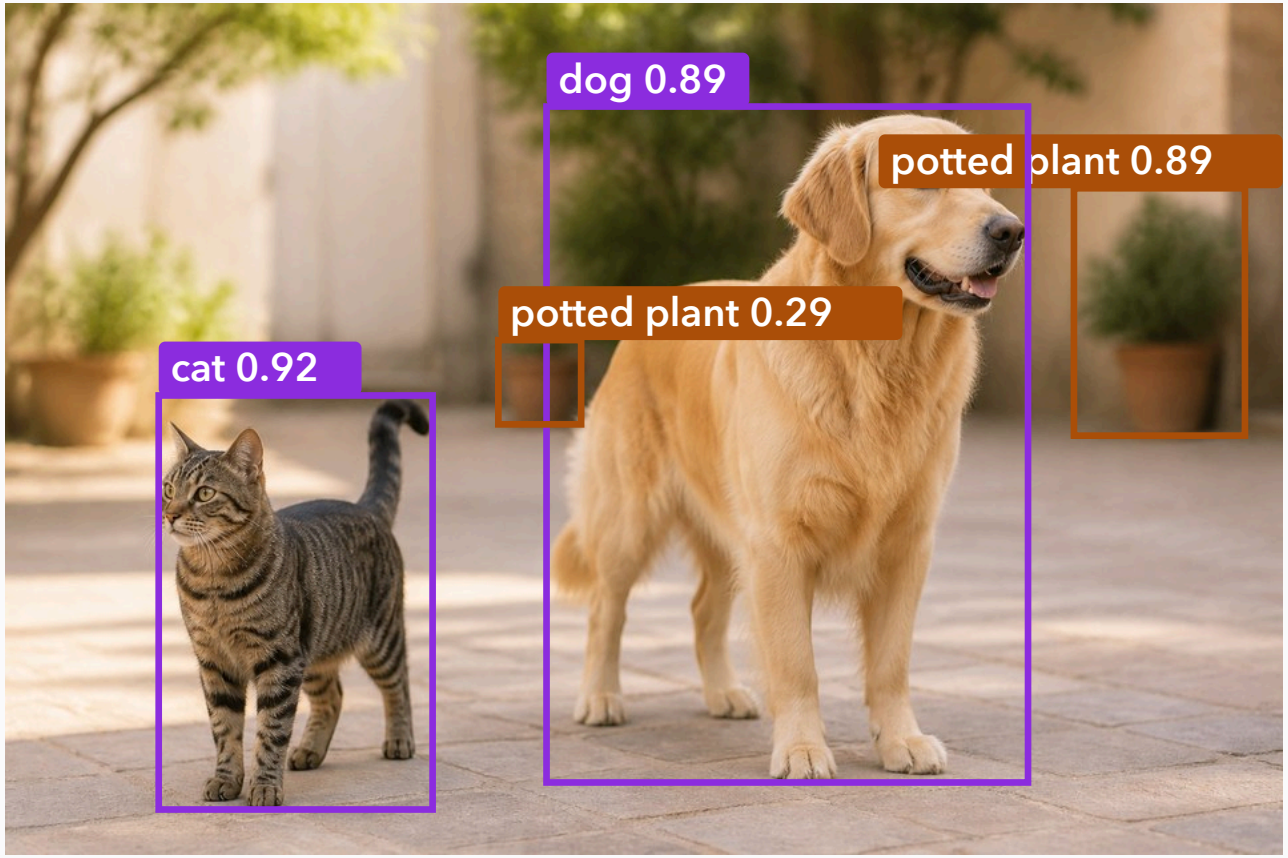
Object detection: from one label to a set of objects

A detector looks at a photo and returns a list: one entry per object, each a class and a box. How can a network with a fixed-size output return a list whose length changes from photo to photo?

ES 667 Deep Learning · Lecture 22
Nipun Batra · IIT Gandhinagar



> The real output of a small detector, YOLOv8n, on today's photo: four objects, each with a class, a score and a box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Part 1 · Why detect objects?

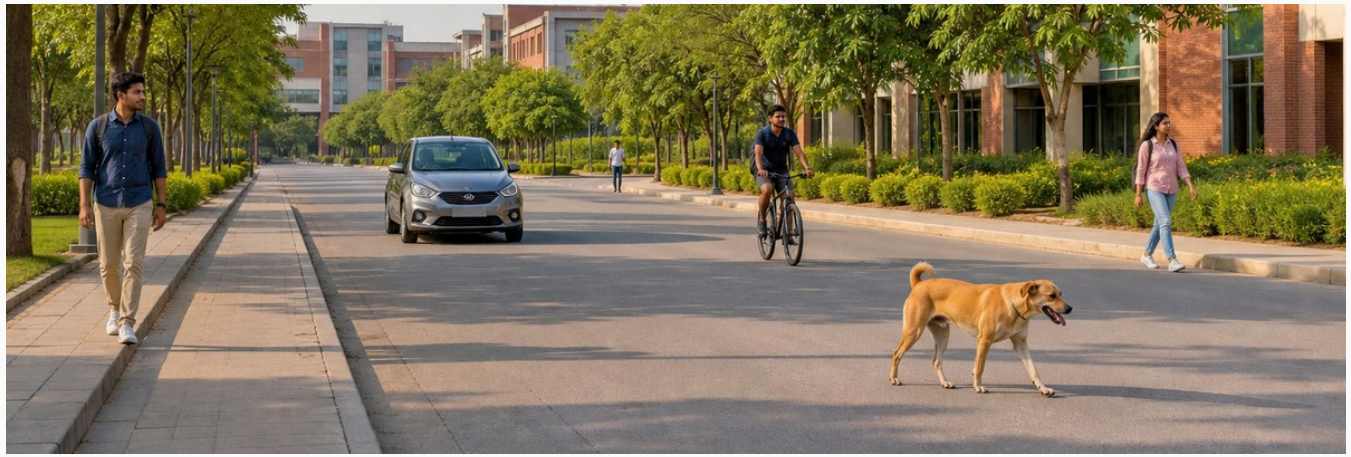
A camera becomes useful when it can say what is where. Start with six jobs that need exactly that.

1. For each of six photos, say what you would want to know, then see the answer.
2. See that every answer has the same shape, and what it is used for.
3. Three things that make the job hard.

> Six jobs that need the same kind of answer, and three reasons it is hard to give.

▶ Notebook, part 1

On the road: who is where?



What must the car know before it brakes or steers?

A car’s camera sees this campus street.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

On the road: who is where?



find people, cyclists, vehicles, animals

use brake or steer; follow each one from frame to frame

US cars that brake automatically for pedestrians had 27% fewer pedestrian crashes, but no fewer on dark, unlit roads (IIHS, 2022).

YOLOv8n: 4 people, a car, a bicycle and a dog, each boxed and scored. The car acts on this list.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

In a crowd: how many people?



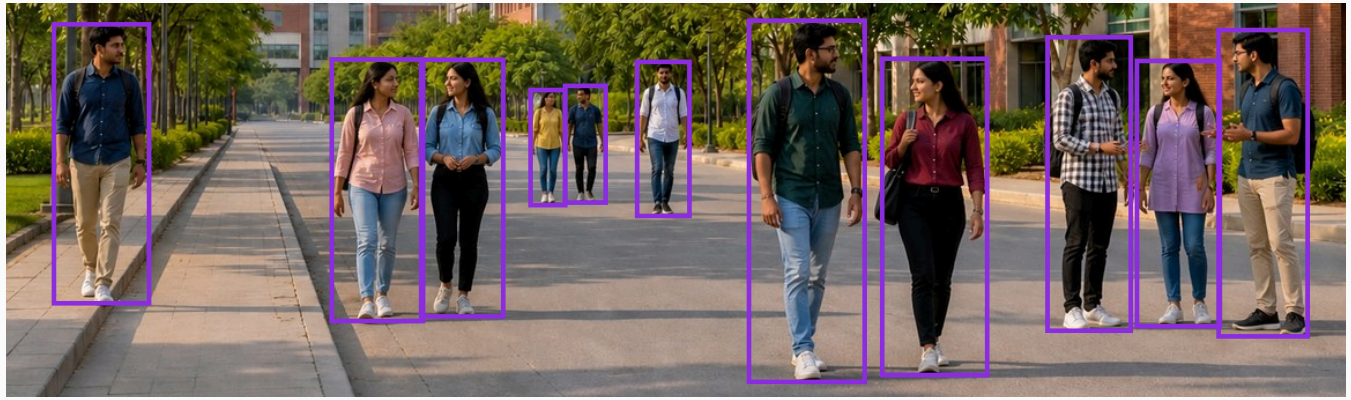
How many people are on this path?

A busy campus path. Count the people before the next click.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

In a crowd: how many people?



find people

use count them; warn when a place gets too crowded

Stations, stadiums and large gatherings count people camera by camera to spot dangerous crowding early.

> YOLOv8n boxes 11 people. The count is the list's length, and each entry also says where.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

At a camera trap: which animals walked past?



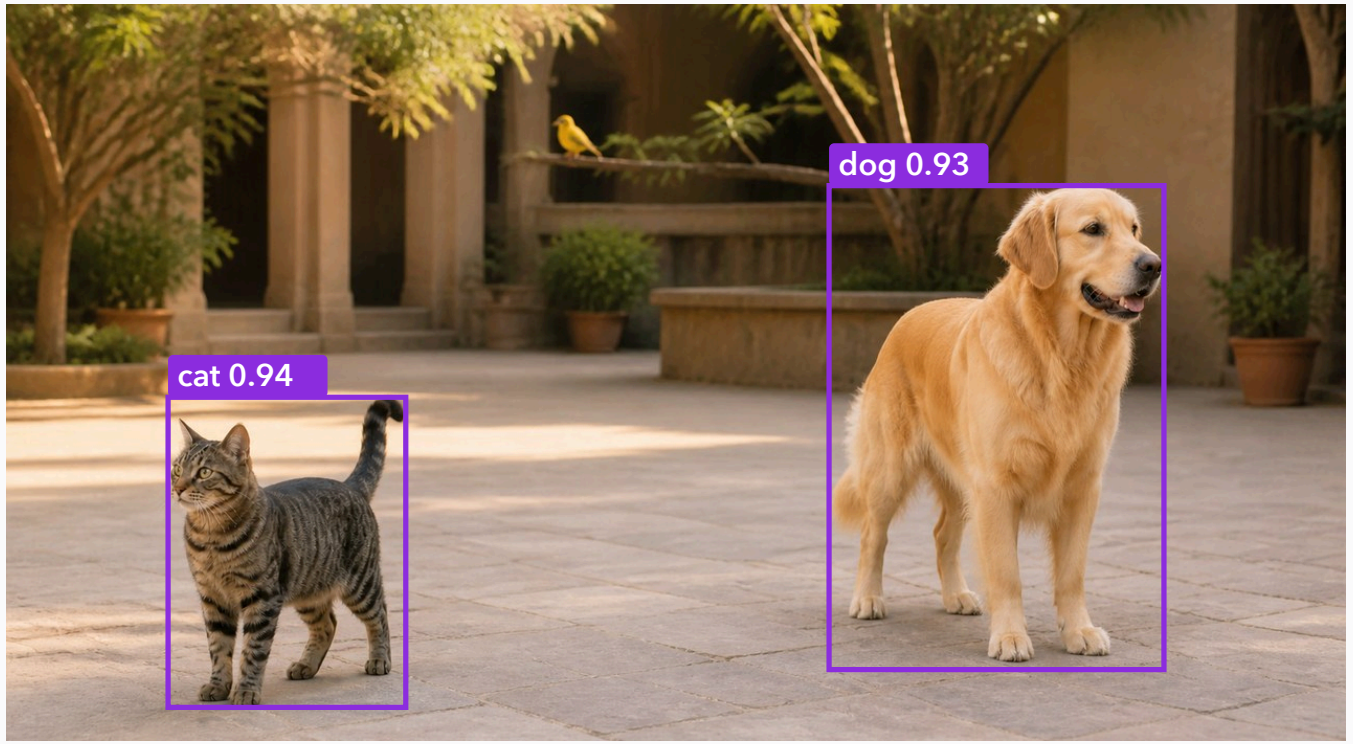
Which animals walked past this camera?

> A camera trap takes a photo whenever something moves.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

At a camera trap: which animals walked past?



find animals by species, people, vehicles

use count each species; skip the empty photos

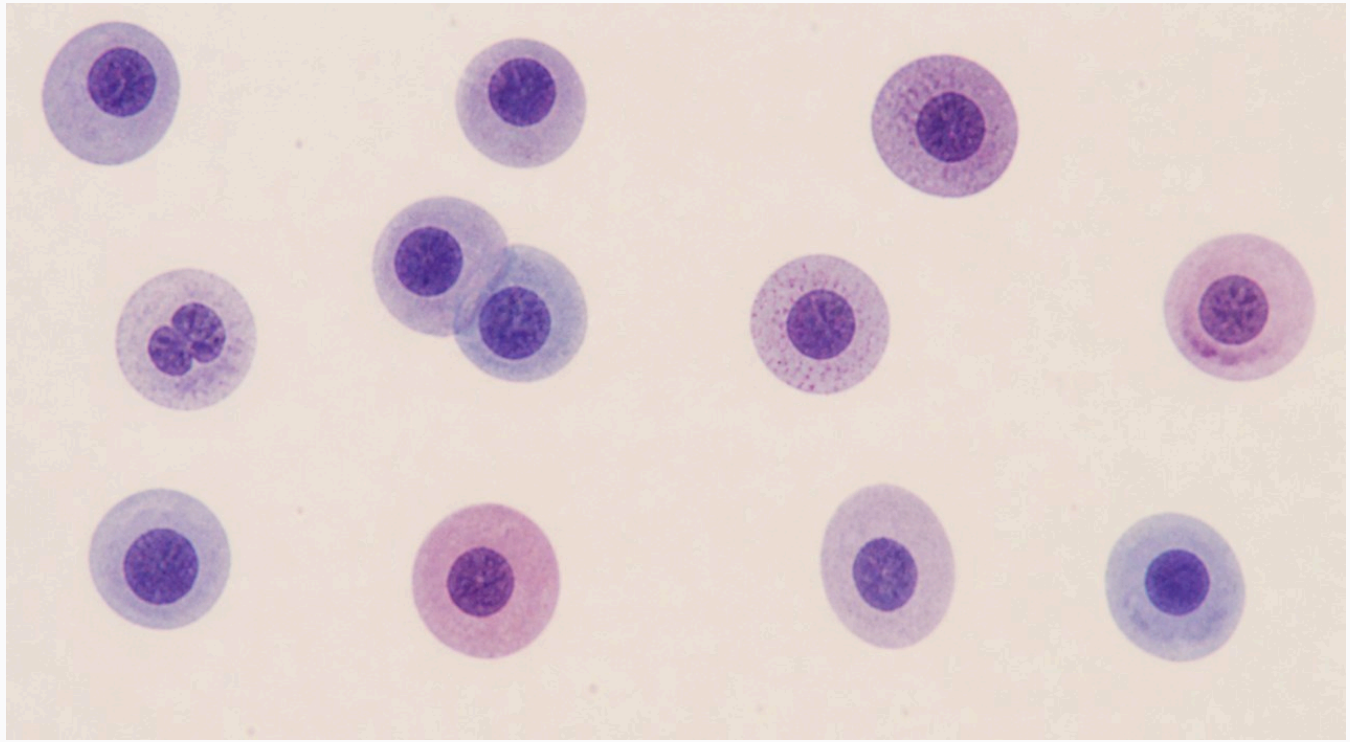
In an Arctic study of 548,627 camera-trap photos, a workflow built on MegaDetector left only 4 to 9% for people to check (Celis et al., 2024).

> YOLOv8n: a cat (0.94) and a dog (0.93). Did it find every animal? We come back to this.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Under a microscope: count the cells



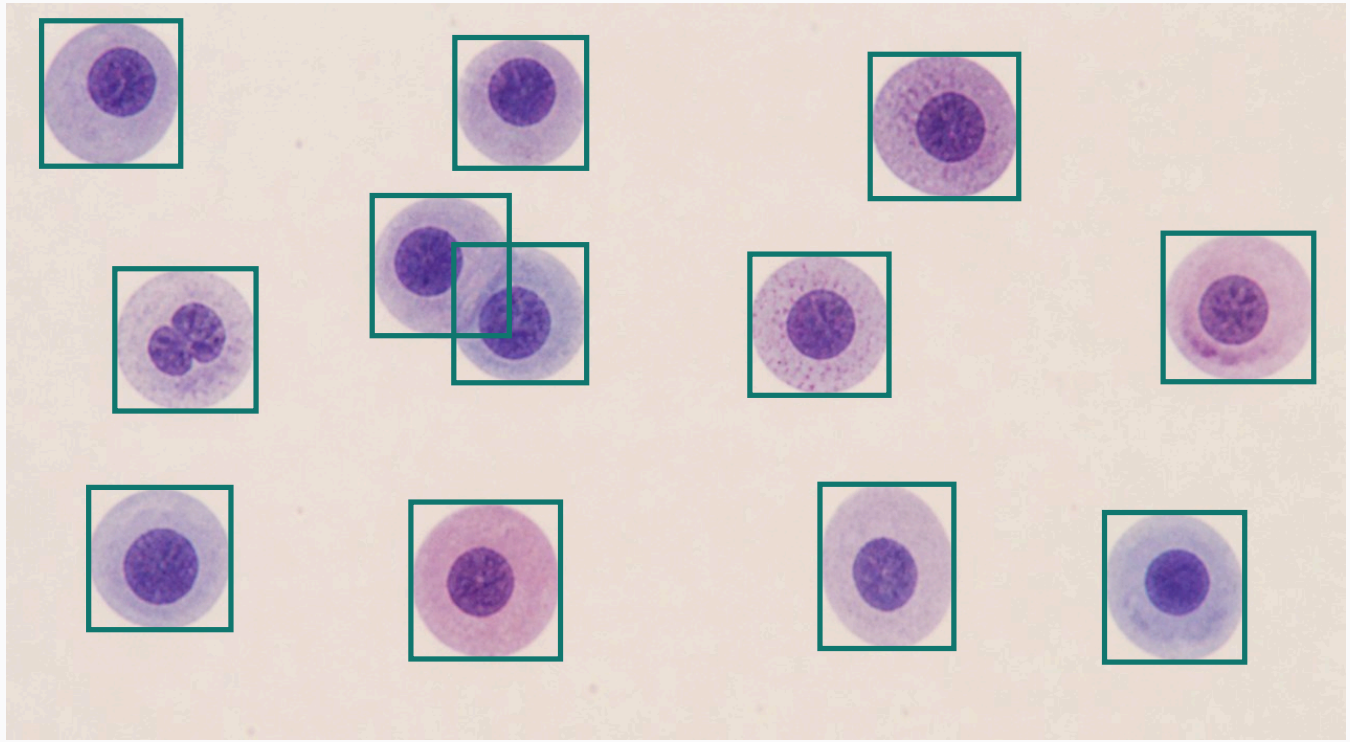
How many cells are there, and of which kinds?

> Stained cells under a microscope, photographed for a lab.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

Under a microscope: count the cells



find cells, by type

use count them; crop each one and classify its type

Cell detectors learn from lists like this: the 2018 Data Science Bowl's 841 images carry 37,333 nuclei, each marked by hand (Nature Methods, 2019).

> 12 cells. These teal boxes are ours: COCO has no class "cell", so no detector in this lecture knows one.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

In a warehouse: where is the parcel?



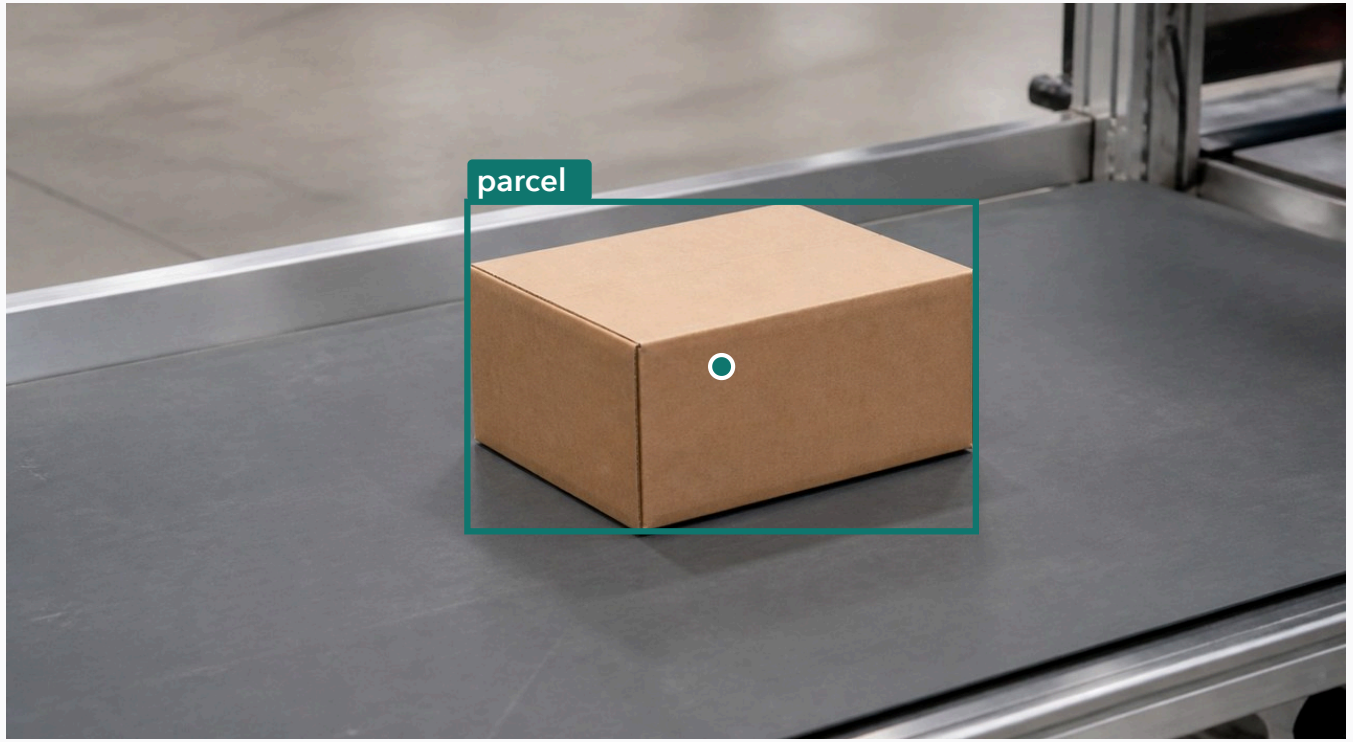
What must the arm know to pick the parcel up?

> A robot arm waits at the end of this conveyor belt.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

In a warehouse: where is the parcel?



find parcels, sacks, envelopes

use aim the gripper at the box; count parcels per hour

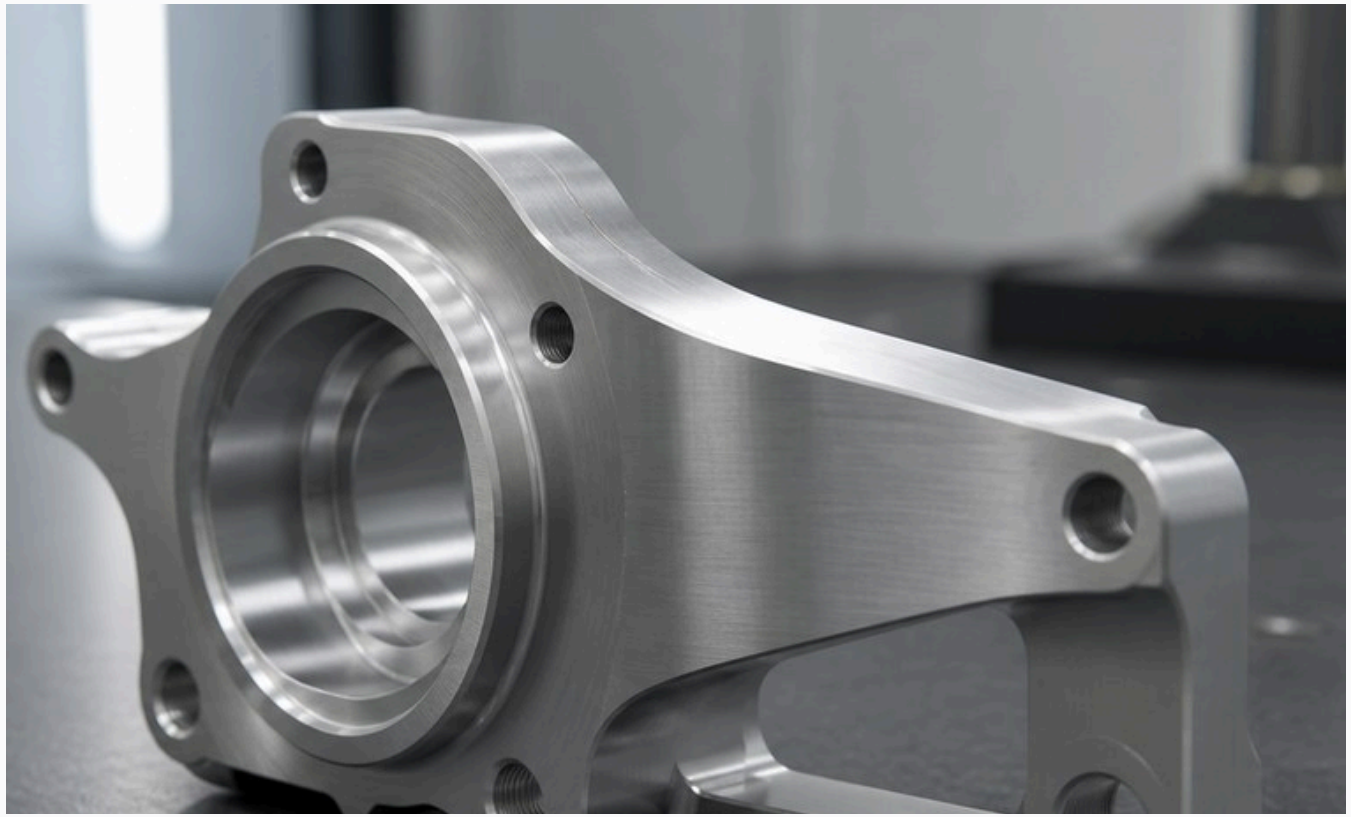
A harder relative: the SKU-110K photos of shop shelves hold about 147 boxed products each, packed edge to edge (Goldman et al., 2019).

One entry: a parcel, 486 × 315 pixels. The arm aims at the box's centre, the dot.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

On a production line: is anything wrong?



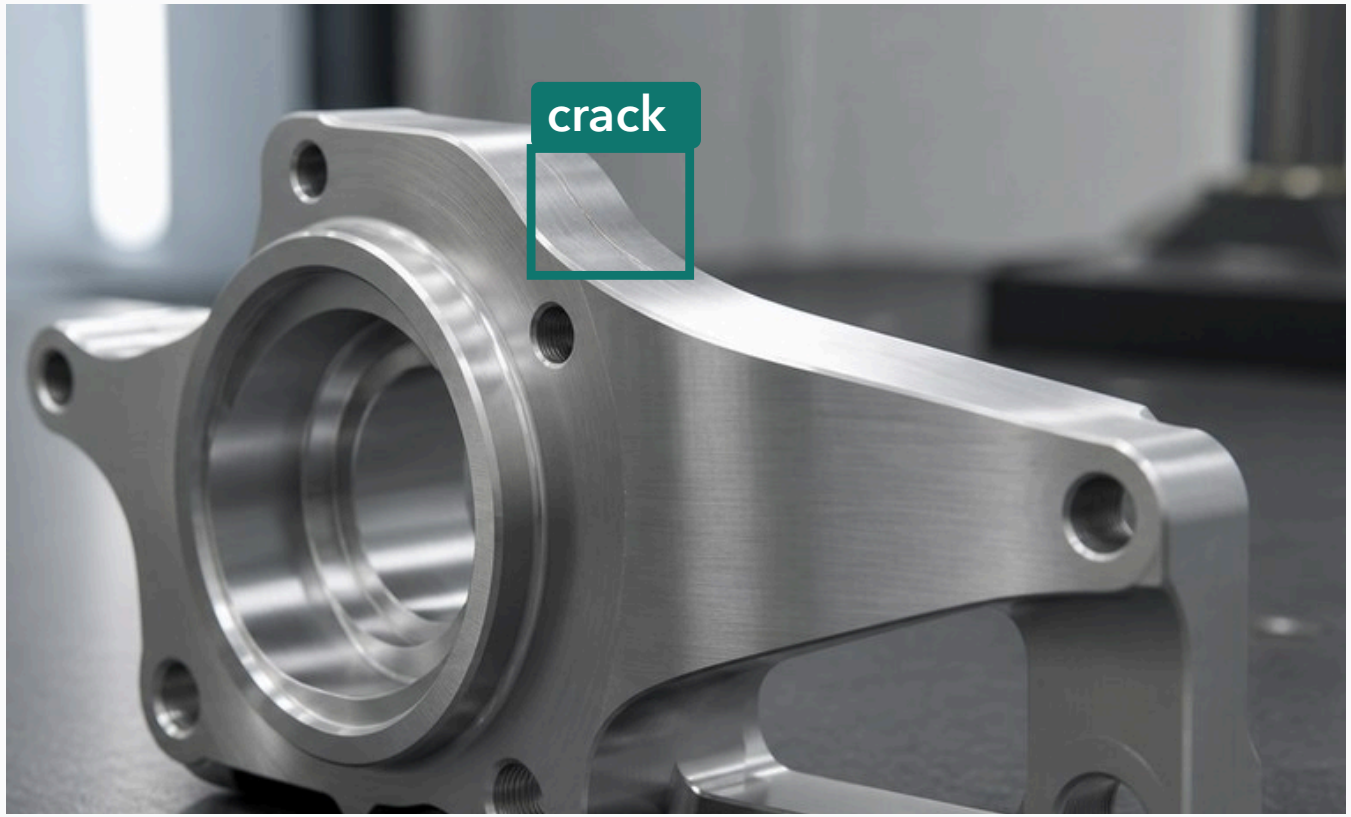
Is anything wrong with this part?

> A part fresh from the machine. Look closely.

Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

On a production line: is anything wrong?



find cracks, scratches, dents, missing holes

use reject the part; show the inspector where to look

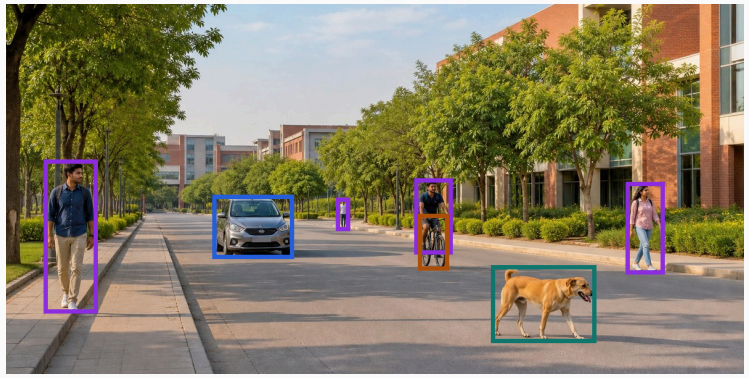
Inspection data sets look like this list: NEU-DET labels six kinds of steel-surface defect, 300 photos of each, every defect boxed.

> A hairline crack on the arm, in a box 90 × 72 pixels: the inspector knows exactly where to look.

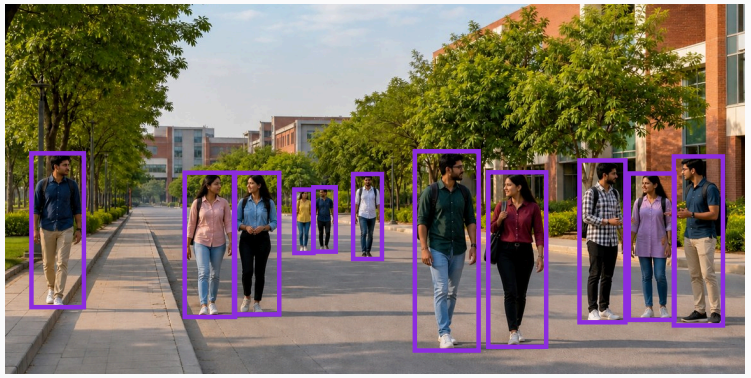
Boxes drawn by us (data/applications.json): COCO has no such class, so no detector here can find it. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

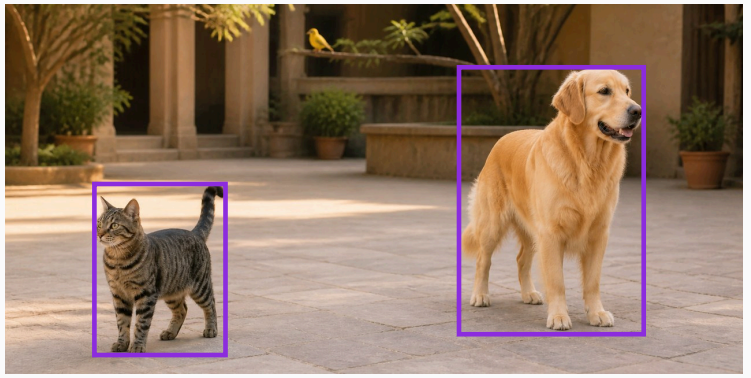
Six jobs, one kind of answer



road · **7** objects



crowd · **11** people



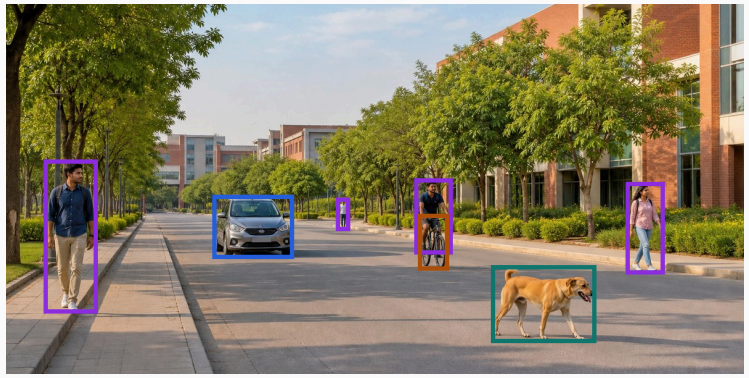
camera trap · **2** animals

Three jobs from everyday photos: 7 objects, 11 people, 2 animals. Purple: a detector’s answer.

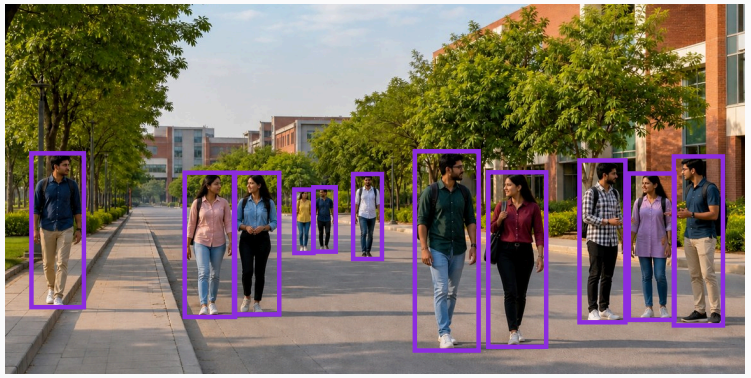
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells, parcel and crack: our annotation.

▶ Notebook, part 1

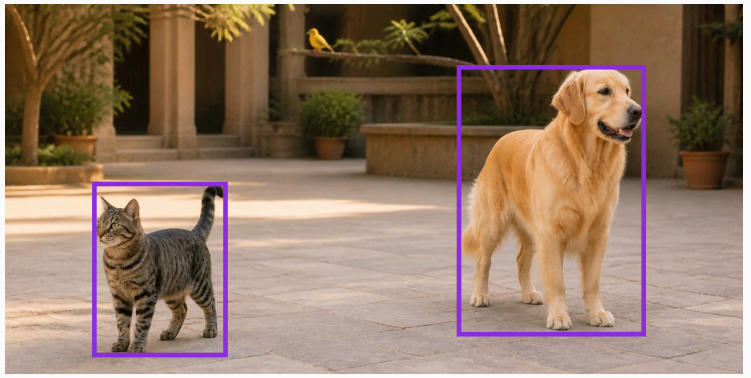
Six jobs, one kind of answer



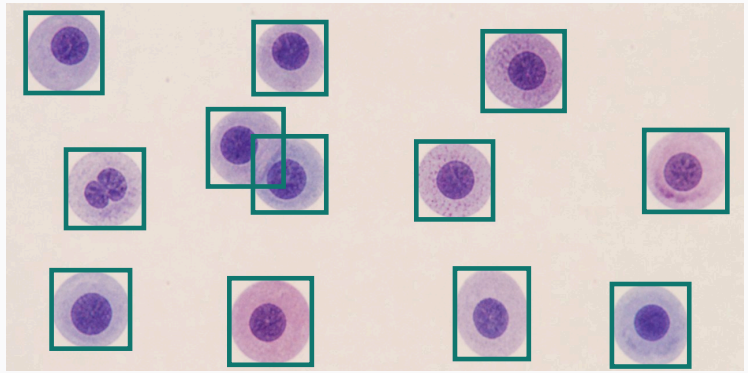
road · **7** objects



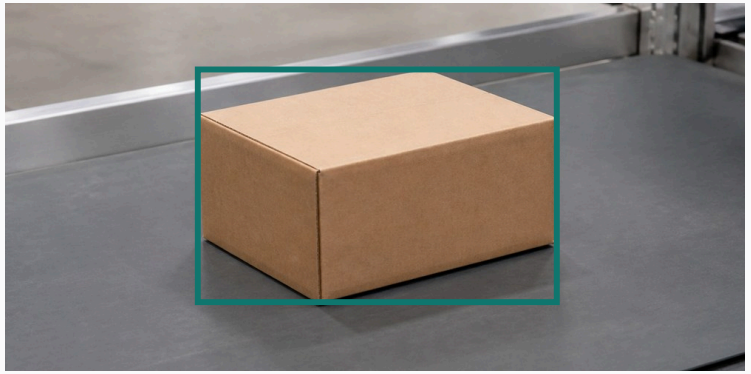
crowd · **11** people



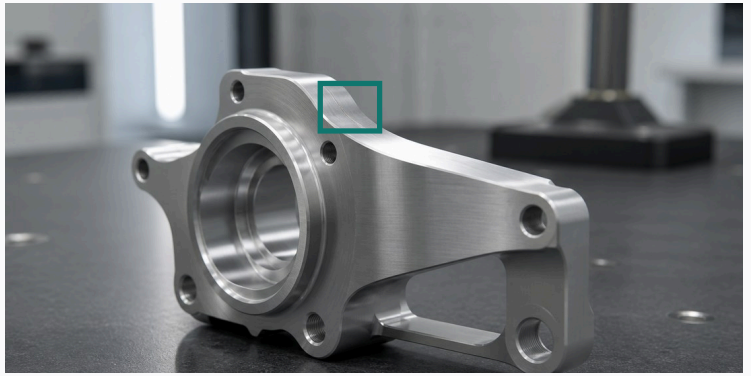
camera trap · **2** animals



microscope · **12** cells



warehouse · **1** parcel



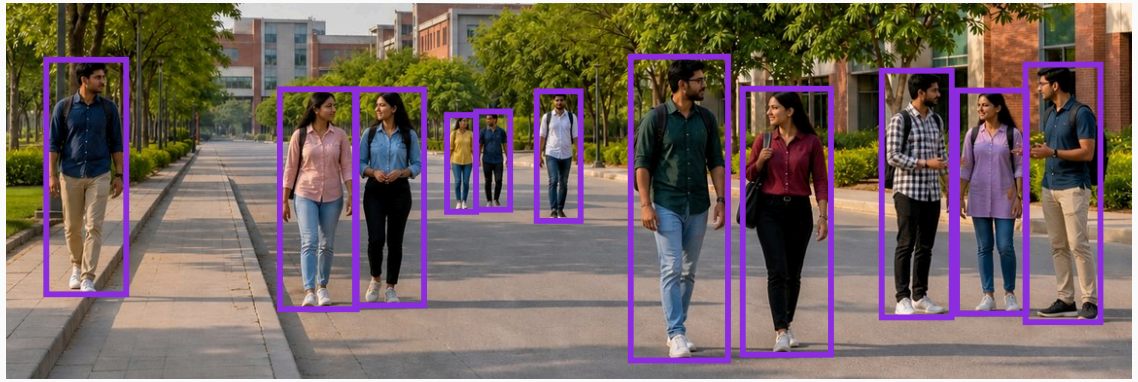
factory · **1** crack

> Three more: 12 cells, 1 parcel, 1 crack (teal: our annotation). Every answer has the same shape: a list, one entry per object, each a class and a box.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells, parcel and crack: our annotation.

▶ Notebook, part 1

What the list is used for



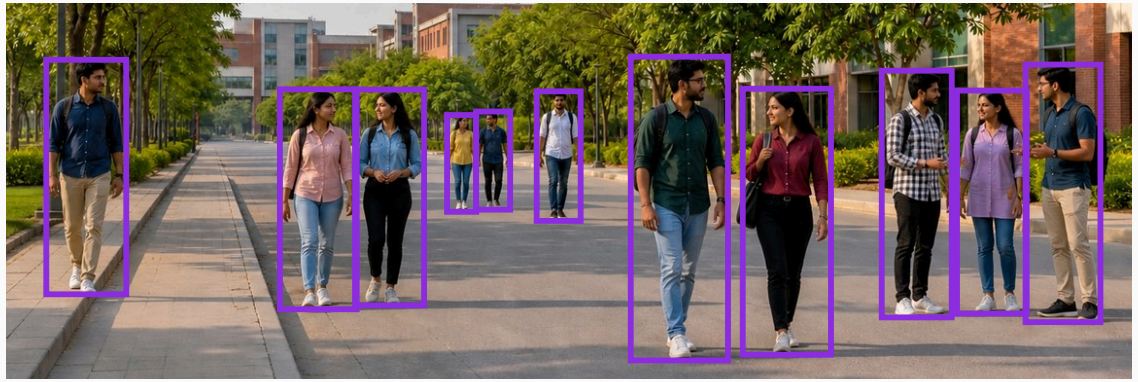
count · 11 people

> Count: the number of entries of a class. Eleven people; twelve cells.

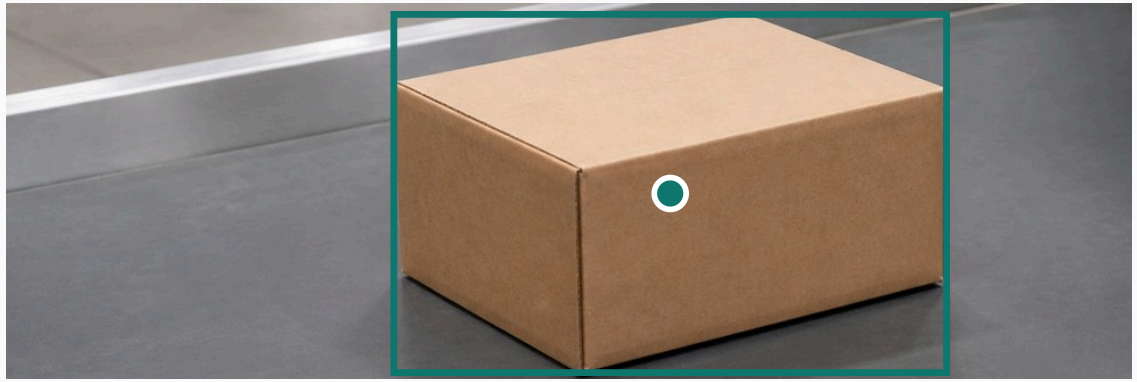
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

What the list is used for



count · 11 people



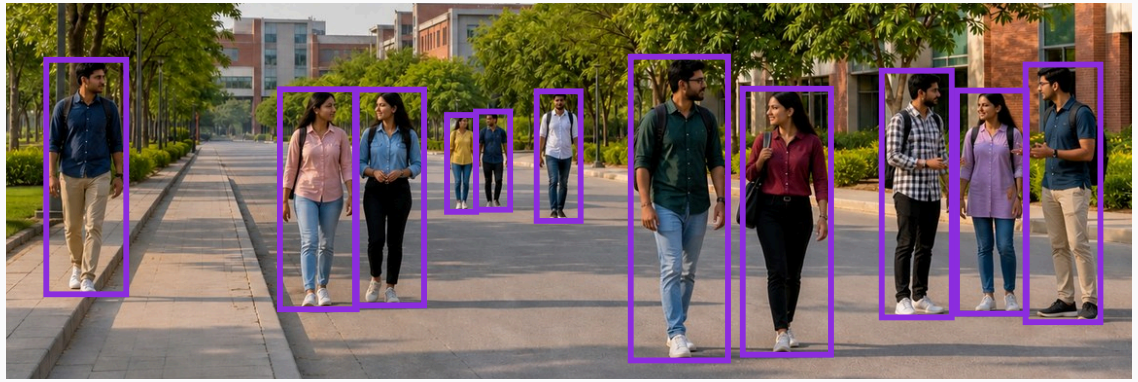
locate · grip at the box's centre

> Locate: the box says where to act. The arm grips at the parcel's centre; a car brakes for the person in its lane.

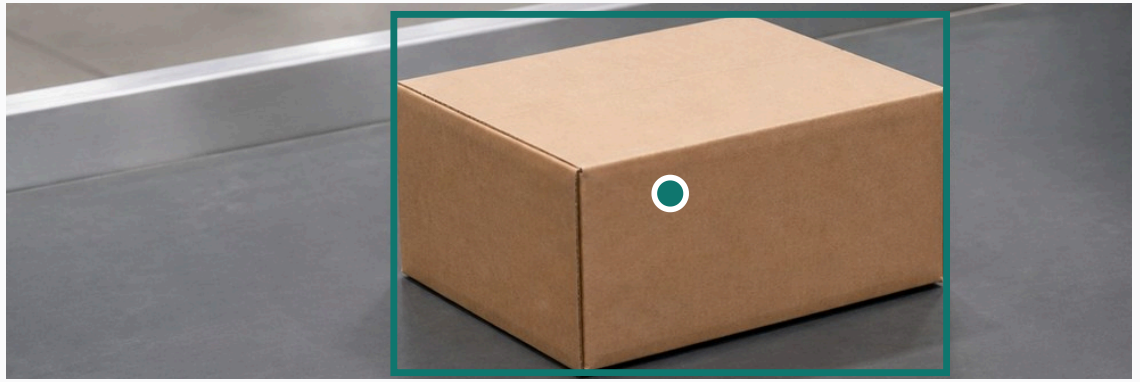
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

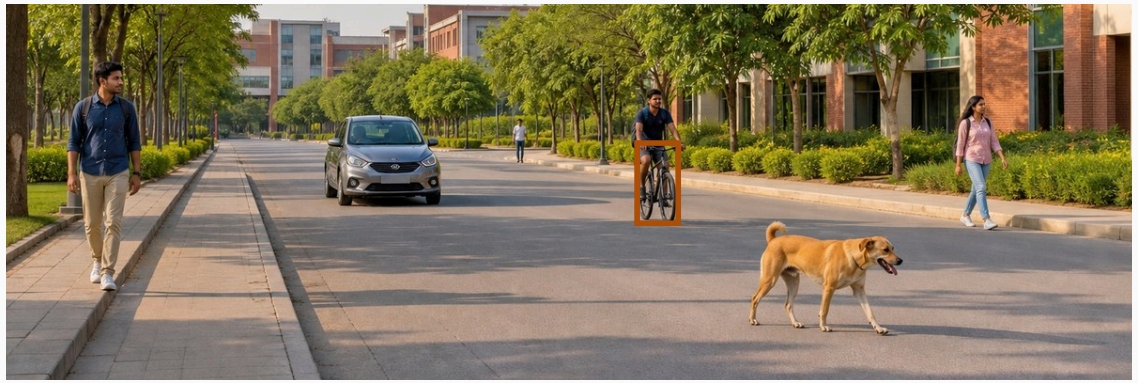
What the list is used for



count · 11 people



locate · grip at the box's centre



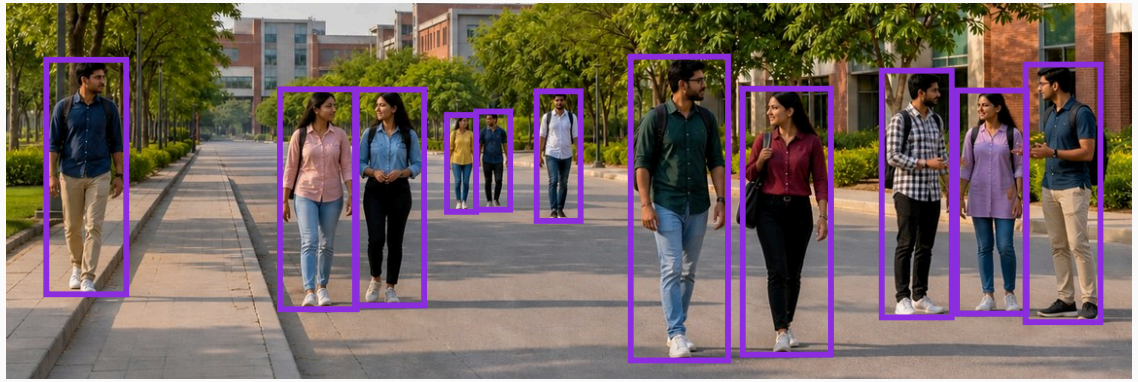
track · link each box to the next frame's

> Track: in a video, run the detector on every frame and link each box to the box that overlaps it most in the next frame.

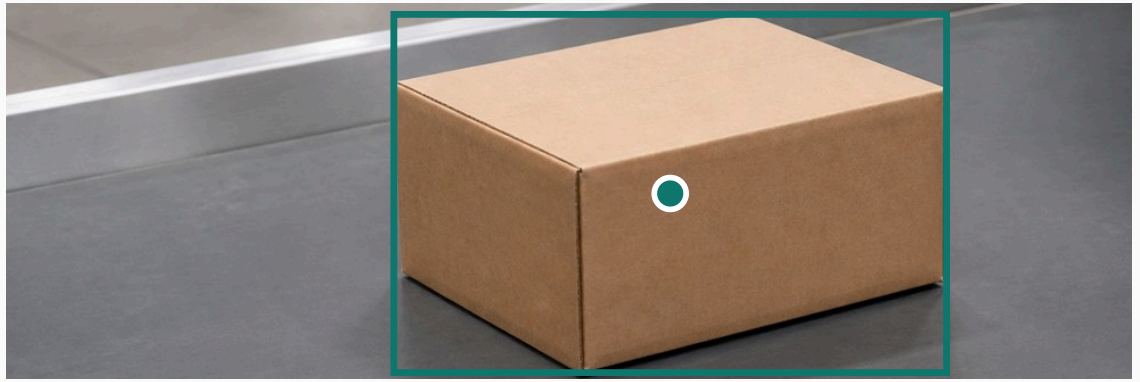
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

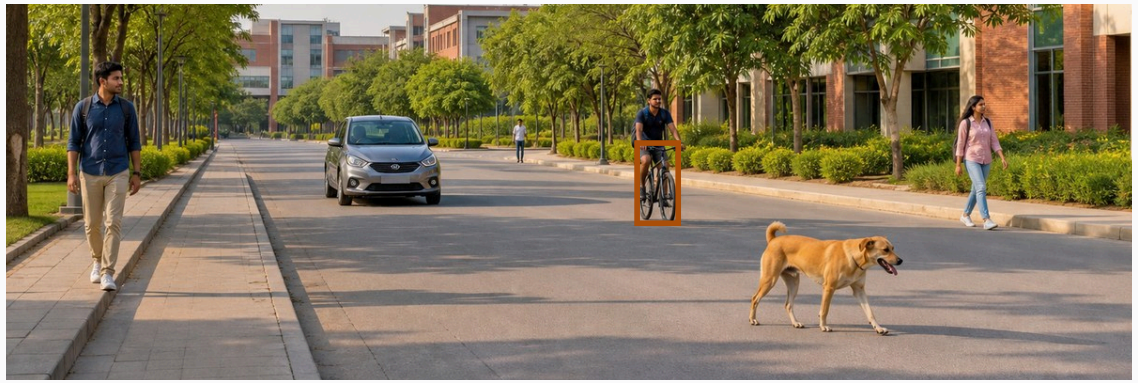
What the list is used for



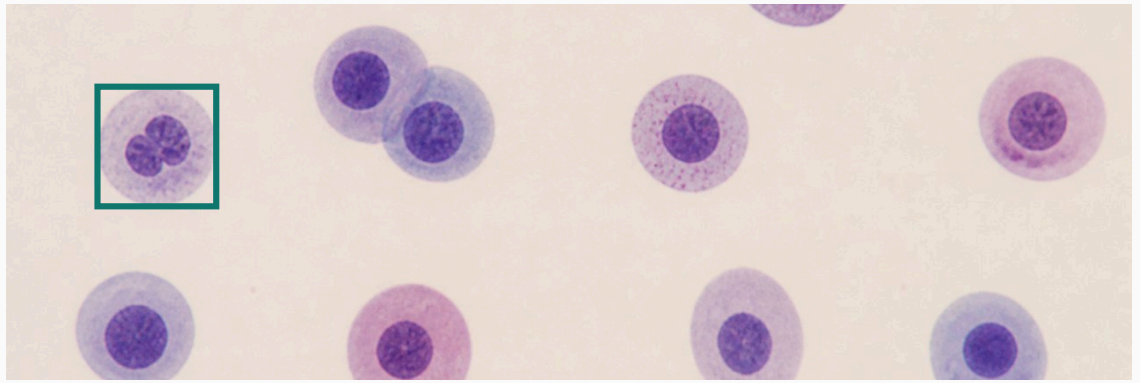
count · 11 people



locate · grip at the box's centre



track · link each box to the next frame's



crop · then name the cell's type

> Crop: cut out each box and pass it to a second model: read a number plate, identify a face, name a cell's type.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells and parcel: our annotation.

▶ Notebook, part 1

Hard, 1: a bus and a distant person in one photo



The bus is 502 pixels wide.

> A bus in the foreground of a street photo.

Boxes: our annotation (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

Hard, 1: a bus and a distant person in one photo



The bus is 502 pixels wide.

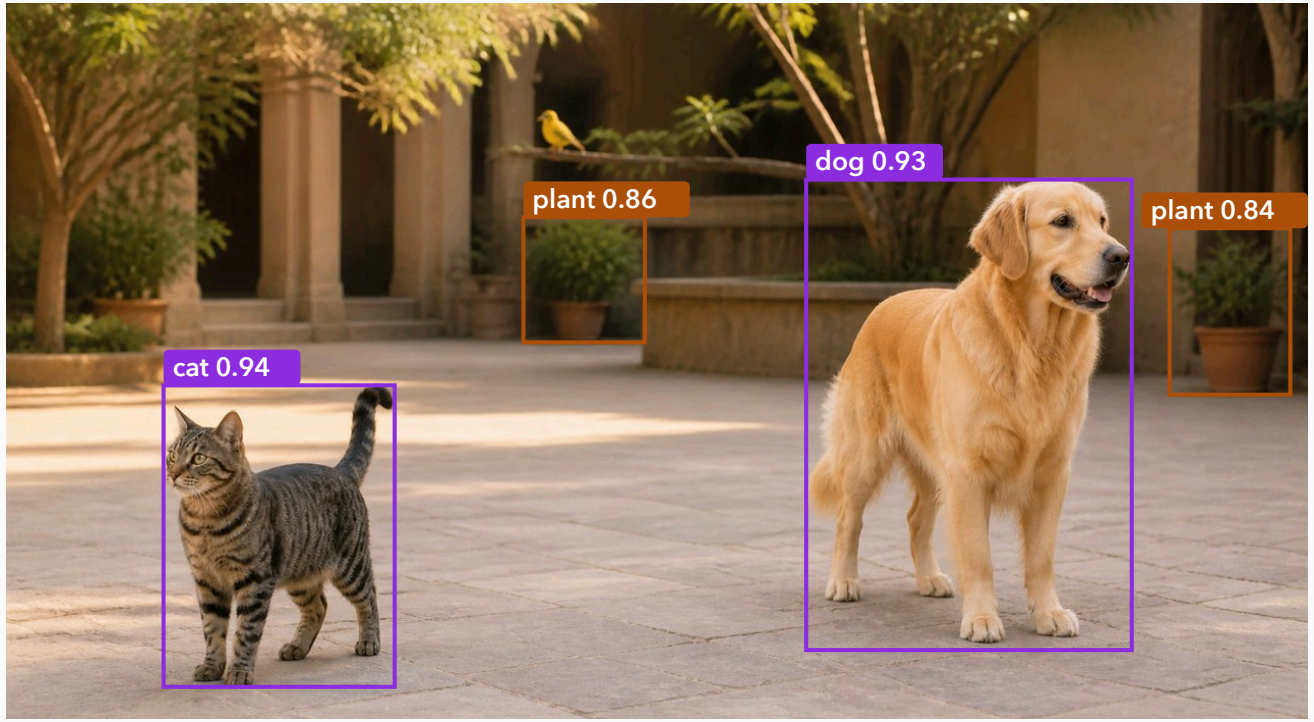
The far pedestrian: 13 pixels, 39 times narrower.

> And a pedestrian far down the same road. A classifier looks for one main thing; a detector must box both, at very different sizes.

Boxes: our annotation (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 1

Hard, 2: small objects are easy to miss



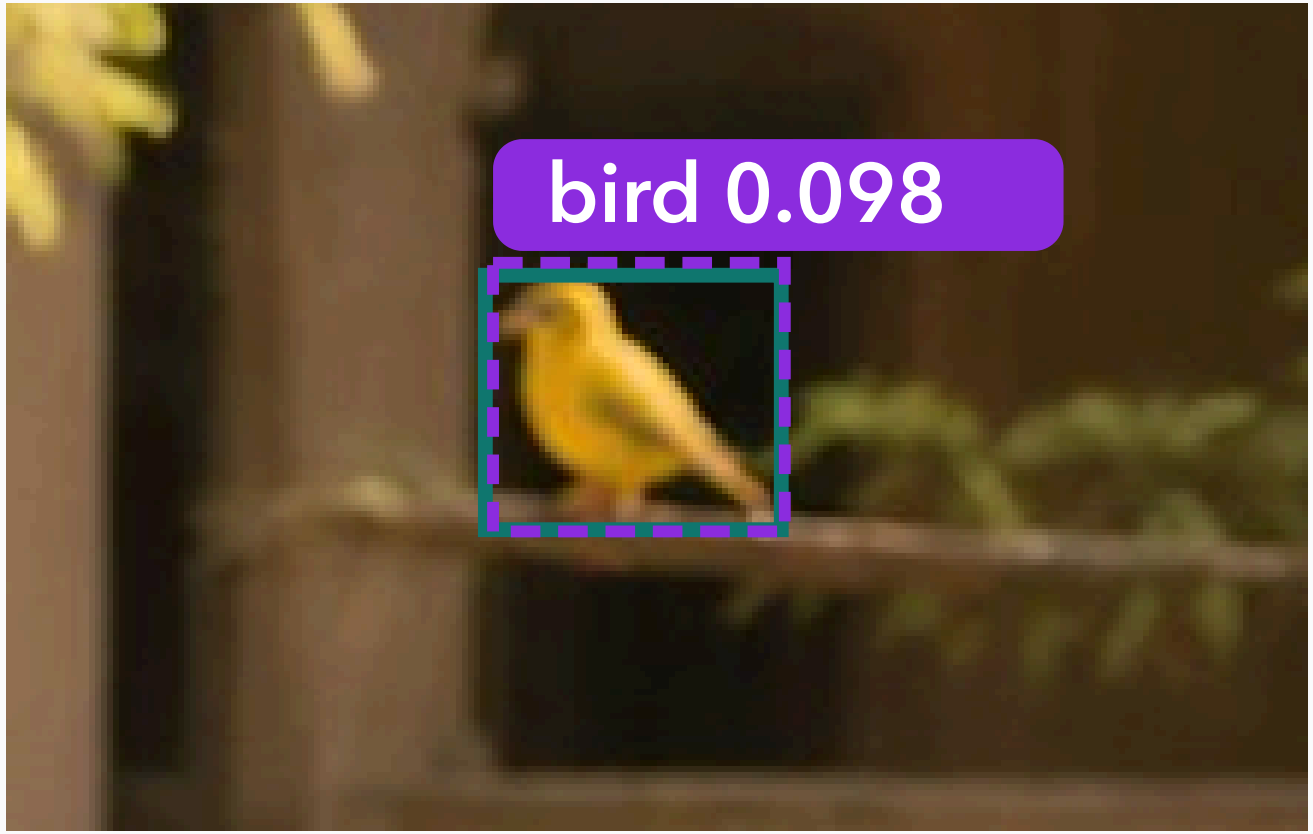
YOLOv8n's four detections: cat, dog, two plants.

> Back to the camera trap. YOLOv8n reports a cat, a dog and two plants. Where is the bird?

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Hard, 2: small objects are easy to miss



The bird is 50 × 43 pixels. One cell did draw a box on it, IoU **0.913** with ours, but scored it **0.098**.

Below the usual cut-off of 0.25, so the box is thrown away.

> Zoom in. One cell boxed the bird almost perfectly but was unsure what it was, so the box was dropped.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Hard, 2: small objects are easy to miss



The bird is 50 × 43 pixels. One cell did draw a box on it, IoU **0.913** with ours, but scored it **0.098**.

Below the usual cut-off of 0.25, so the box is thrown away.

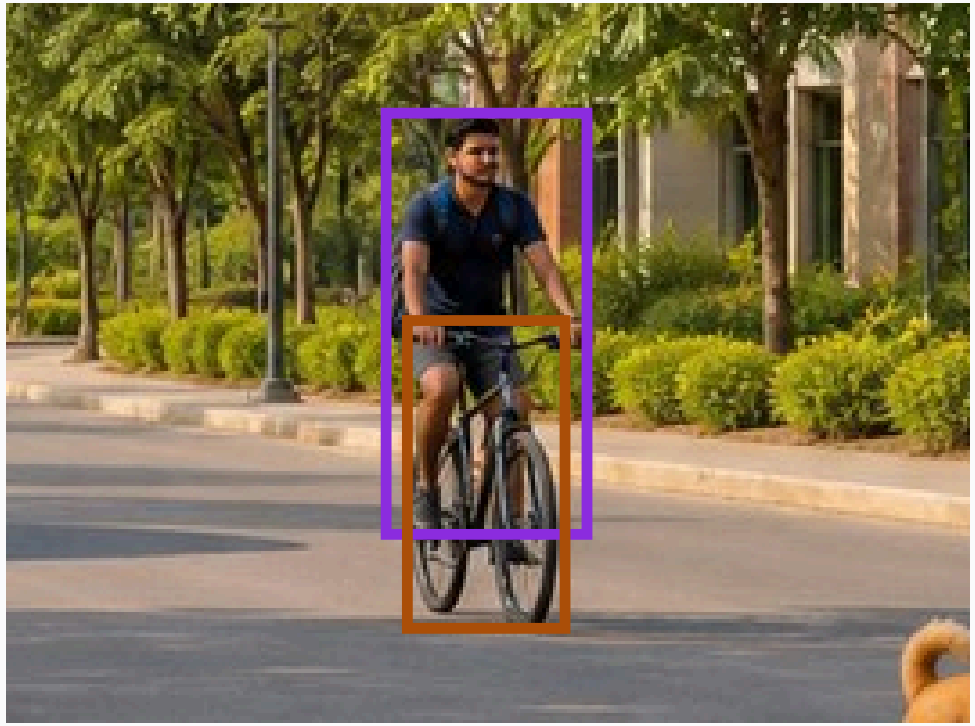
Its next guess for that cell: banana, 0.059, also small and yellow.

> With so few pixels the network has little detail to go on, so small objects get low, unsure scores.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 1

Hard, 3: objects overlap



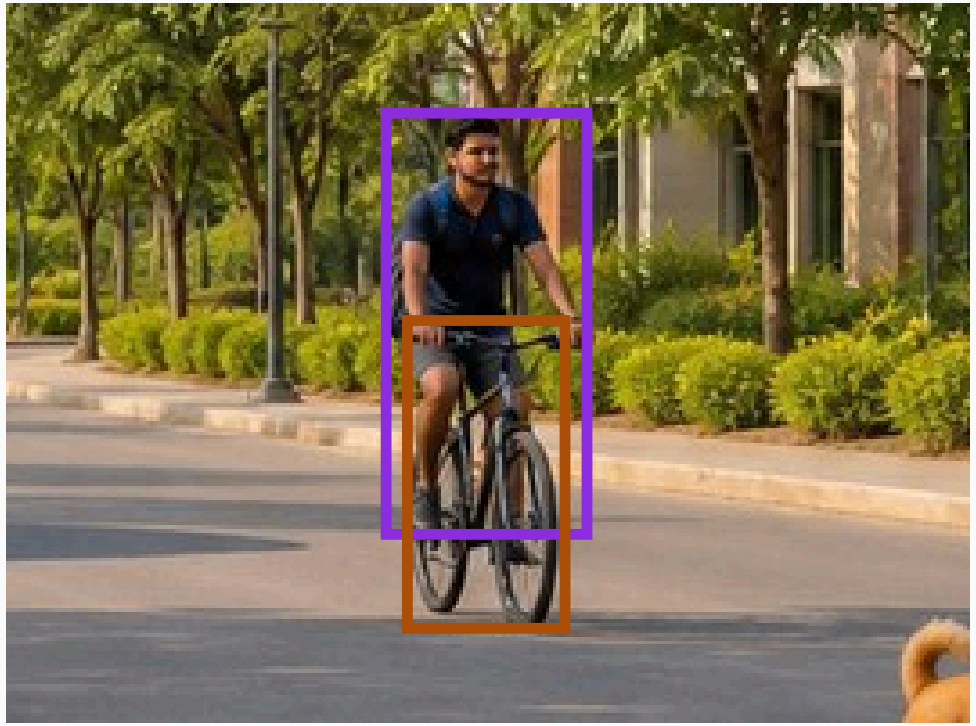
a cyclist: a person and a bicycle

> A cyclist is two objects in one place: a person box and a bicycle box, one largely on top of the other. Both are correct.

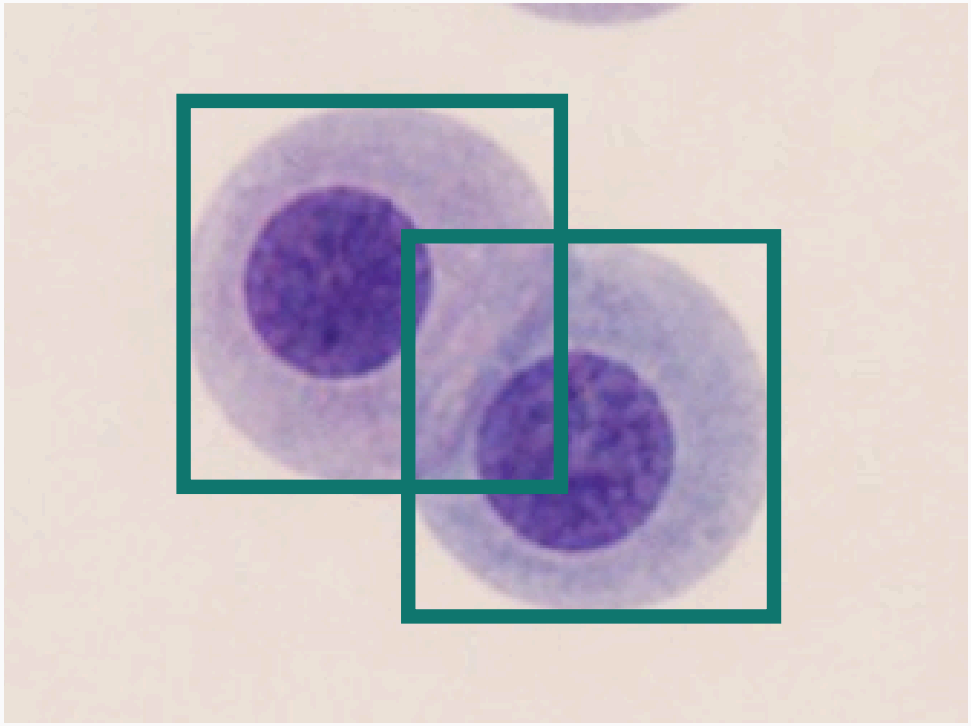
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells: our annotation.

▶ Notebook, part 1

Hard, 3: objects overlap



a cyclist: a person and a bicycle



two touching cells

> Two touching cells: their boxes overlap too. A detector must keep both entries, and yet must not report one object twice.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json. Cells: our annotation.

Part 1 in one slide

1. Detection answers what is where: one (class, box, score) per object.
2. The same kind of answer works for six very different jobs.
3. The list is used to count, locate, track and crop.
4. Hard: sizes vary 40×; small objects get unsure scores; objects overlap.

Next: why a classifier cannot give this list, and what a network would have to output.

▶ Notebook, part 2

Part 2 · From one label to a list

A classifier answers with one word. We need a list. Why is a list hard for a network?

1. What a classifier says about one photo, and what we want instead.
2. Two awkward facts about the list: its length changes, and its order means nothing.
3. The route through the rest of the lecture.

> A classifier gives one label for the whole photo; detection needs one entry per object.

▶ Notebook, part 2

What is in this photo?



> Look at the photo. In one sentence, what is in it?

ResNet-50 with torchvision's IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 2

What is in this photo?



ImageNet class	probability
golden retriever	0.192
flat-coated retriever	0.015
tiger cat	0.013
Labrador retriever	0.012
tabby	0.010

A classifier answers with one label for the whole photo. ResNet-50, trained on ImageNet: golden retriever, 0.19. Every row is a guess about the same single label.

ResNet-50 with torchvision's IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 2

What is in this photo?



ImageNet class	probability
golden retriever	0.192
flat-coated retriever	0.015
tiger cat	0.013
Labrador retriever	0.012
tabby	0.010

Its 118 dog classes together hold 0.39; the five cat classes, 0.03.

> The cat appears only as a weak runner-up, tiger cat at 0.013. The answer cannot say “a dog and a cat”, how many, or where.

ResNet-50 with torchvision’s IMAGENET1K_V2 weights, run on this photo (scripts/run_models.py).

▶ Notebook, part 2

What we want: a list, one entry per object



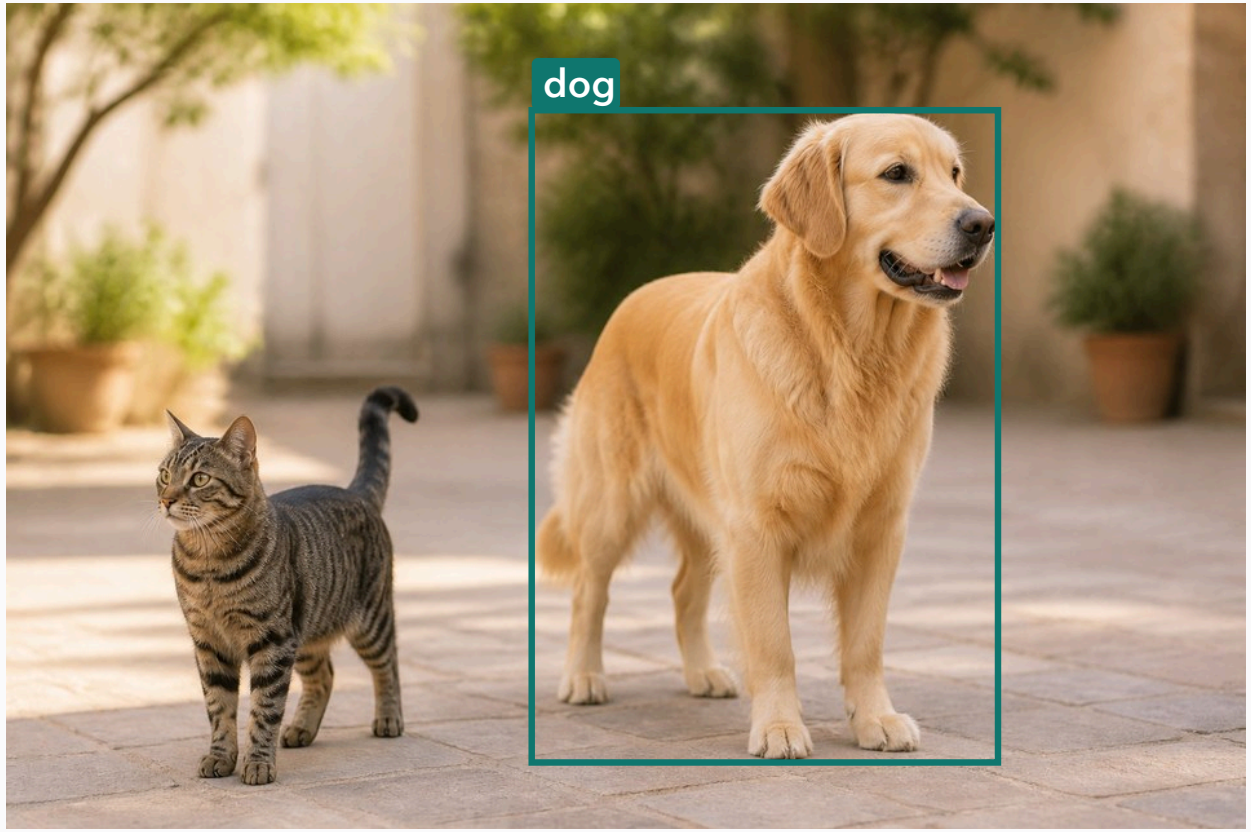
class	box: left, top, right, bottom (pixels)

> Detection answers with a list. Each entry is one object: a class, and a box around it.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

What we want: a list, one entry per object



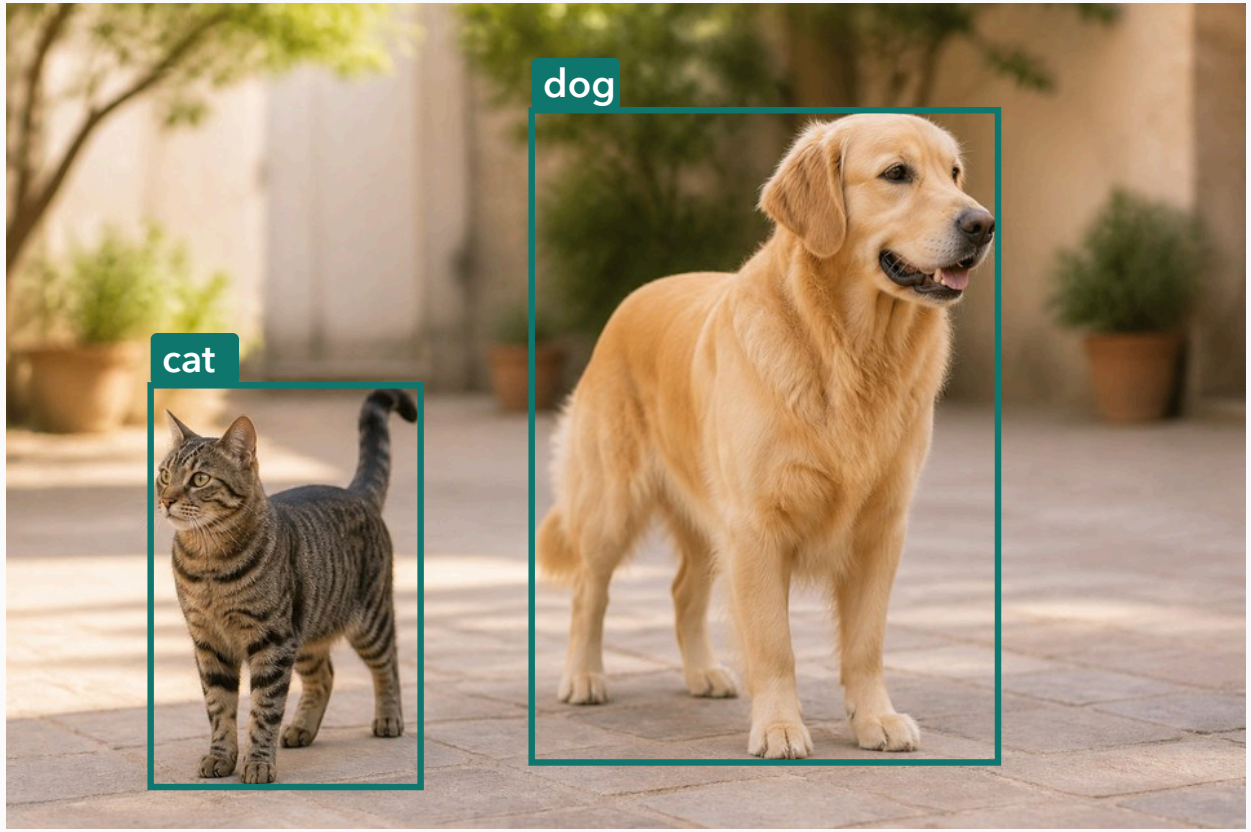
class	box: left, top, right, bottom (pixels)
dog	(407, 83, 768, 588)

First entry: dog. Its box has left edge 407, top 83, right 768 and bottom 588, in pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

What we want: a list, one entry per object



class	box: left, top, right, bottom (pixels)
dog	(407, 83, 768, 588)
cat	(112, 296, 321, 607)

The boxes are our annotation of this photo, in pixels from its top-left corner.

> Second entry: cat. Each object gets one entry, so a photo with five objects gets five.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

Four tasks, four kinds of answer



Classification: one label

> Classification: one label for the whole photo.

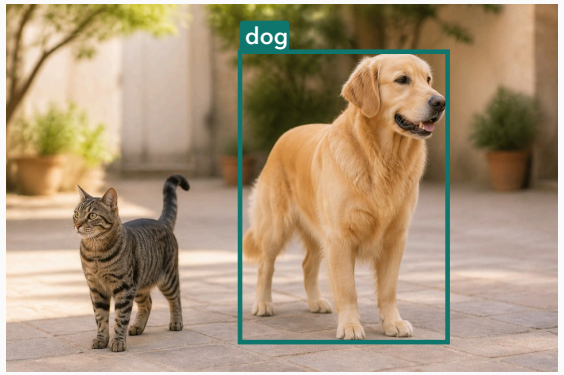
Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

Four tasks, four kinds of answer



Classification: one label



Localisation: one label, one box

> Localisation: one object, its label and one box.

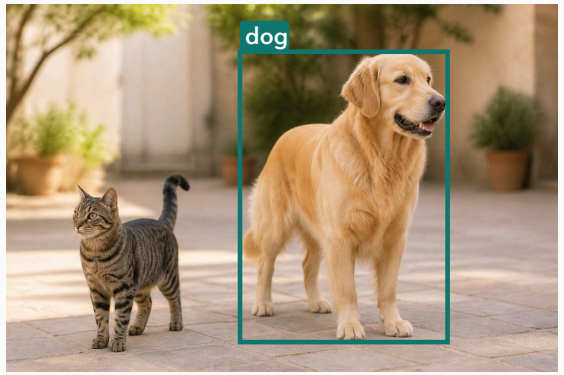
Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

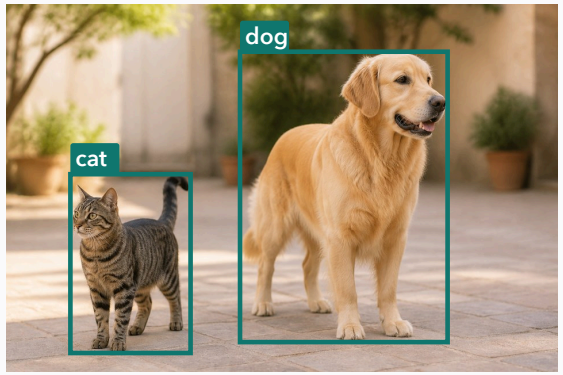
Four tasks, four kinds of answer



Classification: one label



Localisation: one label, one box



Detection: a class and a box per object

> Detection: every object, each with a class and a box. Today's task.

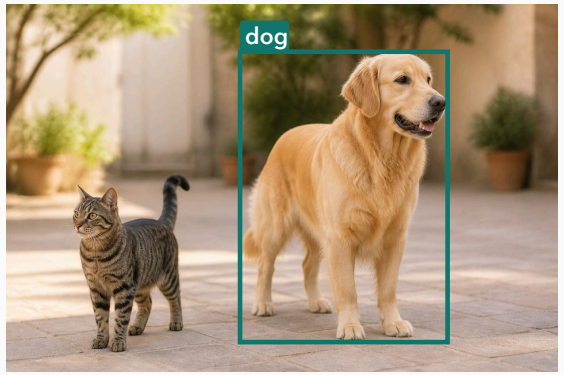
Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

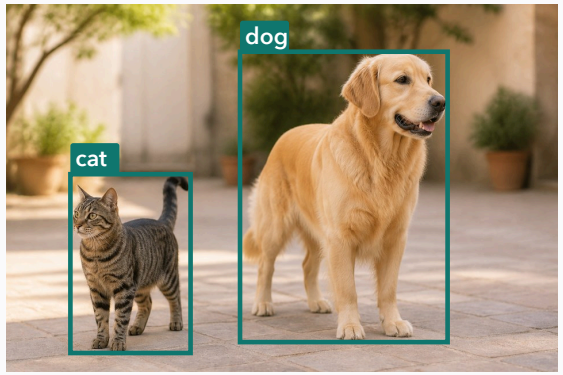
Four tasks, four kinds of answer



Classification: one label



Localisation: one label, one box



Detection: a class and a box per object



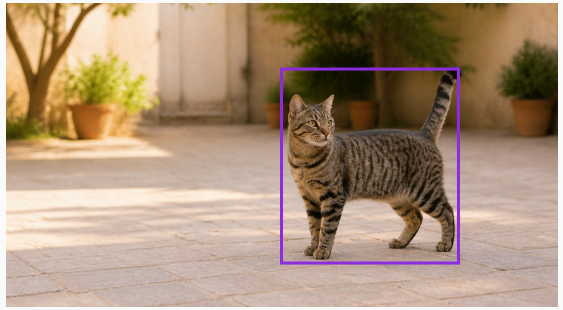
Segmentation: which pixels (Lecture 23)

> Segmentation: which pixels belong to each object. The masks here are from a real YOLOv8n-seg run; this is next lecture.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

The number of objects changes from photo to photo



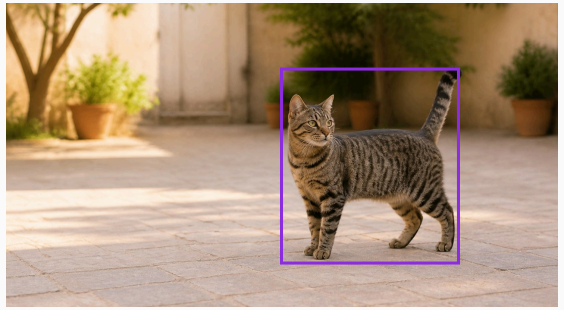
1 · one cat

> One cat: a list of one.

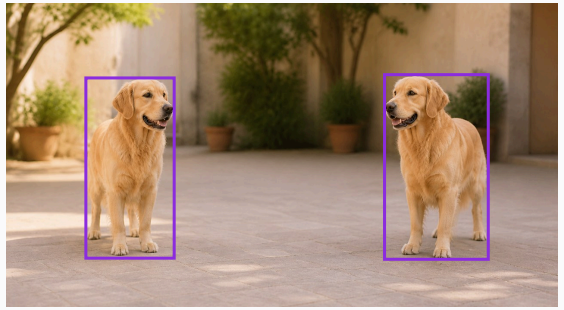
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

The number of objects changes from photo to photo



1 · one cat



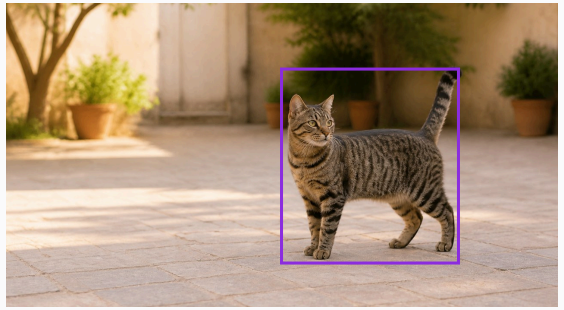
2 · two dogs

> Two dogs: a list of two.

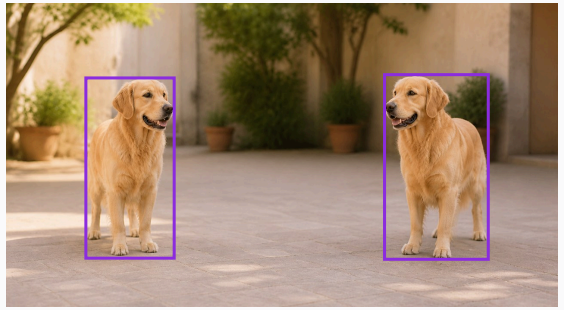
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

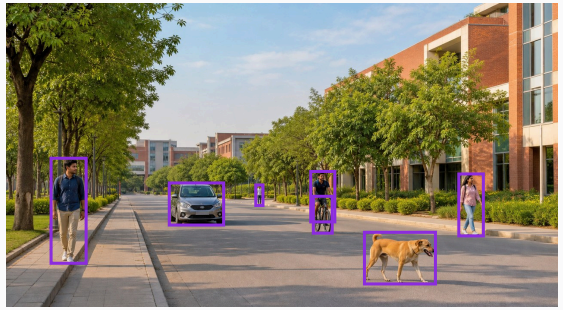
The number of objects changes from photo to photo



1 · one cat



2 · two dogs



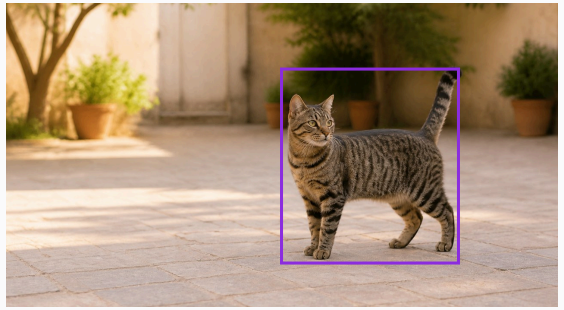
7 · a street

A street: seven.

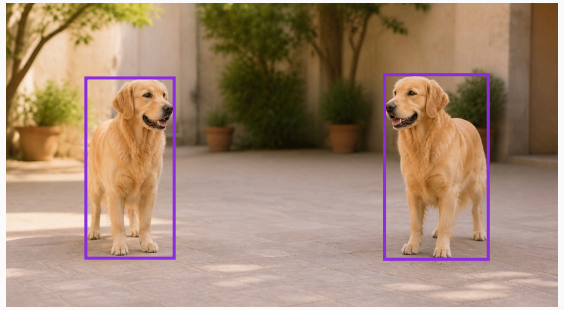
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

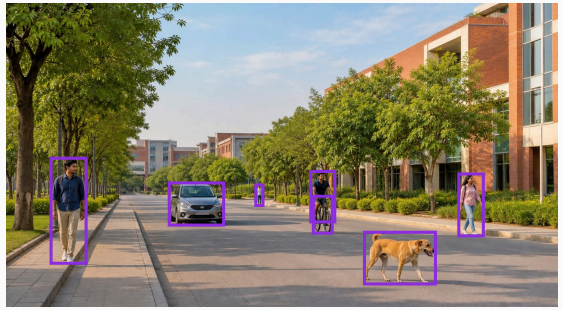
The number of objects changes from photo to photo



1 · one cat



2 · two dogs



7 · a street



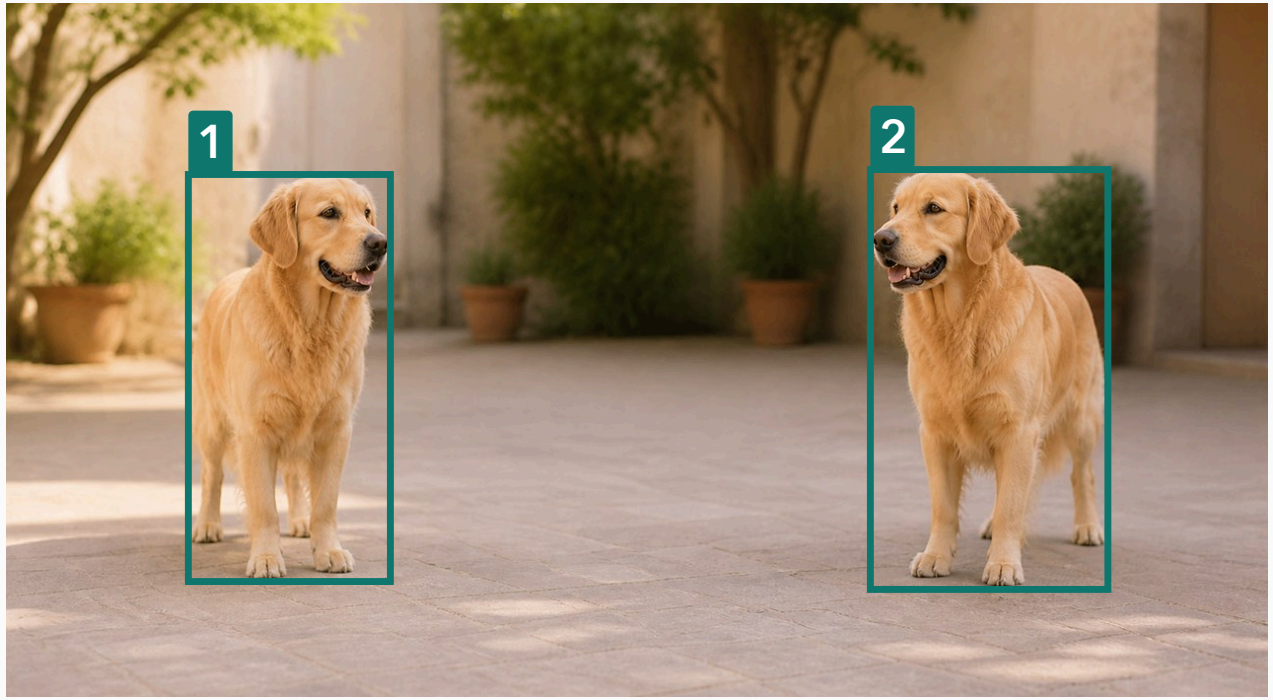
11 · a path of people

A busy path: eleven people. The length of the answer depends on the photo.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 2

A list with no order: a set



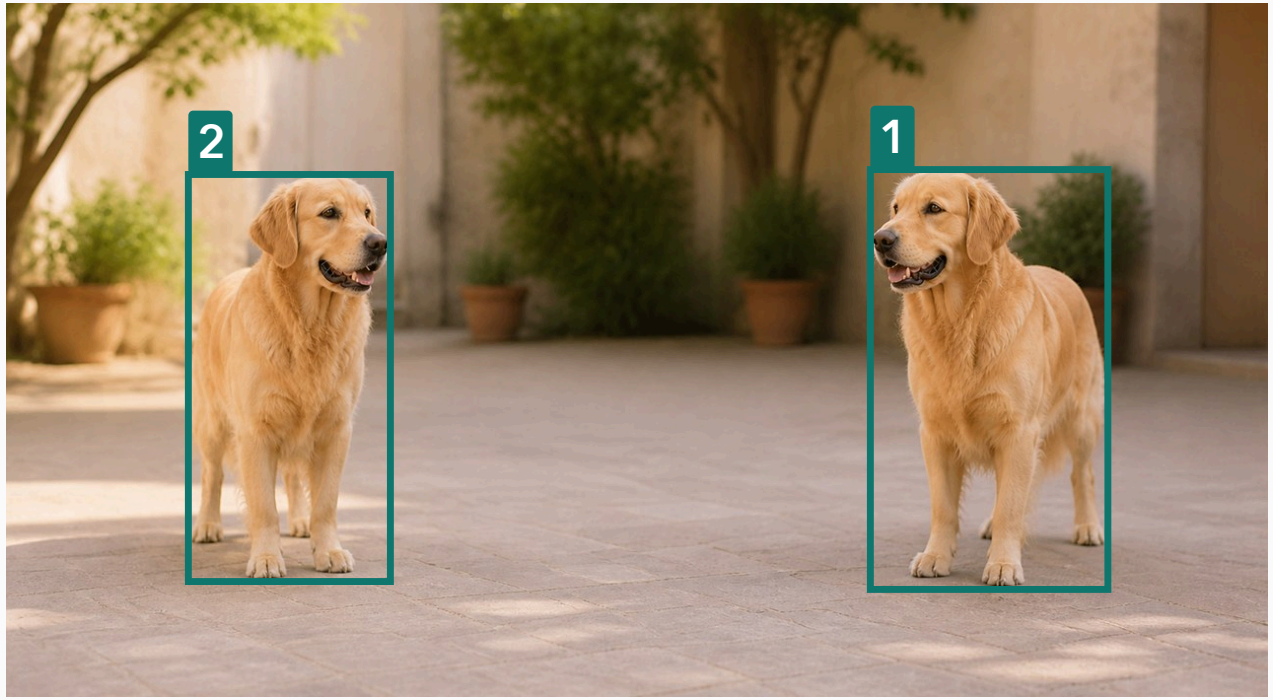
$$Y = \{(c_1, b_1), \ldots, (c_N, b_N)\}$$

> Number the two dogs: left one first.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

A list with no order: a set



$$Y = \{(c_1, b_1), \ldots, (c_N, b_N)\}$$

Number the dogs either way and the answer is the same. What must not change is which class goes with which box.

> Or right one first. Both lists describe the same scene: the answer is a set of N (class, box) pairs, N varies, and order means nothing.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 2

A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



A classifier's last layer always has the same number of outputs: 1,000 for ImageNet, for every photo.

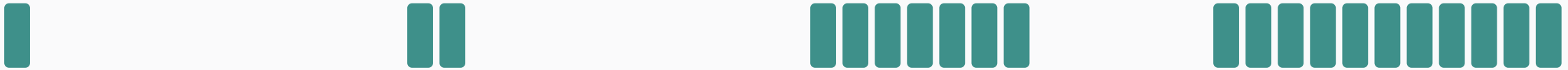
▶ Notebook, part 2

A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



the answer we need: N entries, N = 1, 2, 7, 11, … in any order



N = 1 N = 2 N = 7 N = 11

> We need a list whose length changes with the photo, and whose order means nothing.

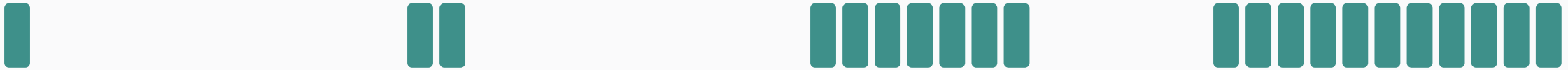
▶ Notebook, part 2

A network's output has a fixed shape

a classifier: always 1,000 numbers (20 drawn), for every photo



the answer we need: N entries, N = 1, 2, 7, 11, … in any order



N = 1 N = 2 N = 7 N = 11

Today's question: **how can a fixed-size output describe a set of any size, in no order?**

> That is the question of the lecture.

▶ Notebook, part 2

Today's route: build a detector step by step

3. **Boxes and IoU**: four numbers for a box; IoU to compare two
4. **One box, then many**: one box works; two animals break it

> First the tools; then a network that outputs boxes, to see where a fixed output breaks.

▶ Notebook, part 2

Today's route: build a detector step by step

3. **Boxes and IoU**: four numbers for a box; IoU to compare two
4. **One box, then many**: one box works; two animals break it
5. **A candidate at every place**: every grid cell writes a box and class scores
6. **NMS**: keep one box per object
7. **Three grids**: small and large objects
8. **Measuring**: precision, recall and AP

> Then YOLO's answer: a candidate at every place, cleaned up by NMS, at three sizes; and how to measure it.

▶ Notebook, part 3

Part 3 · Describe a box, compare two boxes

Every later step needs two small tools.

1. Write a box as four numbers, in two common formats, and see how a labelled photo is stored.
2. Measure how well two boxes agree: intersection over union, IoU, computed by hand.
3. Try IoU live, then write it in six lines of PyTorch.

> Two tools: four numbers for a box, and IoU to compare two boxes.

▶ Notebook, part 3

Positions in a photo: pixels from the top-left corner



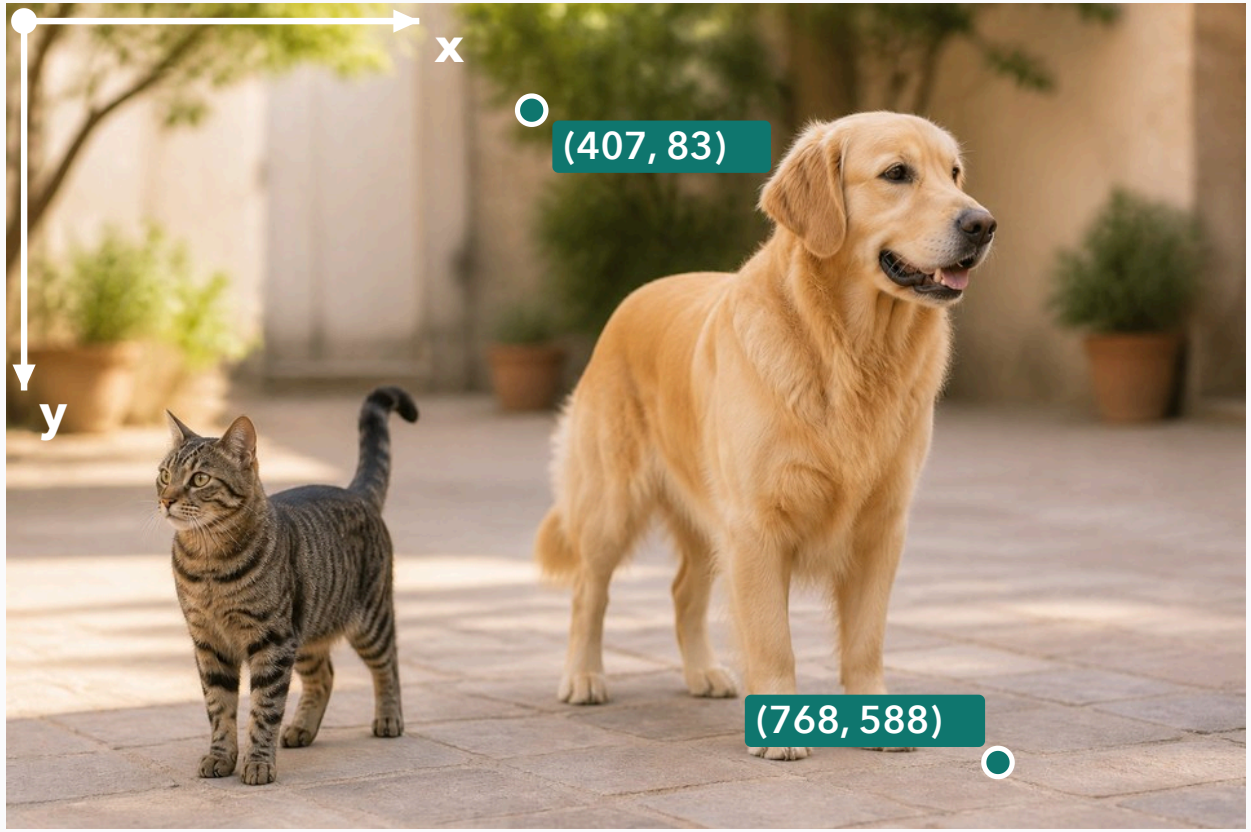
The photo is 960 pixels wide and 640 tall. A point is (x, y): x counts right from the left edge, y counts **down** from the top.

> Positions are counted in pixels from the top-left corner. Unlike on a school graph, y grows downwards.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Positions in a photo: pixels from the top-left corner



The photo is 960 pixels wide and 640 tall. A point is (x, y): x counts right from the left edge, y counts **down** from the top.

Two corners of the dog: top-left (407, 83), bottom-right (768, 588).

Two corners of the dog's box. The bottom-right one has the larger y, because it is lower in the photo.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

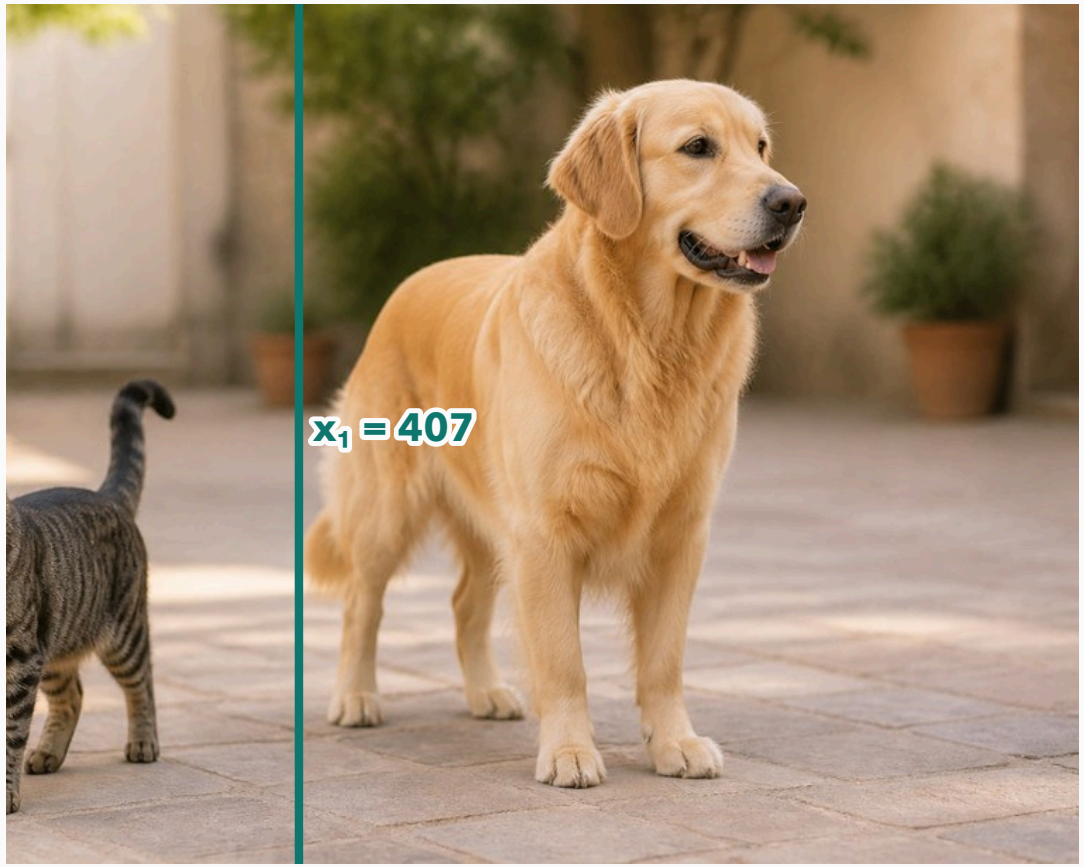
$$(x_1,\ y_1,\ x_2,\ y_2)$$

A box is the region between two vertical and two horizontal lines. Draw them one at a time.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

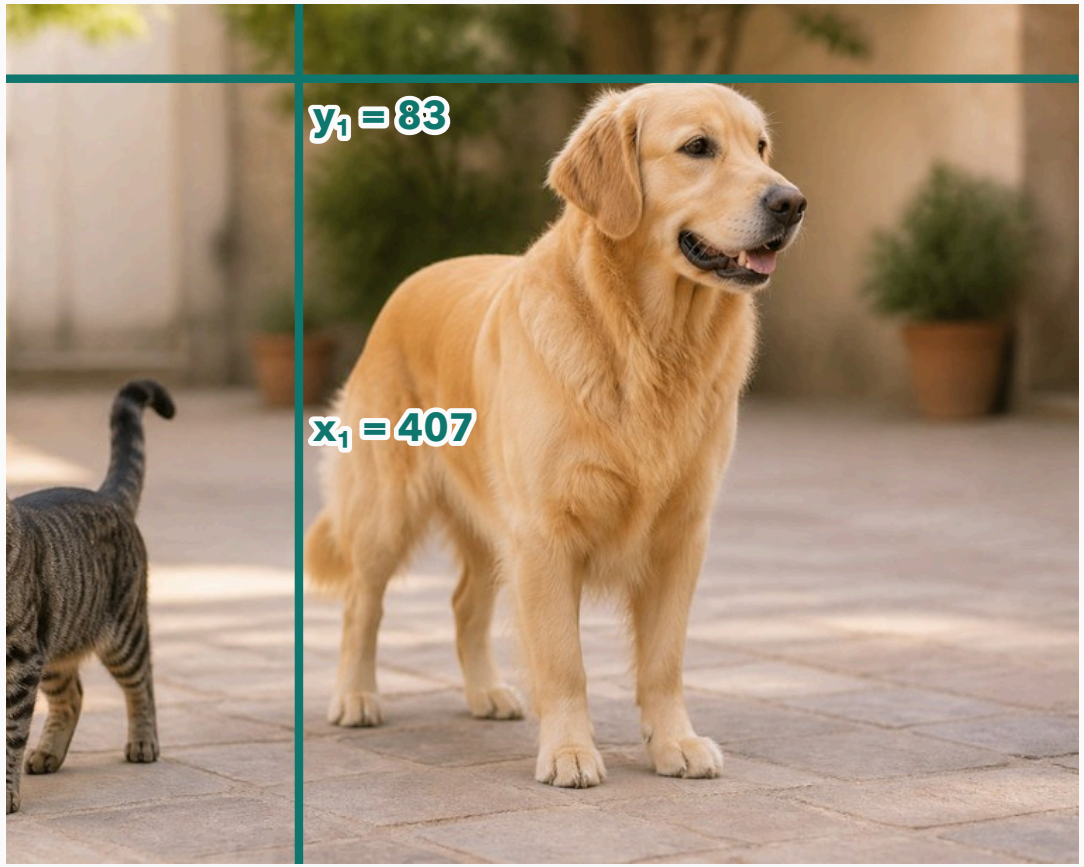
$$(x_1,\ y_1,\ x_2,\ y_2)$$

> Left edge: every point of the dog has x at least 407.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

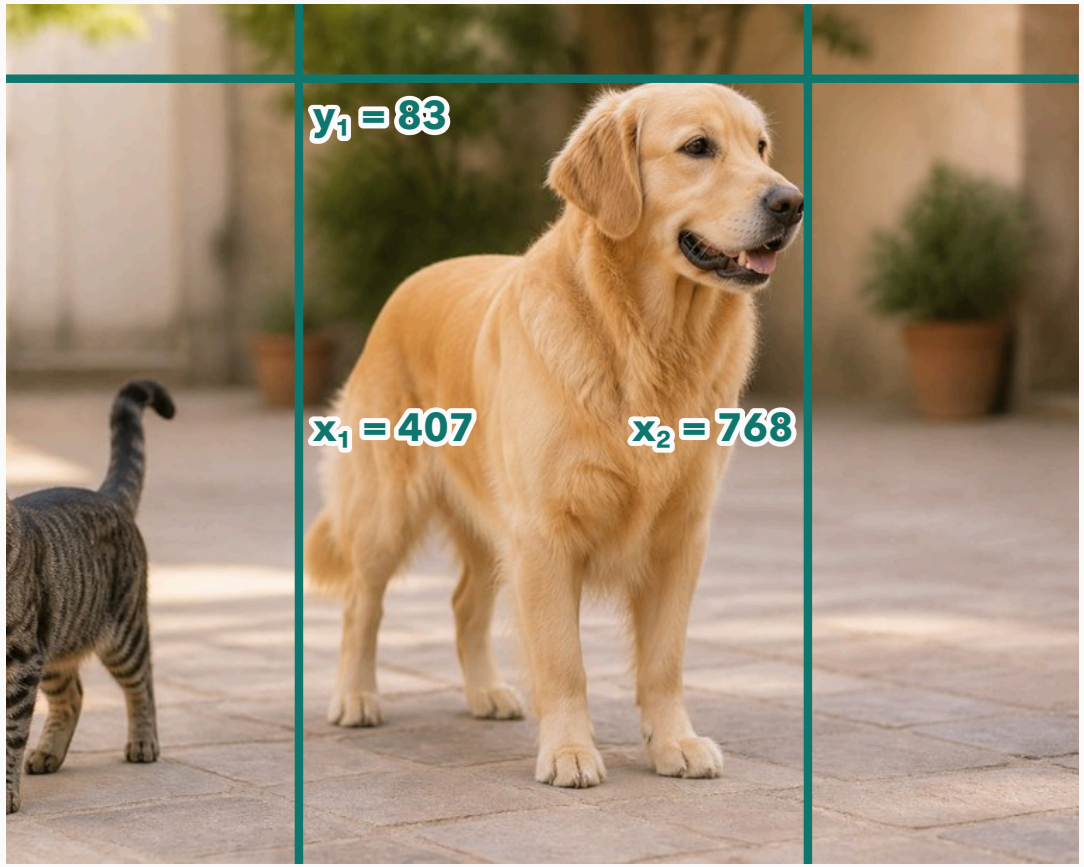
$$(x_1,\ y_1,\ x_2,\ y_2)$$

> Top edge: y at least 83, the tips of the ears.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

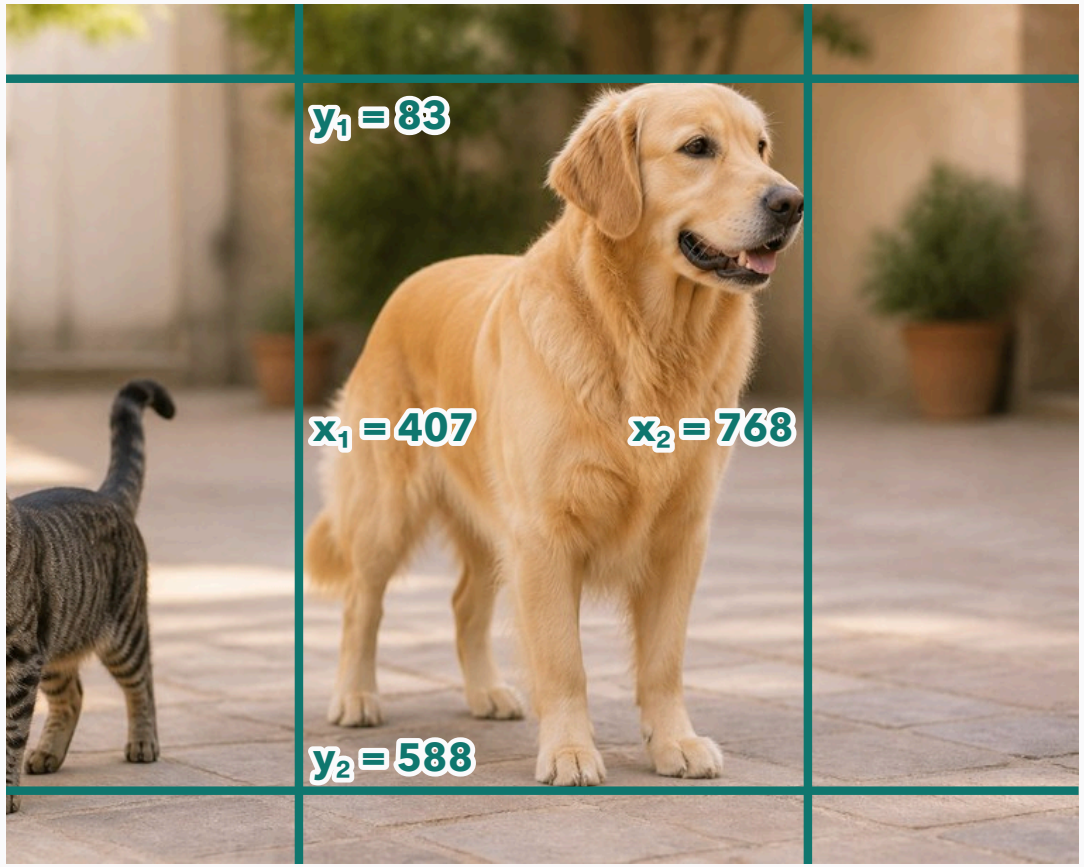
$$(x_1,\ y_1,\ x_2,\ y_2)$$

> Right edge: x at most 768, the nose.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

A box is four numbers: its left, top, right and bottom edges



Corner format:

$$(x_1,\ y_1,\ x_2,\ y_2)$$

$$= (407,\ 83,\ 768,\ 588)$$

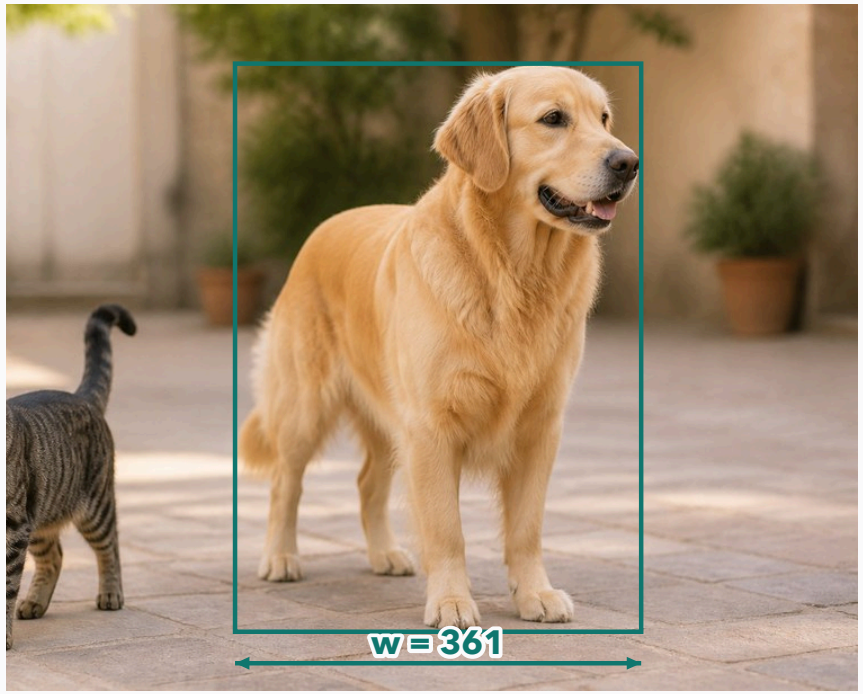
(x_1, y_1) is the top-left corner, (x_2, y_2) the bottom-right. torchvision uses this format.

> Bottom edge: y at most 588, the paws. Four numbers: (407, 83, 768, 588).

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The box's size: width and height



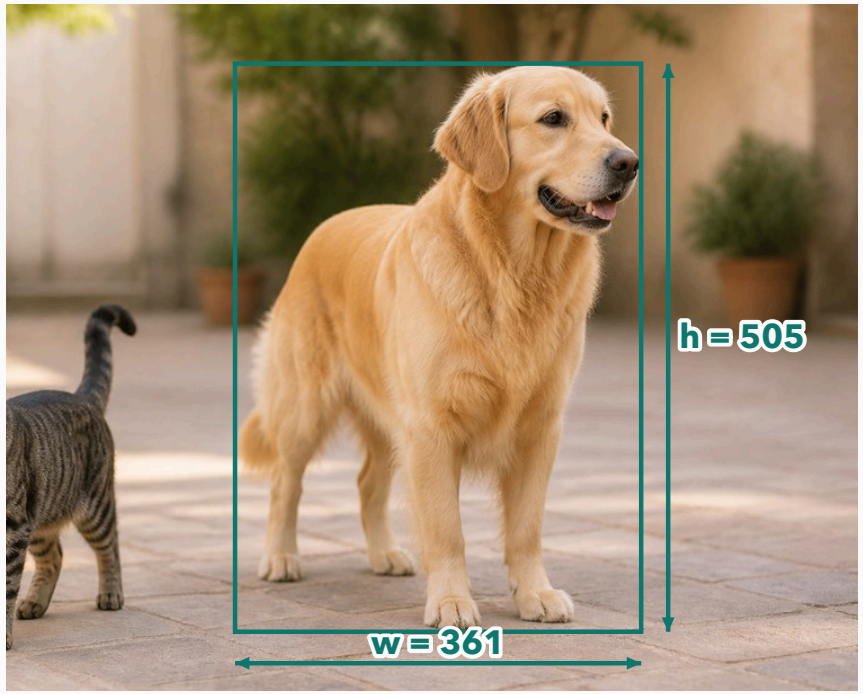
$$w = x_2 - x_1 = 768 - 407 = 361$$

> Width: the right edge minus the left edge, 768 − 407 = 361 pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The box's size: width and height



$$w = x_2 - x_1 = 768 - 407 = 361$$

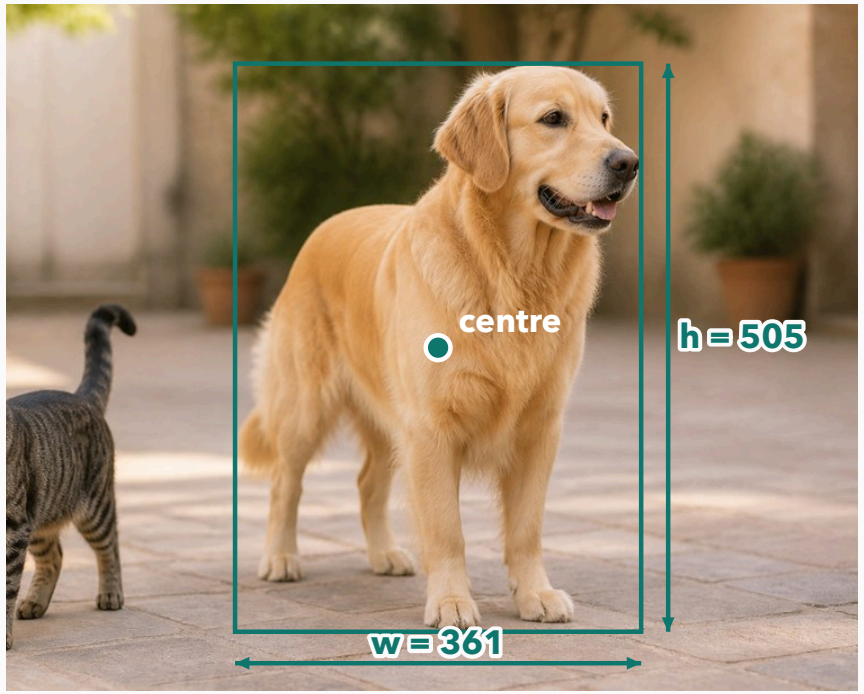
$$h = y_2 - y_1 = 588 - 83 = 505$$

Height: the bottom minus the top, 588 − 83 = 505. Bottom minus top, because y grows downwards.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same box as a centre and a size



$$c_x = x_1 + \frac{w}{2} = 407 + \frac{361}{2} = 587.5$$

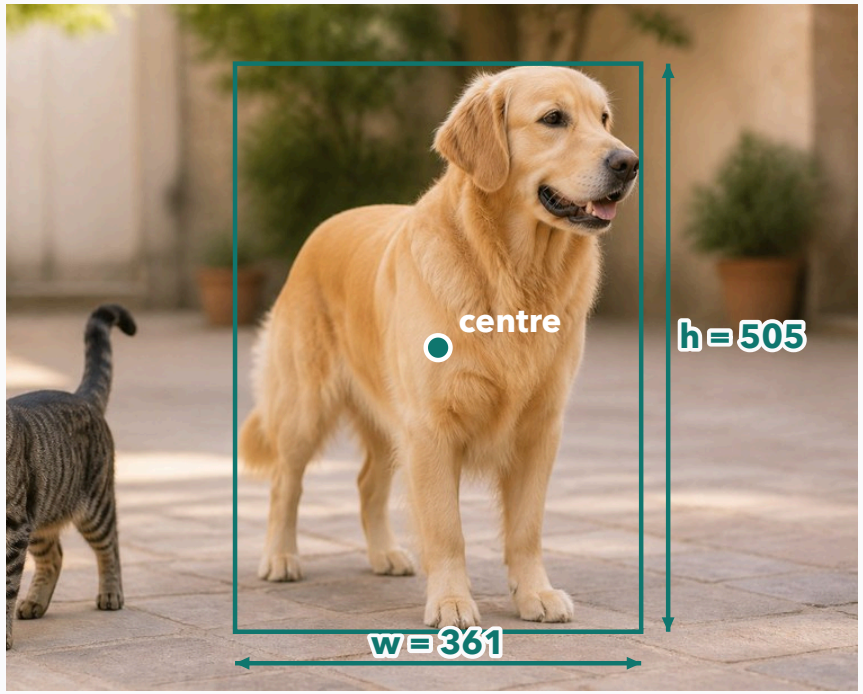
$$c_y = y_1 + \frac{h}{2} = 83 + \frac{505}{2} = 335.5$$

> The centre is halfway across and halfway down: start at the left edge and add half the width; start at the top and add half the height.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same box as a centre and a size



$$c_x = x_1 + \frac{w}{2} = 407 + \frac{361}{2} = 587.5$$

$$c_y = y_1 + \frac{h}{2} = 83 + \frac{505}{2} = 335.5$$

$$(c_x, c_y, w, h) = (587.5,\ 335.5,\ 361,\ 505)$$

> Same box, second format: centre (587.5, 335.5), size 361 × 505. Either format can be computed from the other.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

> Divide horizontal numbers by the photo's width, vertical ones by its height.

▶ Notebook, part 3

Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

$$= (0.612, 0.524, 0.376, 0.789)$$

> The dog: (0.612, 0.524, 0.376, 0.789). Every number is between 0 and 1.

▶ Notebook, part 3

Divide by the photo's size: numbers between 0 and 1

$$\left(\frac{c_x}{W}, \frac{c_y}{H}, \frac{w}{W}, \frac{h}{H}\right) = \left(\frac{587.5}{960}, \frac{335.5}{640}, \frac{361}{960}, \frac{505}{640}\right)$$

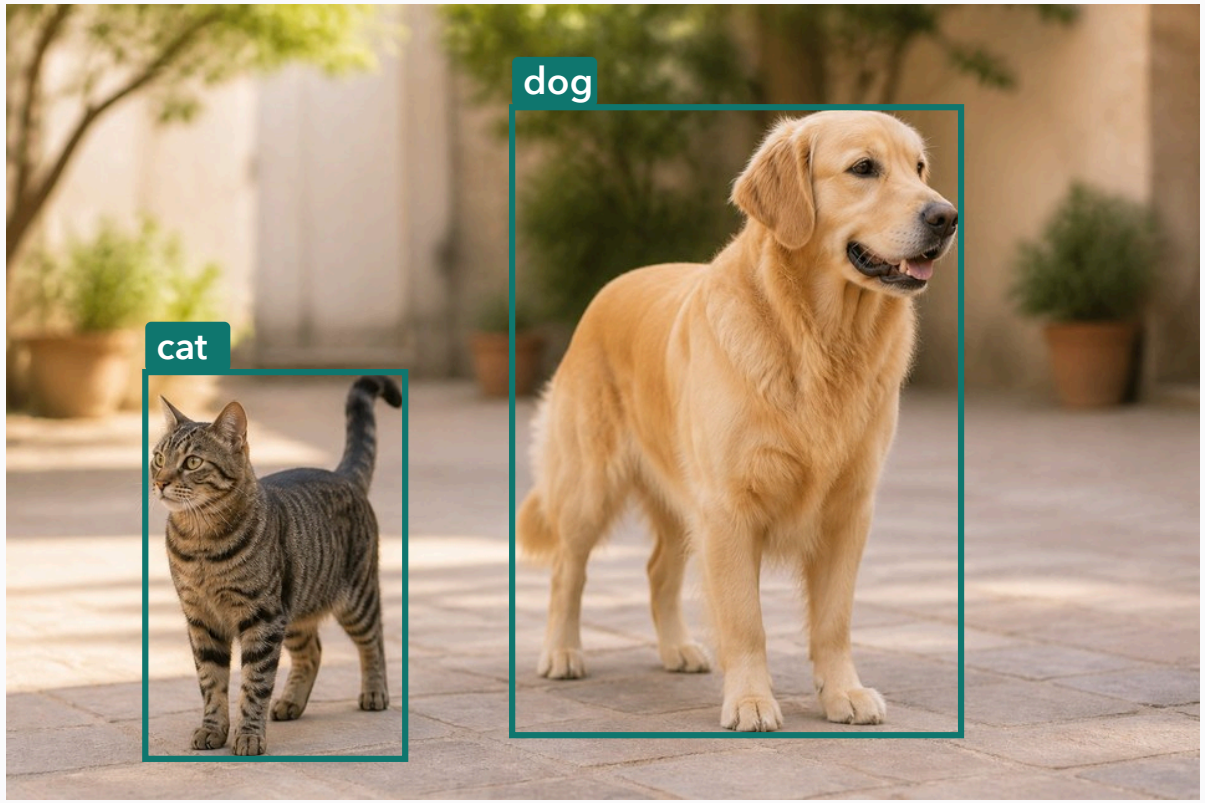
$$= (0.612, 0.524, 0.376, 0.789)$$

Halve the photo to 480 × 320: every pixel number halves, the normalised numbers do not change. The dog's centre is 61% of the way across and 52% of the way down, at any resolution.

> These numbers survive resizing, which is why label files and many networks use them.

▶ Notebook, part 3

A labelled photo on disk: one line per object



`cat-dog.txt`, next to `cat-dog.jpg`:

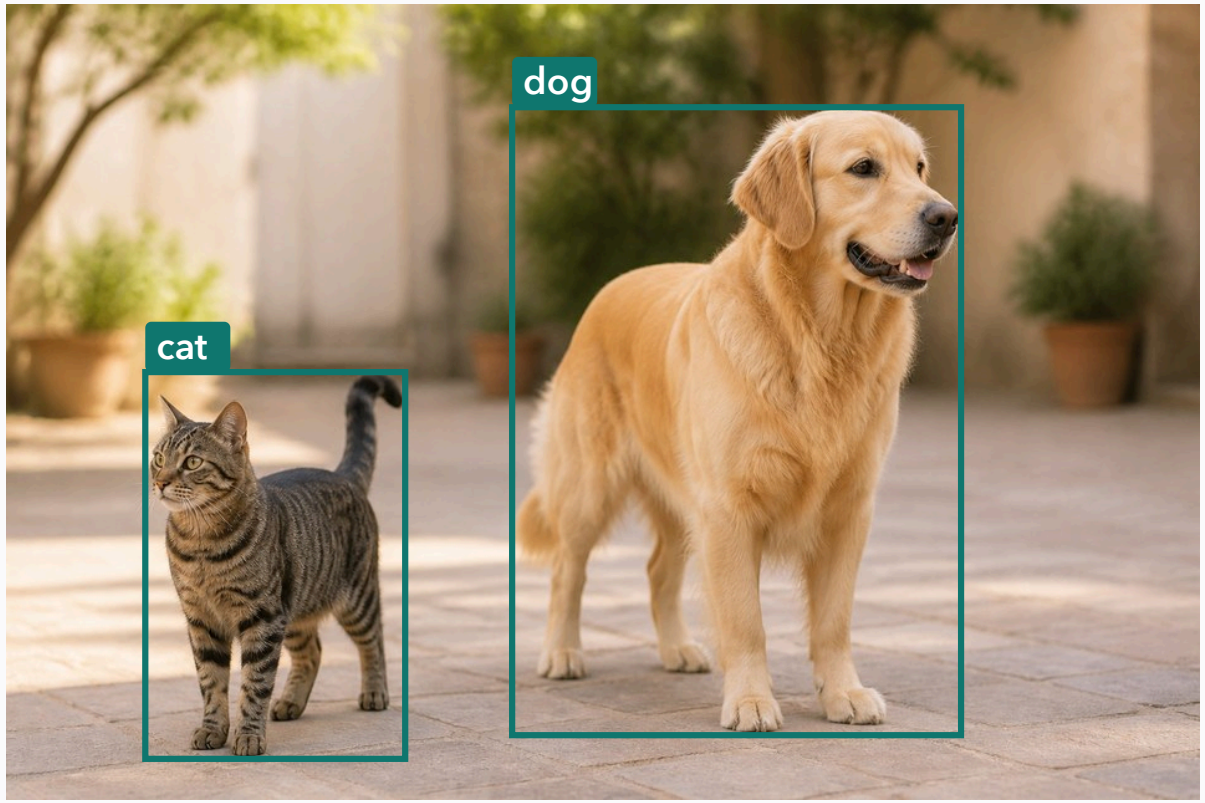
```
16 0.612 0.524 0.376 0.789
15 0.226 0.705 0.218 0.486
```

A training photo is stored with a small text file: one line per object. This is the YOLO format, for our two annotated animals.

Our annotation of the photo (data/annotations.json), written in the YOLO label format.

▶ Notebook, part 3

A labelled photo on disk: one line per object



`cat-dog.txt`, next to `cat-dog.jpg`:

```
16 0.612 0.524 0.376 0.789
15 0.226 0.705 0.218 0.486
```

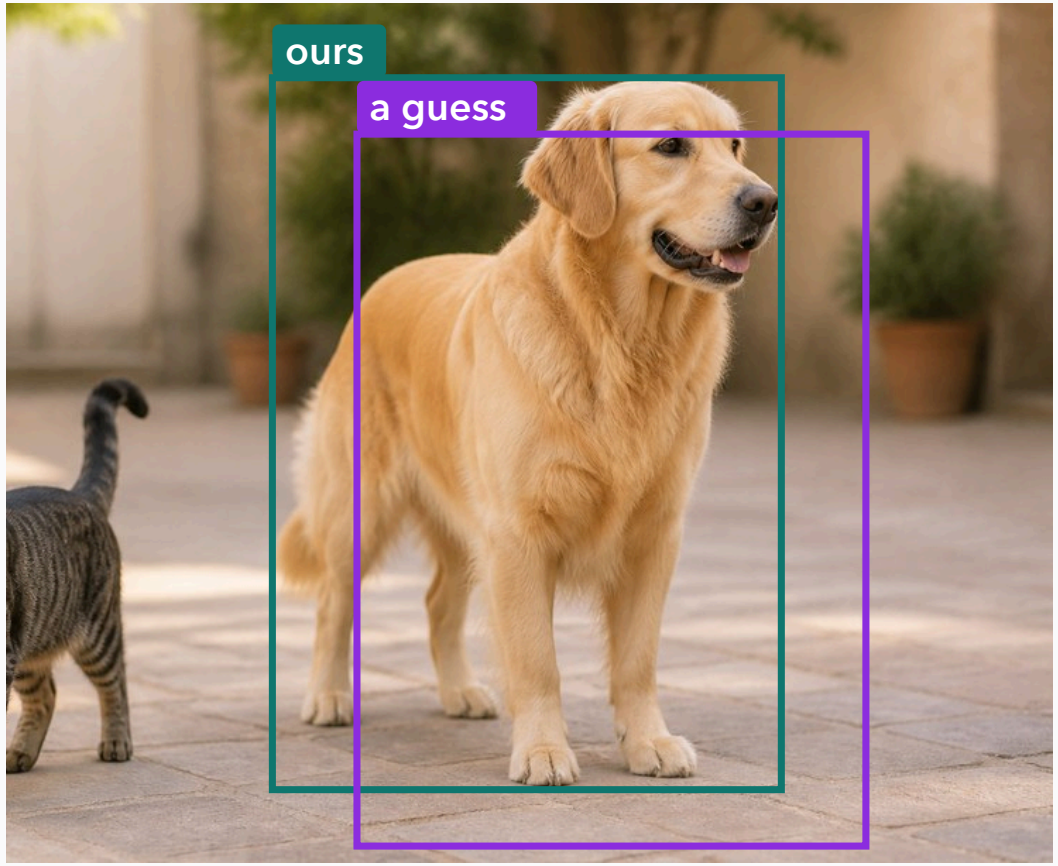
	class	c_x	c_y	w	h
line 1	16 = dog	0.612	0.524	0.376	0.789
line 2	15 = cat	0.226	0.705	0.218	0.486

A class number (COCO counts from 0: 15 is cat, 16 is dog), then the normalised centre and size. A detector learns to write these lines.

Our annotation of the photo (data/annotations.json), written in the YOLO label format.

▶ Notebook, part 3

Two boxes for one dog: how well do they agree?



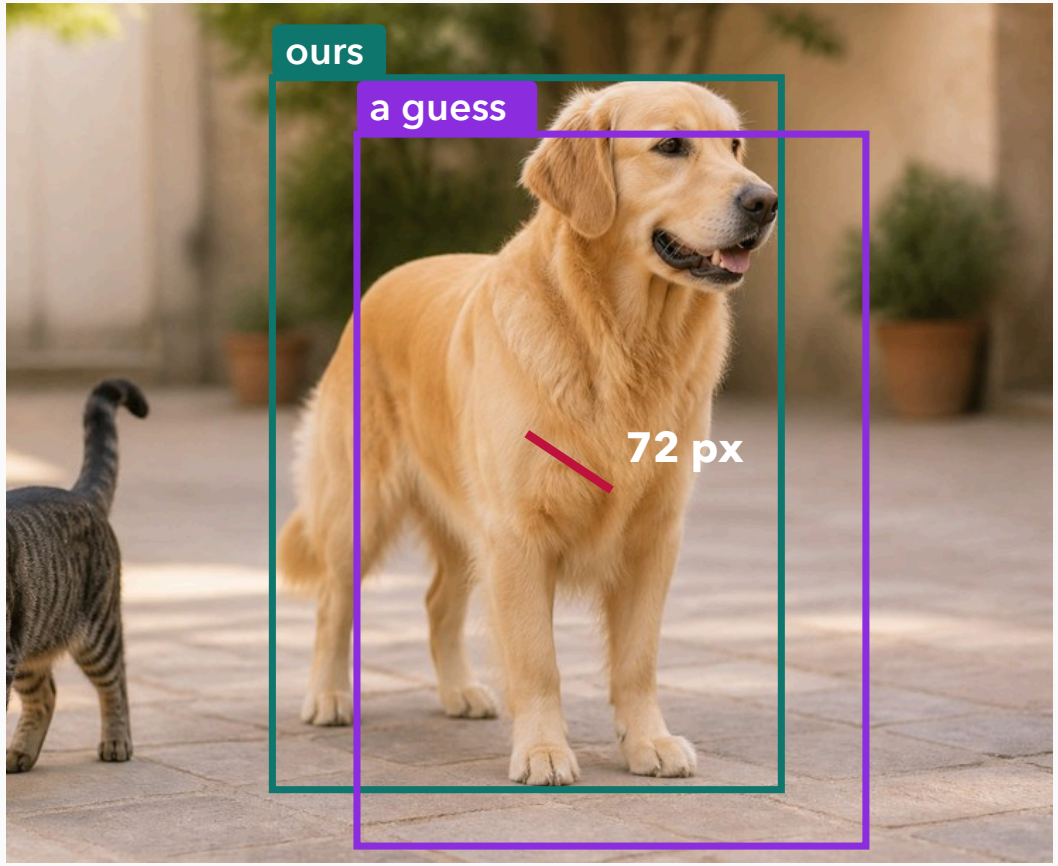
Teal: our box. Purple: a guess, moved 60 px right and 40 px down.

> Teal is our box for the dog. Purple is a guess, moved 60 px right and 40 px down. How good is the guess?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Two boxes for one dog: how well do they agree?



Teal: our box. Purple: a guess, moved 60 px right and 40 px down.

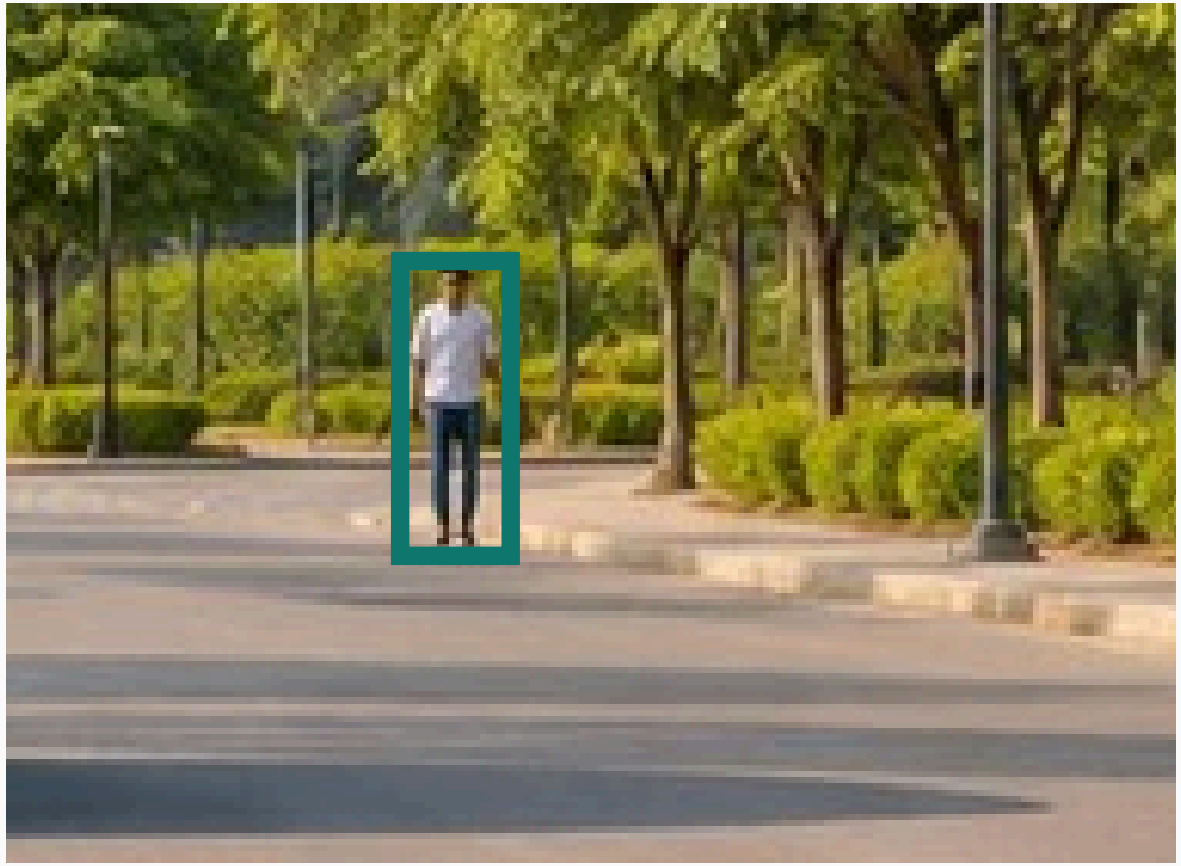
Idea 1: the distance between the centres, $\sqrt{(60^2 + 40^2)} = 72$ px.

> One idea: measure how far apart the centres are. Here 72 px. Is that a lot?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same slip on a small object



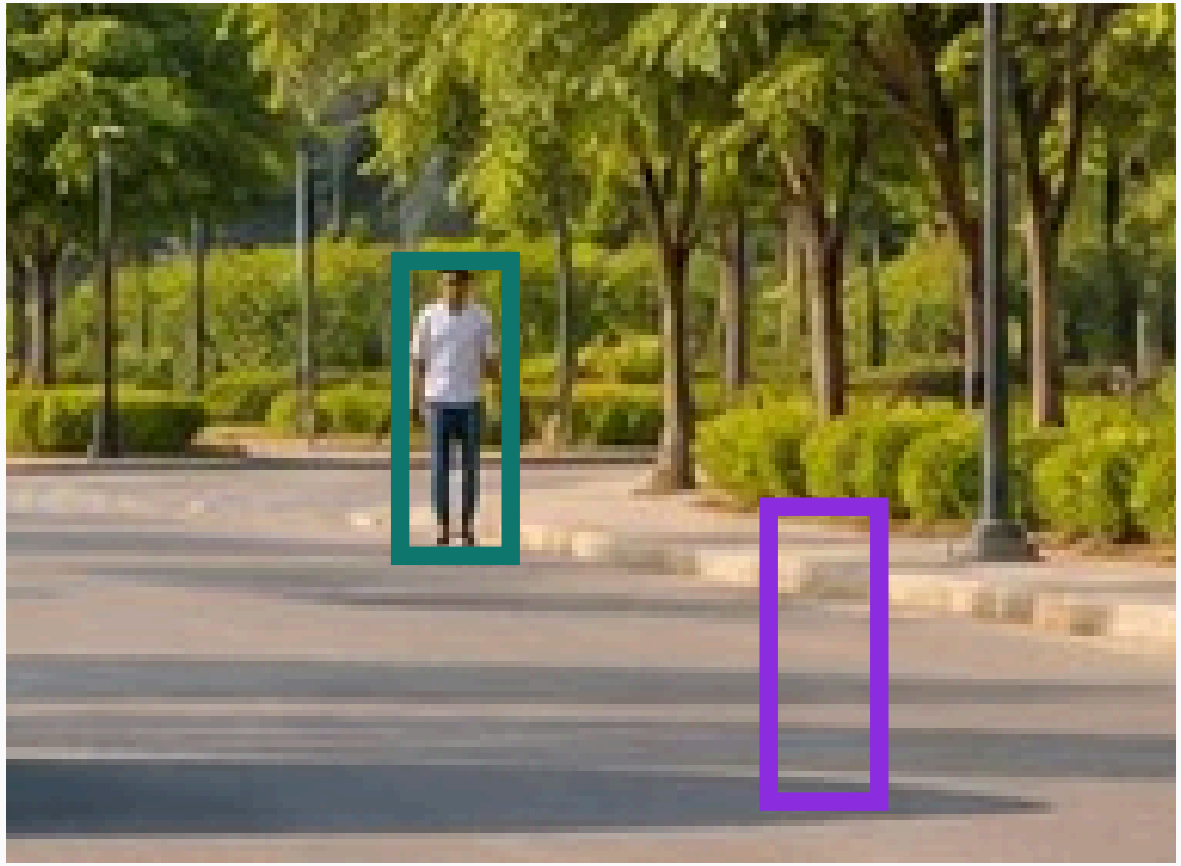
A person far down the street, 18 px wide.

> A person far down the street: 18 pixels wide.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

The same slip on a small object



A person far down the street, 18 px wide.

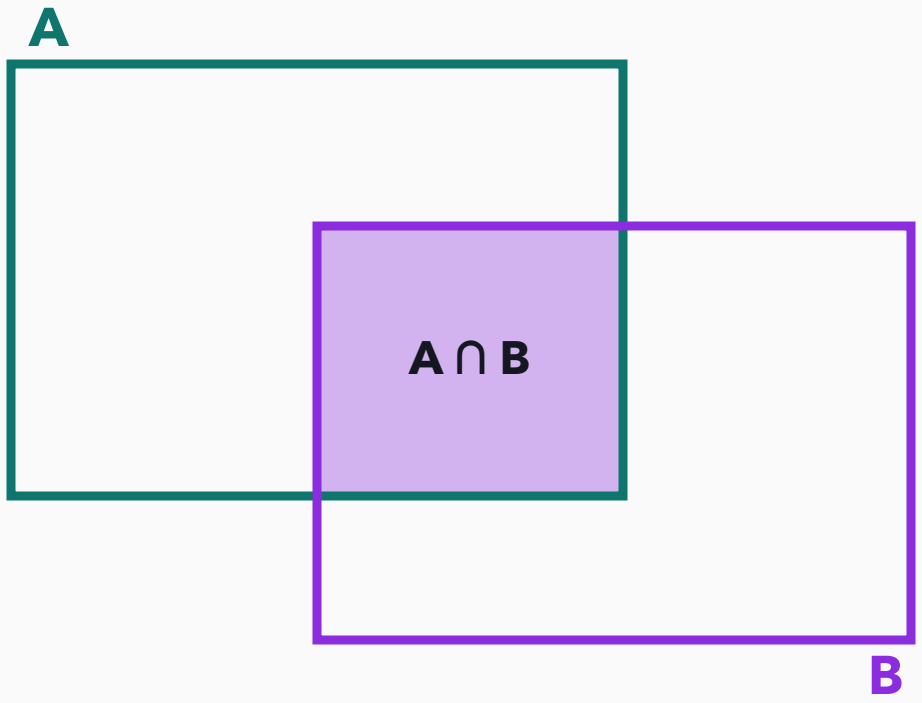
Move the box by the same 60 px right and 40 px down: it no longer touches the person.

> The same 72-pixel slip misses the person entirely. A small error for the dog is a miss here, so compare overlap instead of distance.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

Intersection over union (IoU)

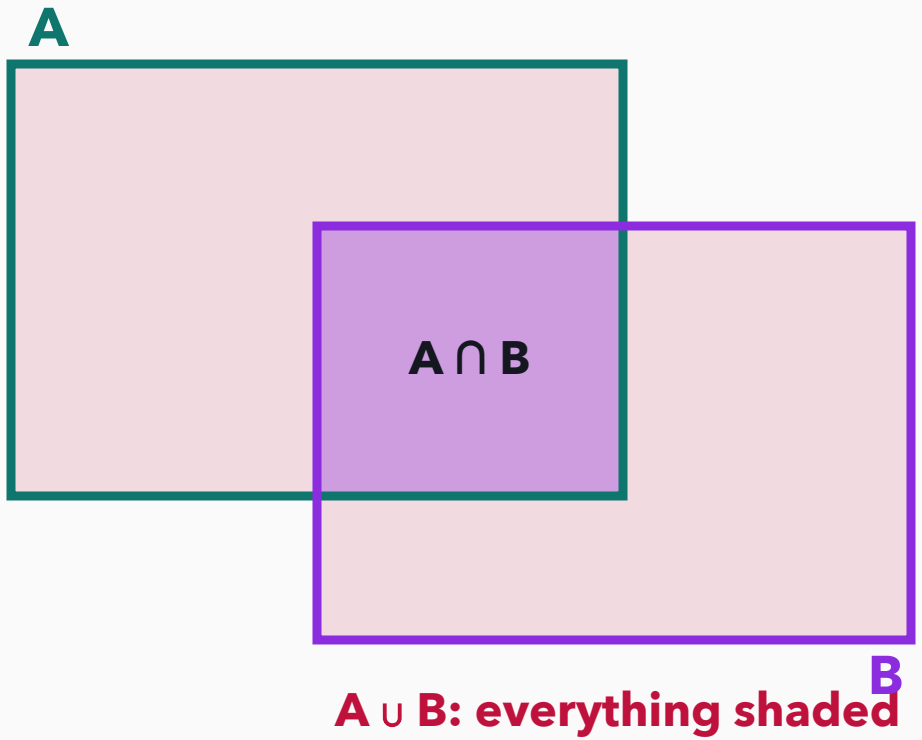


Intersection: the area covered by both boxes.

> The intersection: the region inside both boxes.

▶ Notebook, part 3

Intersection over union (IoU)



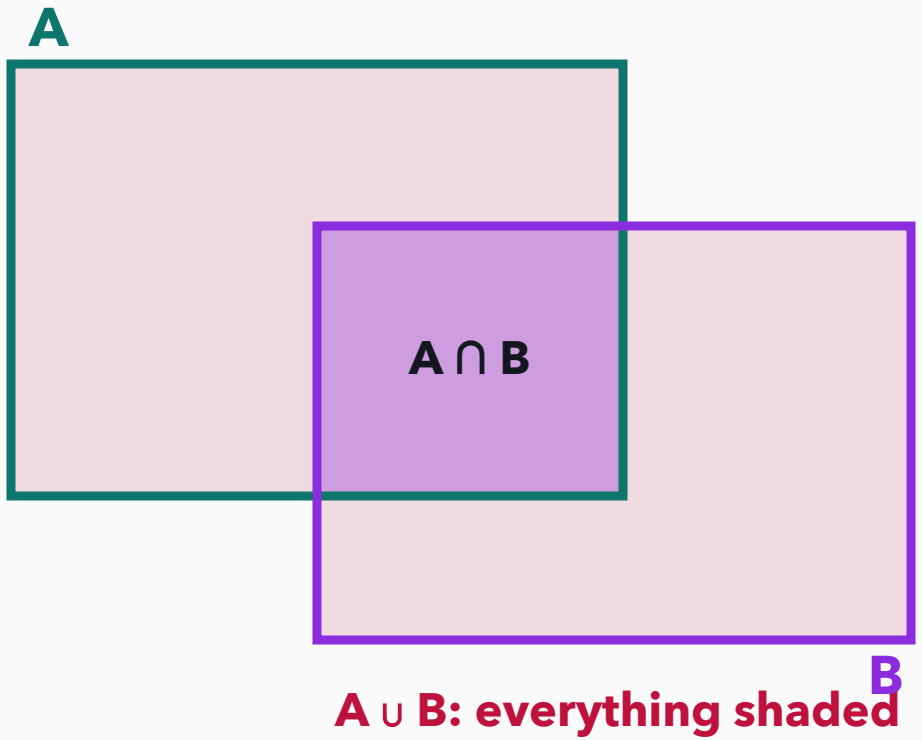
Intersection: the area covered by both boxes.

Union: the area covered by at least one of them.

> The union: the region inside either box.

▶ Notebook, part 3

Intersection over union (IoU)



Intersection: the area covered by both boxes.

Union: the area covered by at least one of them.

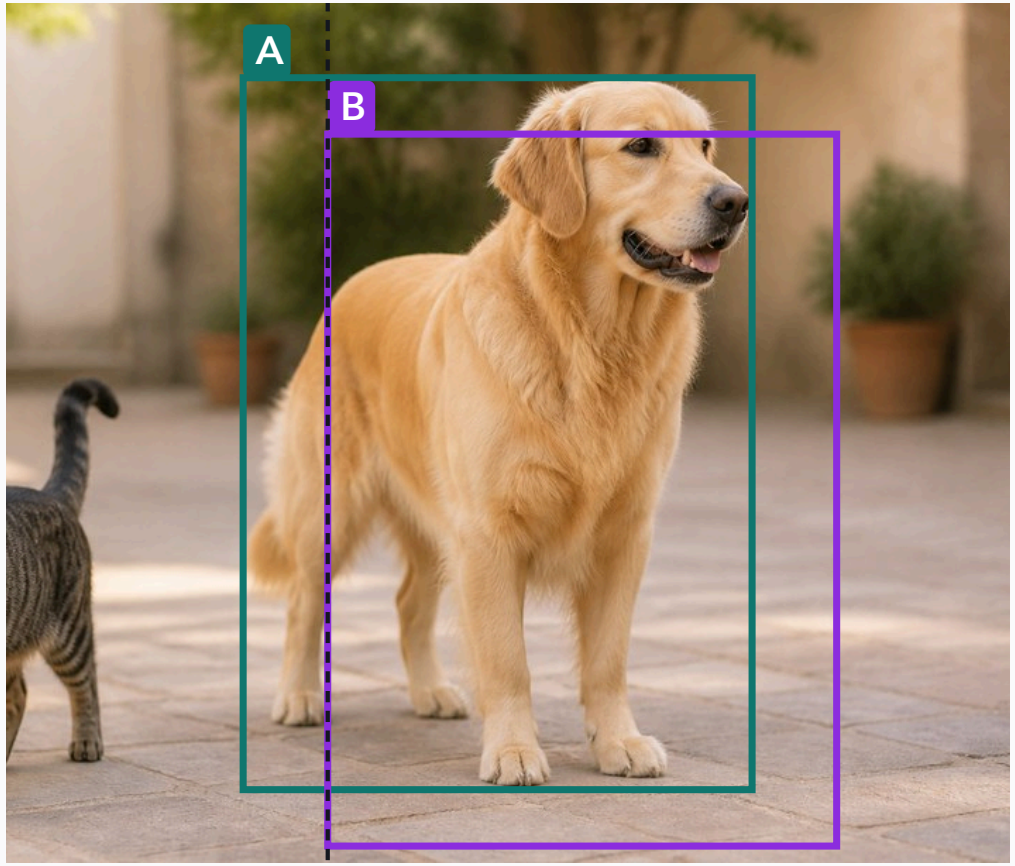
$$\mathrm{IoU}(A, B) = \frac{\mathrm{area}(A \cap B)}{\mathrm{area}(A \cup B)}$$

Identical boxes: IoU = 1. Boxes that do not touch: IoU = 0.

IoU divides the first by the second. It is a fraction of area, so it does not care how big the boxes are.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



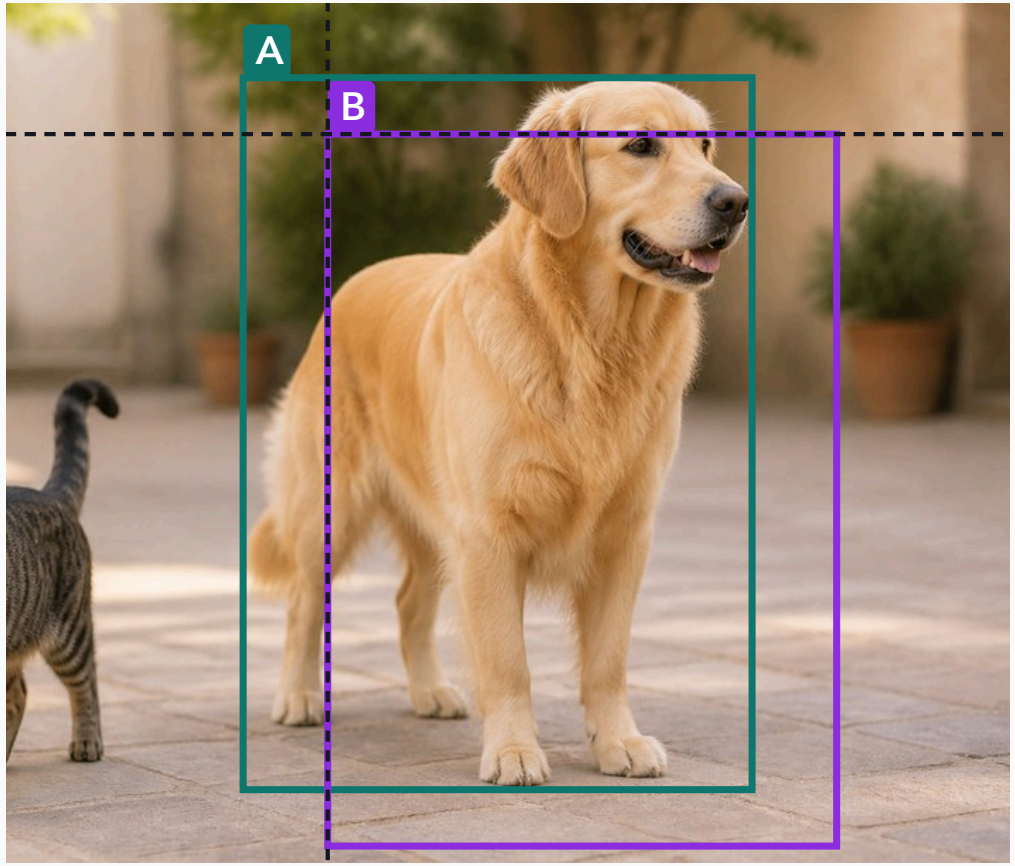
$$\text{left} = \max(407,\ 467) = 467$$

> The overlap starts where both boxes have started: the larger of the two left edges.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



$$\text{left} = \max(407,\ 467) = 467$$

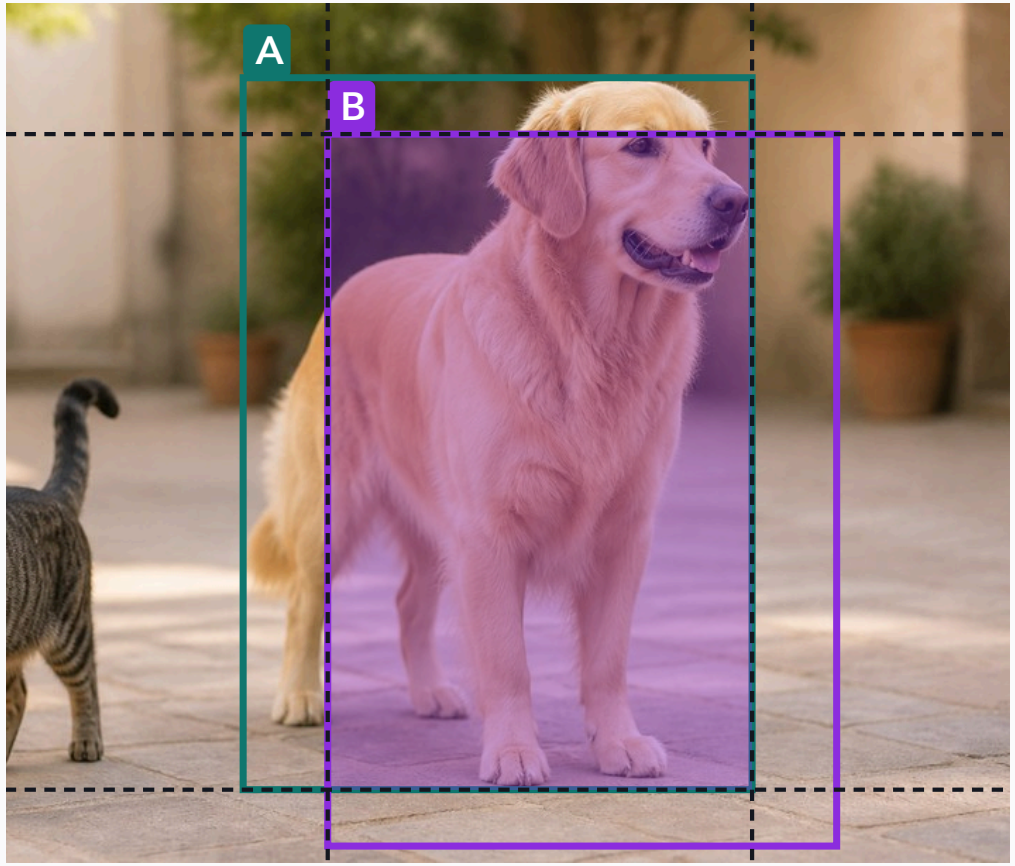
$$\text{top} = \max(83,\ 123) = 123$$

> Likewise its top is the larger of the two tops.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 1: the overlap rectangle



$$\text{left} = \max(407,\ 467) = 467$$

$$\text{top} = \max(83,\ 123) = 123$$

$$\text{right} = \min(768,\ 828) = 768$$

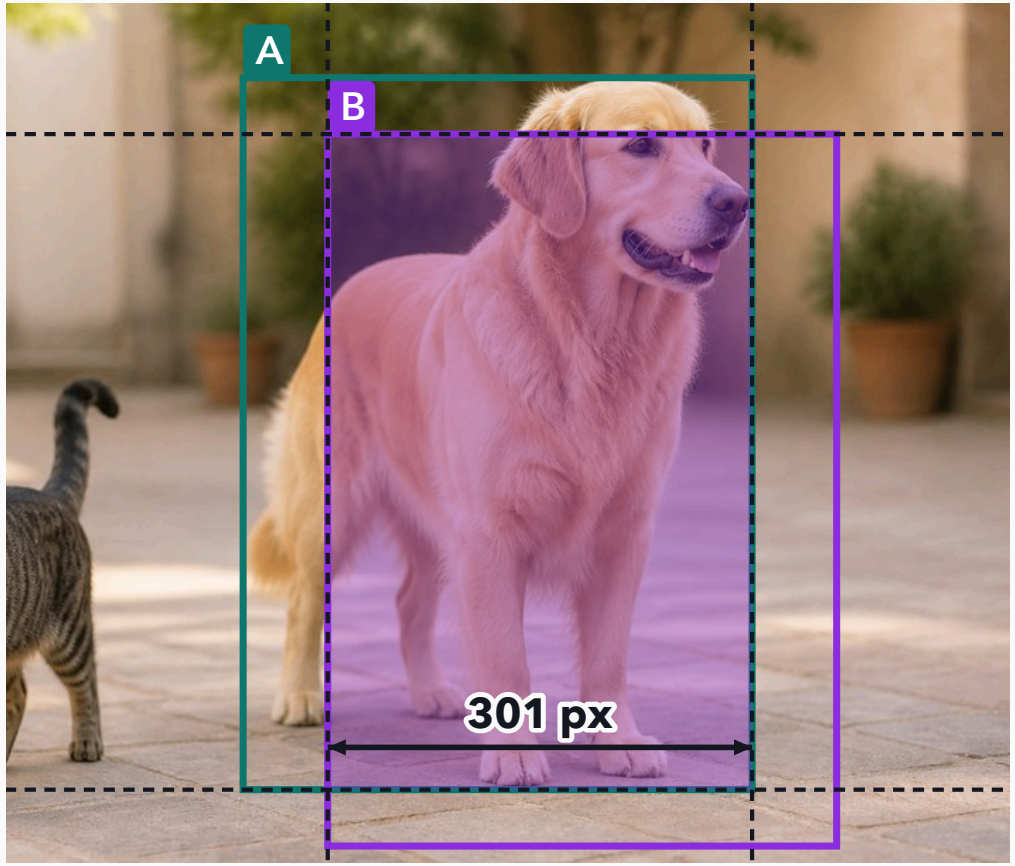
$$\text{bottom} = \min(588,\ 628) = 588$$

> It ends where the first box ends: the smaller right edge and the smaller bottom edge. The shaded rectangle is the overlap.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 2: the overlap's area



	left	top	right	bottom
overlap	467	123	768	588

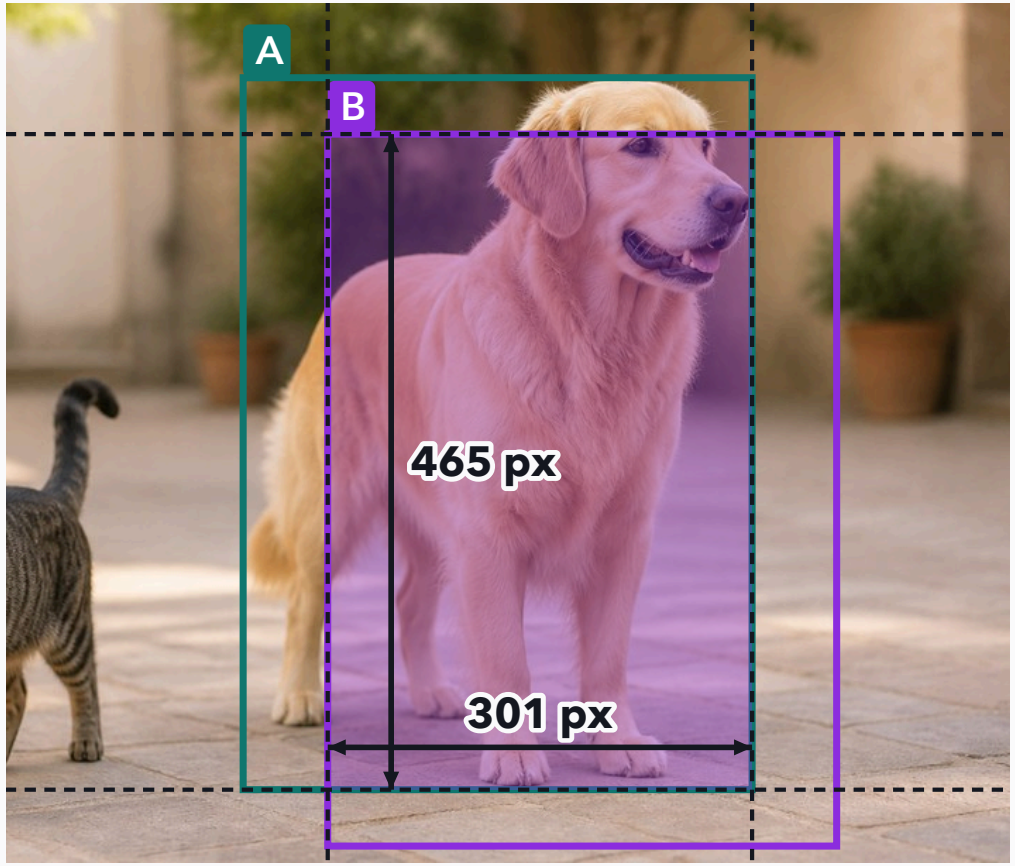
$$\text{width} = \text{right} - \text{left} = 768 - 467 = 301$$

> Measure the overlap the way we measured the dog's box. Width first: its right edge minus its left edge.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 2: the overlap's area



	left	top	right	bottom
overlap	467	123	768	588

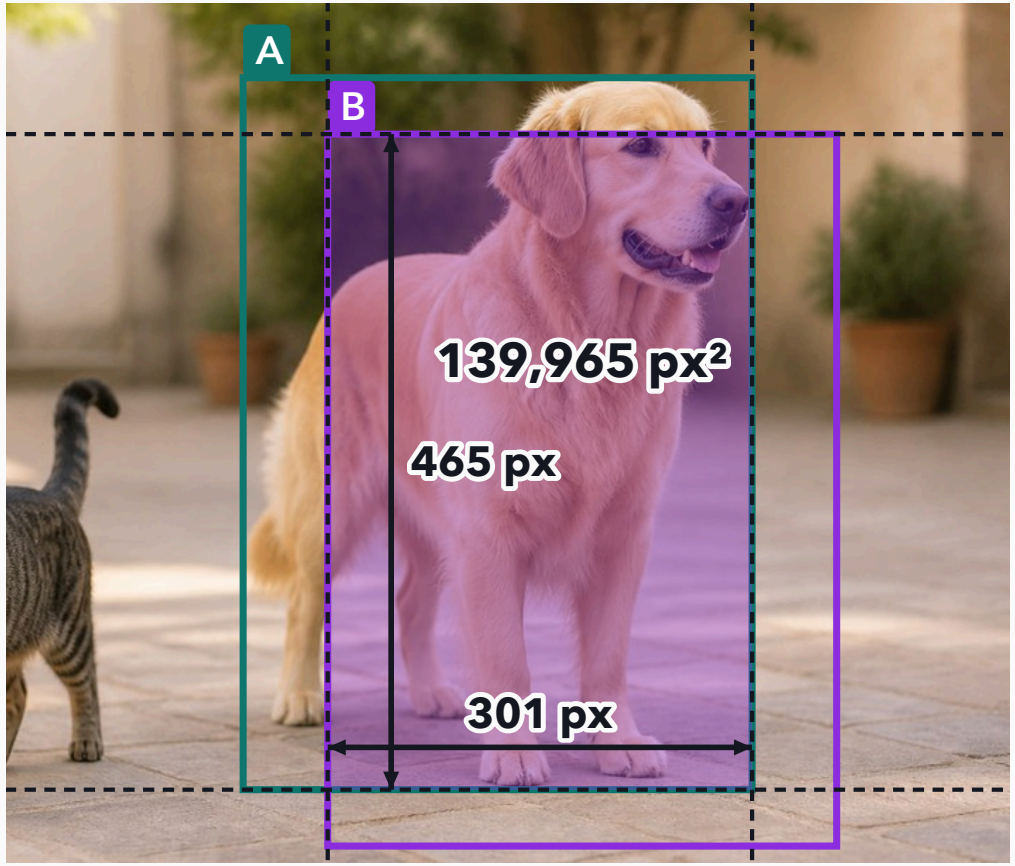
$$\text{width} = \text{right} - \text{left} = 768 - 467 = 301$$

$$\text{height} = \text{bottom} - \text{top} = 588 - 123 = 465$$

> Then the height: bottom minus top. A negative width or height would mean the boxes do not touch, and the overlap is 0.

▶ Notebook, part 3

IoU, step 2: the overlap's area



	left	top	right	bottom
overlap	467	123	768	588

$$\text{width} = \text{right} - \text{left} = 768 - 467 = 301$$

$$\text{height} = \text{bottom} - \text{top} = 588 - 123 = 465$$

$$\text{area} = \text{width} \times \text{height} = 301 \times 465 = 139{,}965$$

So the two boxes share 139,965 square pixels. This is the top of the IoU fraction.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 3

IoU, step 3: the union, then the ratio

$$\text{area}(A) = (768 - 407) \times (588 - 83) = 361 \times 505 = 182{,}305$$

$$\text{area}(B) = (828 - 467) \times (628 - 123) = 361 \times 505 = 182{,}305$$

Each box's own area. The guess has the same size as our box; it is only moved.

▶ Notebook, part 3

IoU, step 3: the union, then the ratio

$$\text{area}(A) = (768 - 407) \times (588 - 83) = 361 \times 505 = 182{,}305$$

$$\text{area}(B) = (828 - 467) \times (628 - 123) = 361 \times 505 = 182{,}305$$

$$\text{union} = 182{,}305 + 182{,}305 - 139{,}965 = 224{,}645$$

> Adding the two areas counts the overlap twice, so subtract it once.

▶ Notebook, part 3

IoU, step 3: the union, then the ratio

$$\text{area}(A) = (768 - 407) \times (588 - 83) = 361 \times 505 = 182{,}305$$

$$\text{area}(B) = (828 - 467) \times (628 - 123) = 361 \times 505 = 182{,}305$$

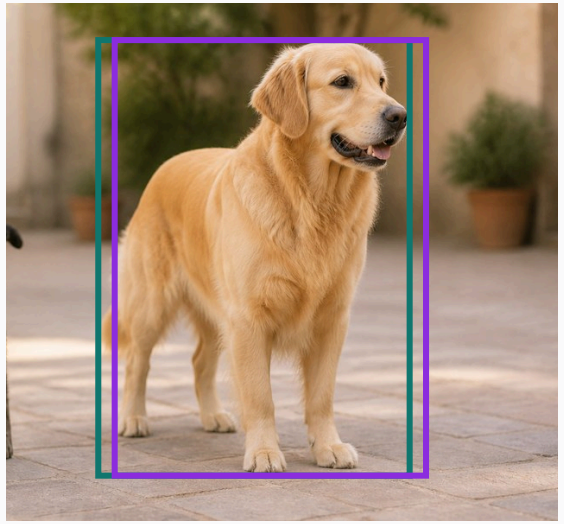
$$\text{union} = 182{,}305 + 182{,}305 - 139{,}965 = 224{,}645$$

$$\text{IoU} = \frac{139{,}965}{224{,}645} = 0.623$$

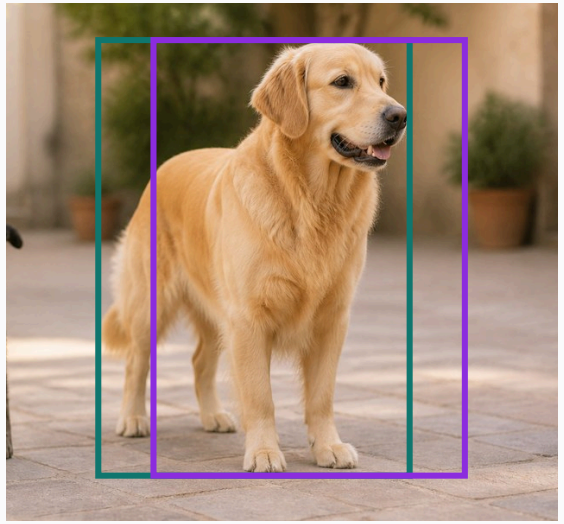
> IoU = 0.623. A move of 60 px right and 40 px down cost more than a third of the score.

▶ Notebook, part 3

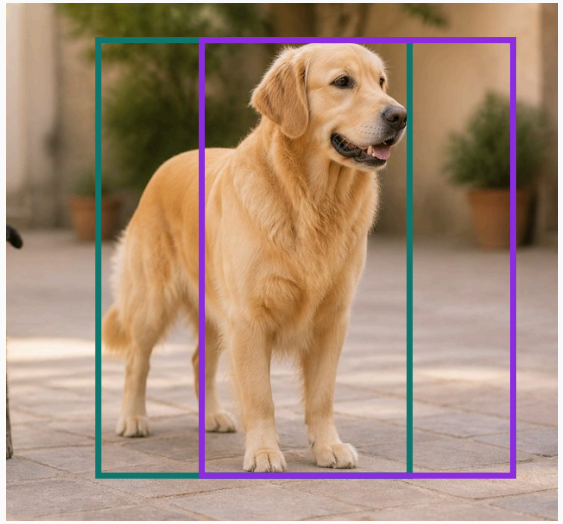
What IoU values look like



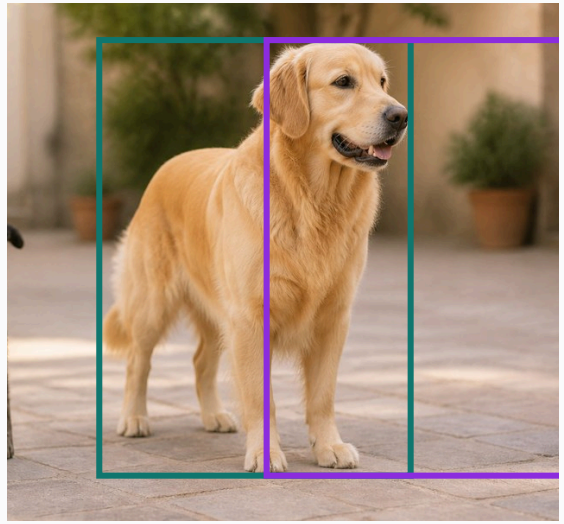
IoU 0.9 · moved 19 px



IoU 0.7 · moved 64 px



IoU 0.5 · moved 120 px



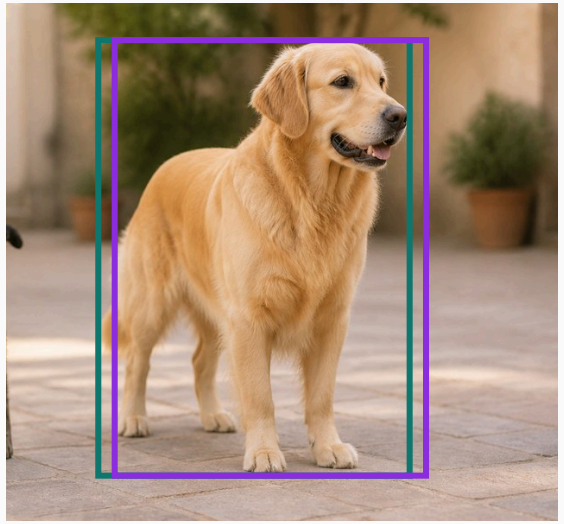
IoU 0.3 · moved 194 px

> The guess slides sideways. At 0.9 the boxes nearly agree; at 0.5 the guess has moved a third of the dog's width; at 0.3, half of it.

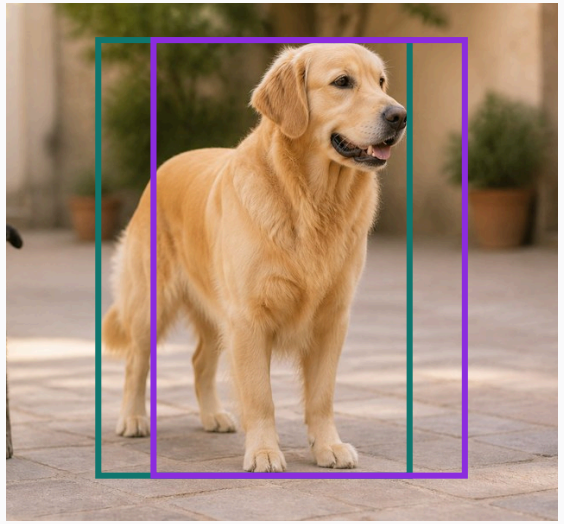
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 3

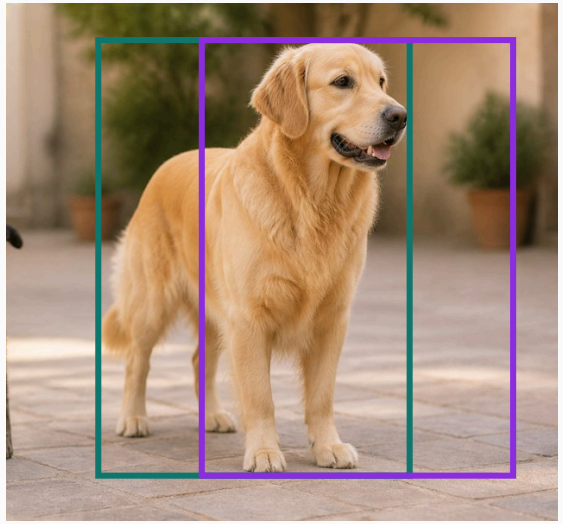
What IoU values look like



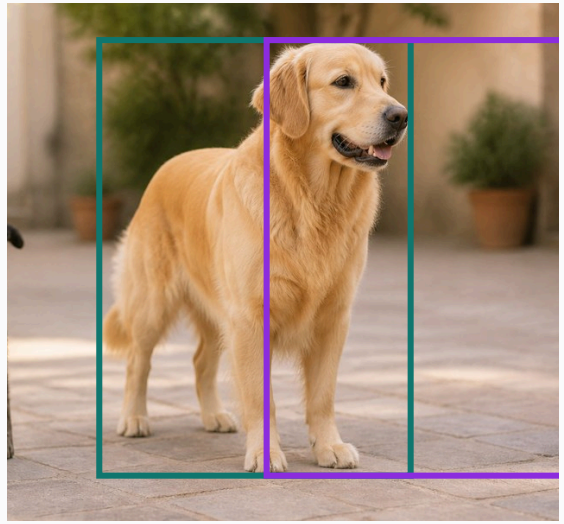
IoU 0.9 · moved 19 px • **hit**



IoU 0.7 · moved 64 px • **hit**



IoU 0.5 · moved 120 px • **hit**



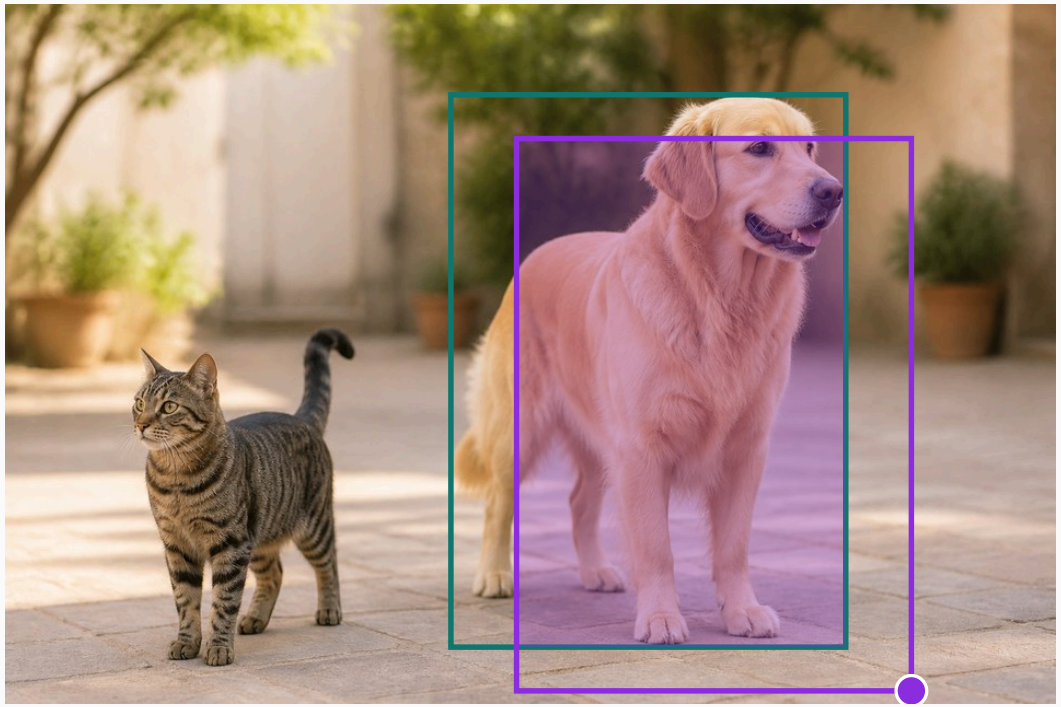
IoU 0.3 · moved 194 px • **miss**

> Most benchmarks call IoU ≥ 0.5 a hit. A trained detector does much better on this dog: 0.983.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 3

Try it: drag a box over the dog



overlap	139,965 px²
union	224,645 px²
IoU	**0.623**

IoU 0.50 that looks fine. Loose, but the whole dog is inside. The box covers 364,800 px², twice the dog box's 182,305 px², so IoU = 1/2.

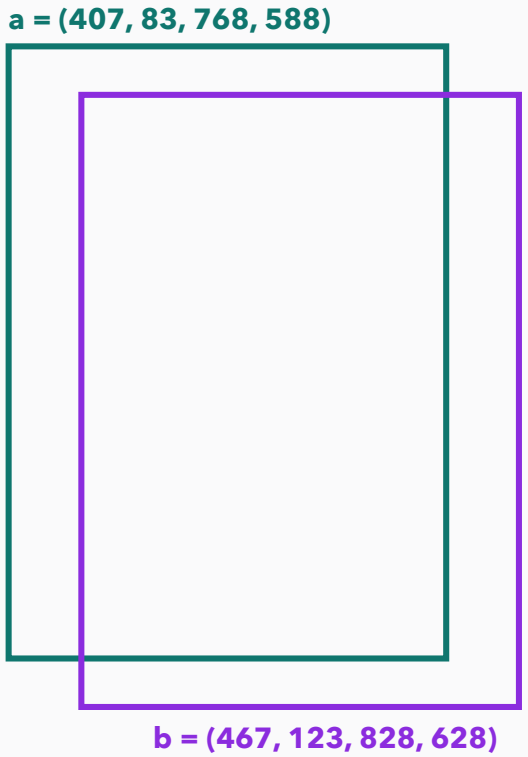
IoU 0.70 that looks wrong. Snug on the body, but the whole head is cut off. The box lies inside the dog's box and keeps 354 of its 505 px of height, so IoU = 354/505 = 0.70.

> Drag the purple box. Can you find a box with IoU 0.5 that looks good? One with IoU 0.7 that looks bad? The two lower buttons show one of each.

▶ Notebook, part 3

IoU in code: every pair at once

```
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
```



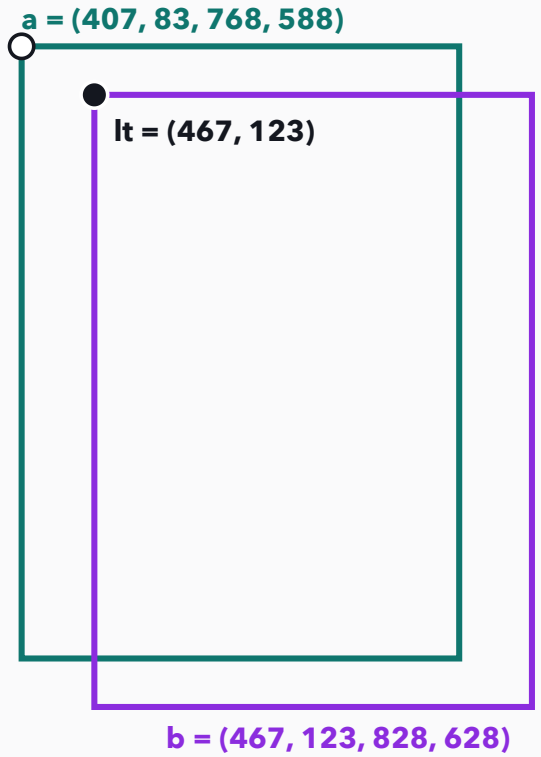
IoU for two lists of boxes in corner format: N boxes in a, M in b. We try it on one pair, our box and the guess.

The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

▶ Notebook, part 3

IoU in code: every pair at once

```
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
```



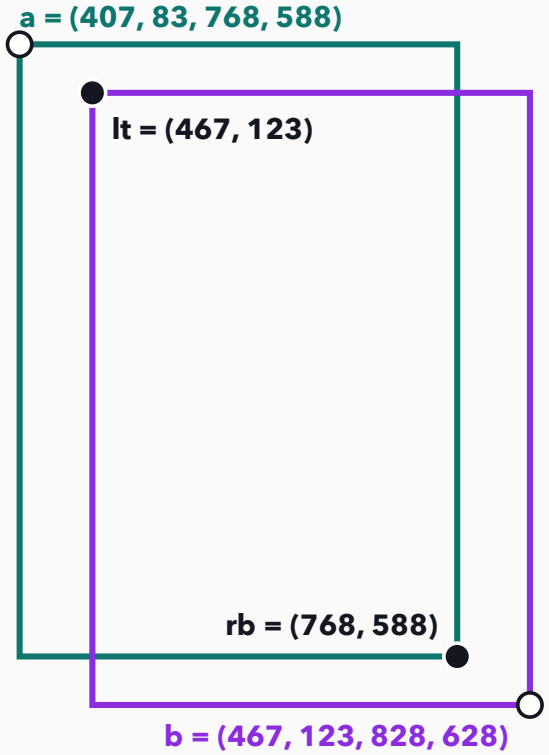
> Line 2 is step 1's max: the larger of the two top-left corners. The None in a[:, None] lets every box in a meet every box in b.

The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

▶ Notebook, part 3

IoU in code: every pair at once

```
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
    rb = torch.min(a[:, None, 2:], b[:, 2:])
```



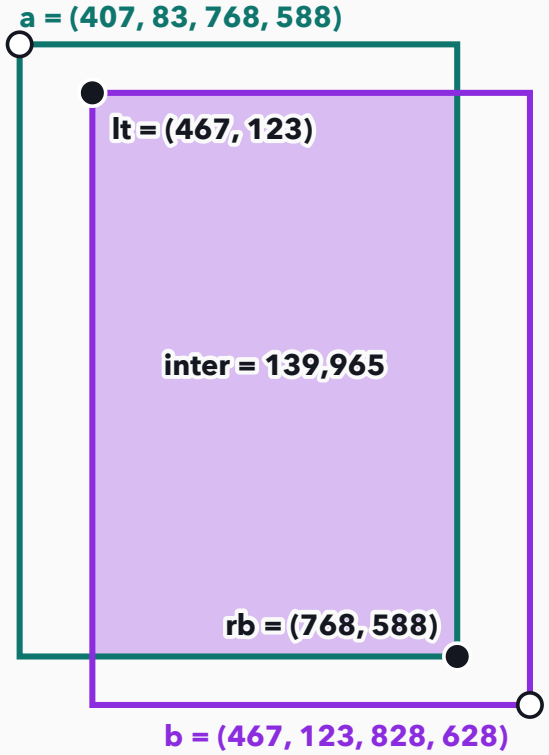
Line 3 takes the smaller of the two bottom-right corners.

The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

▶ Notebook, part 3

IoU in code: every pair at once

```python
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
    rb = torch.min(a[:, None, 2:], b[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
```



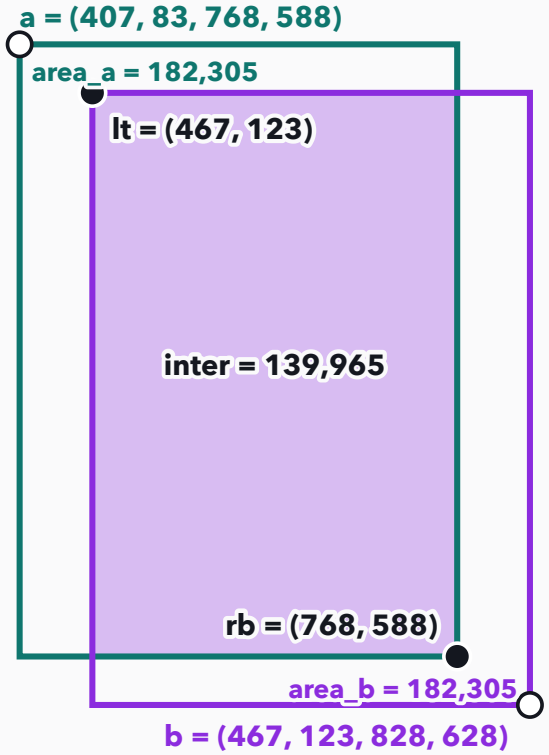
> Subtract the corners for the width and height, set a negative one to 0, and multiply: the overlap.

The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

▶ Notebook, part 3

IoU in code: every pair at once

```python
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
    rb = torch.min(a[:, None, 2:], b[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
    area_a = (a[:, 2:] - a[:, :2]).prod(-1)
    area_b = (b[:, 2:] - b[:, :2]).prod(-1)
```



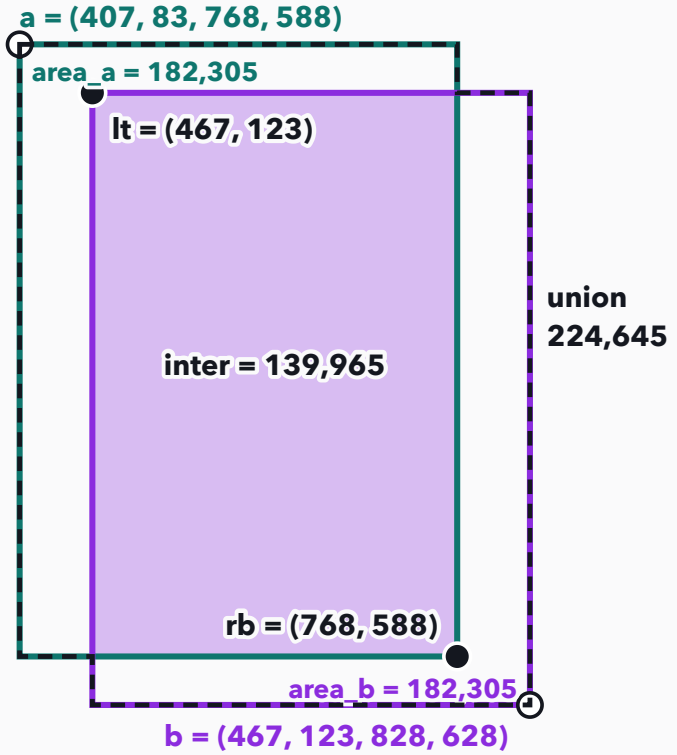
> Each box's own area, the same way: bottom-right minus top-left, multiplied.

The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

▶ Notebook, part 3

IoU in code: every pair at once

```
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
    rb = torch.min(a[:, None, 2:], b[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
    area_a = (a[:, 2:] - a[:, :2]).prod(-1)
    area_b = (b[:, 2:] - b[:, :2]).prod(-1)
    union = area_a[:, None] + area_b - inter
```



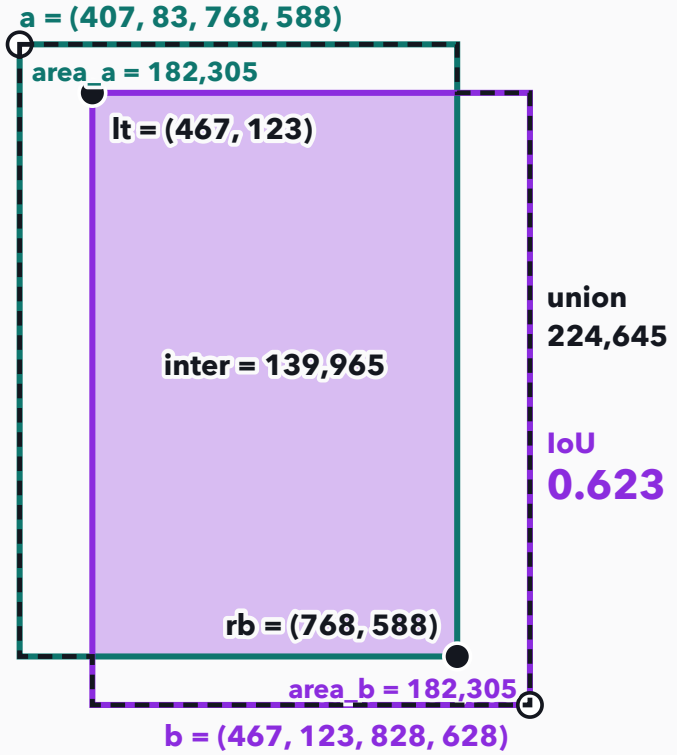
> The union, as in step 3: both areas added, minus the overlap they both contain.

The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

▶ Notebook, part 3

IoU in code: every pair at once

```python
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
    rb = torch.min(a[:, None, 2:], b[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
    area_a = (a[:, 2:] - a[:, :2]).prod(-1)
    area_b = (b[:, 2:] - b[:, :2]).prod(-1)
    union = area_a[:, None] + area_b - inter
    return inter / union
```



Divide: 139,965 / 224,645 = 0.623.

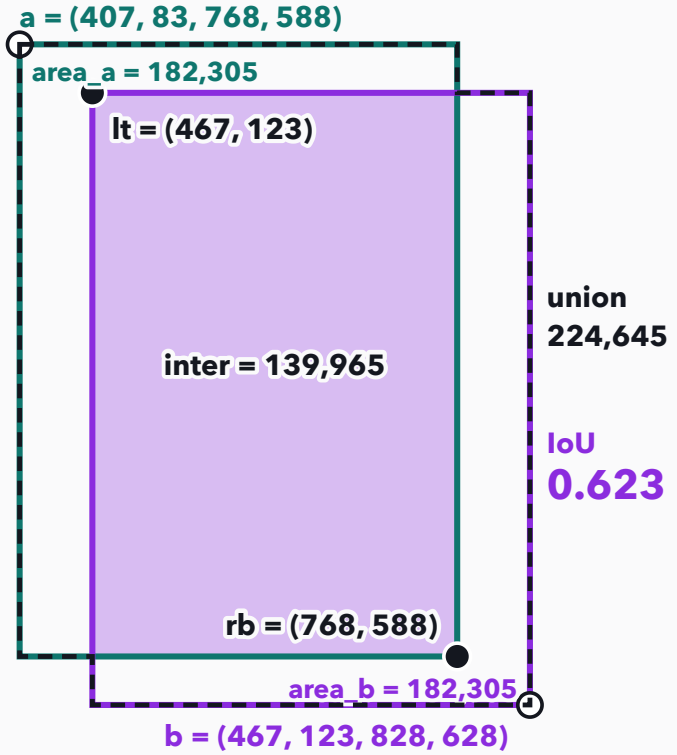
The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

▶ Notebook, part 3

IoU in code: every pair at once

```python
def box_iou(a, b):  # a: (N, 4), b: (M, 4)
    lt = torch.max(a[:, None, :2], b[:, :2])
    rb = torch.min(a[:, None, 2:], b[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(-1)
    area_a = (a[:, 2:] - a[:, :2]).prod(-1)
    area_b = (b[:, 2:] - b[:, :2]).prod(-1)
    union = area_a[:, None] + area_b - inter
    return inter / union

a = torch.tensor([[407., 83., 768., 588.]])
b = torch.tensor([[467., 123., 828., 628.]])
box_iou(a, b)        # tensor([[0.6230]])
```



Run it on our pair: 0.6230, as by hand. With 31 boxes in a and 2 in b, it returns a 31 × 2 matrix: every pair at once.

The two boxes from step 1: our annotation of the dog and the guess moved 60 px right and 40 px down.

Part 3 in one slide

1. A box is four numbers: corners (407, 83, 768, 588), or centre and size.
2. A label file holds one line per object: a class and four numbers.
3. IoU = overlap ÷ union: 1 for the same box, 0 for boxes apart. The 60-px guess: 0.623.
4. IoU ≥ 0.5 is the usual “found it”.

> **Next:** ask a network for one box, then several, and see where it breaks.

▶ Notebook, part 4

Part 4 · One box is easy; several are not

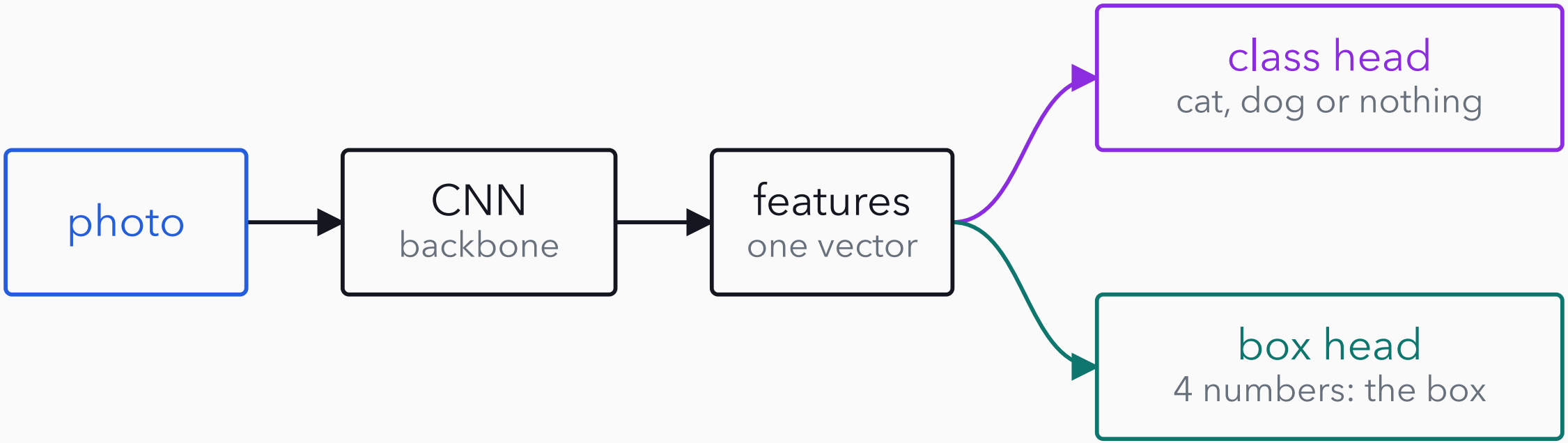
One object: a classifier with four more outputs for the box. Two objects show why detection needs more.

1. Add a box head to a classifier.
2. Write the answer as numbers, with “?” where there is nothing to learn.
3. The loss: compare only where it counts.
4. Two animals, one box: it hedges.

> One object is easy: a classifier with four more outputs. Two objects show why detection needs more than that.

▶ Notebook, part 4

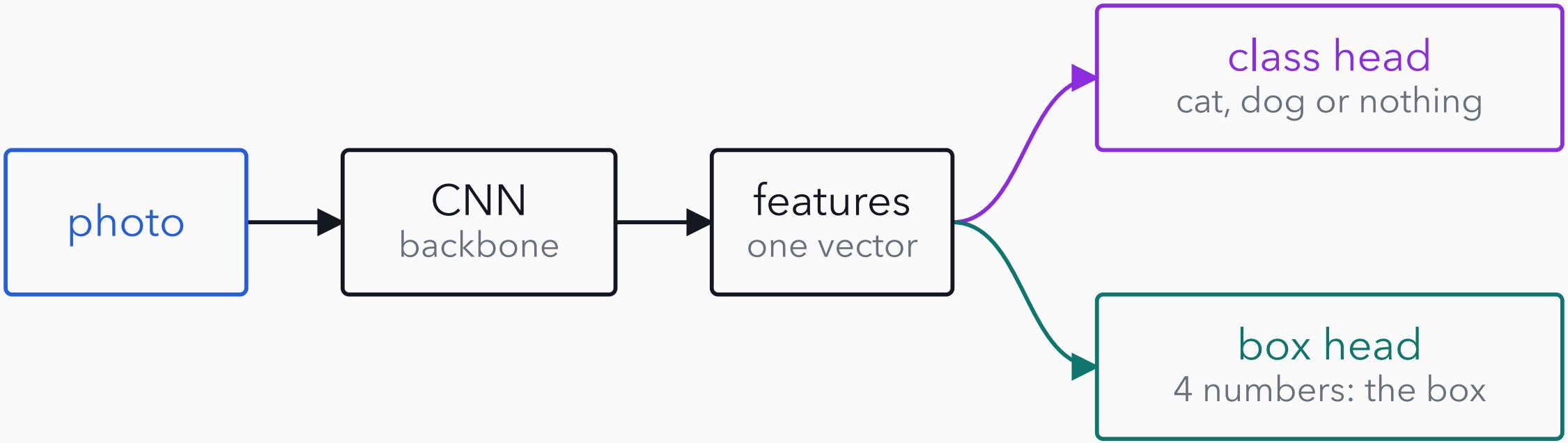
One object: a box head beside the class head



> Start from a classifier: a CNN turns the photo into one feature vector, and a layer turns that into class scores.

▶ Notebook, part 4

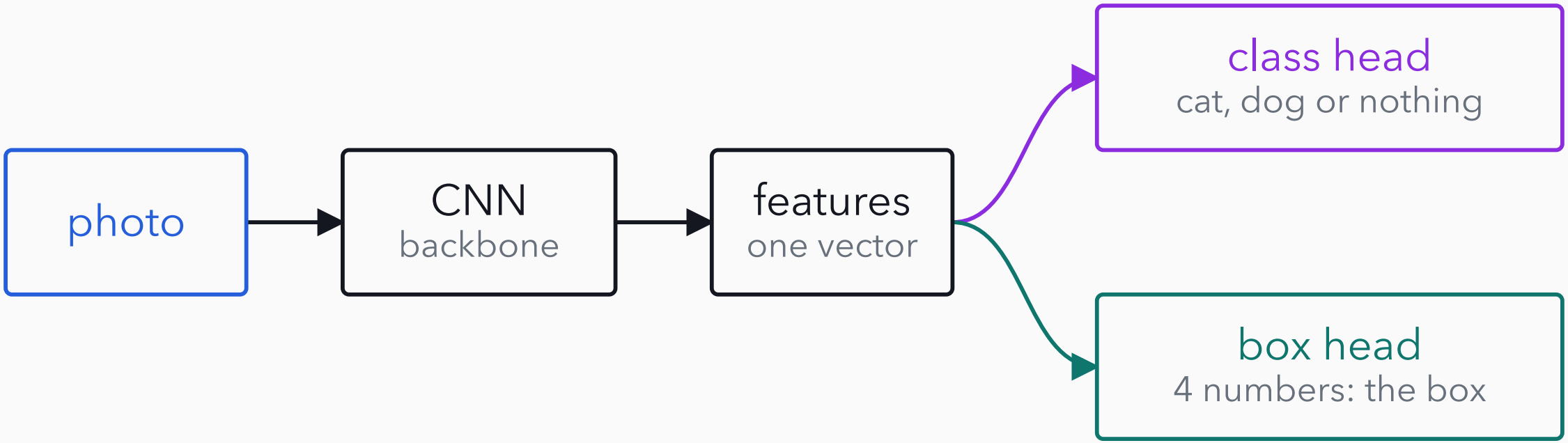
One object: a box head beside the class head



Add a second small layer on the same vector: four numbers for the box.

▶ Notebook, part 4

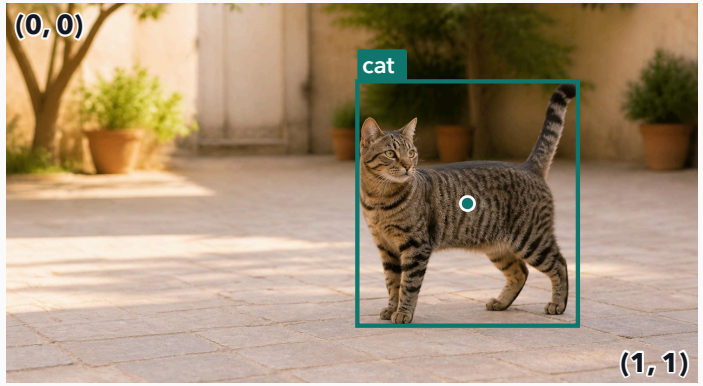
One object: a box head beside the class head



Train both at once, comparing each output with the annotation.

▶ Notebook, part 4

Write the answer as numbers: the target for one photo



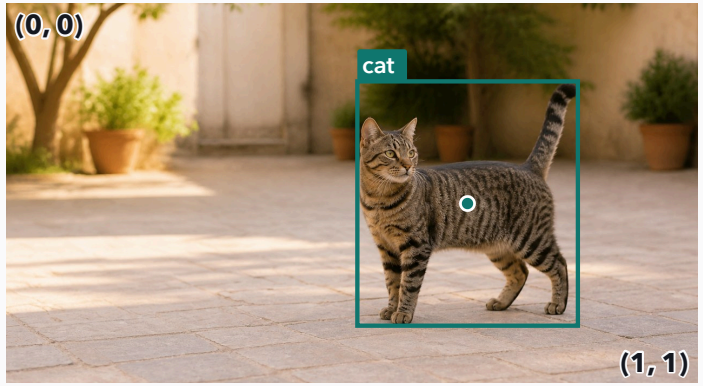
object?	b_x	b_y	b_w	b_h	cat	dog
1	0.67	0.53	0.32	0.64	1	0

> The answer as numbers: object? 1, then the box as fractions of the photo (0 at the top-left, 1 at the bottom-right), then cat 1, dog 0.

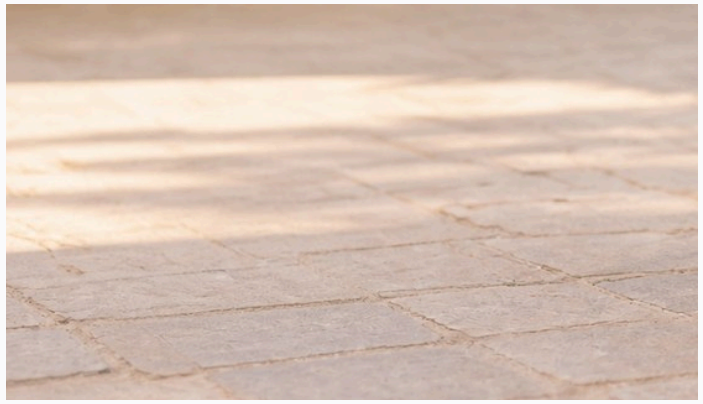
Our annotation of the single-cat photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Write the answer as numbers: the target for one photo



object?	b_x	b_y	b_w	b_h	cat	dog
1	0.67	0.53	0.32	0.64	1	0



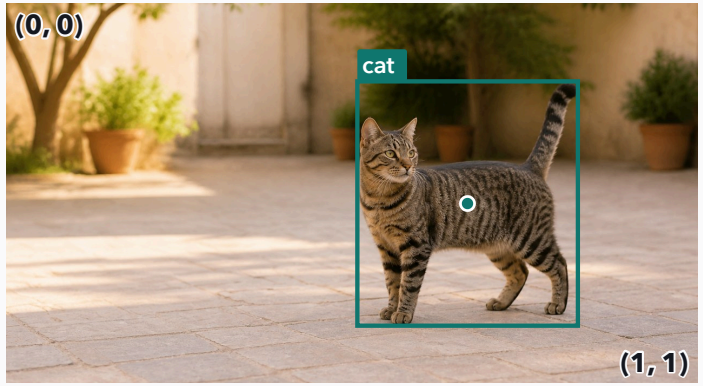
object?	b_x	b_y	b_w	b_h	cat	dog
0	?	?	?	?	?	?

> A photo of empty floor: object? is 0. The box and the class have no right answer, so Ng writes “?”: don’t care.

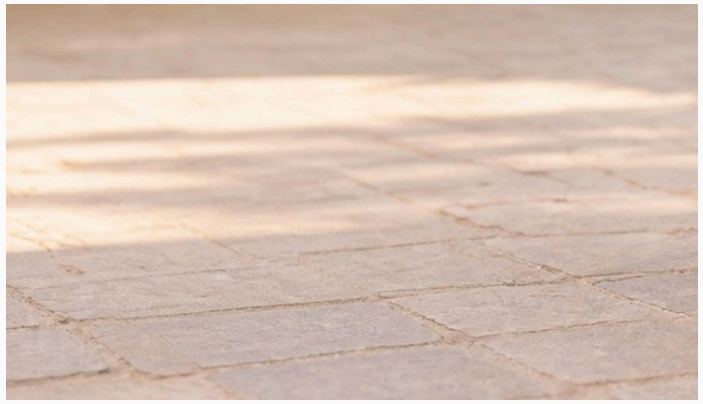
Our annotation of the single-cat photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Write the answer as numbers: the target for one photo



object?	b_x	b_y	b_w	b_h	cat	dog
1	0.67	0.53	0.32	0.64	1	0



object?	b_x	b_y	b_w	b_h	cat	dog
0	?	?	?	?	?	?

Only “object?” is learned here: each ? is left out of the loss.

> Only entries with a right answer are learned: for the floor, just object?.

Our annotation of the single-cat photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

The loss: compare only where it counts

cat photo

object?	b_x	b_y	b_w	b_h	cat	dog
1	0.67	0.53	0.32	0.64	1	0

$$\text{object} = 1: \quad \mathcal{L} = \sum_{i=1}^{7} (\hat{y}_i - y_i)^2$$

> When there is an object, all seven outputs are compared with the target. Ng adds up the squared differences.

▶ Notebook, part 4

The loss: compare only where it counts

	object?	b_x	b_y	b_w	b_h	cat	dog
cat photo	1	0.67	0.53	0.32	0.64	1	0

$$\text{object} = 1: \quad \mathcal{L} = \sum_{i=1}^{7} (\hat{y}_i - y_i)^2$$

	object?	b_x	b_y	b_w	b_h	cat	dog
empty floor	0	?	?	?	?	?	?

$$\text{object} = 0: \quad \mathcal{L} = (\hat{y}_1 - y_1)^2$$

> When there is none, only object? is compared. The "?" entries are skipped: whatever the network writes there costs nothing.

▶ Notebook, part 4

The loss: compare only where it counts

	object?	b_x	b_y	b_w	b_h	cat	dog
cat photo	1	0.67	0.53	0.32	0.64	1	0

$$\text{object} = 1: \quad \mathcal{L} = \sum_{i=1}^{7} (\hat{y}_i - y_i)^2$$

	object?	b_x	b_y	b_w	b_h	cat	dog
empty floor	0	?	?	?	?	?	?

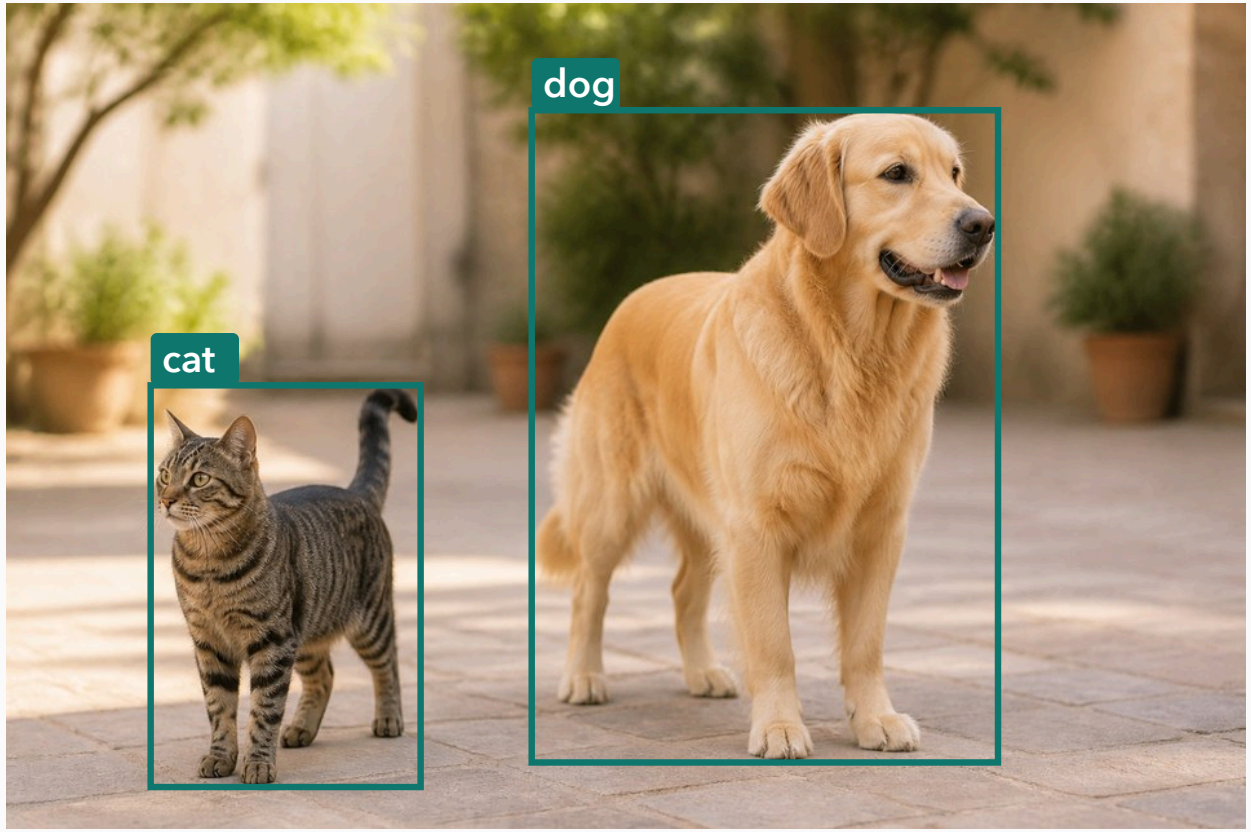
$$\text{object} = 0: \quad \mathcal{L} = (\hat{y}_1 - y_1)^2$$

In practice: cross-entropy for object? and the classes, and an L1 or squared loss for the box.

> Real detectors pick a better loss for each kind of entry, but the rule stays the same: skip the "?".

▶ Notebook, part 4

Two animals, one box: which one should it draw?



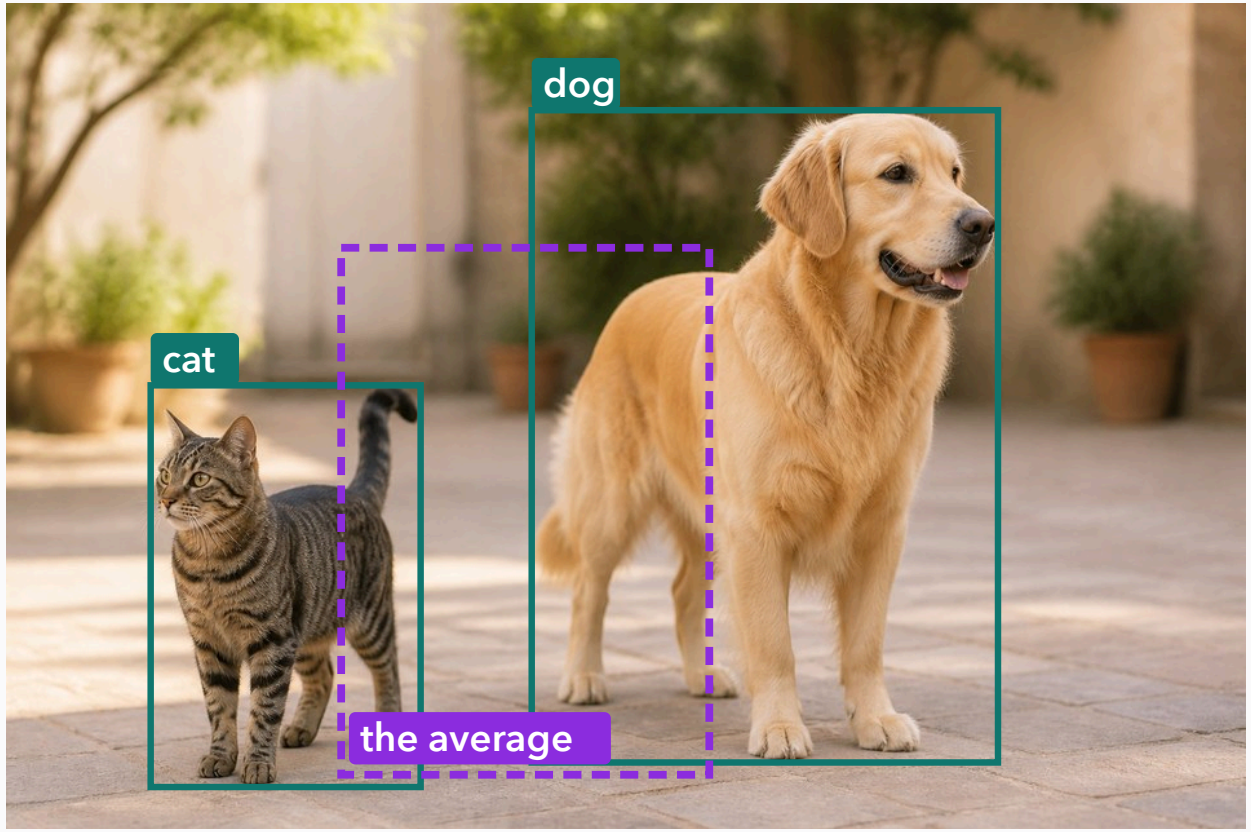
The annotation lists both animals. The head has room for one box.

> One box head, two animals. Which should it draw?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Two animals, one box: which one should it draw?



The annotation lists both animals. The head has room for one box.

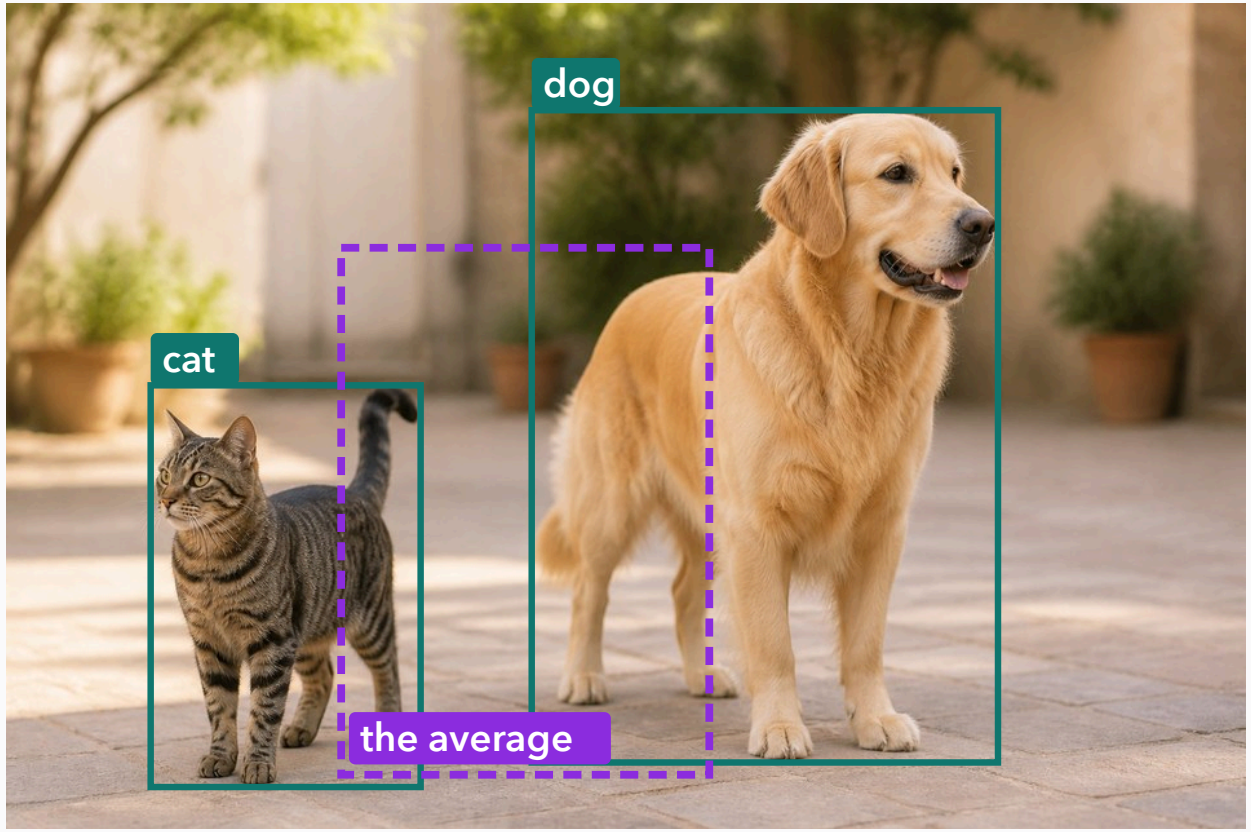
Its safest single answer lies in between: the average box.

> If some annotations list the dog first and some the cat, a squared error is smallest for the average of the two boxes: the network learns to hedge.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 4

Two animals, one box: which one should it draw?



The annotation lists both animals. The head has room for one box.

Its safest single answer lies in between: the average box.

$$\mathrm{IoU}(\mathrm{avg}, \mathrm{dog}) = 0.225$$
$$\mathrm{IoU}(\mathrm{avg}, \mathrm{cat}) = 0.114$$

> A box between two objects describes neither of them.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

Part 4 in one slide

1. One object: a CNN with a class head and a box head writes a 7-number record.
2. The target uses "?" wherever there is nothing to learn, and the loss skips those entries.
3. Two animals, one record: it learns the average box, which finds neither (IoU 0.23, 0.11).

Next: give every place in the photo its own record.

▶ Notebook, part 5

Part 5 · A candidate at every place

Tie each output to a place: a grid of cells over the photo, each writing one record.

1. Slide a classifier over the photo, and see what is wrong with that.
2. YOLO: grid the photo; every cell predicts a record. Its structure, target, code and loss.
3. The real grid: read its boxes out and keep the confident ones; the whole algorithm.

> Every cell of a grid writes a record. We work out its target and its loss, then read its box back out.

▶ Notebook, part 5

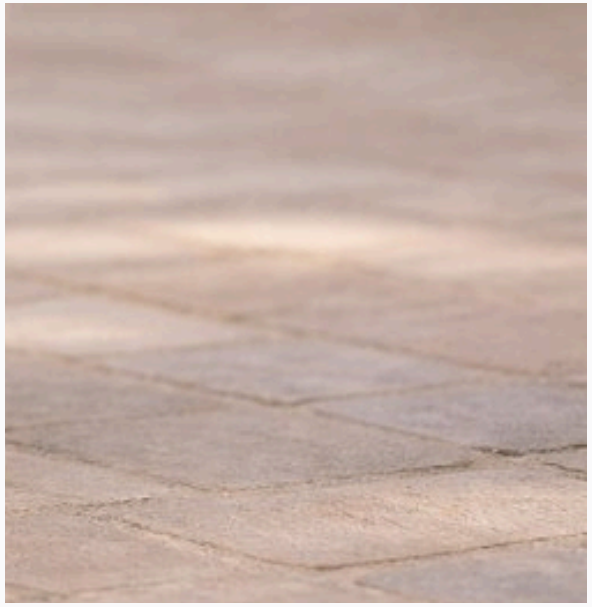
Start from a tool we have: a classifier for crops



dog



cat



nothing

> Three crops cut from the courtyard: a dog, a cat, and floor with nothing on it.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

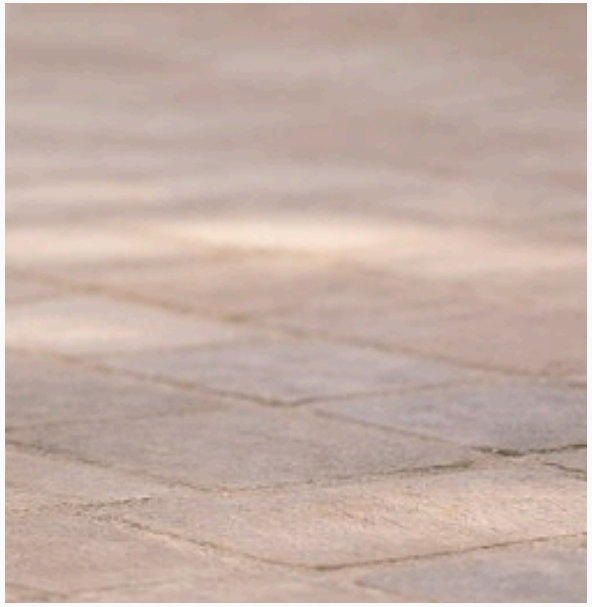
Start from a tool we have: a classifier for crops



dog



cat



nothing

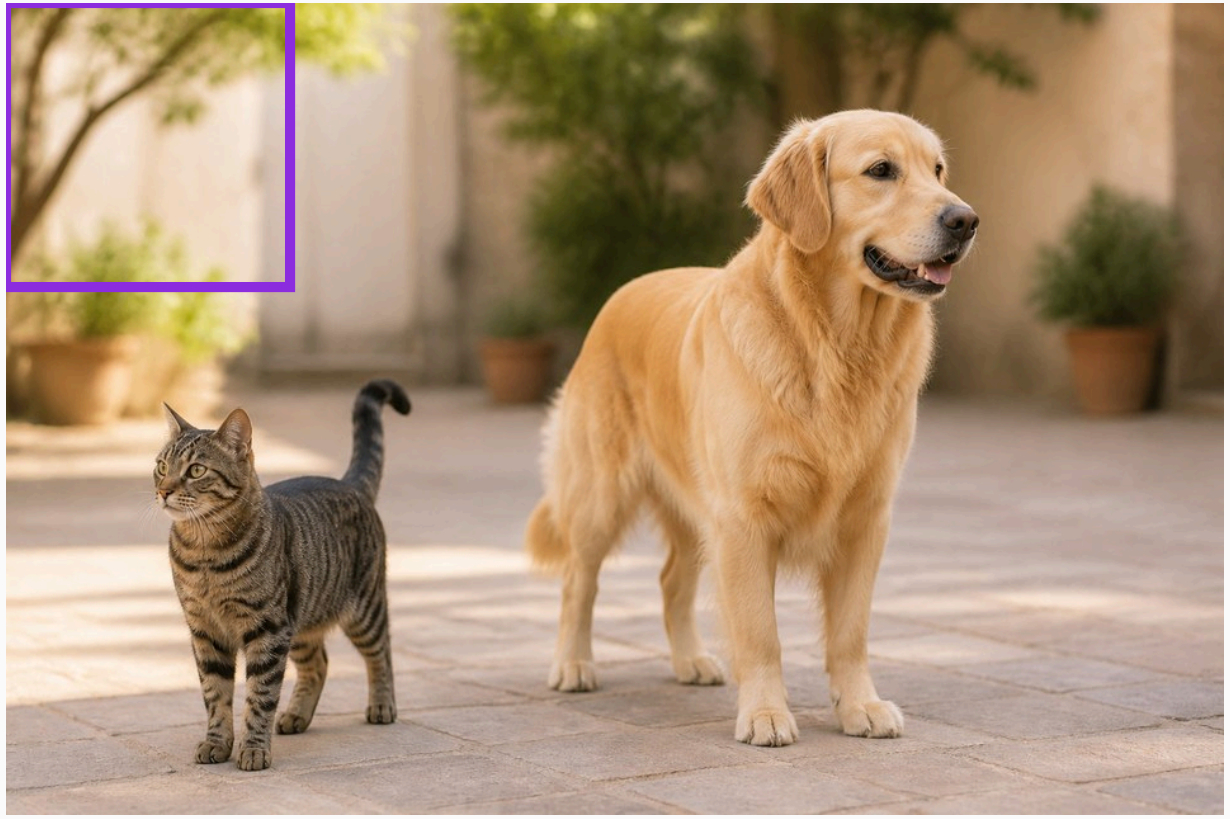
Train a classifier on many crops like these. Shown one crop, it answers: dog, cat, or nothing.

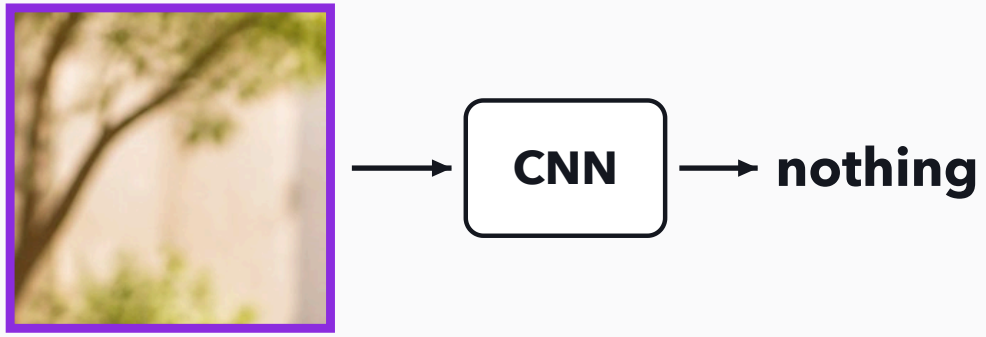
> It cannot say where anything is in a whole photo. Can we use it to find objects anyway?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



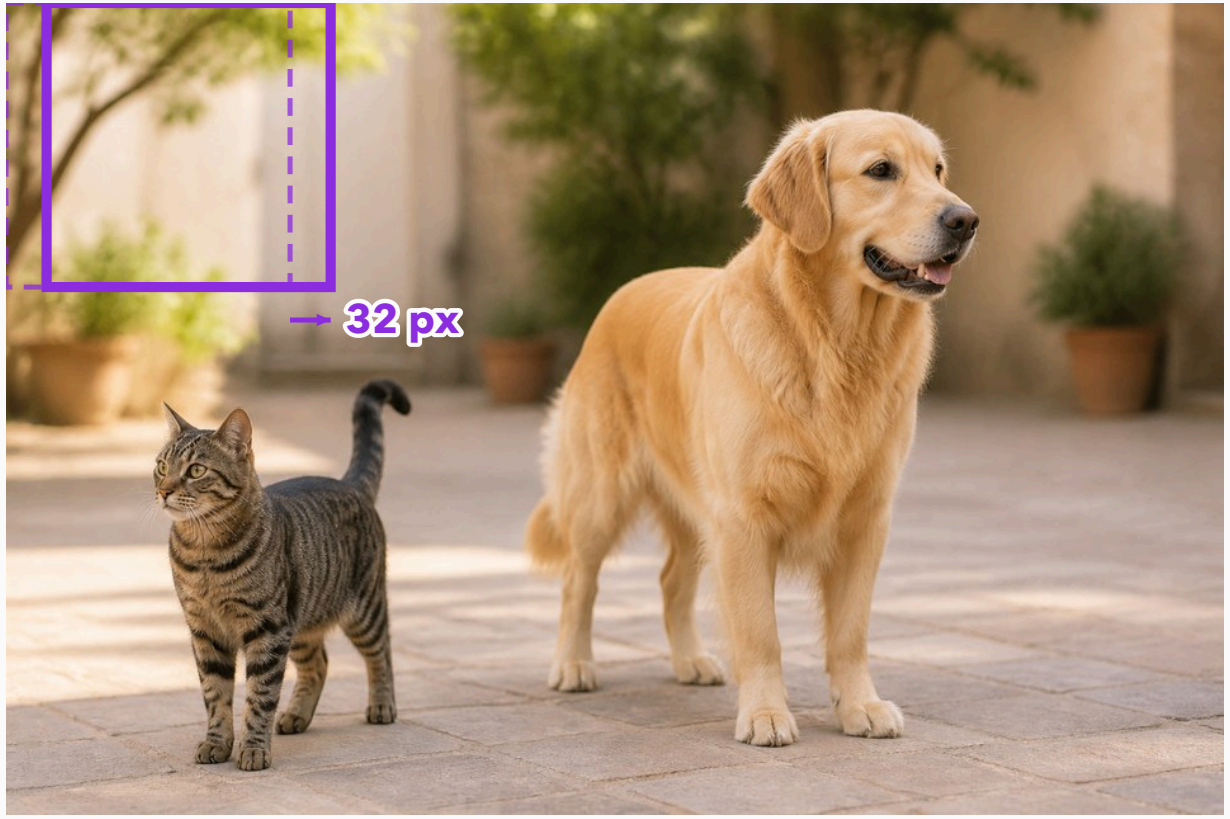


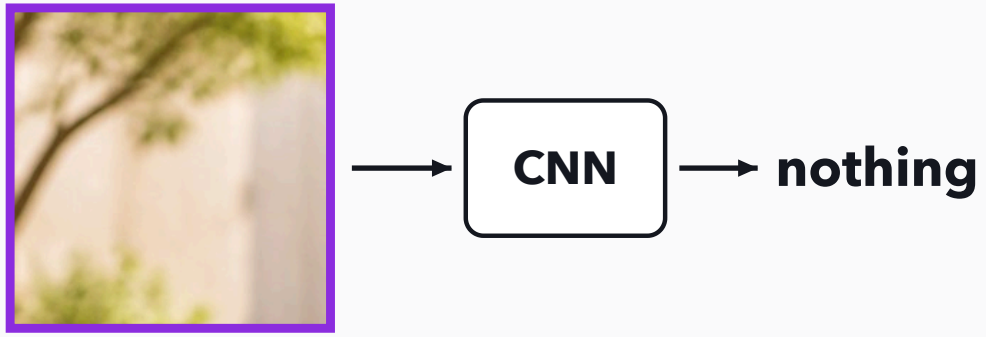
> Put a 224 × 224 window in the top-left corner, crop it, and give the crop to the classifier: nothing there.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



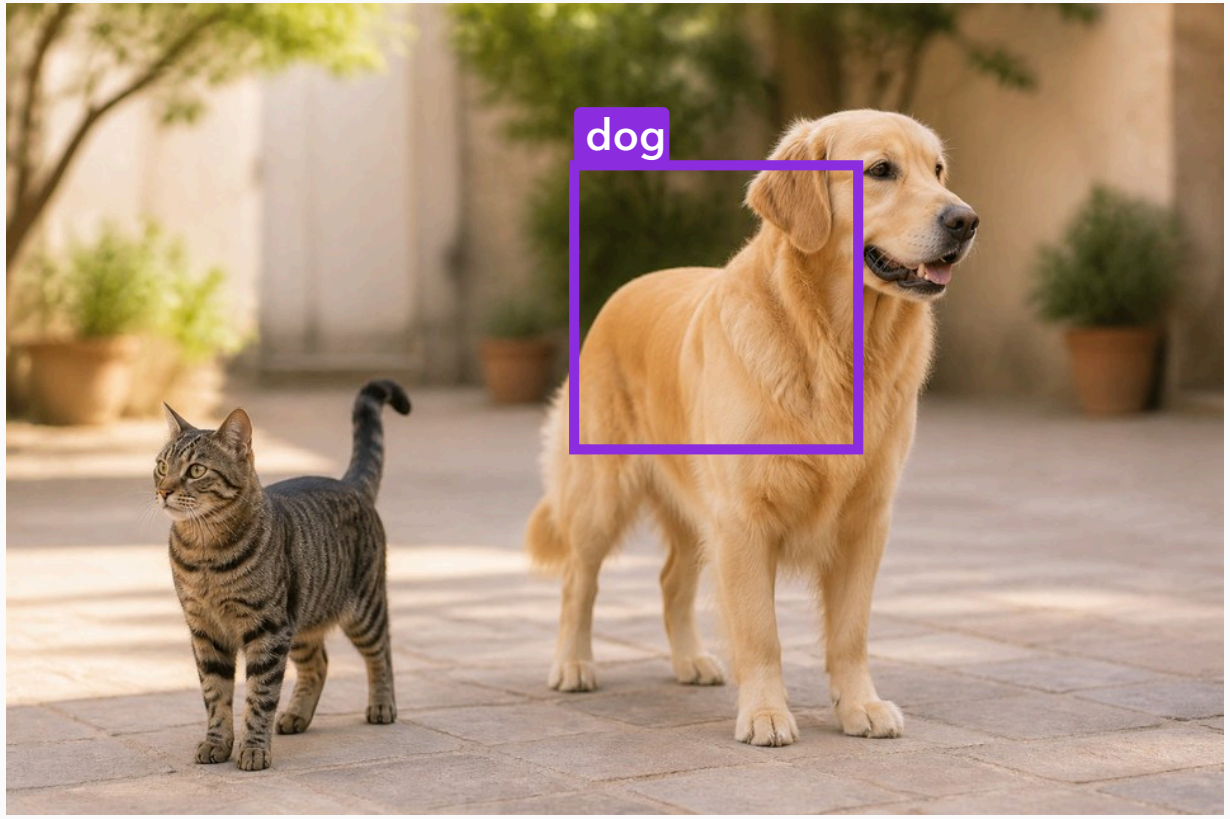


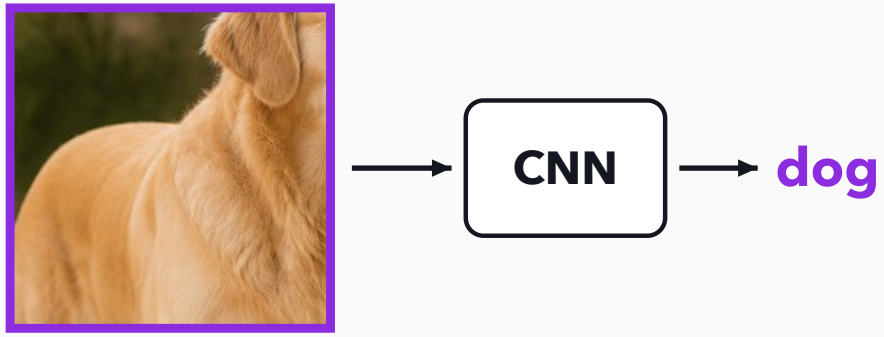
> Move the window 32 pixels to the right and classify again: nothing. Two neighbouring crops share 86% of their pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



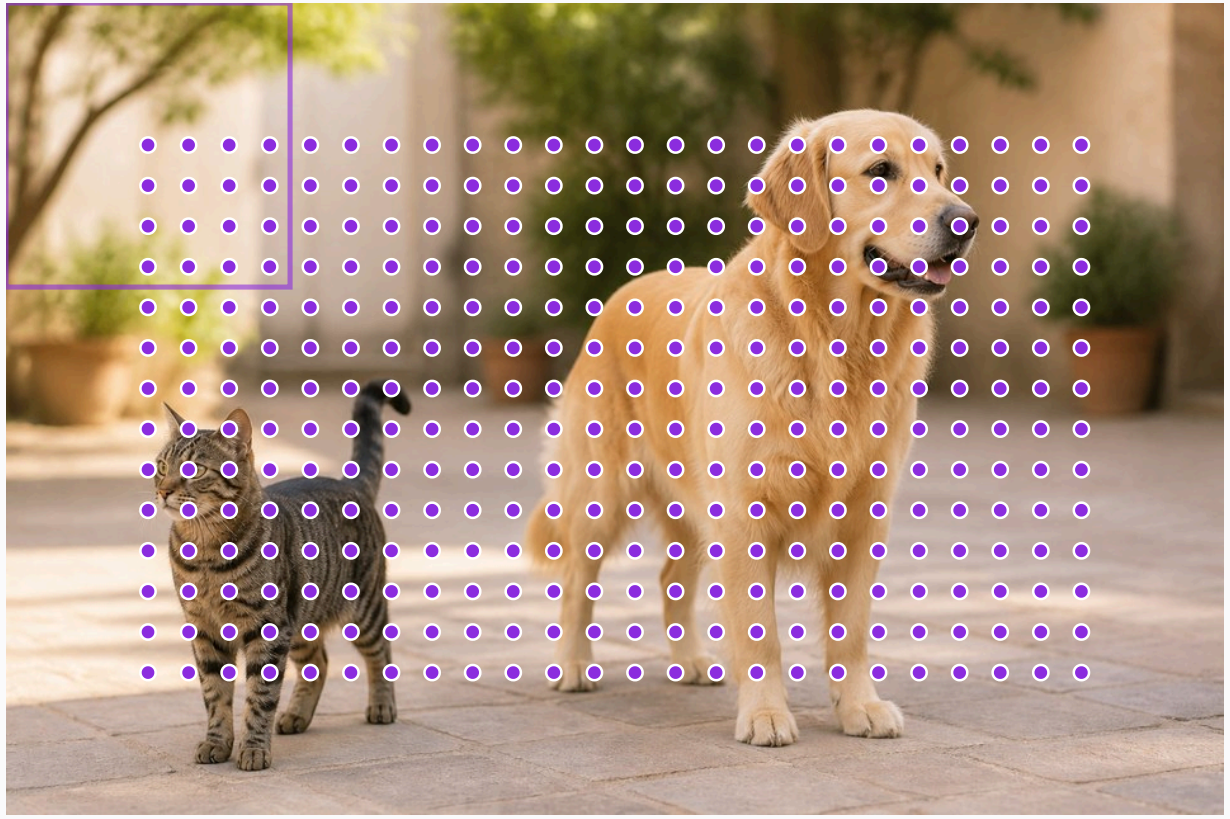


> Keep sliding, across and then down, a row at a time. When the window sits on the dog, the classifier says dog.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo



24 × 14 = 336

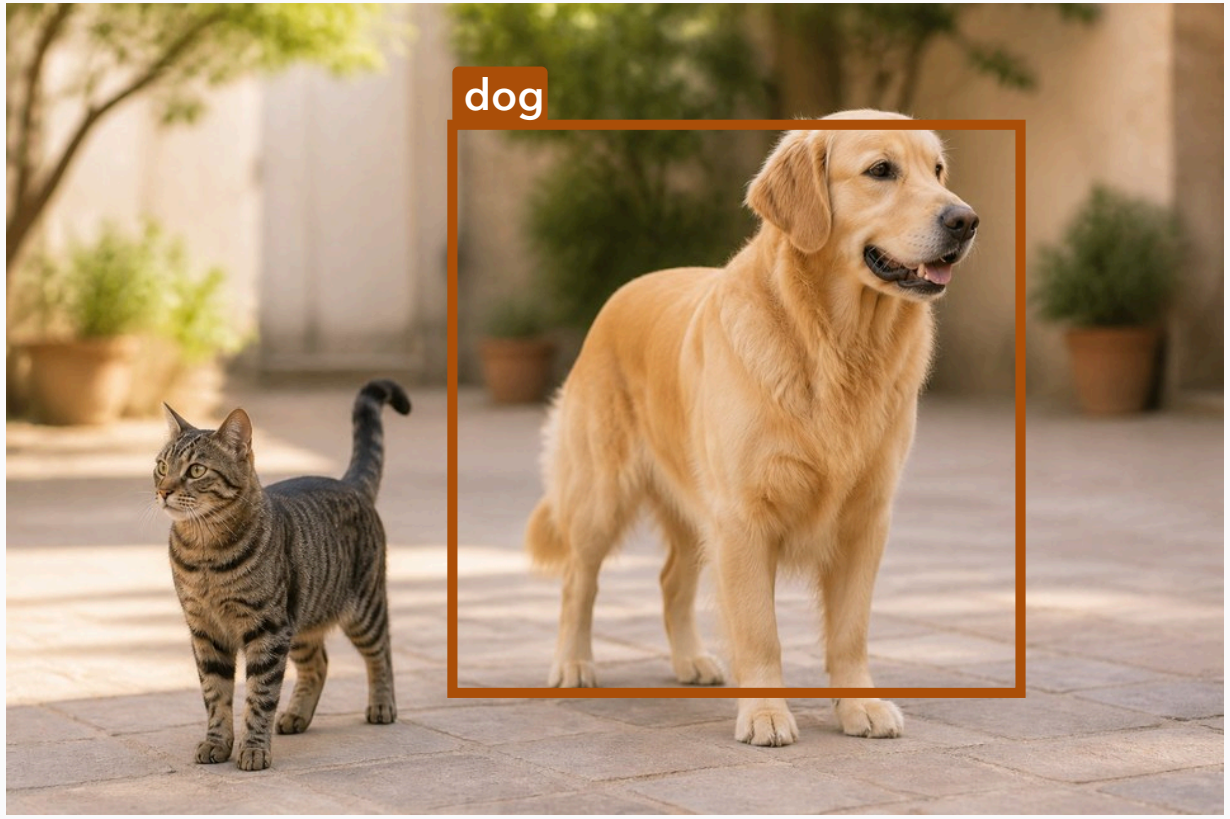
crops, one per dot: each is a full run of the classifier

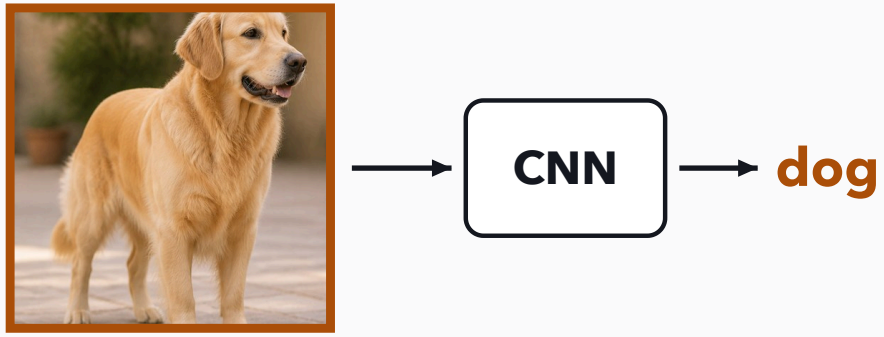
> One window size gives 24 × 14 = 336 crops, one per dot: 336 full runs of the classifier for one photo.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Slide the classifier over the photo





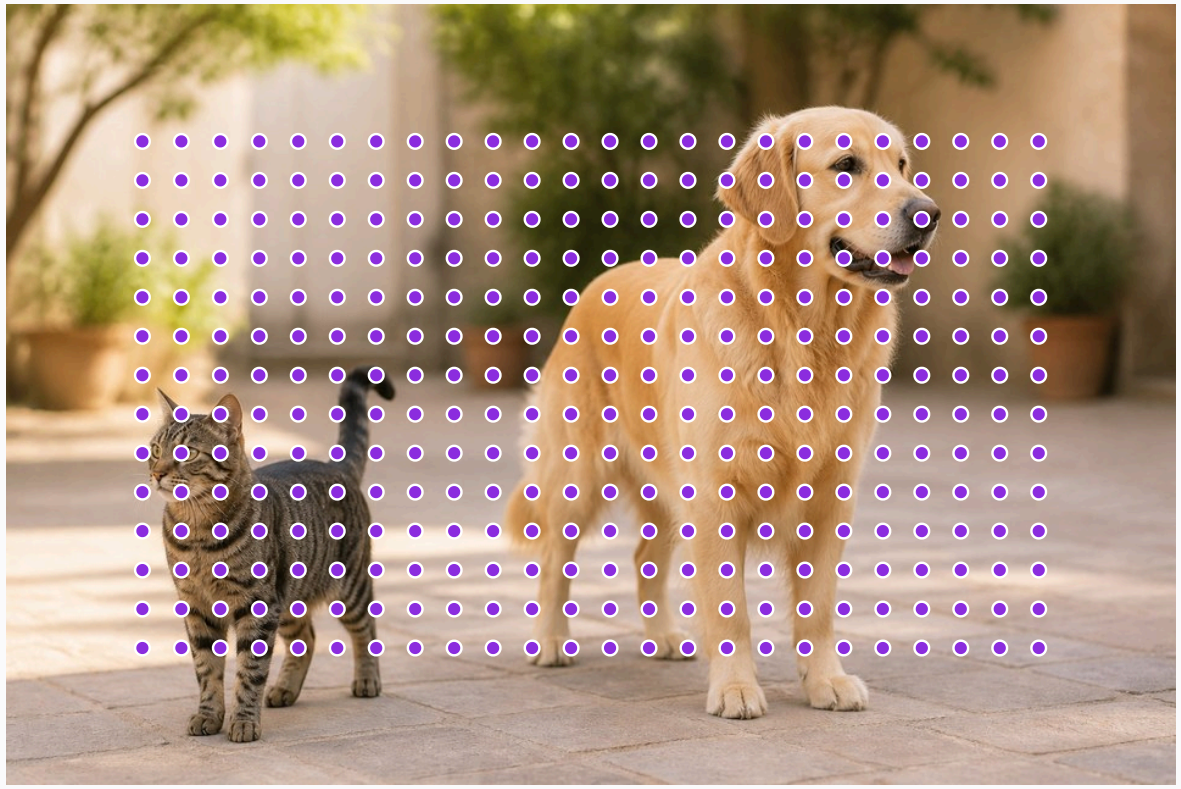
448 × 448 windows, shrunk to 224 × 224: 17 × 7 = 119 more crops

> Objects come in many sizes, so slide a bigger window too, shrinking each crop to the classifier's size. Every window size adds another full set of runs.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

What is wrong with sliding windows?



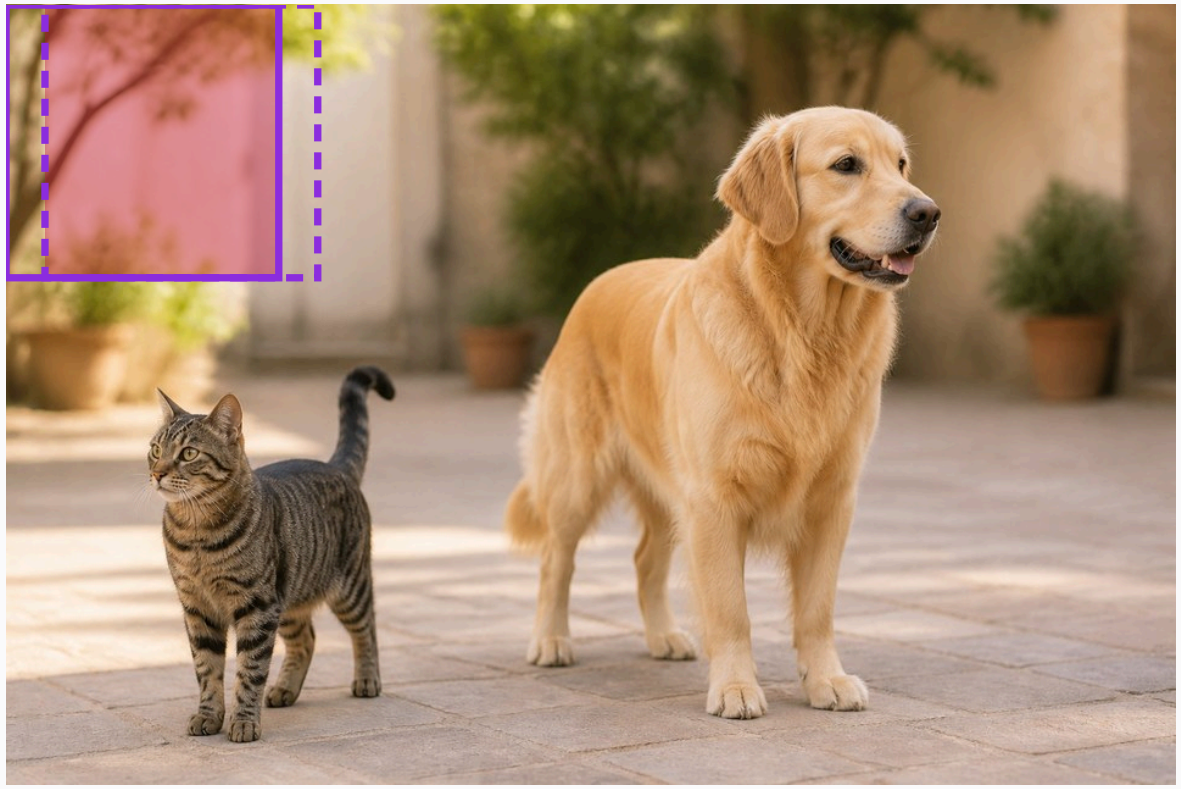
1. It is slow: 336 crops, each a full run of the CNN (28 s for VGG16 on a laptop).

> First, speed: one window size already means 336 separate runs of the classifier. VGG16 needs 28 s for them on a laptop, for one photo.

VGG16 (torchvision, ImageNet) run on this photo by scripts/run_models.py; timed on a laptop CPU. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

What is wrong with sliding windows?



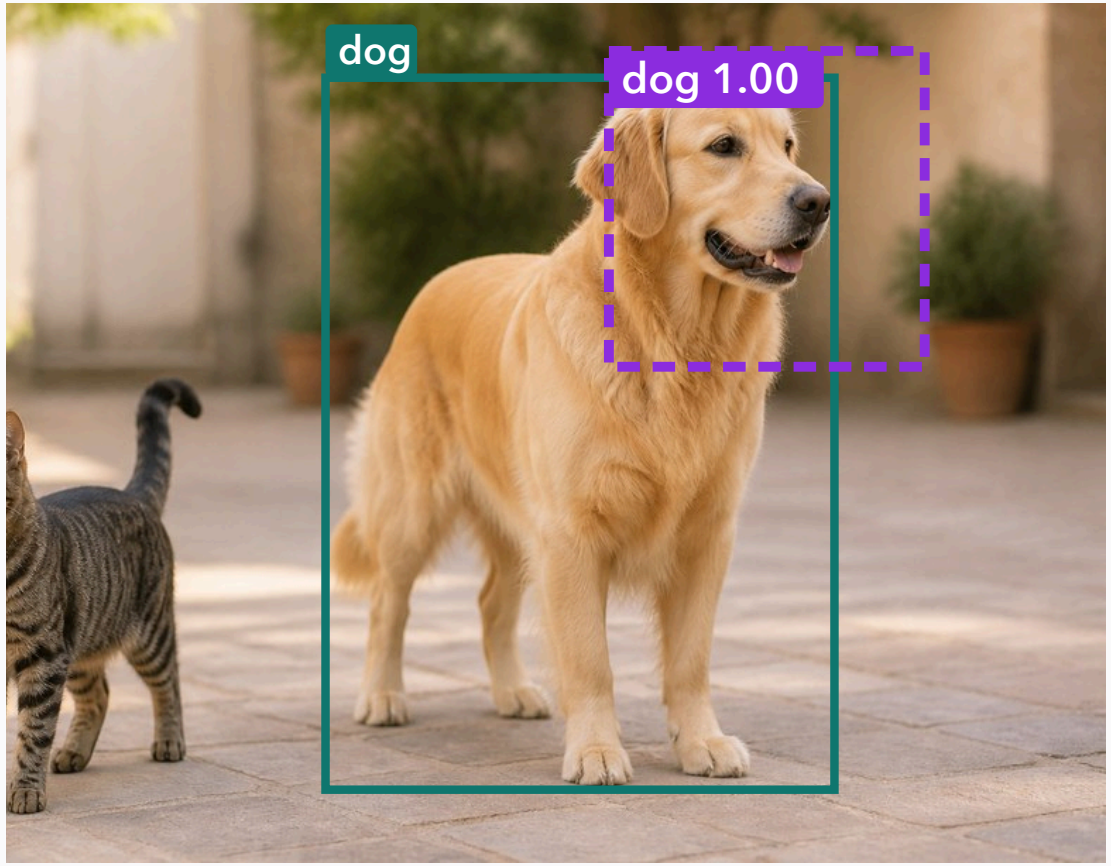
1. It is slow: 336 crops, each a full run of the CNN (28 s for VGG16 on a laptop).
2. It repeats work: neighbouring crops share 86% of their pixels.

> Second, repeated work: two neighbouring crops share 86% of their pixels, yet each crop is processed from scratch.

VGG16 (torchvision, ImageNet) run on this photo by scripts/run_models.py; timed on a laptop CPU. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

What is wrong with sliding windows?



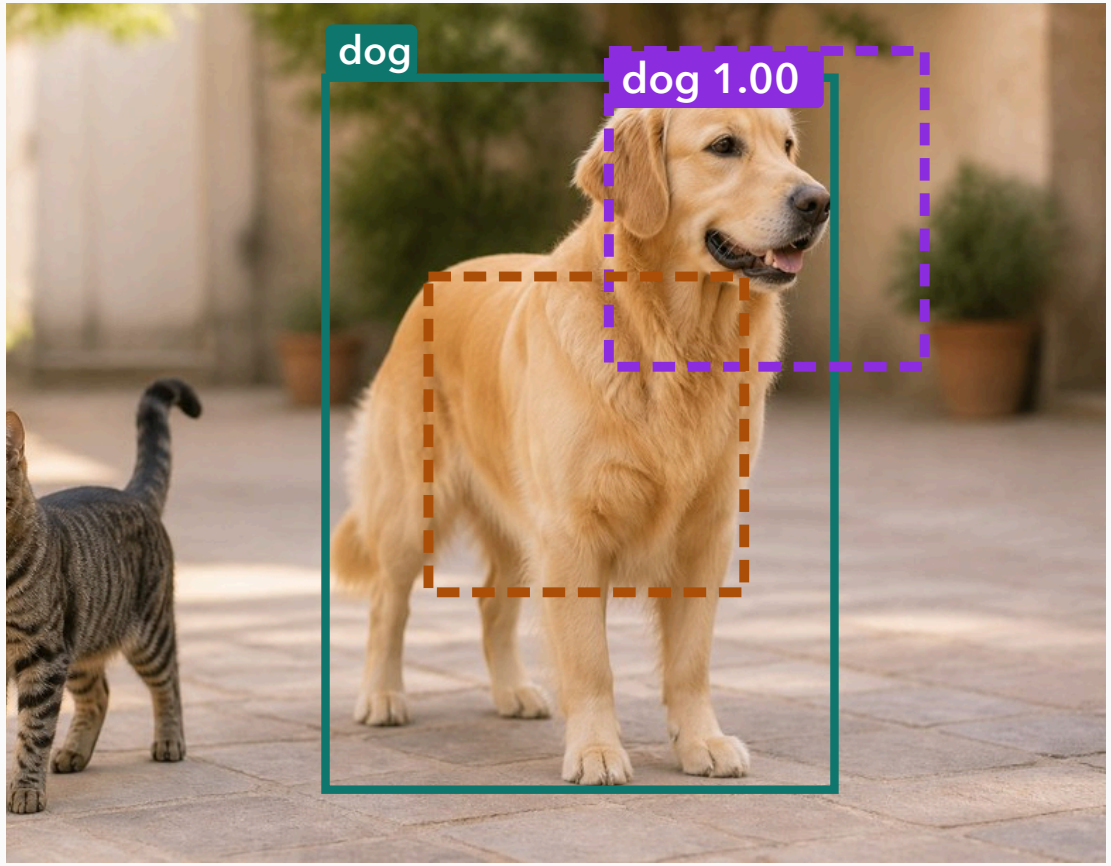
1. It is slow: 336 crops, each a full run of the CNN (28 s for VGG16 on a laptop).
2. It repeats work: neighbouring crops share 86% of their pixels.
3. Its box is only the window: the surest window holds the dog's head (IoU 0.16).

> Third, the box is only the window. The window VGG16 is surest about holds just the dog's head: IoU 0.16 with the dog's box.

VGG16 (torchvision, ImageNet) run on this photo by scripts/run_models.py; timed on a laptop CPU. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

What is wrong with sliding windows?



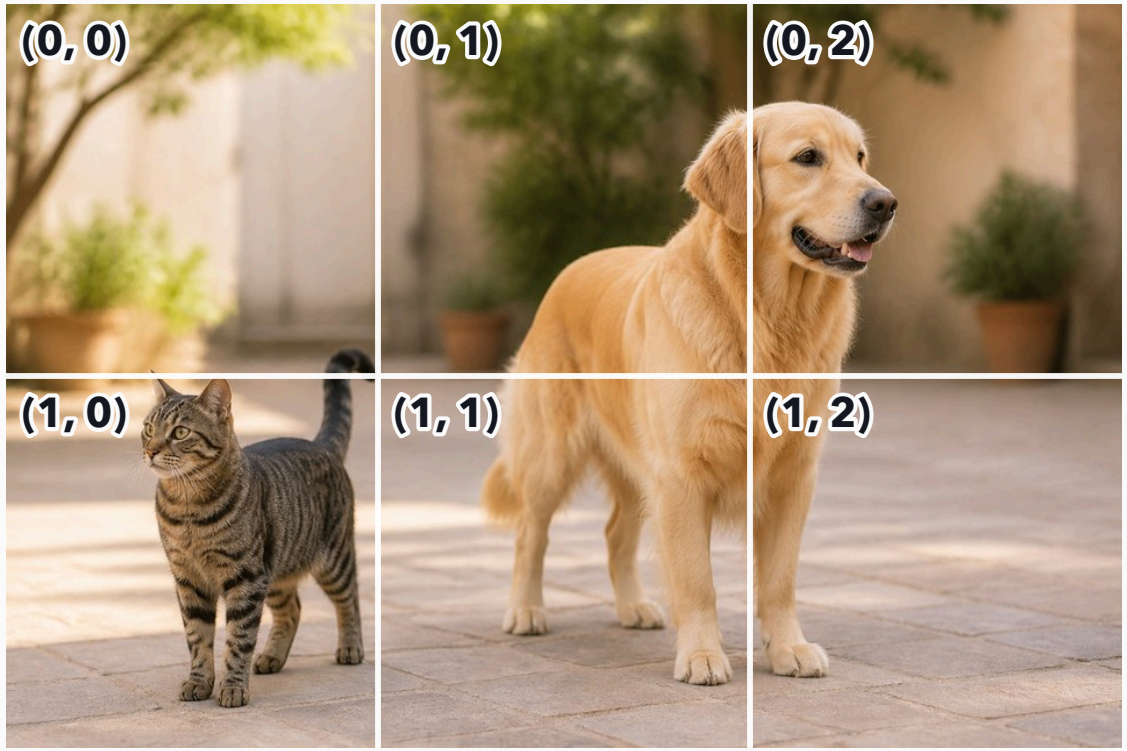
1. It is slow: 336 crops, each a full run of the CNN (28 s for VGG16 on a laptop).
2. It repeats work: neighbouring crops share 86% of their pixels.
3. Its box is only the window: the surest window holds the dog's head (IoU 0.16).
4. Fitting better costs more crops: the best 224-pixel window reaches only IoU 0.28.

> Even the best-placed window reaches IoU 0.28. Fitting better needs more sizes and shapes: more crops still. We want one pass that writes the box.

VGG16 (torchvision, ImageNet) run on this photo by scripts/run_models.py; timed on a laptop CPU. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

YOLO: grid the photo, and let every cell predict a record



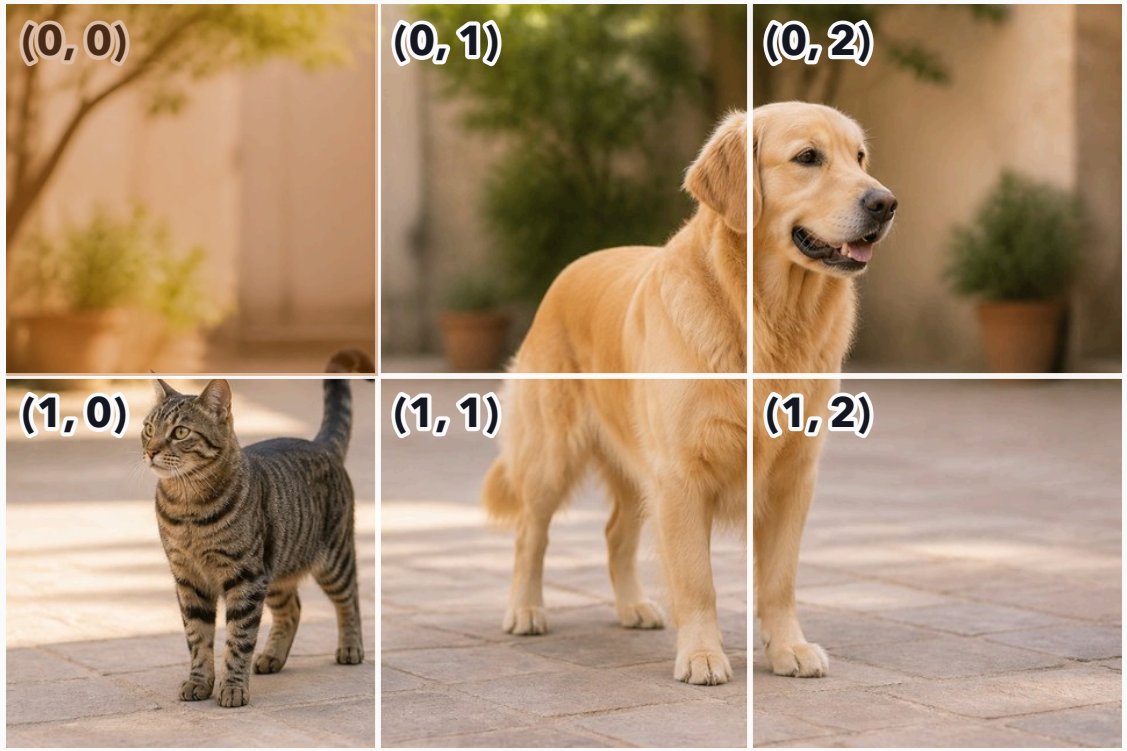
$$960 \div 3 = 320, \quad 640 \div 2 = 320$$

> YOLO's idea: cut the photo into a grid and let every cell predict. Ours has 3 × 2 cells of 320 pixels; rows and columns count from 0.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

YOLO: grid the photo, and let every cell predict a record



cell (0, 0) predicts:

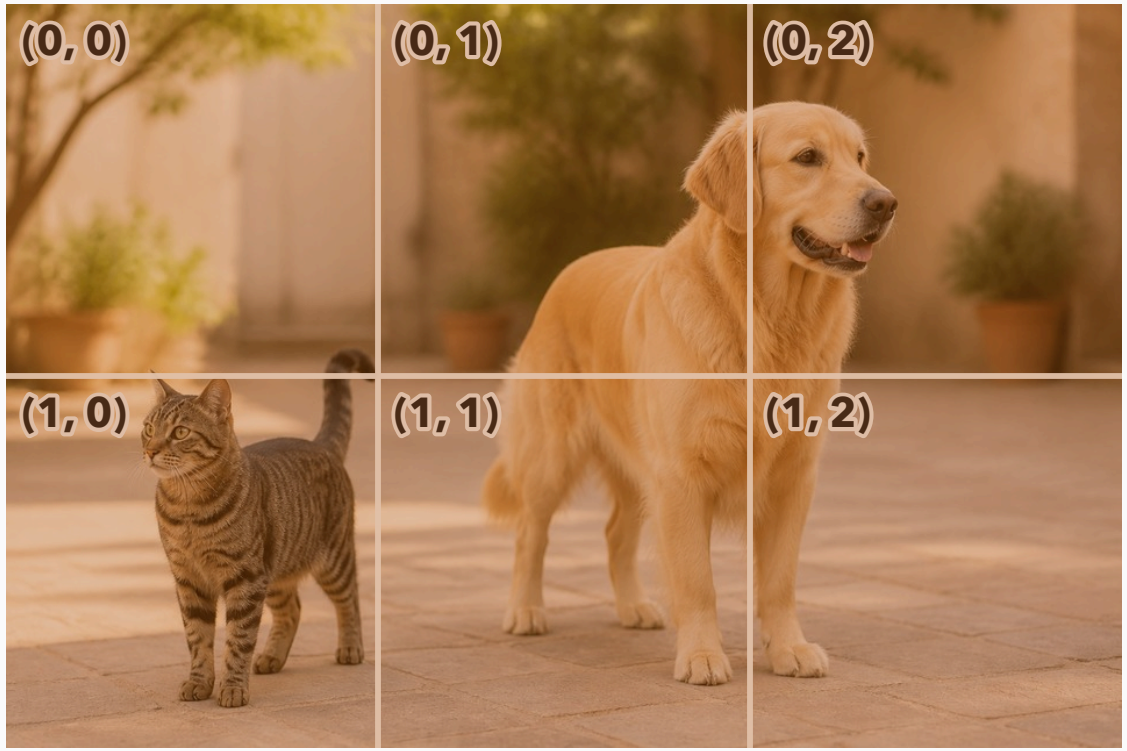
object?	
b_x	
b_y	
b_w	
b_h	
cat	
dog	

> Each cell predicts Part 4's record, a vector of 7 numbers about that cell: object?, the box b_x, b_y, b_w, b_h, and a cat and a dog score.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

YOLO: grid the photo, and let every cell predict a record



cell (0, 0) predicts:

- **object?**
- b_x
- b_y
- b_w
- b_h
- **cat**
- **dog**

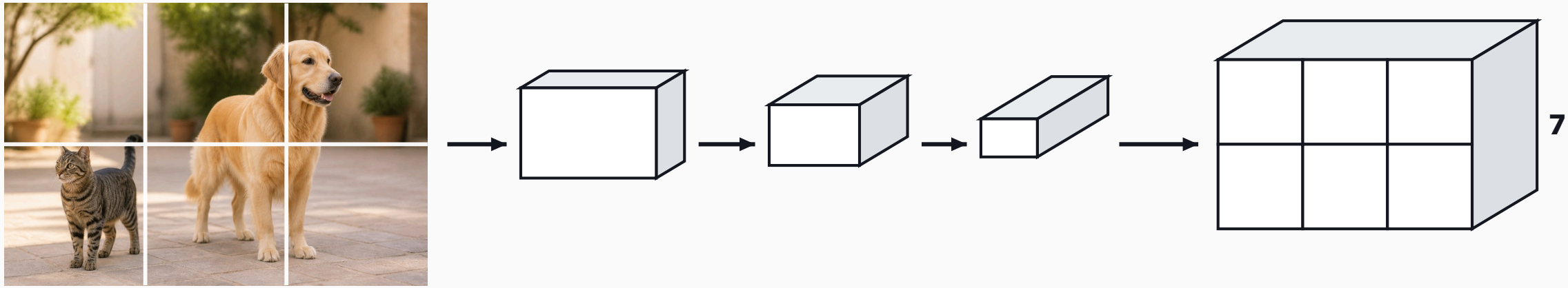
$$\underbrace{6}_{\text{cells}} \times \underbrace{7}_{\text{numbers}} = 42$$

> Six cells with seven numbers each: 42 numbers describe the whole photo.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The structure: one CNN pass writes all six records



960 × 640 × 3

convolutions and pooling

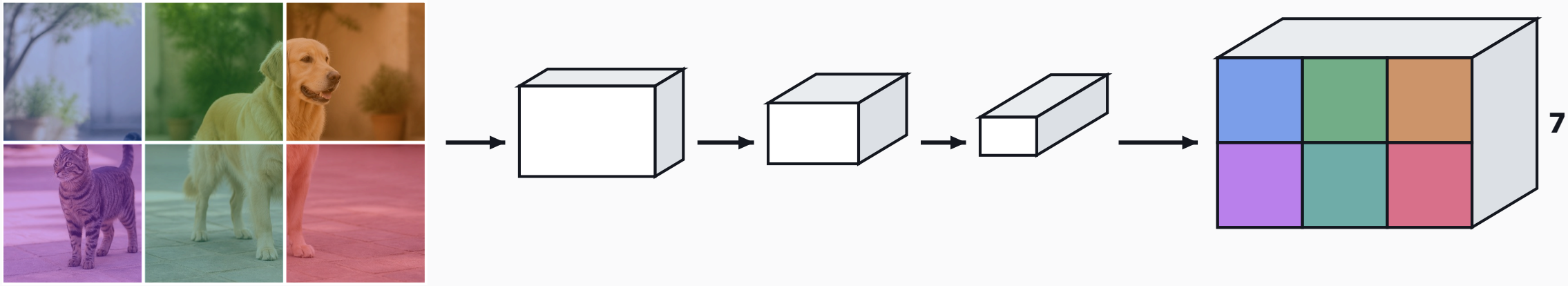
2 × 3 × 7

One CNN reads the whole photo once. Its output is a 2 × 3 block with 7 numbers at each position: one record per cell. You Only Look Once.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The structure: one CNN pass writes all six records



960 × 640 × 3

convolutions and pooling

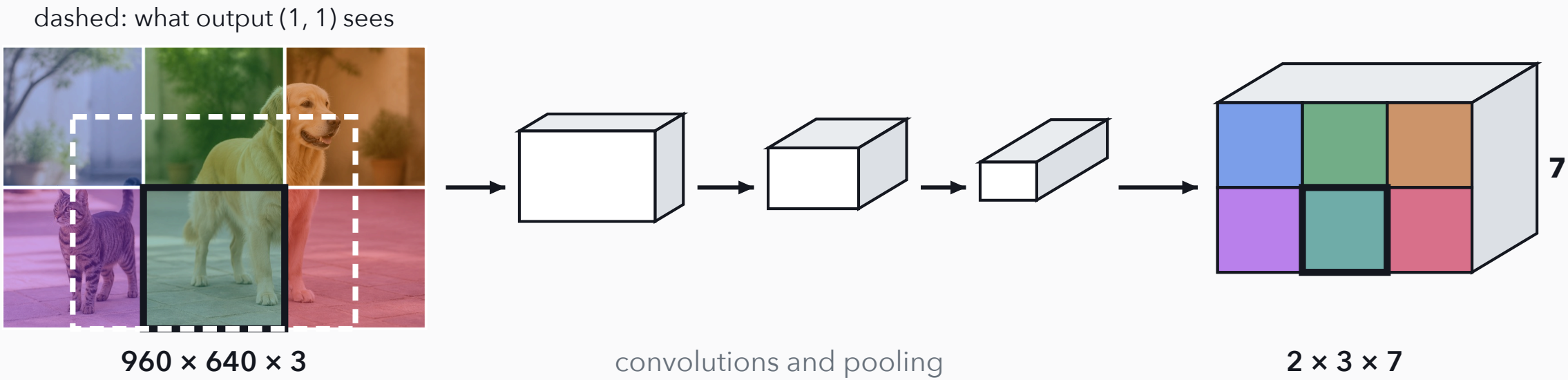
2 × 3 × 7

> Convolutions keep the photo's layout, so each output position belongs to one cell: top-left to top-left, and so on. The colours show the six pairs.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The structure: one CNN pass writes all six records

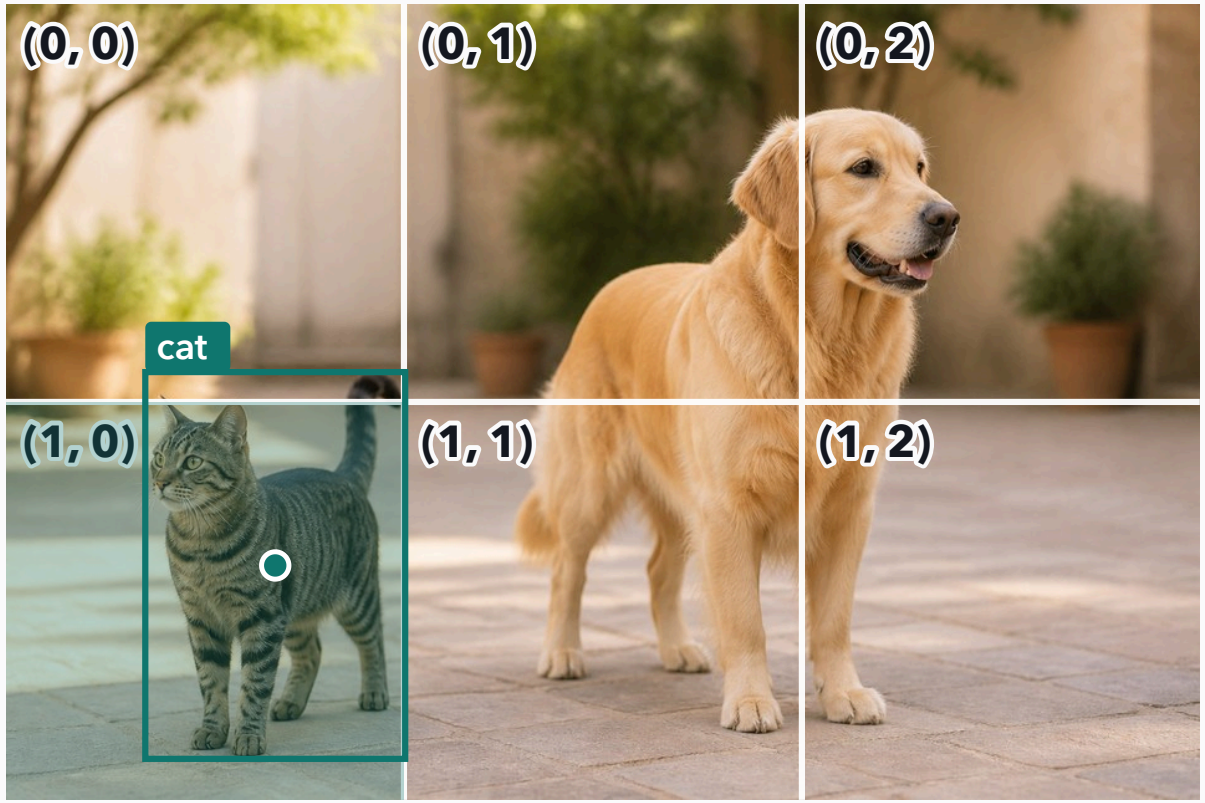


> Each position sees much more than its own cell, so a cell can describe an object bigger than itself, like the dog.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Which cell answers for an animal? The one that holds its centre



$$\text{cat's centre } (217,\ 452)$$

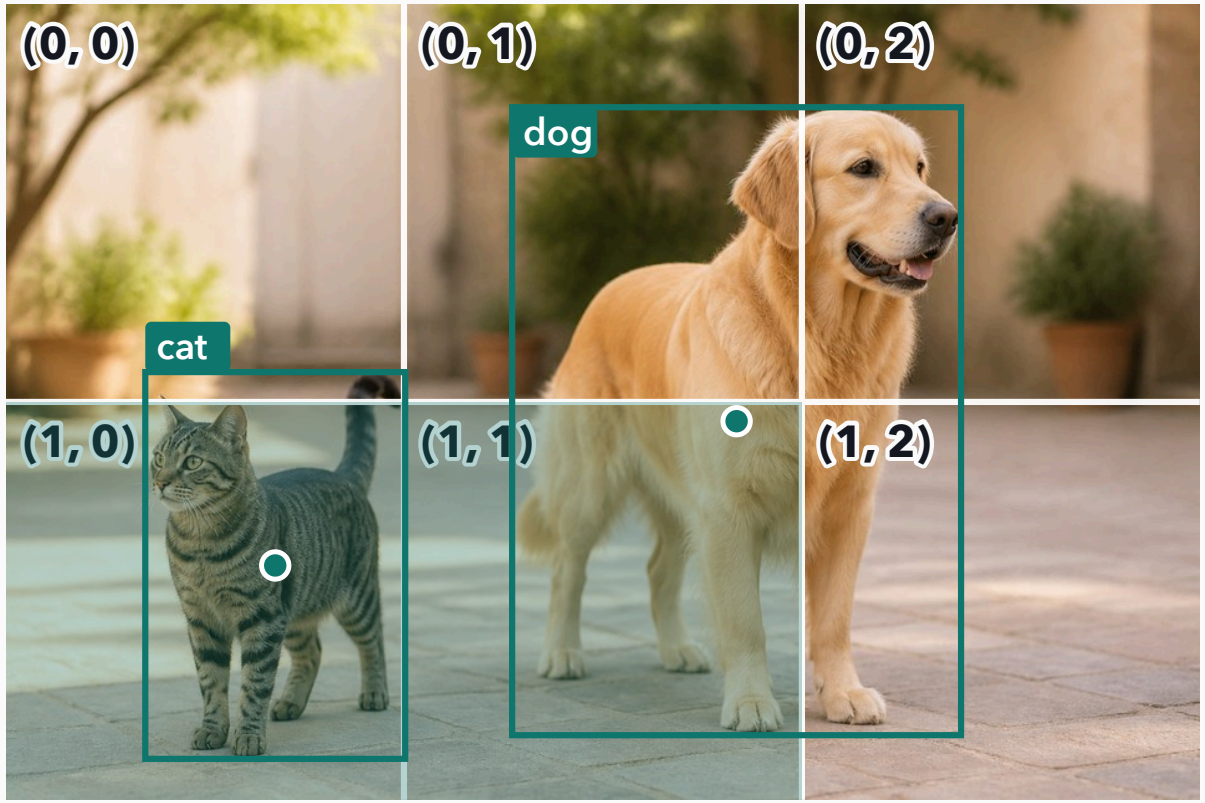
$$\text{row } \lfloor 452/320 \rfloor = 1, \quad \text{column } \lfloor 217/320 \rfloor = 0$$

> Which cell answers for the cat? Ng's rule: the one that holds the centre of its box. The cat's centre (217, 452) is in row 1, column 0.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Which cell answers for an animal? The one that holds its centre



$$\text{cat's centre } (217,\ 452)$$

$$\text{row } \lfloor 452/320 \rfloor = 1, \quad \text{column } \lfloor 217/320 \rfloor = 0$$

$$\text{dog's centre } (588,\ 336)$$

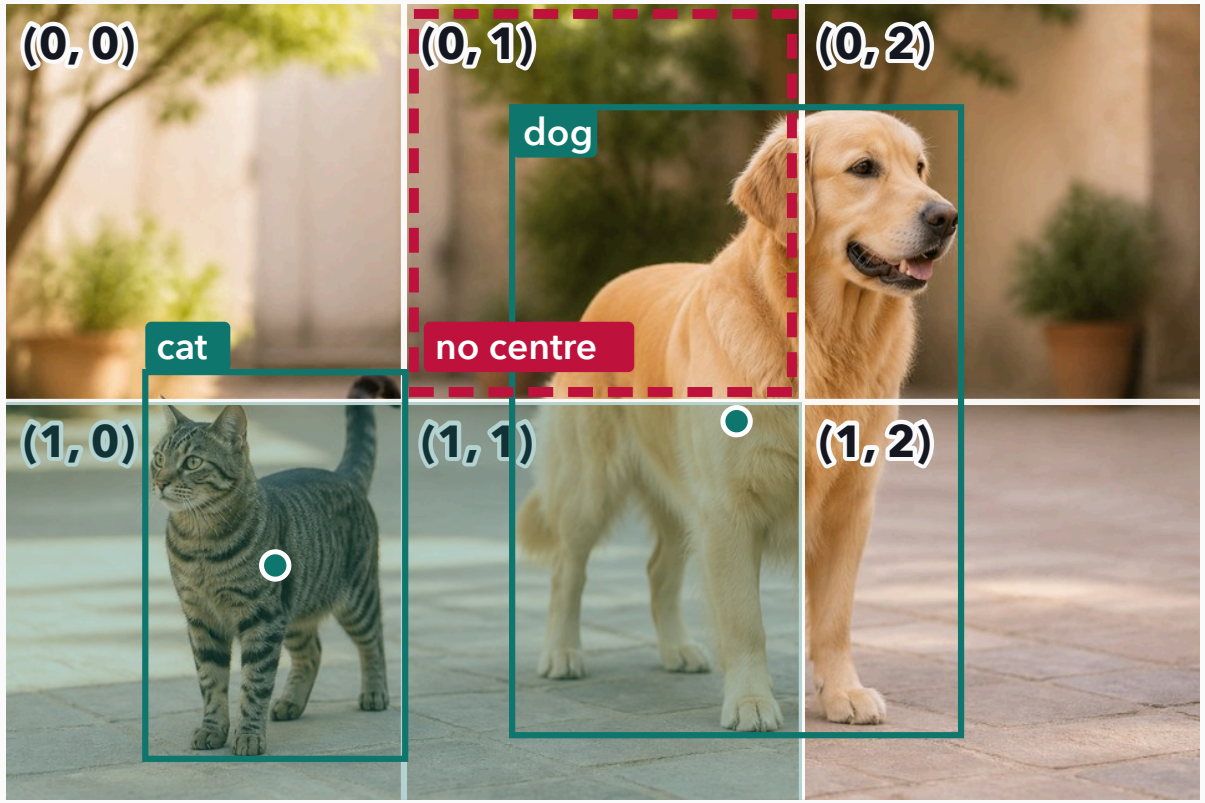
$$\text{row } \lfloor 336/320 \rfloor = 1, \quad \text{column } \lfloor 588/320 \rfloor = 1$$

> The dog's centre (588, 336) is just 16 pixels below the line between the rows, so cell (1, 1) answers for the dog.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Which cell answers for an animal? The one that holds its centre



$$\text{cat's centre } (217,\ 452)$$

$$\text{row } \lfloor 452/320 \rfloor = 1, \quad \text{column } \lfloor 217/320 \rfloor = 0$$

$$\text{dog's centre } (588,\ 336)$$

$$\text{row } \lfloor 336/320 \rfloor = 1, \quad \text{column } \lfloor 588/320 \rfloor = 1$$

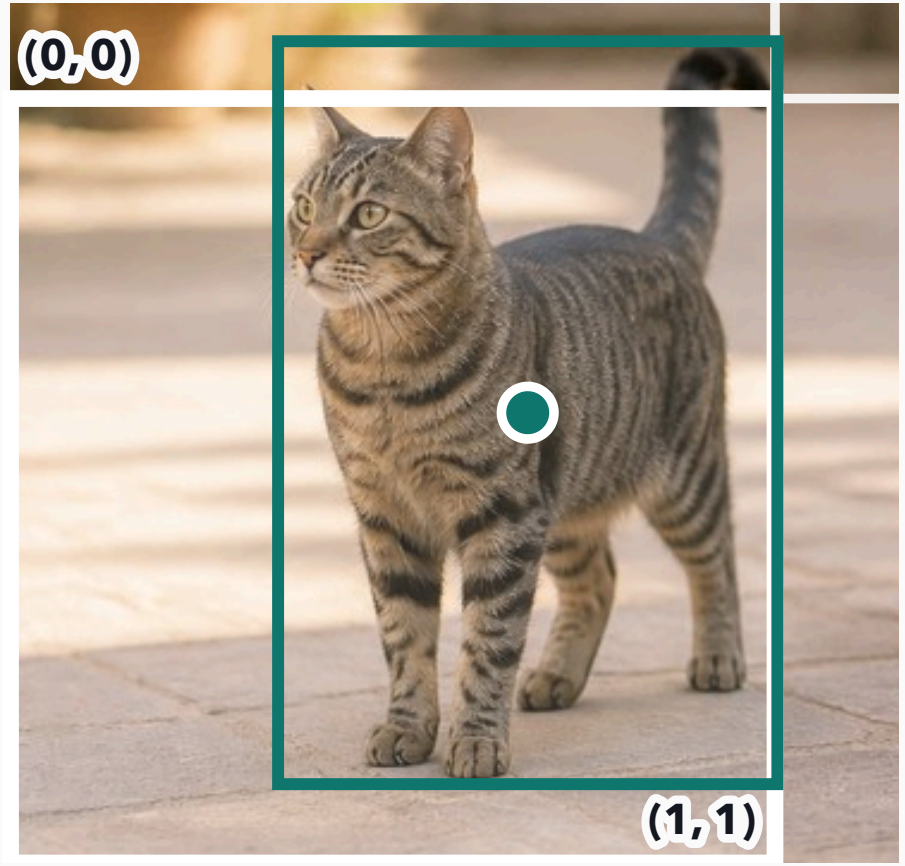
$$\text{cell } (0, 1) : \ \text{object?} = 0$$

> Cell (0, 1) holds the dog's head but not its centre, so it answers "no object". Each animal has exactly one cell.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The cat's box, measured in its cell



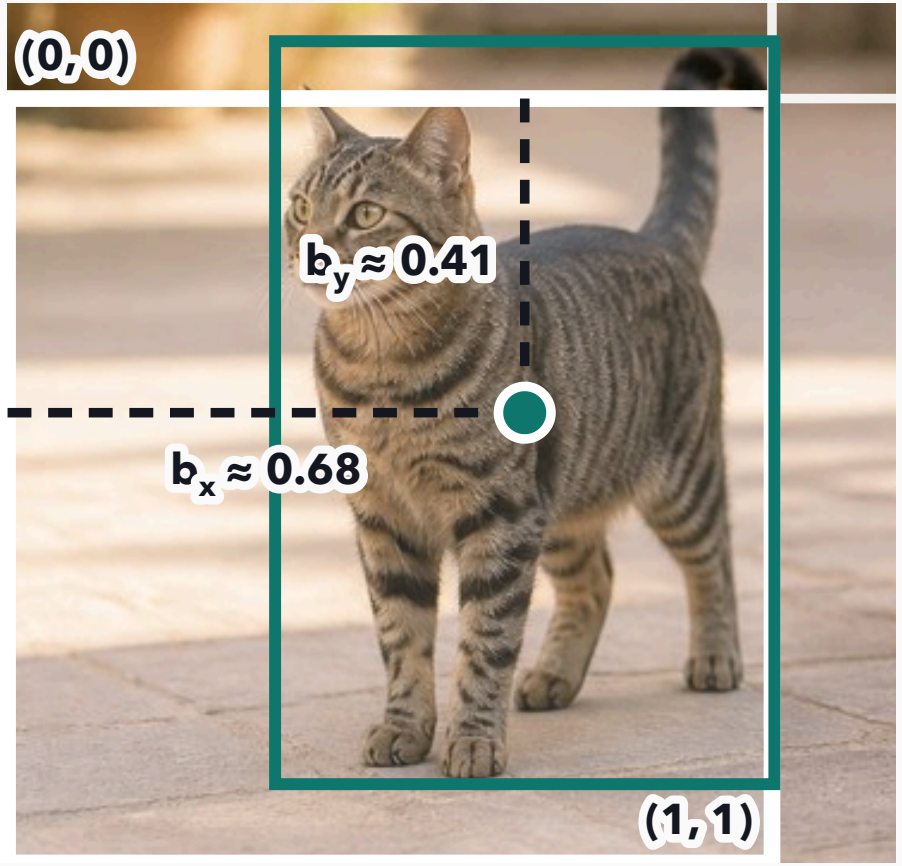
$$\text{cell } (1,0):\ x \text{ from } 0 \text{ to } 320,\ y \text{ from } 320 \text{ to } 640$$

> Ng measures the box inside its cell. The cat's cell gets its own coordinates: (0, 0) at its top-left corner and (1, 1) at its bottom-right.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The cat's box, measured in its cell



$$\text{cell } (1,0):\ x \text{ from } 0 \text{ to } 320,\ y \text{ from } 320 \text{ to } 640$$

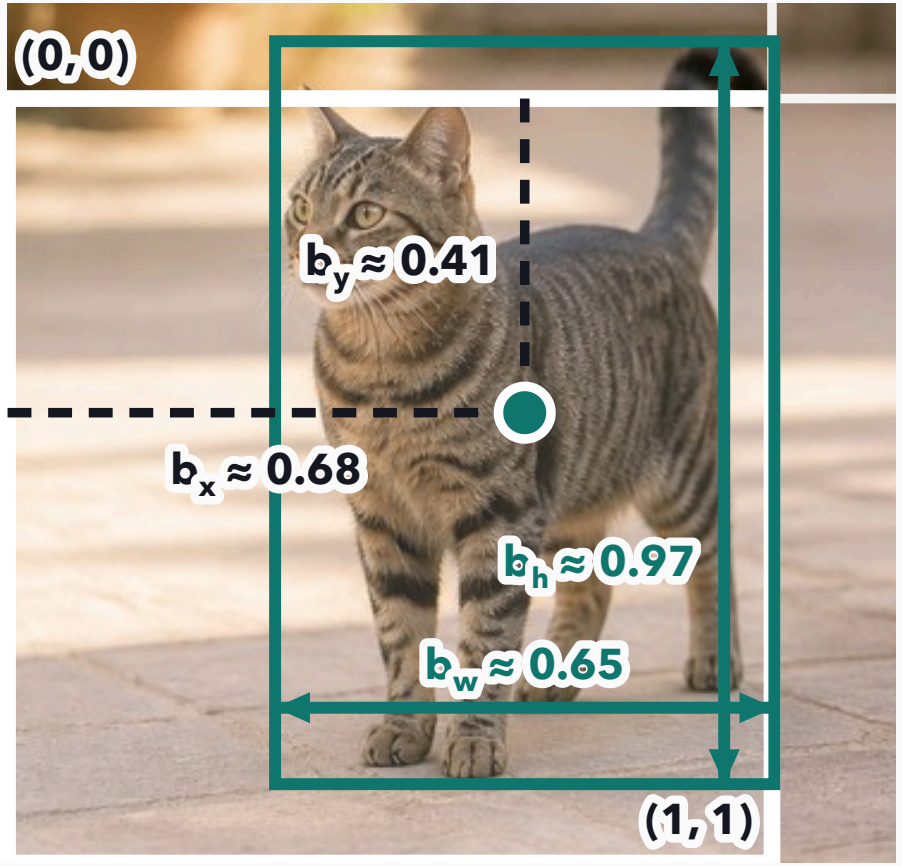
$$b_x = \frac{217 - 0}{320} \approx 0.68, \quad b_y = \frac{452 - 320}{320} \approx 0.41$$

> The cat's centre is 217 pixels from the cell's left edge and 132 below its top: b_x ≈ 0.68 and b_y ≈ 0.41 of a cell.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The cat's box, measured in its cell



$$\text{cell } (1,0):\ x \text{ from } 0 \text{ to } 320,\ y \text{ from } 320 \text{ to } 640$$

$$b_x = \frac{217 - 0}{320} \approx 0.68, \quad b_y = \frac{452 - 320}{320} \approx 0.41$$

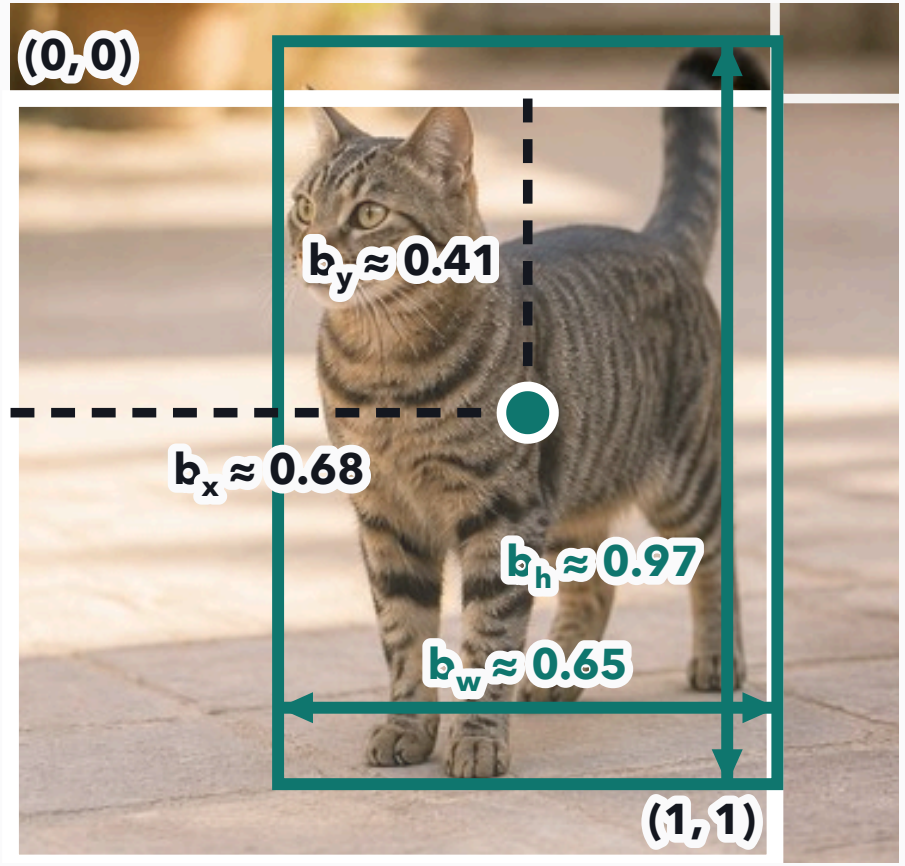
$$b_w = \frac{209}{320} \approx 0.65, \quad b_h = \frac{311}{320} \approx 0.97$$

> The size is counted in cells too. The cat is 209 pixels wide, 0.65 of a cell, and 311 pixels tall, 0.97 of a cell.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The cat's box, measured in its cell



$$\text{cell } (1,0):\ x \text{ from } 0 \text{ to } 320,\ y \text{ from } 320 \text{ to } 640$$

$$b_x = \frac{217-0}{320} \approx 0.68, \quad b_y = \frac{452-320}{320} \approx 0.41$$

$$b_w = \frac{209}{320} \approx 0.65, \quad b_h = \frac{311}{320} \approx 0.97$$

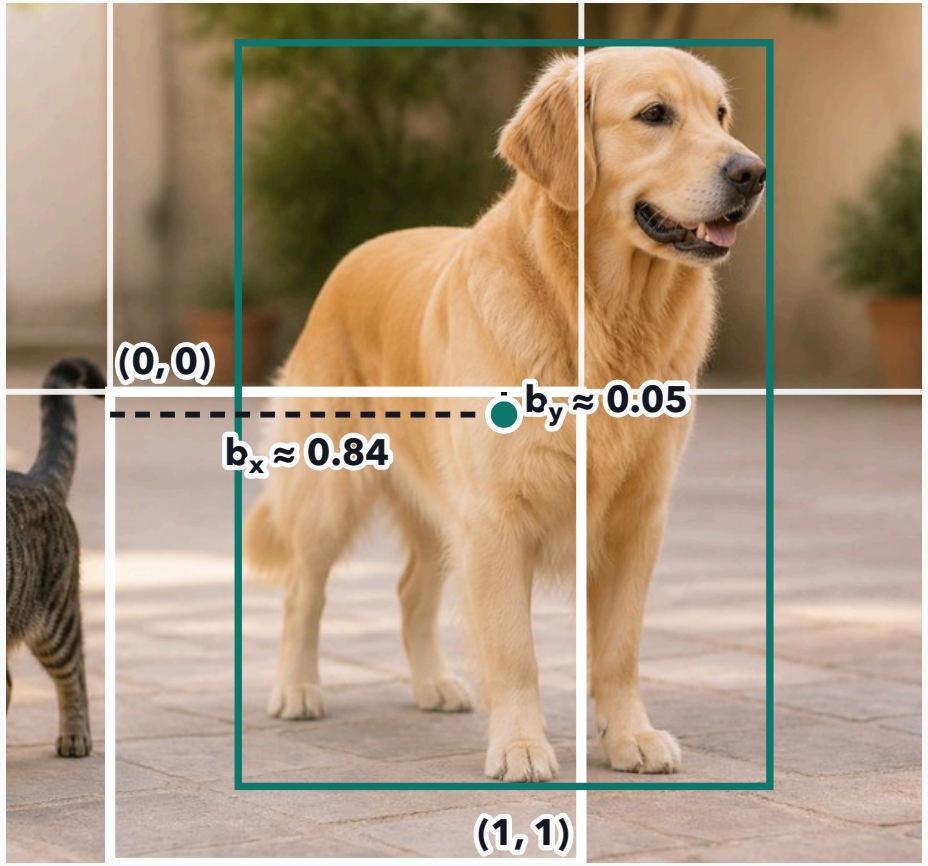
object?	b_x	b_y	b_w	b_h	cat	dog
1	0.68	0.41	0.65	0.97	1	0

So the cat's cell writes object? 1, the box (0.68, 0.41, 0.65, 0.97), cat 1 and dog 0.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The dog’s box is bigger than its cell



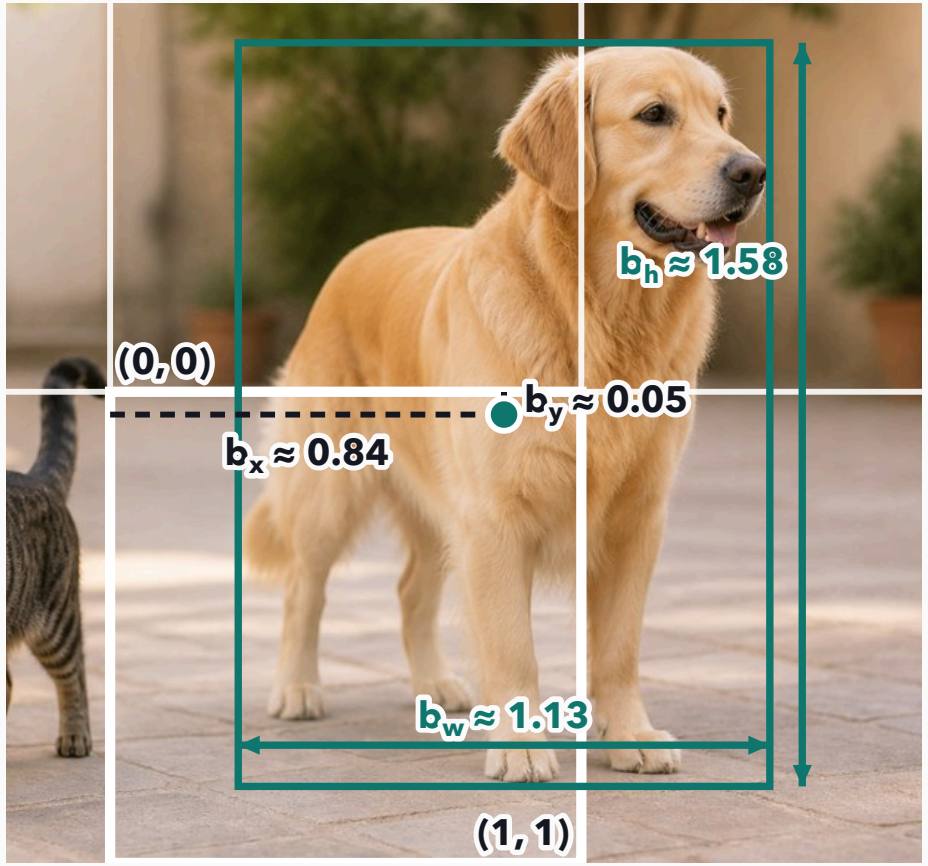
$$b_x = \frac{588 - 320}{320} \approx 0.84, \quad b_y = \frac{336 - 320}{320} \approx 0.05$$

> The dog’s centre is 268 pixels from its cell’s left edge and only 16 below its top: b_x ≈ 0.84 and b_y ≈ 0.05.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The dog's box is bigger than its cell



$$b_x = \frac{588 - 320}{320} \approx 0.84, \quad b_y = \frac{336 - 320}{320} \approx 0.05$$

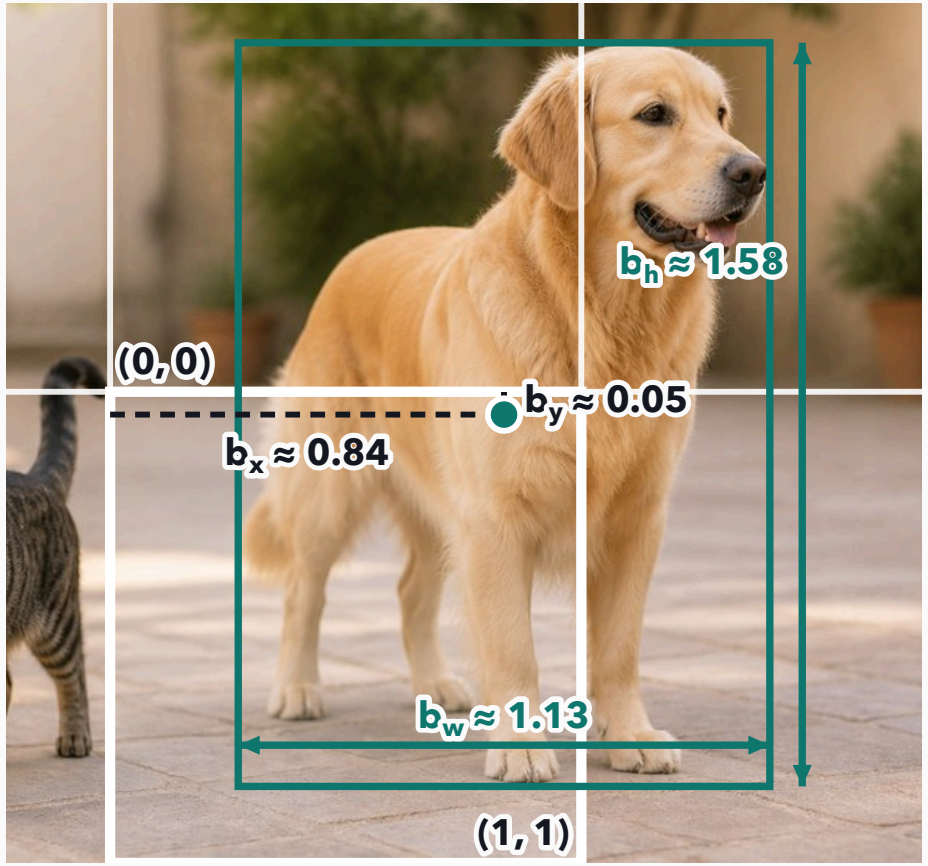
$$b_w = \frac{361}{320} \approx 1.13, \quad b_h = \frac{505}{320} \approx 1.58$$

> The dog is wider and taller than its cell: b_w ≈ 1.13 and b_h ≈ 1.58 cells. A box may spill out of its cell, but its centre may not.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The dog's box is bigger than its cell



$$b_x = \frac{588 - 320}{320} \approx 0.84, \quad b_y = \frac{336 - 320}{320} \approx 0.05$$

$$b_w = \frac{361}{320} \approx 1.13, \quad b_h = \frac{505}{320} \approx 1.58$$

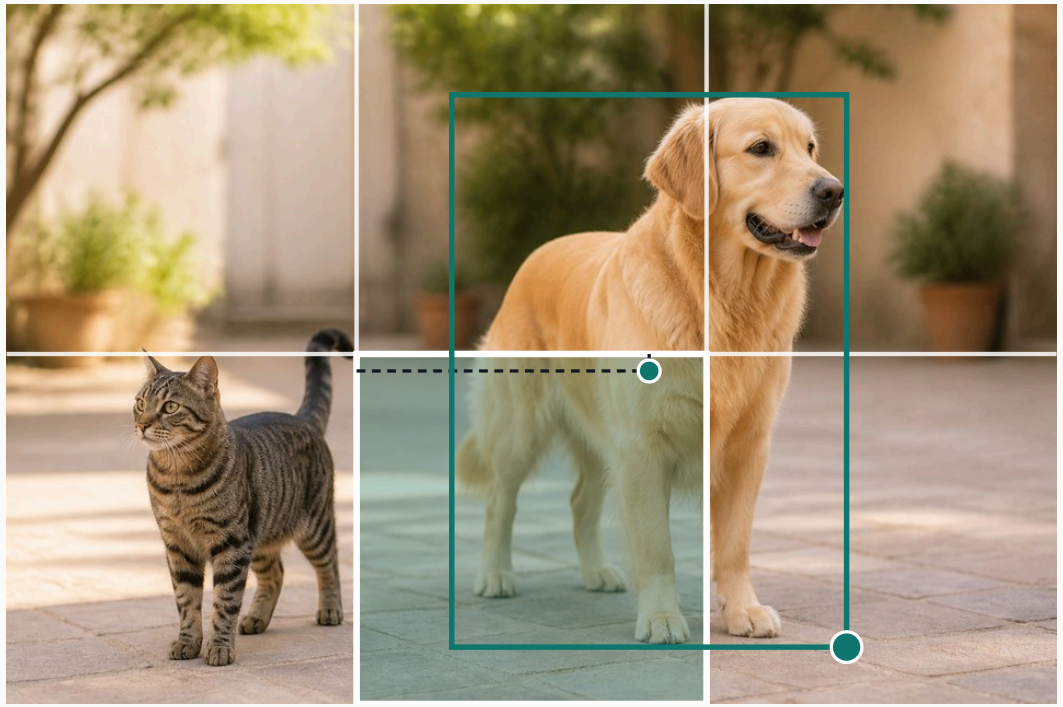
object?	b_x	b_y	b_w	b_h	cat	dog
1	0.84	0.05	1.13	1.58	0	1

So the dog's cell writes object? 1, the box (0.84, 0.05, 1.13, 1.58), cat 0 and dog 1.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Try it: move a box, and watch its cell write the record



cell (1, 1) holds the centre and writes:

object?	1
b_x	0.84
b_y	0.05
b_w	1.13
b_h	1.58
cat	0
dog	1

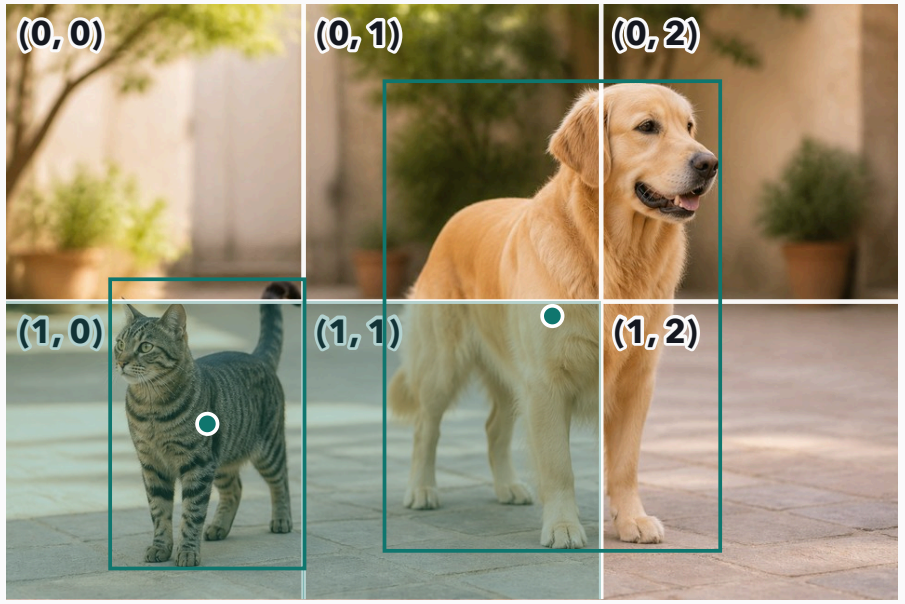
The other 5 cells: object? 0, the rest "?".

Drag the box or its corner dot. The cell holding the centre lights up and writes the record. Try the finer grids: the same box, counted in smaller cells.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The target for the whole photo: six records



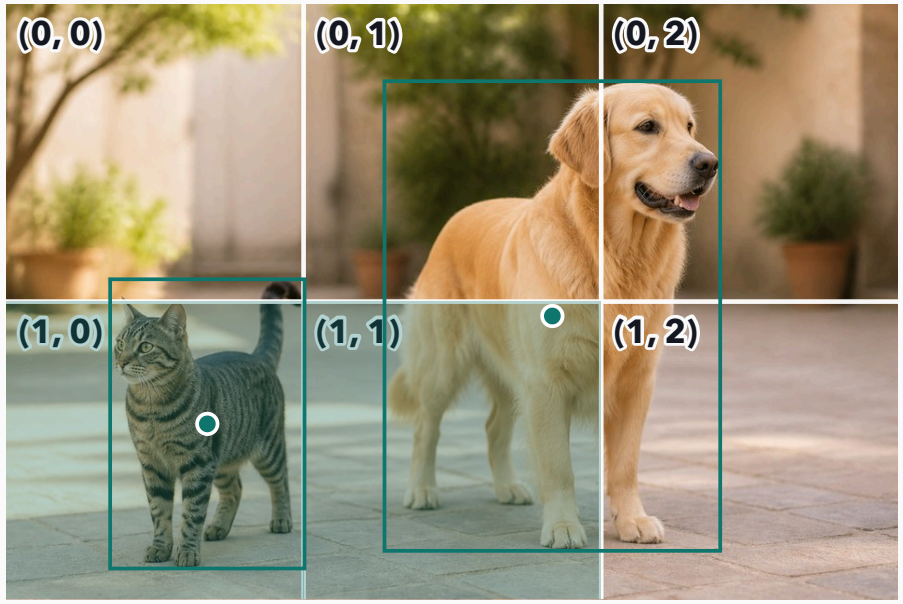
	object?	b_x	b_y	b_w	b_h	cat	dog
cell (0, 0)							
cell (0, 1)							
cell (0, 2)							
cell (1, 0)	1	0.68	0.41	0.65	0.97	1	0
cell (1, 1)	1	0.84	0.05	1.13	1.58	0	1
cell (1, 2)							

> The whole target, one record per cell. The cat's cell and the dog's cell hold the records we just wrote.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The target for the whole photo: six records



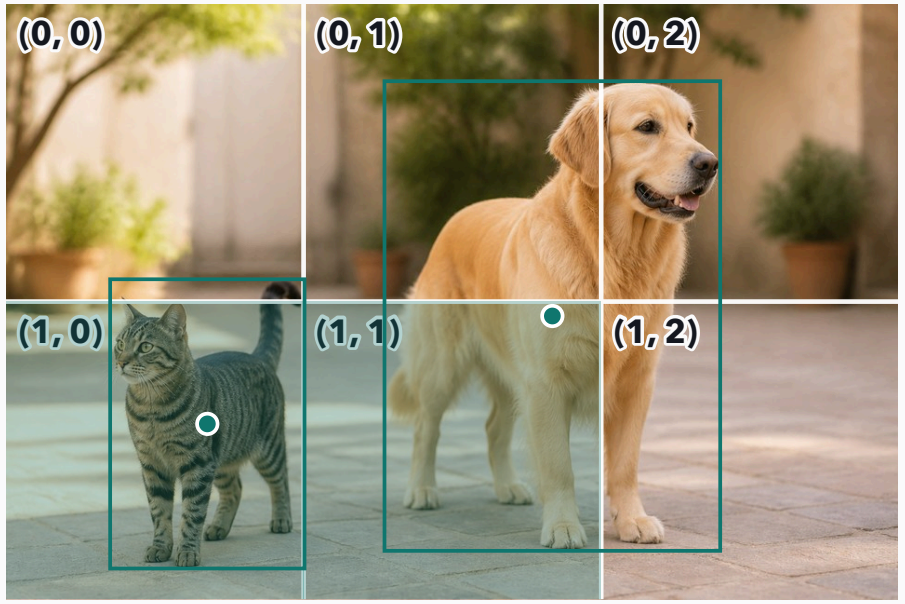
	object?	b_x	b_y	b_w	b_h	cat	dog
cell (0, 0)	0	?	?	?	?	?	?
cell (0, 1)	0	?	?	?	?	?	?
cell (0, 2)	0	?	?	?	?	?	?
cell (1, 0)	1	0.68	0.41	0.65	0.97	1	0
cell (1, 1)	1	0.84	0.05	1.13	1.58	0	1
cell (1, 2)	0	?	?	?	?	?	?

> The other four cells hold no centre, not even (0, 1) with the dog's head: object? 0, and "?" everywhere else.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

The target for the whole photo: six records



	object?	b_x	b_y	b_w	b_h	cat	dog
cell (0, 0)	0	?	?	?	?	?	?
cell (0, 1)	0	?	?	?	?	?	?
cell (0, 2)	0	?	?	?	?	?	?
cell (1, 0)	1	0.68	0.41	0.65	0.97	1	0
cell (1, 1)	1	0.84	0.05	1.13	1.58	0	1
cell (1, 2)	0	?	?	?	?	?	?

$6 \times 7 = 42$ numbers: the target

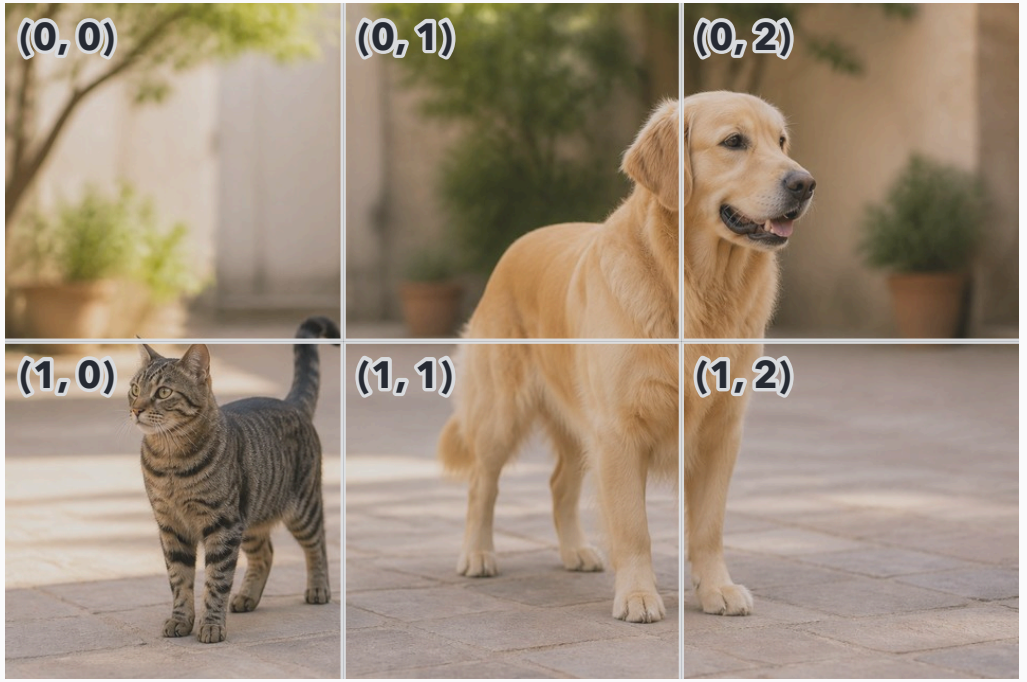
> Six records of seven numbers: these 42 numbers are what the network should write for this photo.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
```



y[1, 1], the dog's cell:

object?	b_x	b_y	b_w	b_h	cat	dog
0	0	0	0	0	0	0

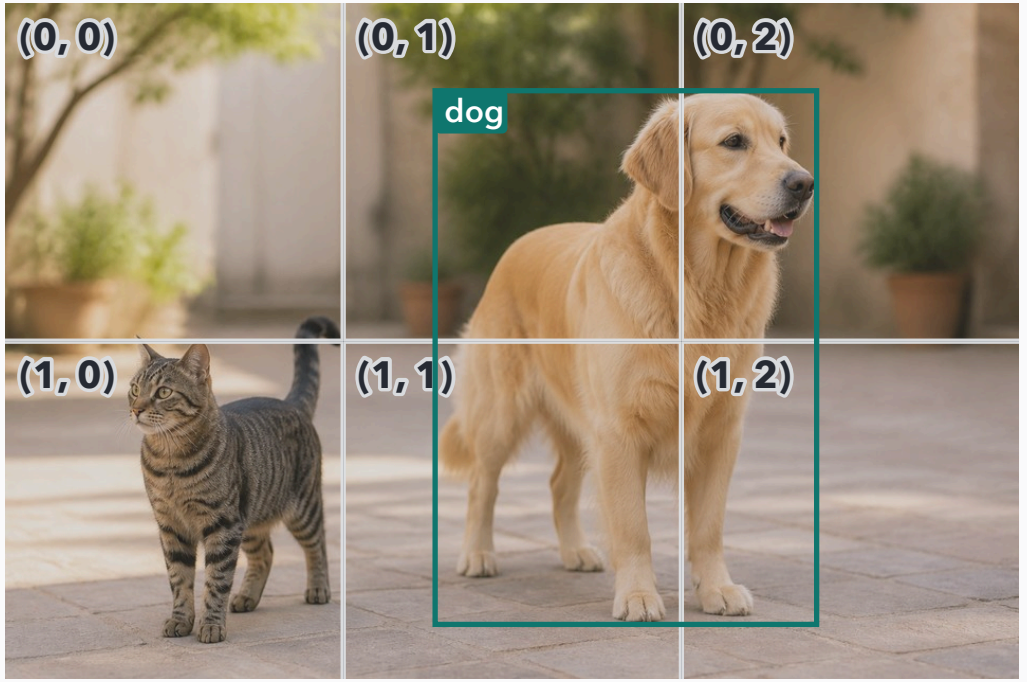
> Build the target in code. Every cell starts as "no object": a 2 × 3 × 7 block of zeros. The "?" entries stay 0; the loss will skip them.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
    for (x1, y1, x2, y2), k in zip(boxes, classes):
```



y[1, 1], the dog's cell:

object?	b_x	b_y	b_w	b_h	cat	dog
0	0	0	0	0	0	0

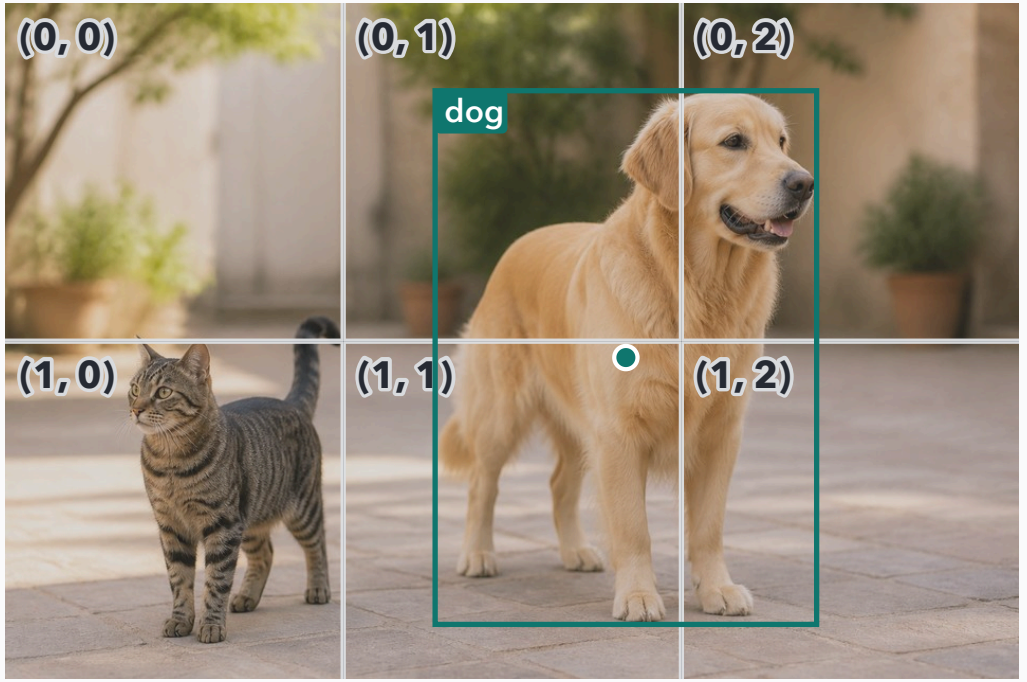
Loop over the annotated objects, each a box and a class. First the dog: box (407, 83, 768, 588), class 1.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
    for (x1, y1, x2, y2), k in zip(boxes, classes):
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
```



y[1, 1], the dog's cell:

object?	b_x	b_y	b_w	b_h	cat	dog
0	0	0	0	0	0	0

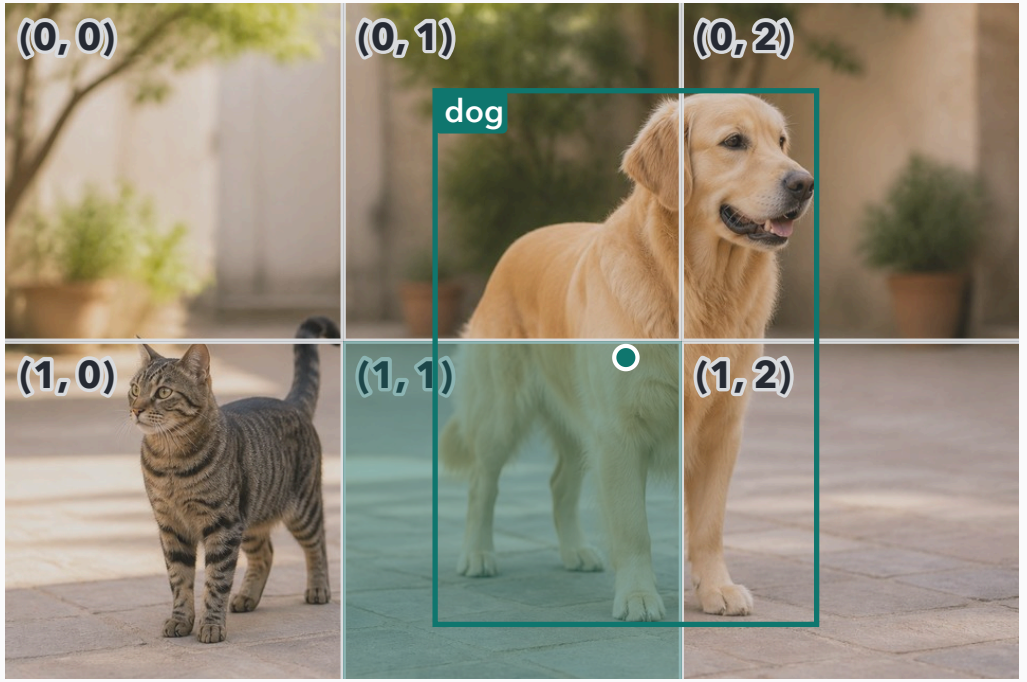
> Its centre is the middle of the box: (588, 336).

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
    for (x1, y1, x2, y2), k in zip(boxes, classes):
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
        r, c = int(cy // s), int(cx // s)
```



y[1, 1], the dog's cell:

object?	b_x	b_y	b_w	b_h	cat	dog
0	0	0	0	0	0	0

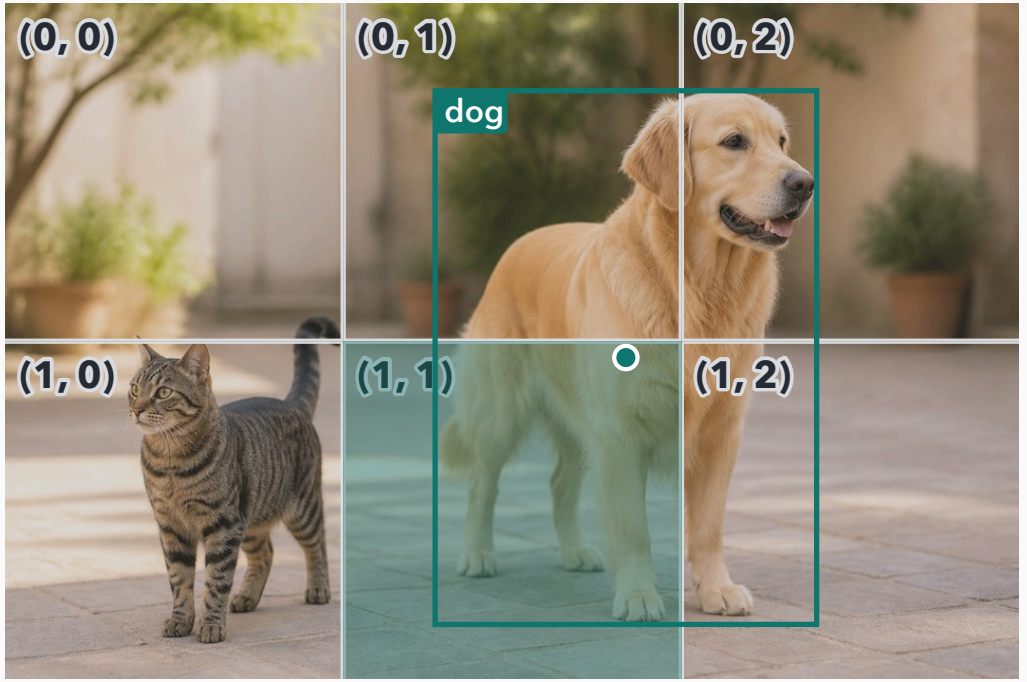
Divide by the cell size, 320, and round down: row 1, column 1. Only this cell will describe the dog.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
    for (x1, y1, x2, y2), k in zip(boxes, classes):
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
        r, c = int(cy // s), int(cx // s)
        y[r, c, 0] = 1                  # object?
```



y[1, 1], the dog's cell:

object?	b_x	b_y	b_w	b_h	cat	dog
1	0	0	0	0	0	0

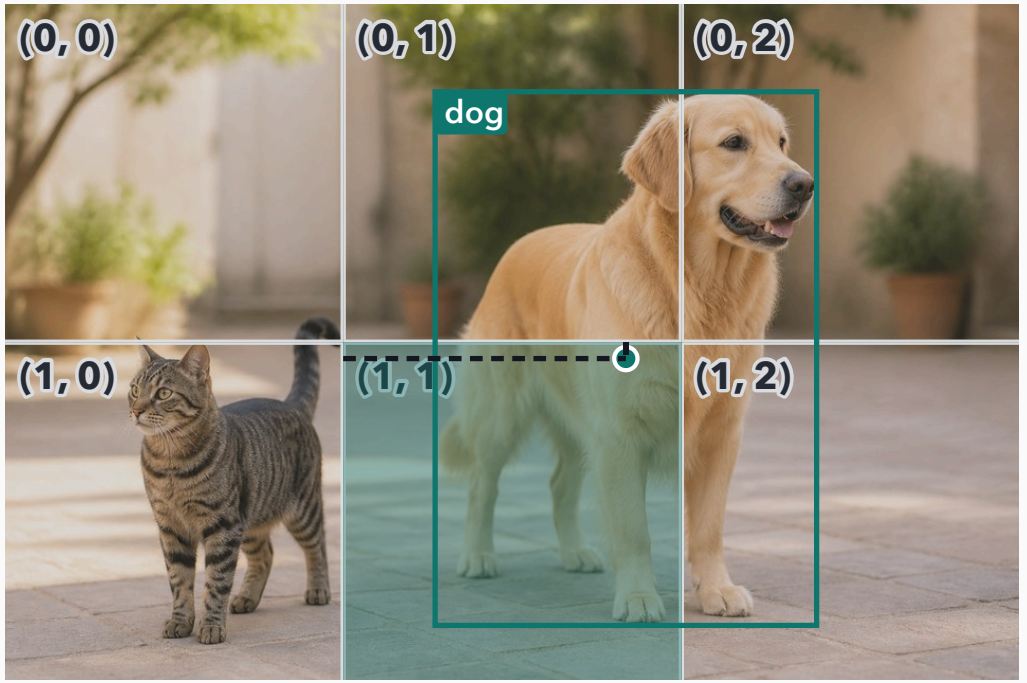
> Set object? to 1 in that cell.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
    for (x1, y1, x2, y2), k in zip(boxes, classes):
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
        r, c = int(cy // s), int(cx // s)
        y[r, c, 0] = 1                # object?
        y[r, c, 1] = cx / s - c       # b_x
        y[r, c, 2] = cy / s - r       # b_y
```



y[1, 1], the dog's cell:

object?	b_x	b_y	b_w	b_h	cat	dog
1	0.84	0.05	0	0	0	0

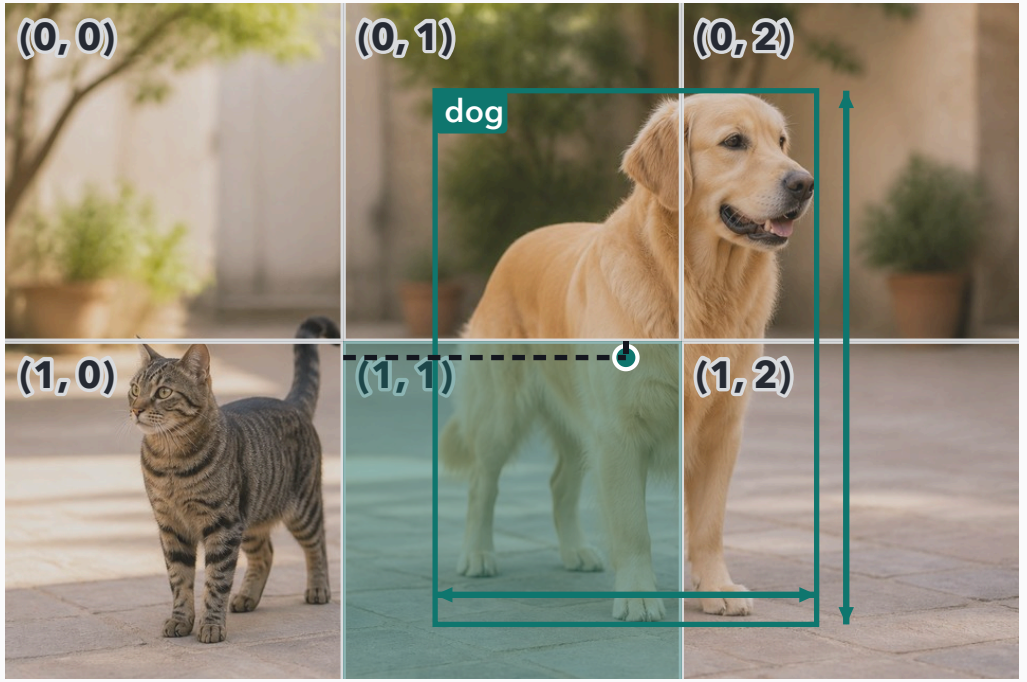
The centre inside the cell: b_x = 588 / 320 − 1 ≈ 0.84 and b_y = 336 / 320 − 1 ≈ 0.05.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
    for (x1, y1, x2, y2), k in zip(boxes, classes):
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
        r, c = int(cy // s), int(cx // s)
        y[r, c, 0] = 1                # object?
        y[r, c, 1] = cx / s - c       # b_x
        y[r, c, 2] = cy / s - r       # b_y
        y[r, c, 3] = (x2 - x1) / s    # b_w
        y[r, c, 4] = (y2 - y1) / s    # b_h
```



y[1, 1], the dog's cell:

object?	b_x	b_y	b_w	b_h	cat	dog
1	0.84	0.05	1.13	1.58	0	0

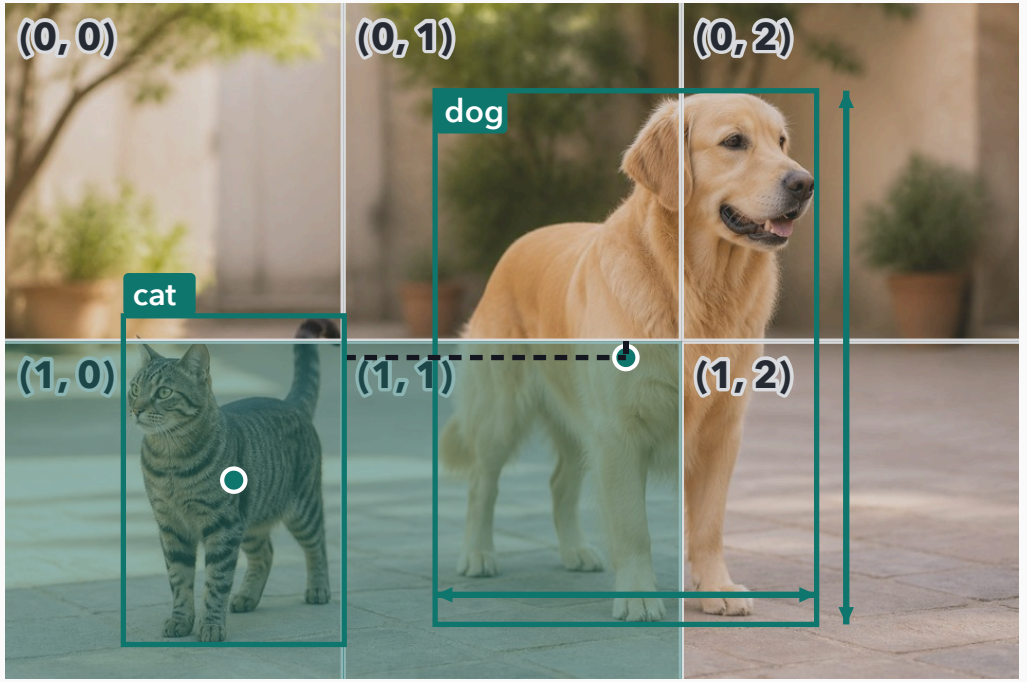
The size in cells: b_w = 361 / 320 ≈ 1.13 and b_h = 505 / 320 ≈ 1.58.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Building the target in code, one line at a time

```python
def build_target(boxes, classes, s=320):
    y = torch.zeros(2, 3, 7)  # object? 0 everywhere
    for (x1, y1, x2, y2), k in zip(boxes, classes):
        cx, cy = (x1 + x2) / 2, (y1 + y2) / 2
        r, c = int(cy // s), int(cx // s)
        y[r, c, 0] = 1                # object?
        y[r, c, 1] = cx / s - c       # b_x
        y[r, c, 2] = cy / s - r       # b_y
        y[r, c, 3] = (x2 - x1) / s    # b_w
        y[r, c, 4] = (y2 - y1) / s    # b_h
        y[r, c, 5 + k] = 1            # cat or dog
    return y
```



y[1, 1], the dog's cell:

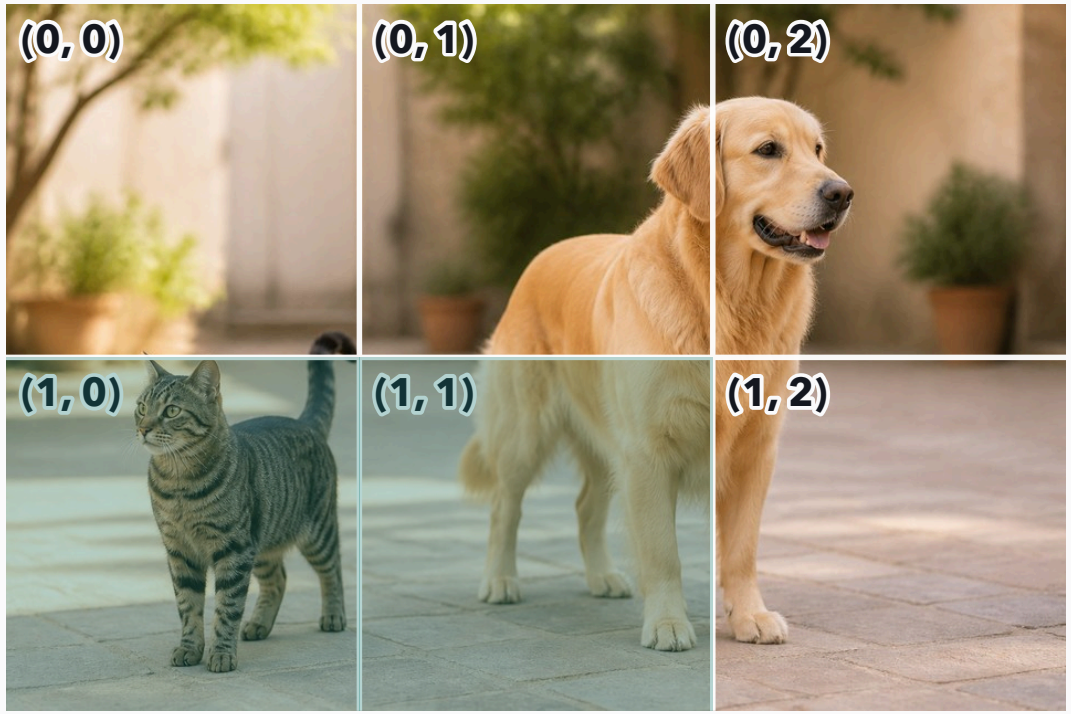
object?	b_x	b_y	b_w	b_h	cat	dog
1	0.84	0.05	1.13	1.58	0	1

Mark the class, dog. The loop does the same for the cat in cell (1, 0), and the other four cells stay all zeros.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Training: Part 4's loss in every cell



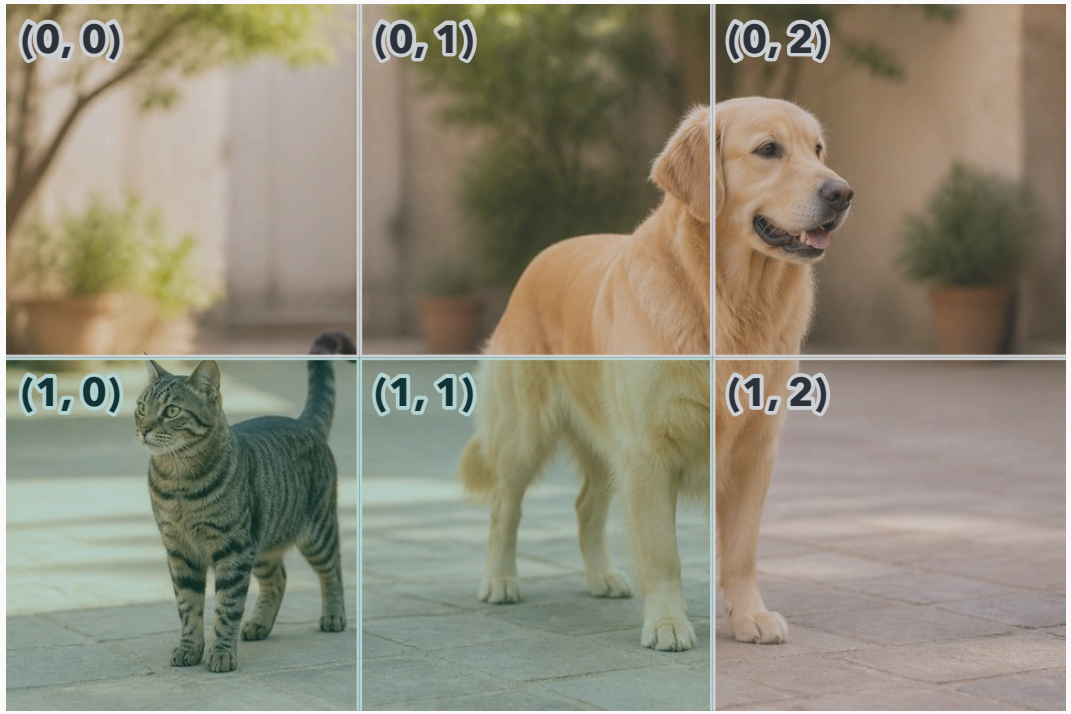
$$\text{cells } (1,0),(1,1): \ \sum_{i=1}^{7}(\hat{y}_i - y_i)^2$$

> Training compares the network's block with the target, one cell at a time. In the two cells with a centre every number counts, as for Part 4's cat photo.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Training: Part 4's loss in every cell



$$\text{cells } (1,0),(1,1): \ \sum_{i=1}^{7}(\hat{y}_i - y_i)^2$$

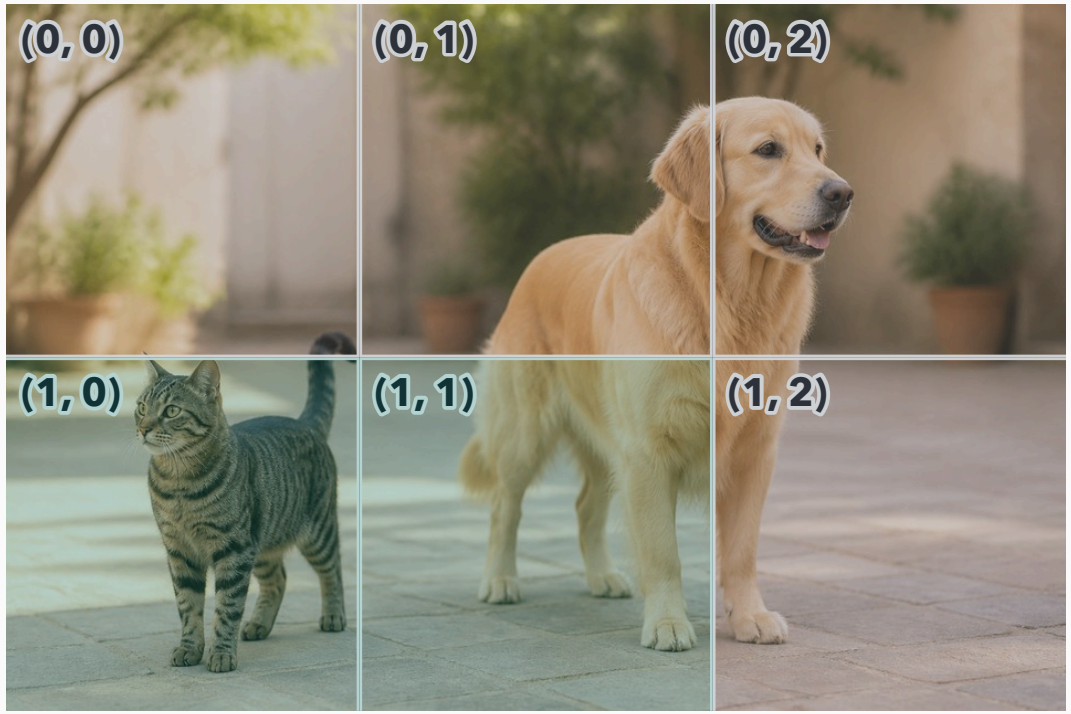
$$\text{other four cells: } (\hat{y}_1 - y_1)^2$$

> In the four empty cells only object? counts, as for Part 4's empty floor.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Training: Part 4's loss in every cell



$$\text{cells } (1,0),(1,1): \ \sum_{i=1}^{7}(\hat{y}_i - y_i)^2$$

$$\text{other four cells: } (\hat{y}_1 - y_1)^2$$

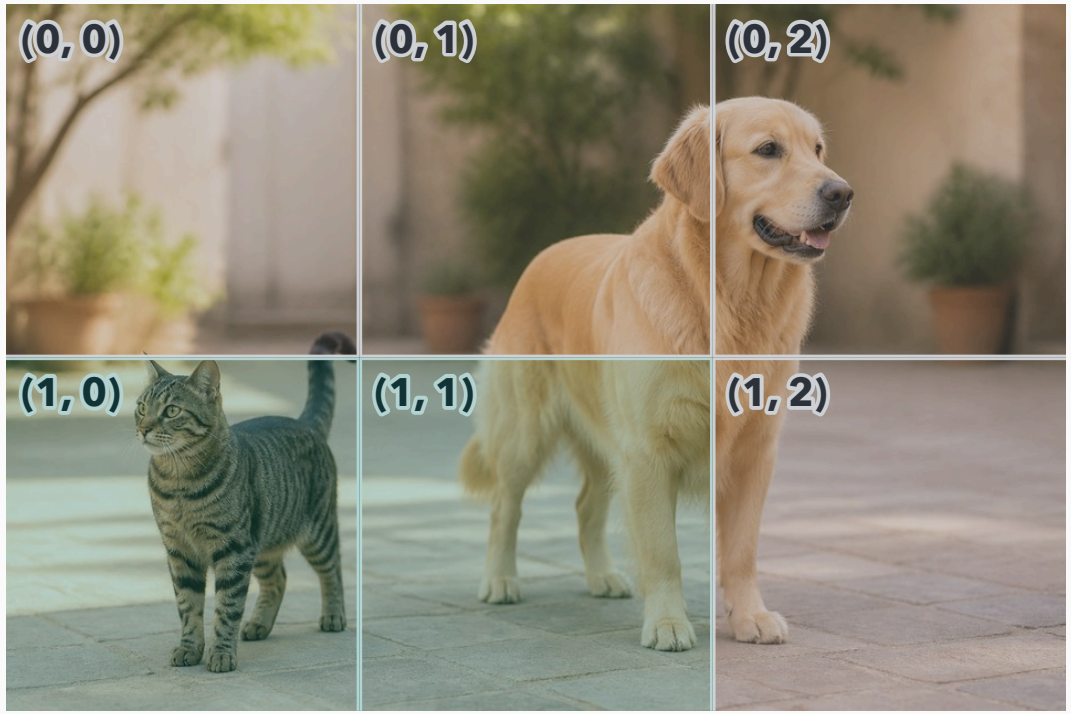
$$\mathcal{L} = \sum_{\text{6 cells}} \mathcal{L}_{\text{cell}}$$

> Add up the six cells' losses and lower the total by gradient descent. One loss trains the whole network.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Training: Part 4's loss in every cell



$$\text{cells } (1,0),(1,1): \ \sum_{i=1}^{7}(\hat{y}_i - y_i)^2$$

$$\text{other four cells: } (\hat{y}_1 - y_1)^2$$

$$\mathcal{L} = \sum_{\text{6 cells}} \mathcal{L}_{\text{cell}}$$

```
m = y[..., 0] == 1
loss = ((p[..., 0] - y[..., 0])**2).sum()
loss += ((p[m, 1:] - y[m, 1:])**2).sum()
```

In code: a mask m marks the cells with a centre. object? is compared in every cell, the other six numbers only where m is true.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Two centres in one cell? Use smaller cells



5

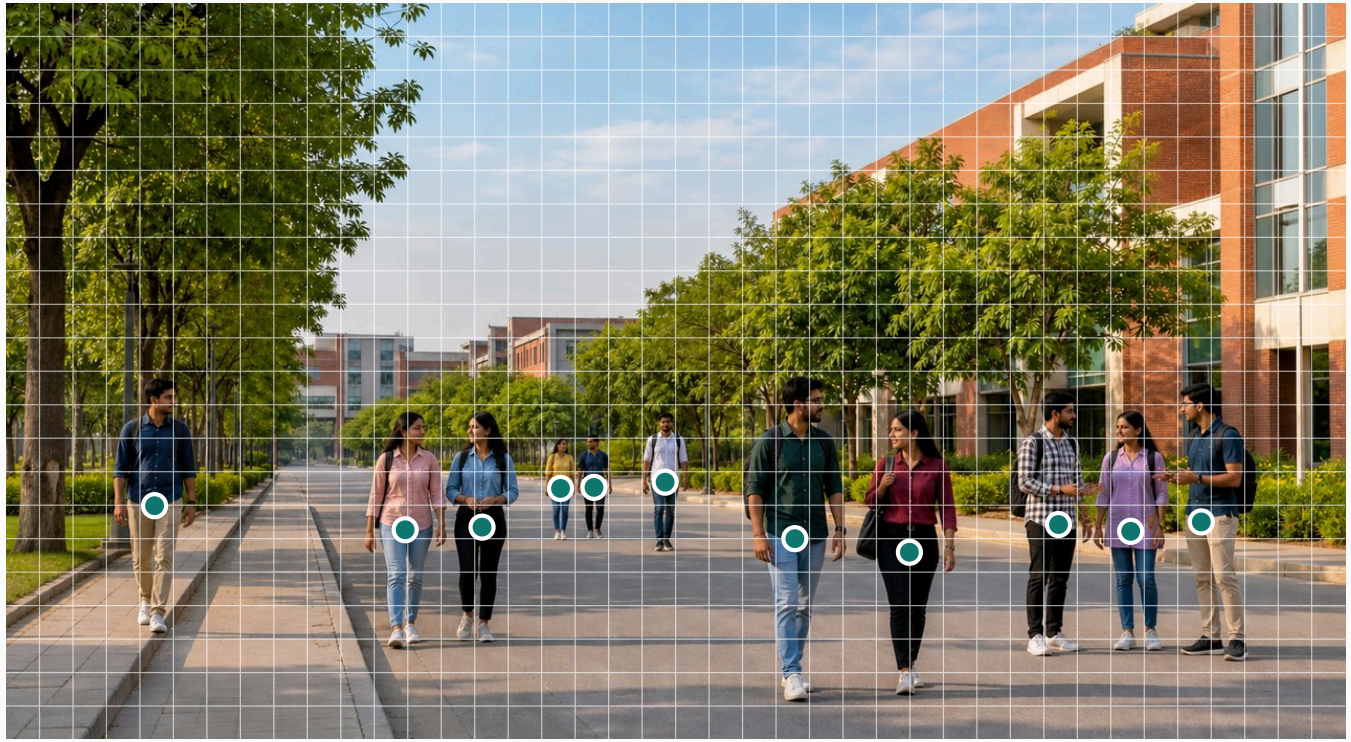
centres in one cell, which writes one record

> A cell writes one record. On the crowd photo a 3 × 2 grid puts 5 people’s centres in one cell, which can report only one of them.

Our annotation of the crowd photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

Two centres in one cell? Use smaller cells



1

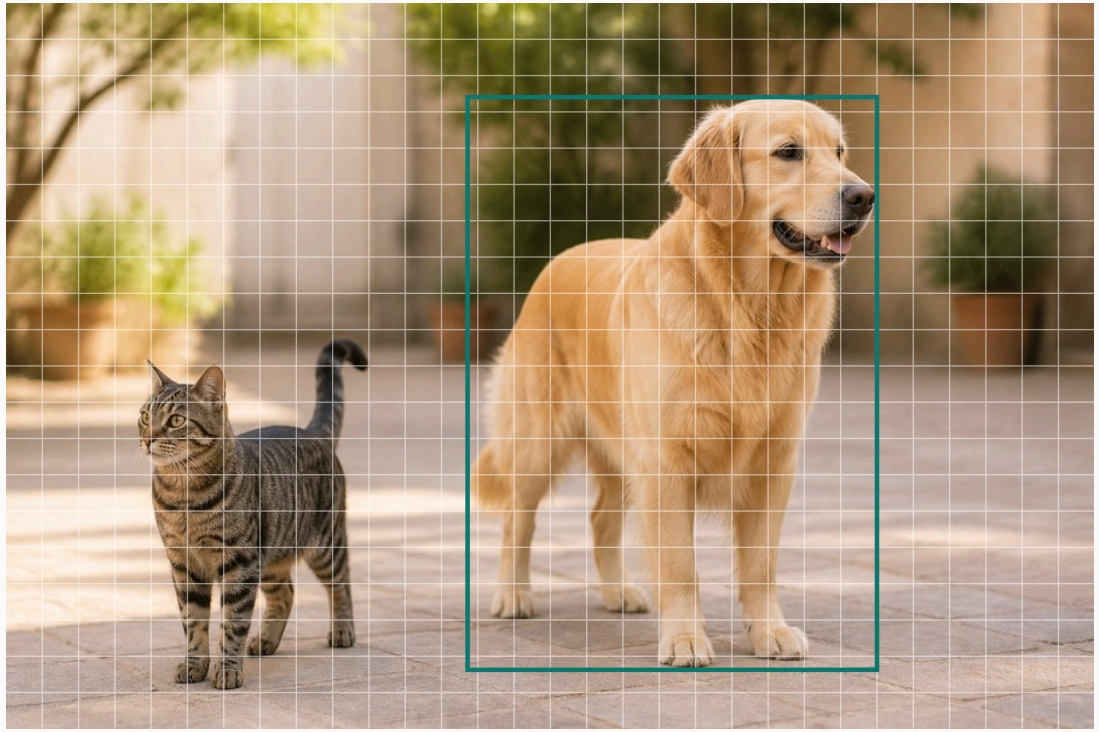
centre per cell, on 40 × 22 cells of 32 px

> Smaller cells fix it: with 32-pixel cells, every person’s centre has a cell of its own. Ng uses 19 × 19 cells for the same reason.

Our annotation of the crowd photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

In practice: small cells of 32 pixels



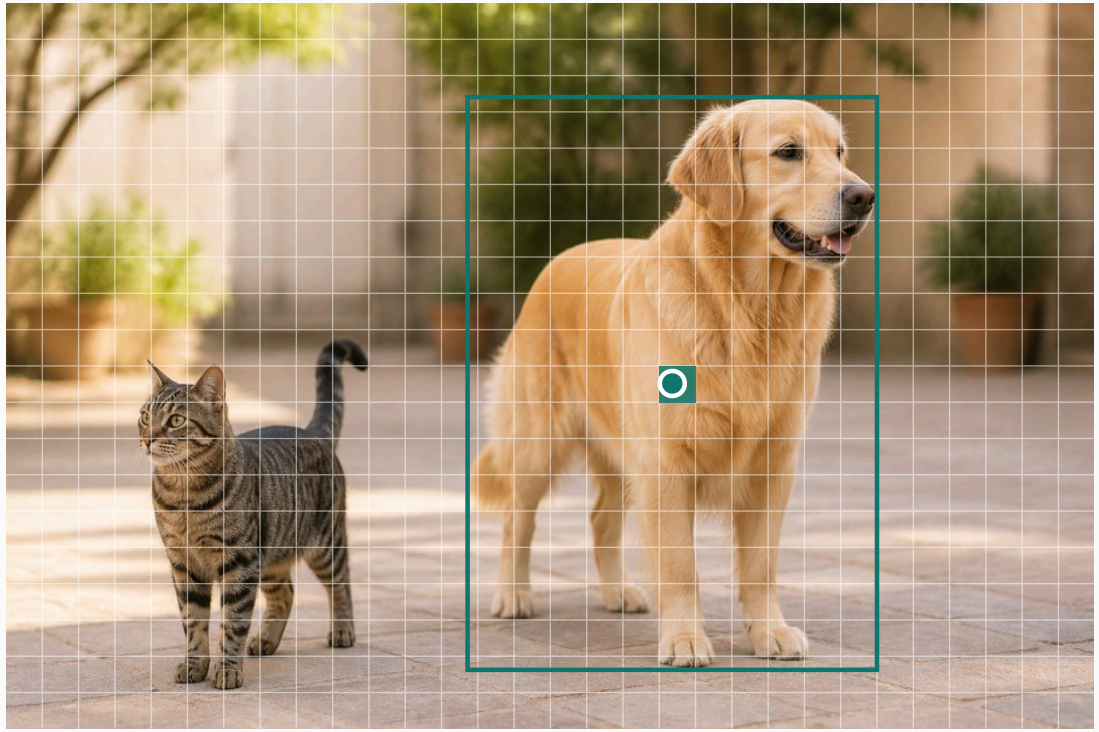
$$960 \div 32 = 30, \quad 640 \div 32 = 20 : \quad 600 \text{ cells}$$

> Real YOLOs use many small cells. YOLOv8n's cells are 32 pixels: 30 × 20 = 600 of them on our photo.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

In practice: small cells of 32 pixels



$$960 \div 32 = 30, \quad 640 \div 32 = 20 : \quad 600 \text{ cells}$$

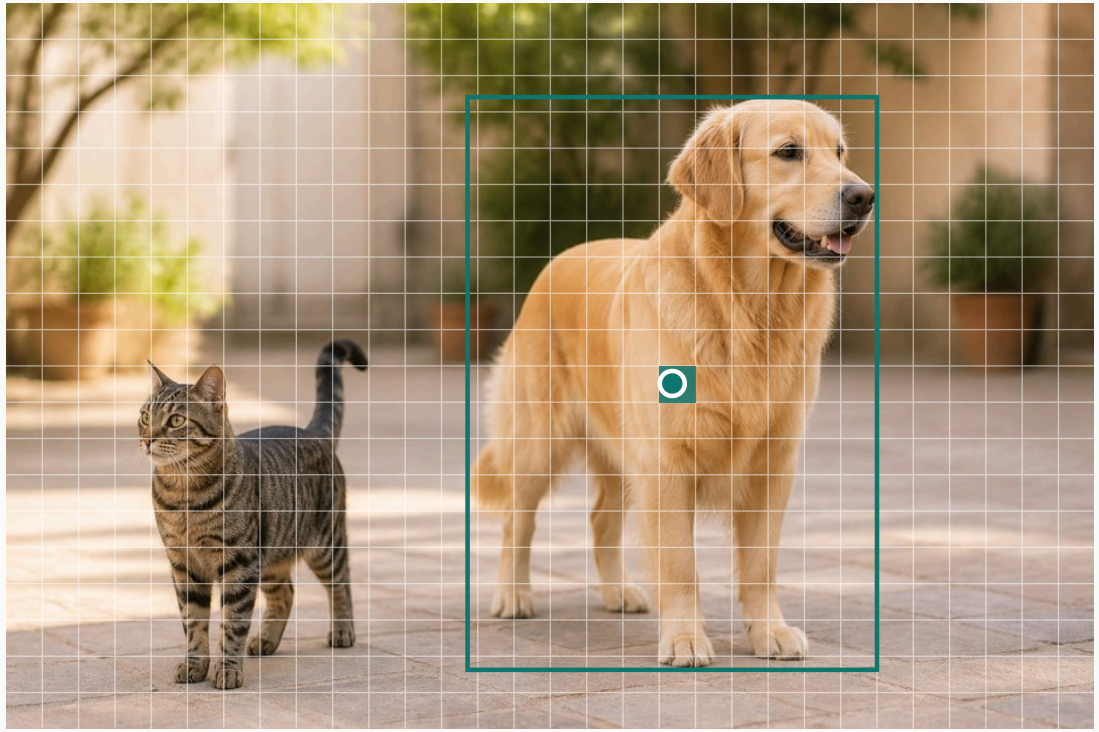
$$588 \div 32 \approx 18.4, \quad 336 \div 32 \approx 10.5 \rightarrow \text{cell } (10, 18)$$

> Same rule, smaller cells: the dog's centre (588, 336) falls in cell (10, 18), row 10 and column 18.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

In practice: small cells of 32 pixels



$$960 \div 32 = 30, \quad 640 \div 32 = 20 : \quad 600 \text{ cells}$$

$$588 \div 32 \approx 18.4, \quad 336 \div 32 \approx 10.5 \;\rightarrow\; \text{cell } (10, 18)$$

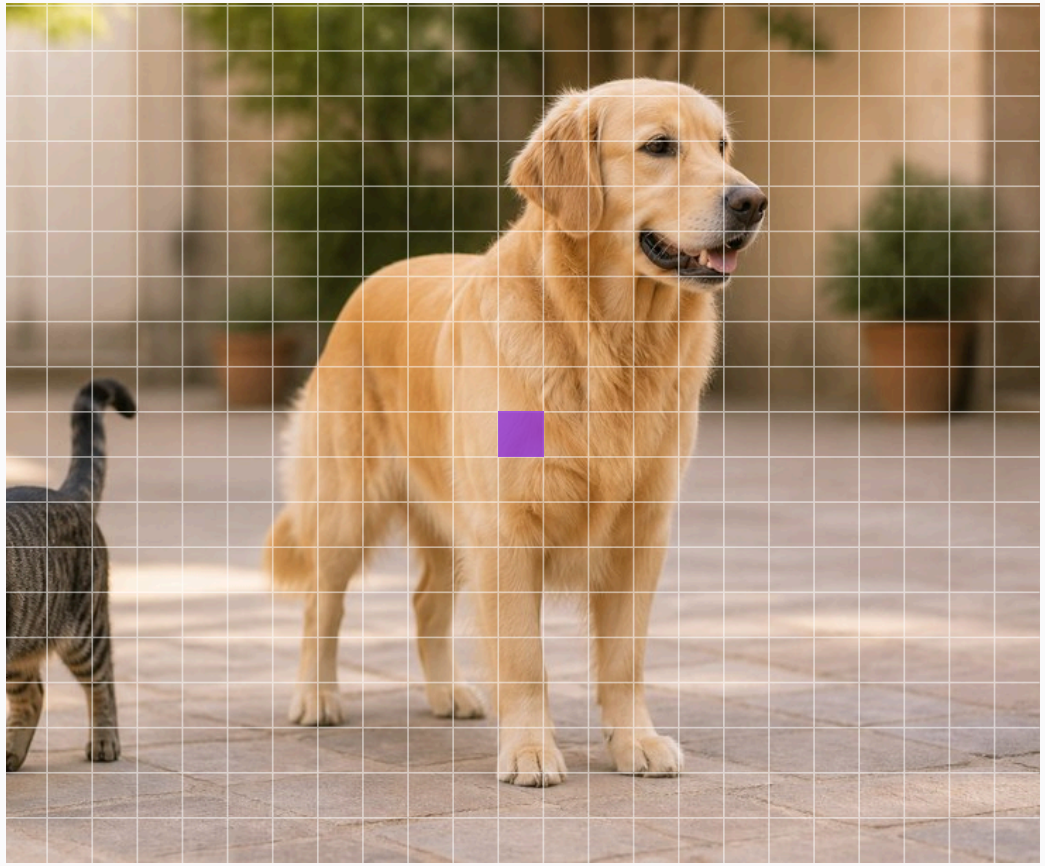
$$(b_x, b_y, b_w, b_h) \approx (0.36,\ 0.48,\ 11.3,\ 15.8)$$

> Its target in that cell's units: the centre at (0.36, 0.48), the size 11.3 × 15.8 cells. A big object spans many small cells.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 5

At test time: read the box back out of the cell



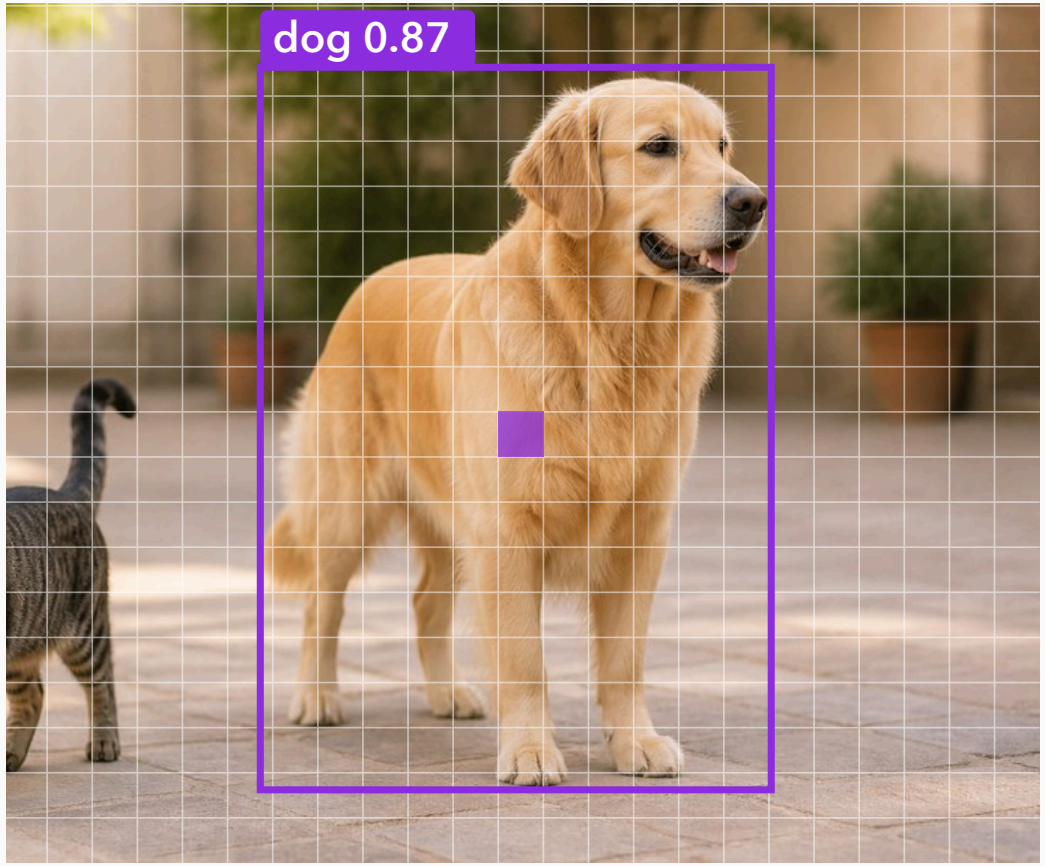
$$(b_x, b_y, b_w, b_h) \approx (0.39,\ 0.37,\ 11.3,\ 16.0), \quad \text{dog } 0.87$$

> At test time there is no annotation, only what the dog's cell writes: four box numbers, and a score for each of 80 classes. Dog scores 0.87.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

At test time: read the box back out of the cell



$$(b_x, b_y, b_w, b_h) \approx (0.39,\ 0.37,\ 11.3,\ 16.0), \quad \text{dog } 0.87$$

$$c_x \approx 576 + 0.39 \times 32 \approx 588$$

$$c_y \approx 320 + 0.37 \times 32 \approx 332$$

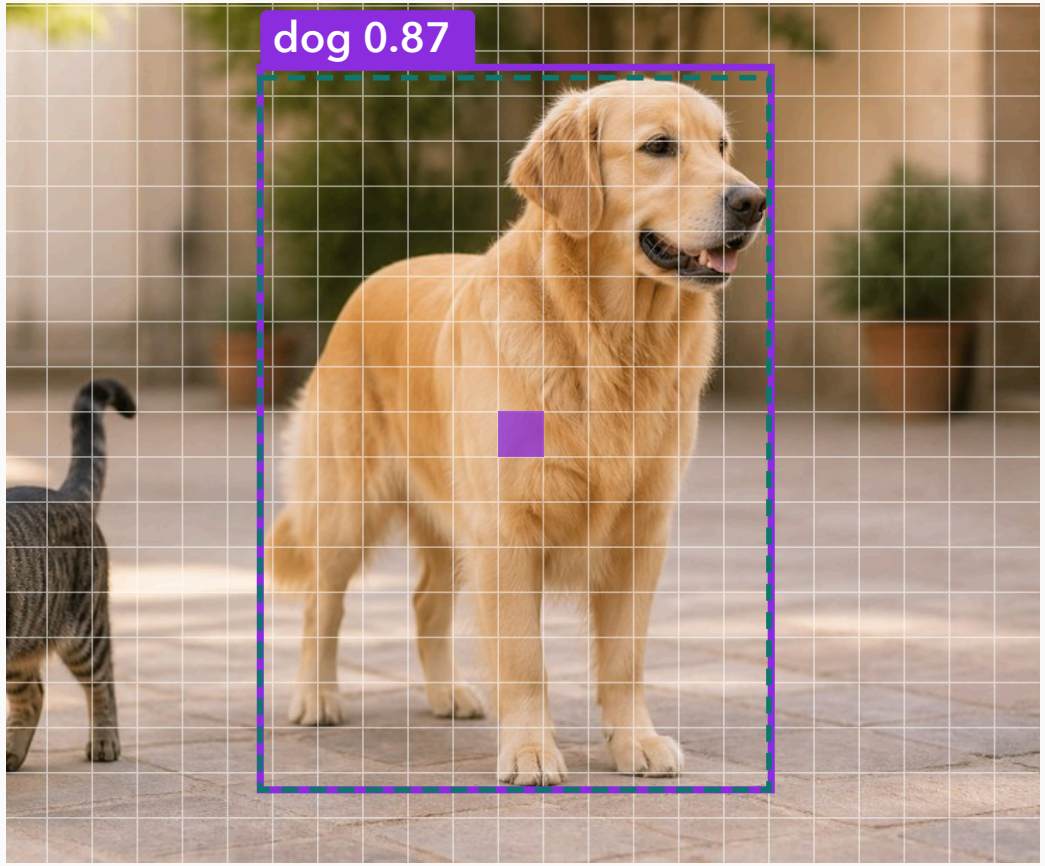
$$\text{size} \approx (11.3 \times 32) \times (16.0 \times 32) \approx 362 \times 512$$

> Undo the cell units: the centre is the cell's corner plus b_x and b_y cells, and the size is b_w by b_h cells, all times 32 pixels.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

At test time: read the box back out of the cell



$$(b_x, b_y, b_w, b_h) \approx (0.39,\ 0.37,\ 11.3,\ 16.0), \quad \text{dog } 0.87$$

$$c_x \approx 576 + 0.39 \times 32 \approx 588$$

$$c_y \approx 320 + 0.37 \times 32 \approx 332$$

$$\text{size} \approx (11.3 \times 32) \times (16.0 \times 32) \approx 362 \times 512$$

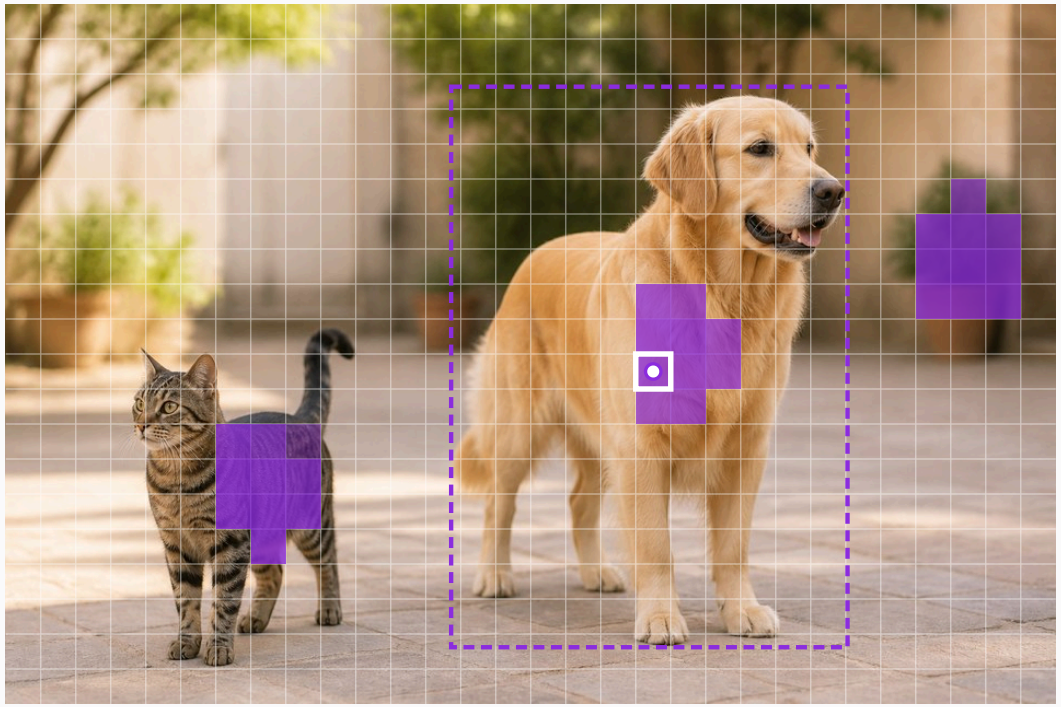
$$\text{IoU}(\text{box}, \text{annotation}) \approx 0.98$$

> The box sits on the dog, IoU 0.98 with our annotation. The target was (0.36, 0.48, 11.3, 15.8); training got close.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a record: point at one



Cell: row 10, column 18

centre (592, 336) px

dog	0.871
cat	0.0001
skateboard	0.0001

its box: (407, 76, 770, 588)

> Point at cells: near the dog they say dog and draw nearly the same box; on the wall every class score is tiny.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a box: keep the confident ones



600

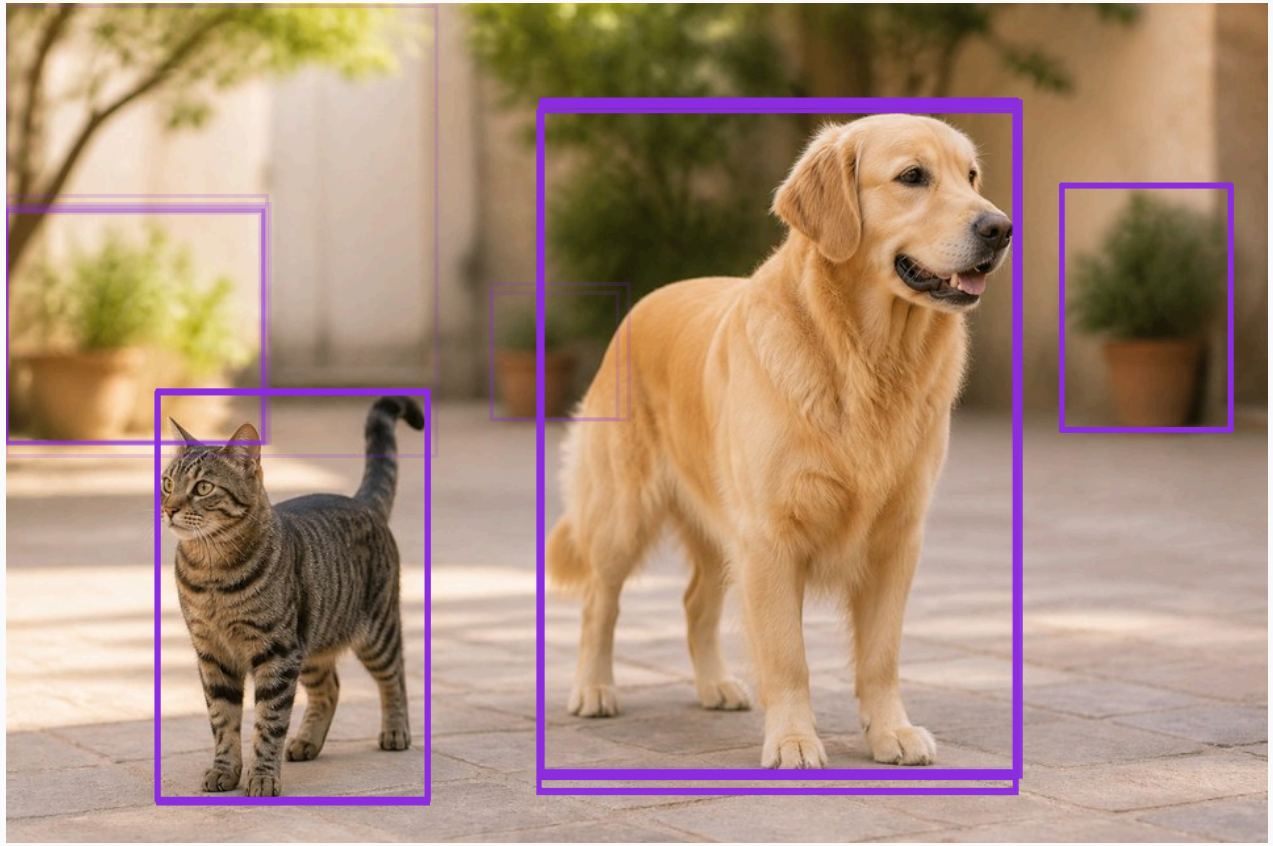
cells, so 600 boxes

> Read out every cell the same way: 600 boxes. The network never writes “?”, so cells on the wall and floor draw boxes too.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a box: keep the confident ones



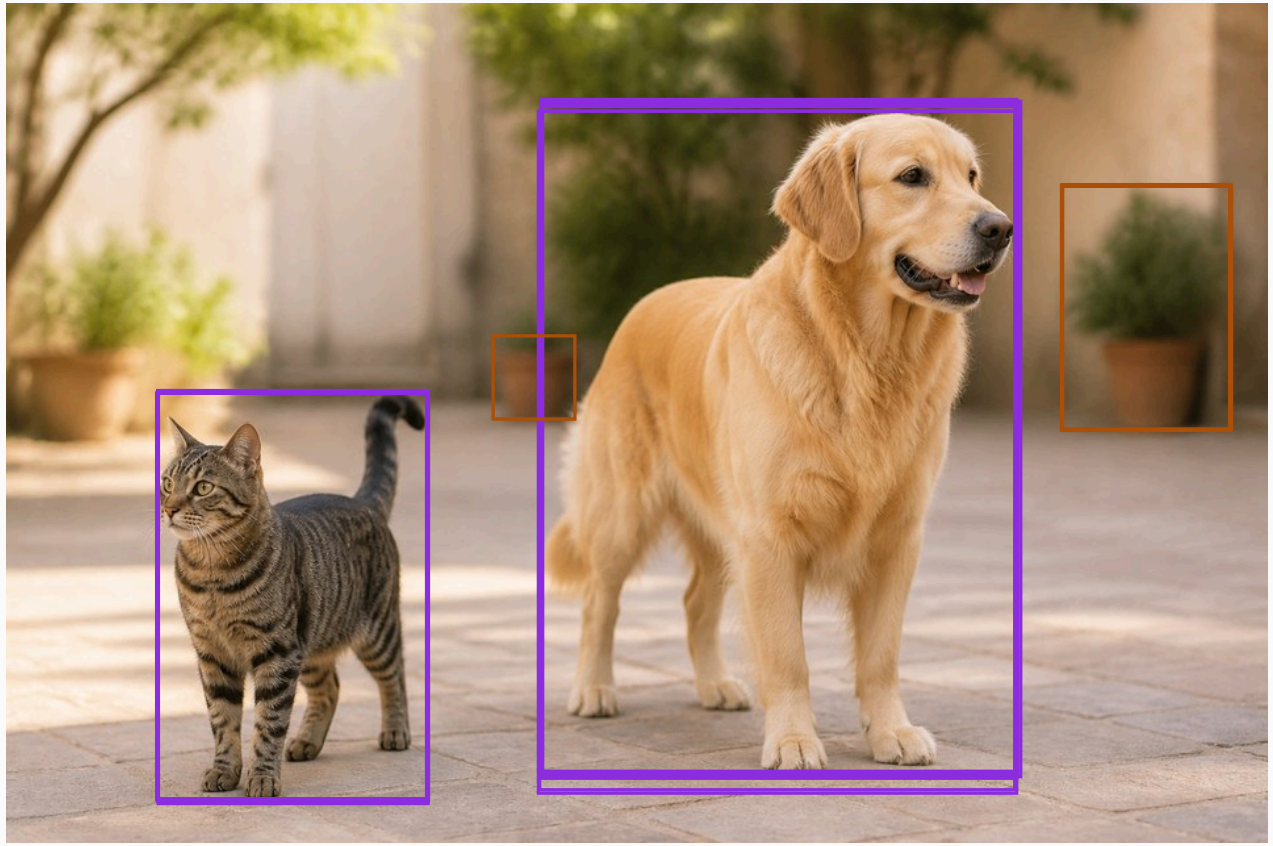
each faded by its best class score

> Fade each box by its cell's best class score. Almost all of them vanish: the score says which boxes to believe.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a box: keep the confident ones



31

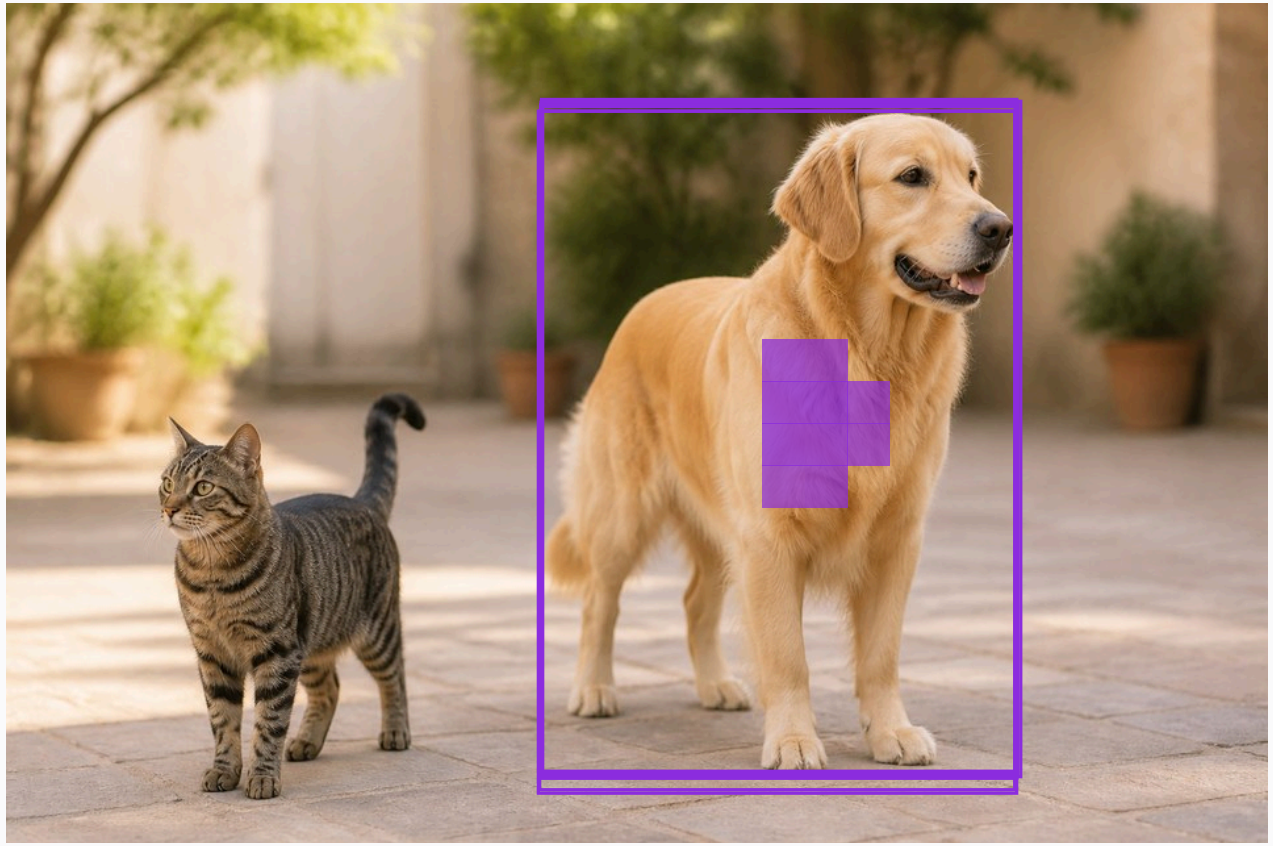
boxes with a score ≥ 0.25: 10 dog, 10 cat, 11 potted plant

> Keep the boxes scoring at least 0.25: 31 of them, 10 on the dog, 10 on the cat and 11 on two potted plants.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

Every cell writes a box: keep the confident ones



10

neighbouring cells, ten dog boxes

> The dog’s ten boxes come from ten neighbouring cells, each of which saw nearly the same dog. To get one box per object, remove the duplicates.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 5

YOLO, the whole algorithm

Training, for each photo

1. Grid the photo.
2. Each object's centre picks a cell: object? 1, its box in cell units, its class.
3. Every other cell: object? 0, the rest "?".
4. Part 4's loss in every cell; gradient descent.

> Training, for every photo in the set: grid it, write each object's record in its centre cell and "no object" elsewhere, then lower Part 4's loss.

▶ Notebook, part 5

YOLO, the whole algorithm

Training, for each photo

1. Grid the photo.
2. Each object's centre picks a cell: object? 1, its box in cell units, its class.
3. Every other cell: object? 0, the rest "?".
4. Part 4's loss in every cell; gradient descent.

Test time, for a new photo

1. One CNN pass: a record per cell.
2. Turn each record's box back into pixels.
3. Keep the boxes that score at least 0.25.
4. Remove duplicates with NMS (Part 6).

> Test time: one pass, every cell writes a record, decode the boxes, keep the confident ones (31 here), and remove the duplicates.

Part 5 in one slide

1. Sliding windows are slow, repeat work, and their boxes are only the windows.
2. YOLO: one CNN pass writes a record per grid cell; the cell that holds an object's centre describes it, in cell units.
3. Training: build the target cell by cell, then Part 4's loss in every cell. Real cells are 32 px.
4. Test time: every cell writes a box; 31 score at least 0.25, for 4 objects.

> **Next:** each object has about ten boxes; keep one.

▶ Notebook, part 6

Part 6 · One box per object: non-maximum suppression

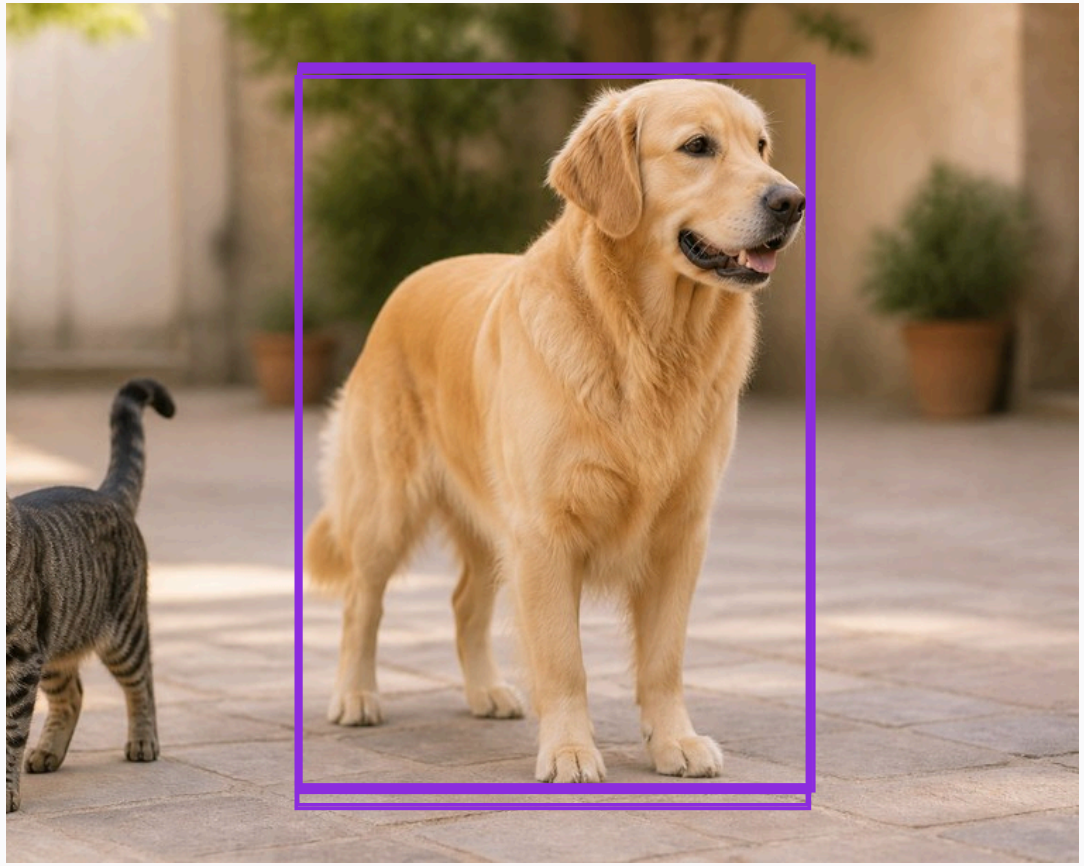
Ten boxes for one dog. Keep the best and delete the ones that overlap it heavily.

1. Run the greedy recipe on the dog's ten real boxes, then on two dogs.
2. See what the IoU threshold trades off, and why classes are kept apart.
3. Try it live; write it in eight lines; then the whole detector in code.

> NMS keeps the highest-scoring box and removes boxes that overlap it.

▶ Notebook, part 6

Ten boxes for one dog



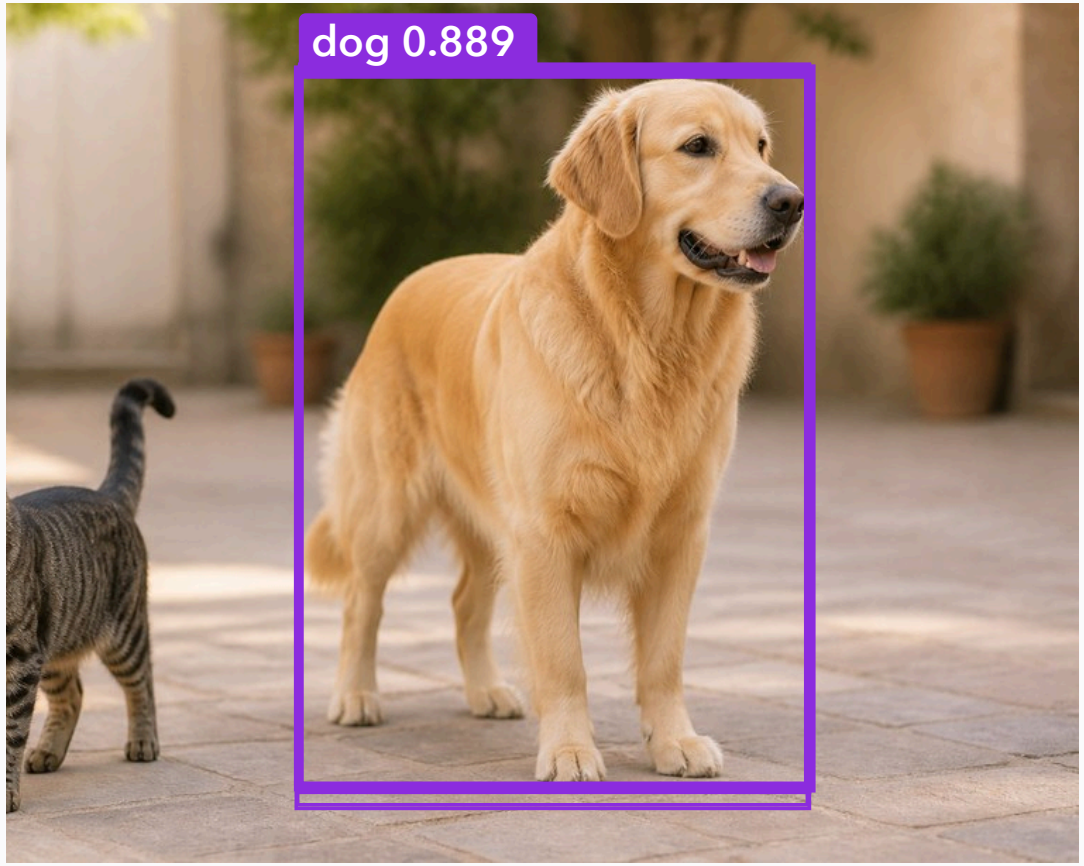
cell (row, col)	score
(9, 19)	**0.889**
(9, 18)	0.888
(10, 18)	0.871
(10, 19)	0.862
(8, 18)	0.822
(10, 20)	0.797
(8, 19)	0.793
(11, 19)	0.792
(9, 20)	0.792
(11, 18)	0.766

> The ten cells that fired for the dog, and their scores, from 0.889 down to 0.766. Their boxes lie almost on top of each other.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Ten boxes for one dog



cell (row, col)	score
(9, 19)	**0.889**
(9, 18)	0.888
(10, 18)	0.871
(10, 19)	0.862
(8, 18)	0.822
(10, 20)	0.797
(8, 19)	0.793
(11, 19)	0.792
(9, 20)	0.792
(11, 18)	0.766

> They describe one dog. Keep the most confident box, from cell (9, 19), and delete the rest.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.

> Sort by score.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.

> The best box is kept: no other box can beat it.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.
3. Delete every remaining box of the same class whose IoU with it is above a threshold (0.7 here).

> Boxes that overlap it heavily describe the same object: delete them. Boxes that overlap little may be other objects: keep them for now.

▶ Notebook, part 6

Non-maximum suppression: the greedy recipe

1. Sort the boxes by score, highest first.
2. Keep the top box.
3. Delete every remaining box of the same class whose IoU with it is above a threshold (0.7 here).
4. Repeat from step 2 with what is left, until nothing is left.

> Then repeat on what is left. Non-maximum suppression removes every box that is not a local maximum.

▶ Notebook, part 6

NMS on the dog's ten boxes

cell	score	IoU with the kept box	fate
(9, 19)	0.889		
(9, 18)	0.888		
(10, 18)	0.871		
(10, 19)	0.862		
(8, 18)	0.822		
(10, 20)	0.797		
(8, 19)	0.793		
(11, 19)	0.792		
(9, 20)	0.792		
(11, 18)	0.766		

> Step 1: the ten boxes sorted by score.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS on the dog's ten boxes

cell	score	IoU with the kept box	fate
(9, 19)	0.889	kept box	**keep**
(9, 18)	0.888		
(10, 18)	0.871		
(10, 19)	0.862		
(8, 18)	0.822		
(10, 20)	0.797		
(8, 19)	0.793		
(11, 19)	0.792		
(9, 20)	0.792		
(11, 18)	0.766		

> Step 2: keep the top one, cell (9, 19).

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS on the dog's ten boxes

cell	score	IoU with the kept box	fate
(9, 19)	0.889	kept box	**keep**
(9, 18)	0.888	0.993	
(10, 18)	0.871	0.988	
(10, 19)	0.862	0.985	
(8, 18)	0.822	0.961	
(10, 20)	0.797	0.981	
(8, 19)	0.793	0.956	
(11, 19)	0.792	0.980	
(9, 20)	0.792	0.996	
(11, 18)	0.766	0.977	

> Step 3: each other box's IoU with it. All are above 0.9: these boxes are the same box, drawn by neighbouring cells.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS on the dog's ten boxes

cell	score	IoU with the kept box	fate
(9, 19)	0.889	kept box	**keep**
(9, 18)	0.888	0.993	**delete (> 0.7)**
(10, 18)	0.871	0.988	**delete (> 0.7)**
(10, 19)	0.862	0.985	**delete (> 0.7)**
(8, 18)	0.822	0.961	**delete (> 0.7)**
(10, 20)	0.797	0.981	**delete (> 0.7)**
(8, 19)	0.793	0.956	**delete (> 0.7)**
(11, 19)	0.792	0.980	**delete (> 0.7)**
(9, 20)	0.792	0.996	**delete (> 0.7)**
(11, 18)	0.766	0.977	**delete (> 0.7)**

> Every IoU is above the 0.7 threshold, so all nine go and one dog box remains.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Two dogs: why the loop keeps the second one



20 dog boxes with score ≥ 0.25.

> Two dogs in one photo: about ten boxes on each.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Two dogs: why the loop keeps the second one



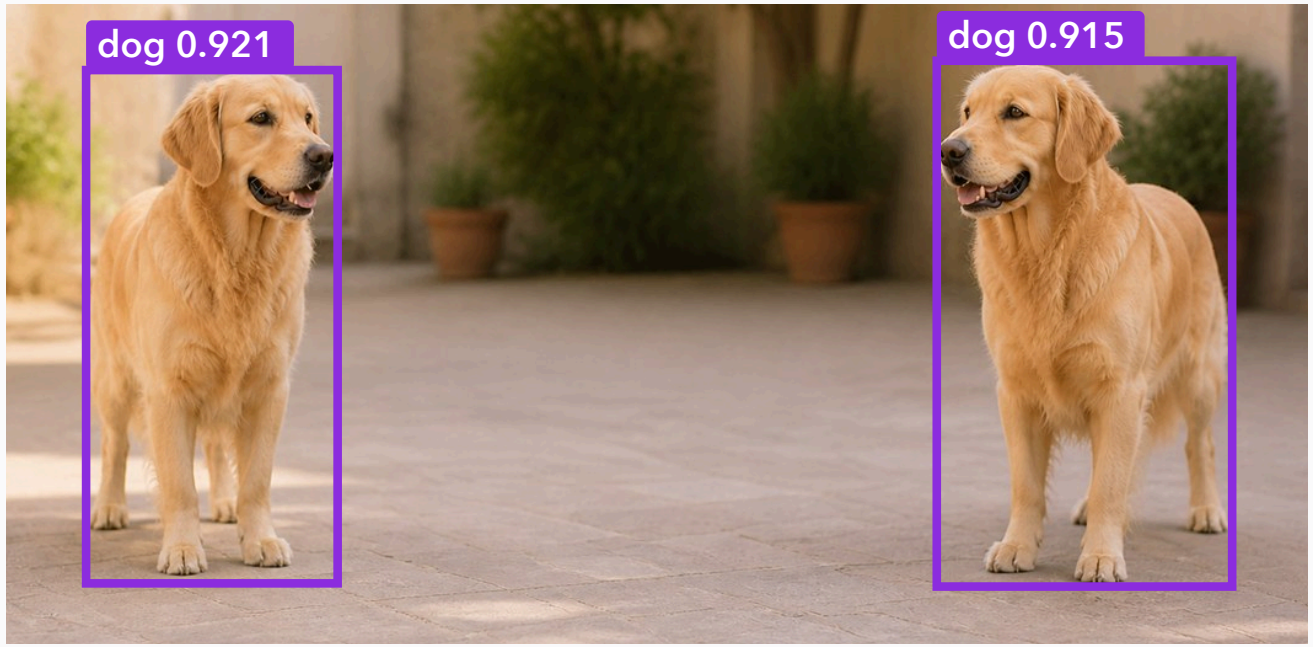
Round 1 keeps 0.921 on the left dog and deletes its 9 twins (IoU ≥ 0.97). The right dog's boxes have IoU **0.0** with it: untouched.

> Round 1 works on the left dog only: the right dog's boxes do not overlap the kept box, so they stay.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Two dogs: why the loop keeps the second one



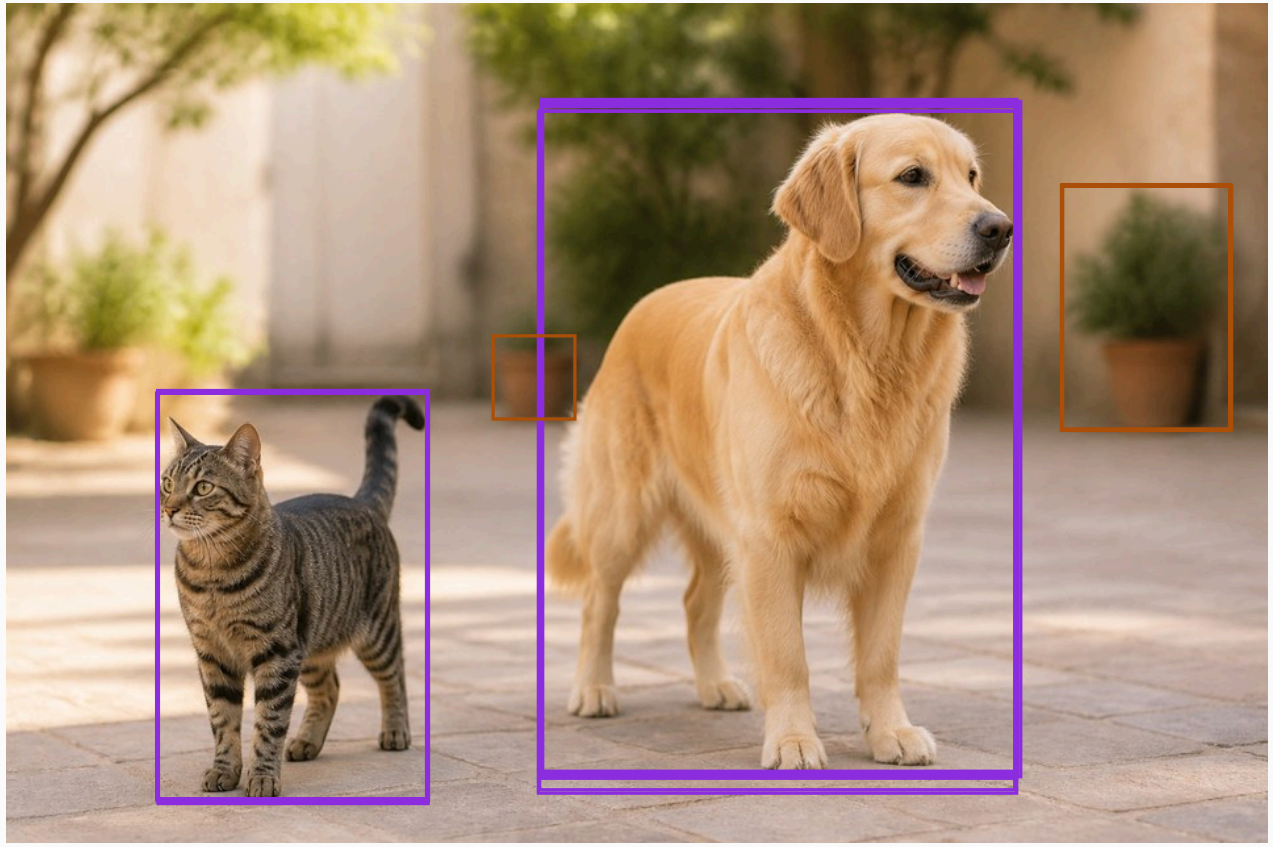
Round 2 keeps 0.915 on the right dog and deletes its 9 twins, leaving one box per dog.

> Round 2 keeps the right dog. Keeping only the single best box overall would have lost a dog; the loop keeps one per object.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

All 31 boxes → 4 detections



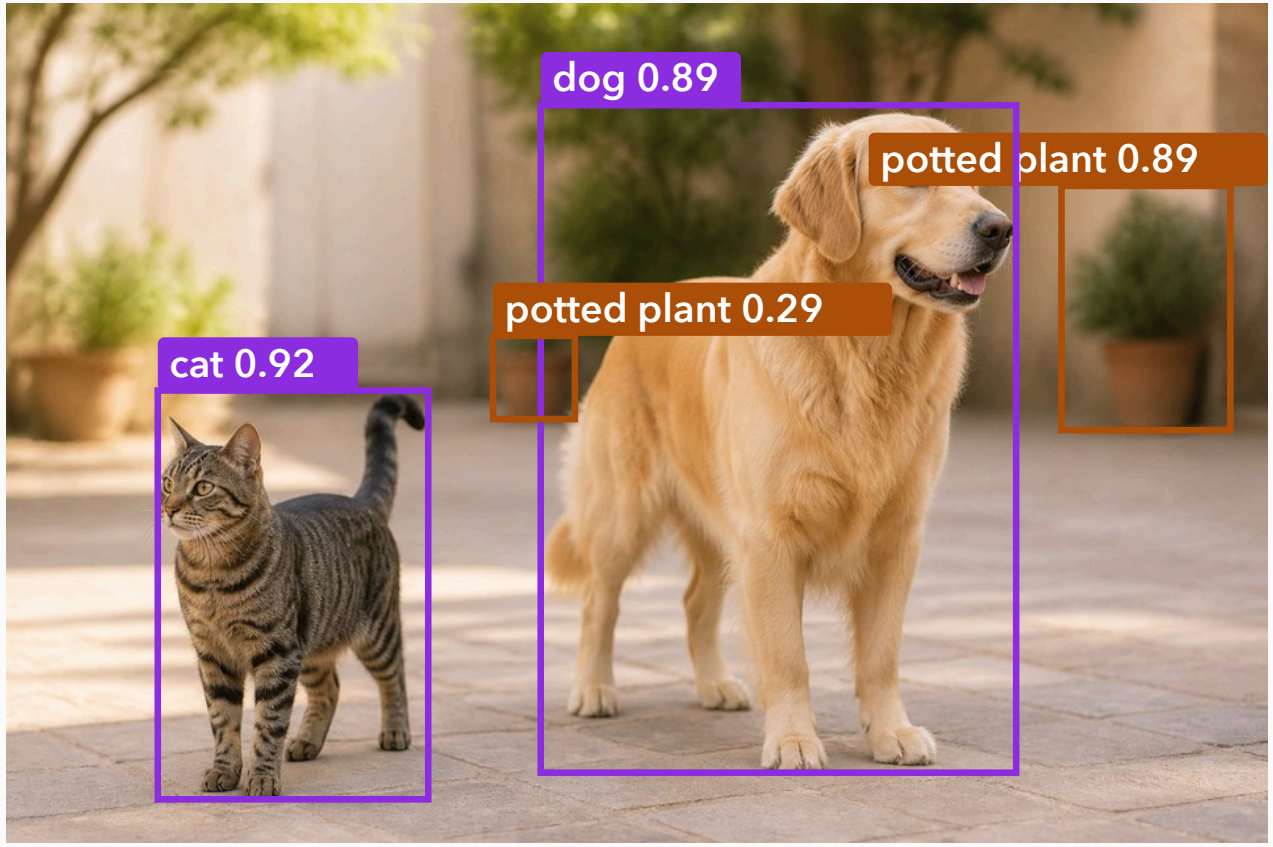
Before: 31 boxes.

> The same loop runs on every class: 10 cat boxes become one, 10 plant boxes become one.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

All 31 boxes → 4 detections



Before: 31 boxes.

After: 4. cat 0.921, potted plant 0.890, dog 0.889, potted plant 0.290.

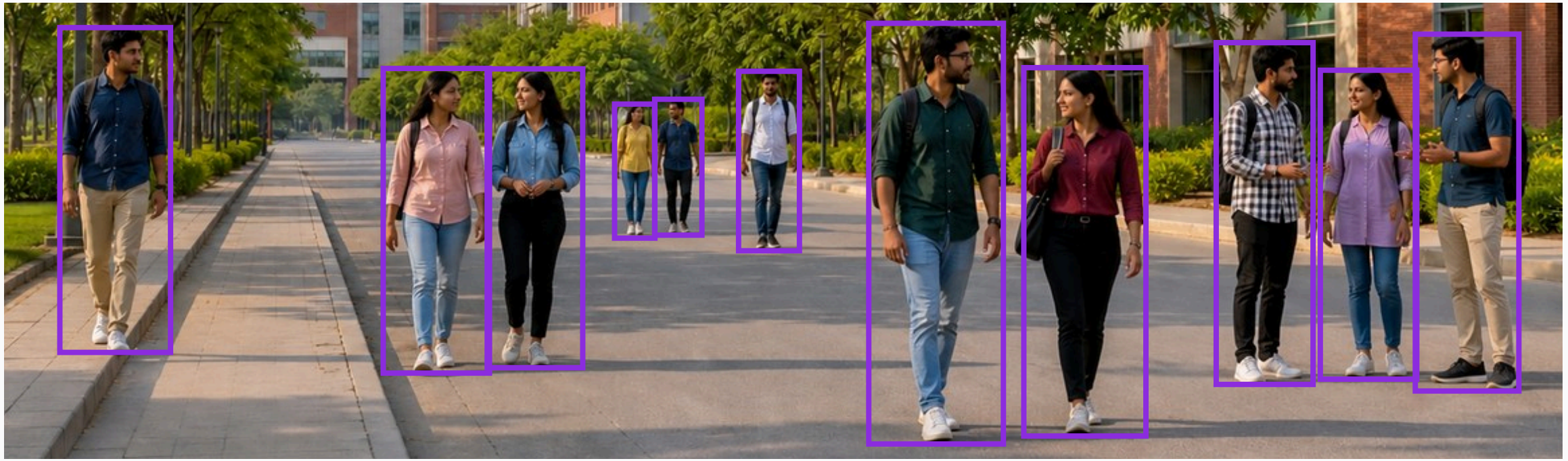
Our eight-line loop keeps exactly what Ultralytics' own post-processing keeps (checked in scripts/science.py).

> Four detections: cat, the big potted plant, dog, and a small plant on a finer grid. A fixed 12,600 records became a set of four.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The IoU threshold is a trade-off



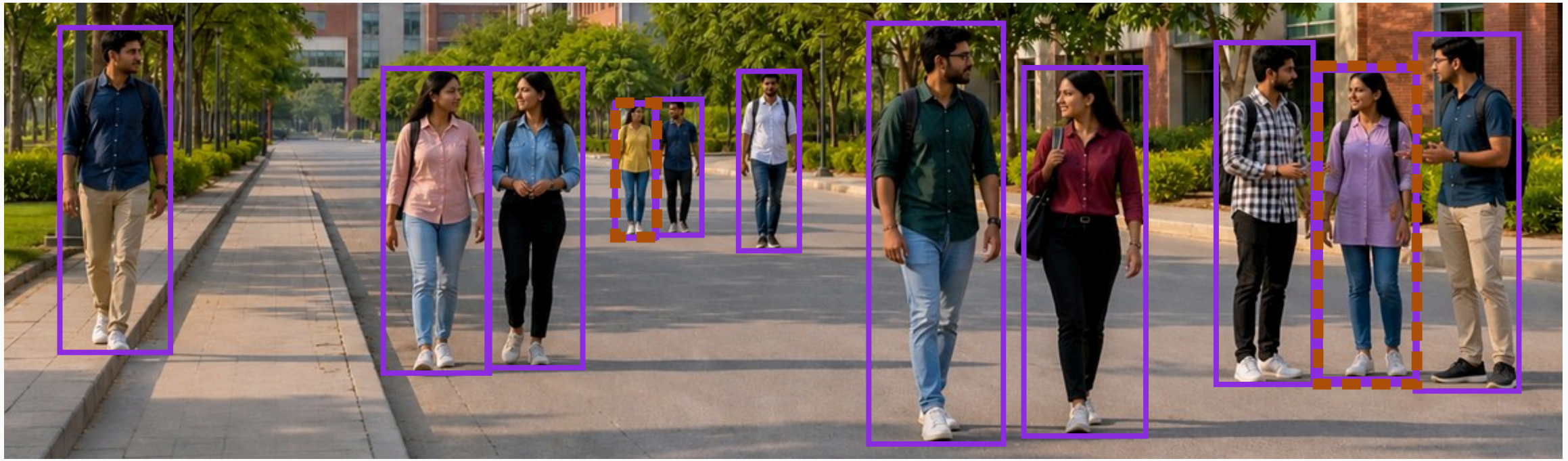
Threshold 0.7: 11 boxes, one per person.

> The crowd photo: 112 person candidates. NMS at 0.7 keeps eleven boxes, one per person.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The IoU threshold is a trade-off



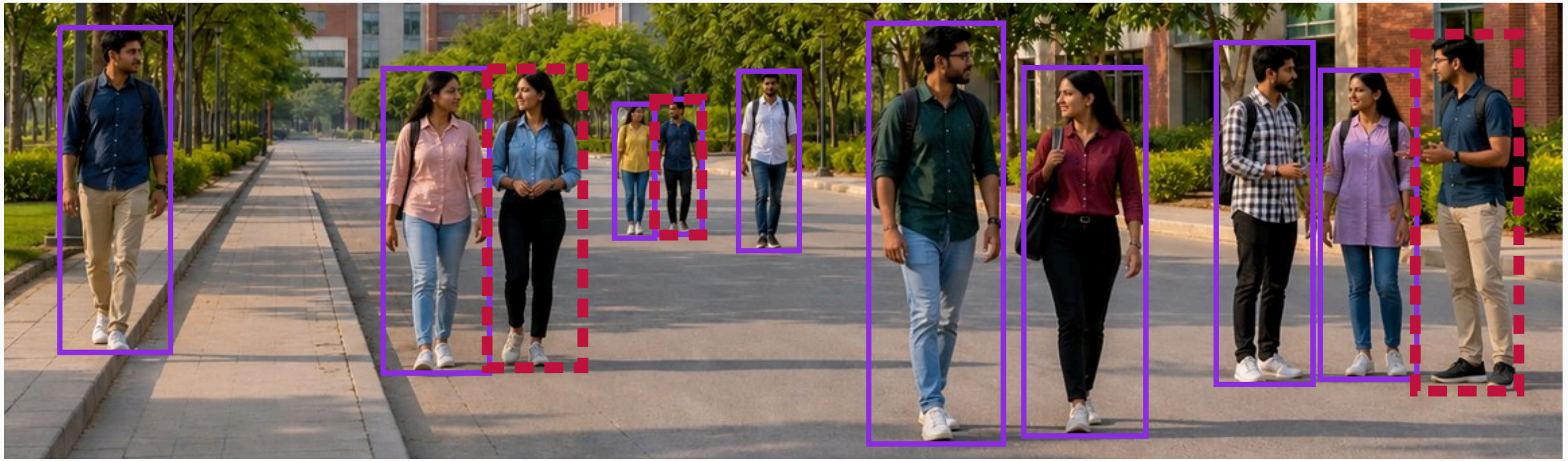
Too high, 0.95: 13 boxes. Two duplicates (amber) survive.

> Raise the threshold to 0.95 and NMS deletes only near-identical boxes: two duplicates slip through.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The IoU threshold is a trade-off



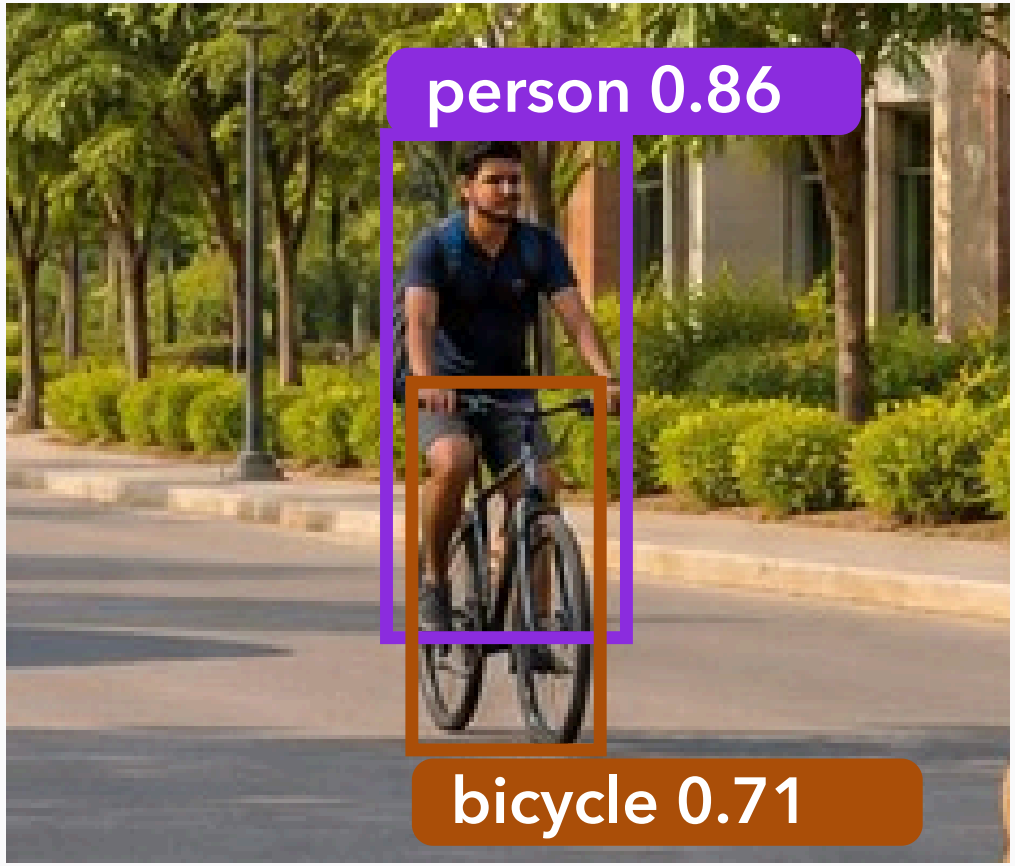
Too low, 0.0: 9 boxes. Two real people (red) are deleted.

> Lower it to 0 and any touching box is deleted, real neighbours included. No single threshold suits every scene.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Boxes of different classes do not suppress each other



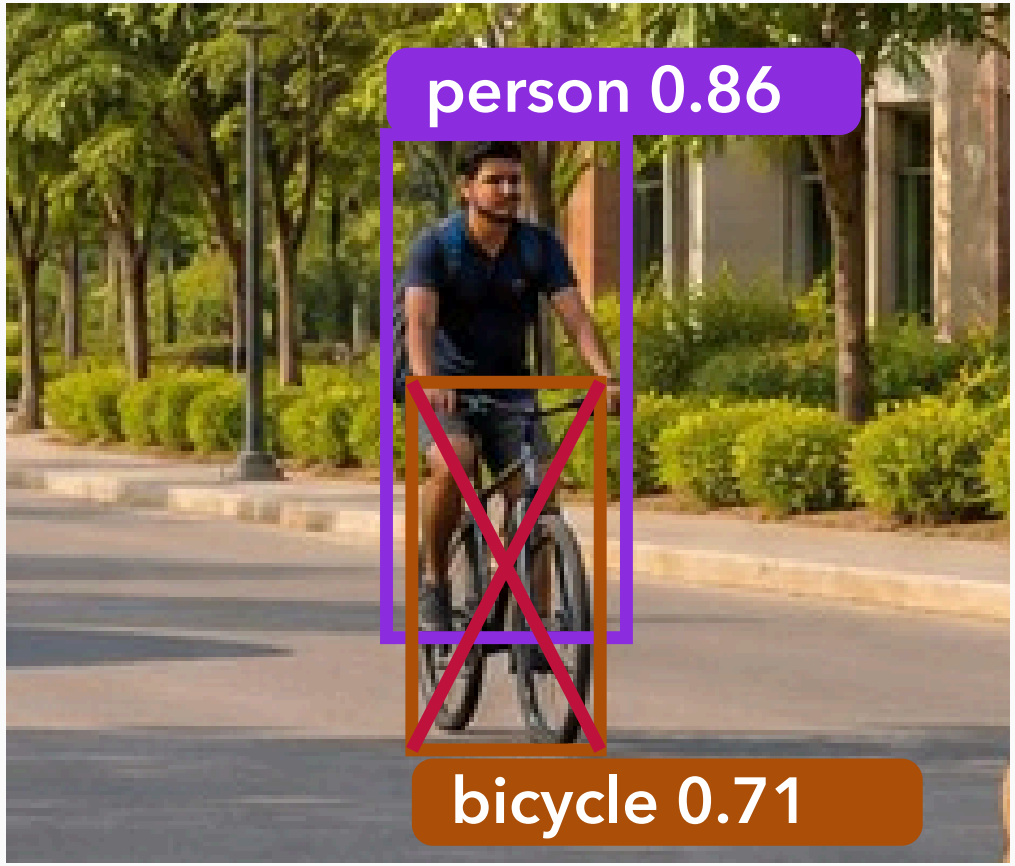
$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

A cyclist: a person box and a bicycle box. They overlap, and both are right.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Boxes of different classes do not suppress each other



$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

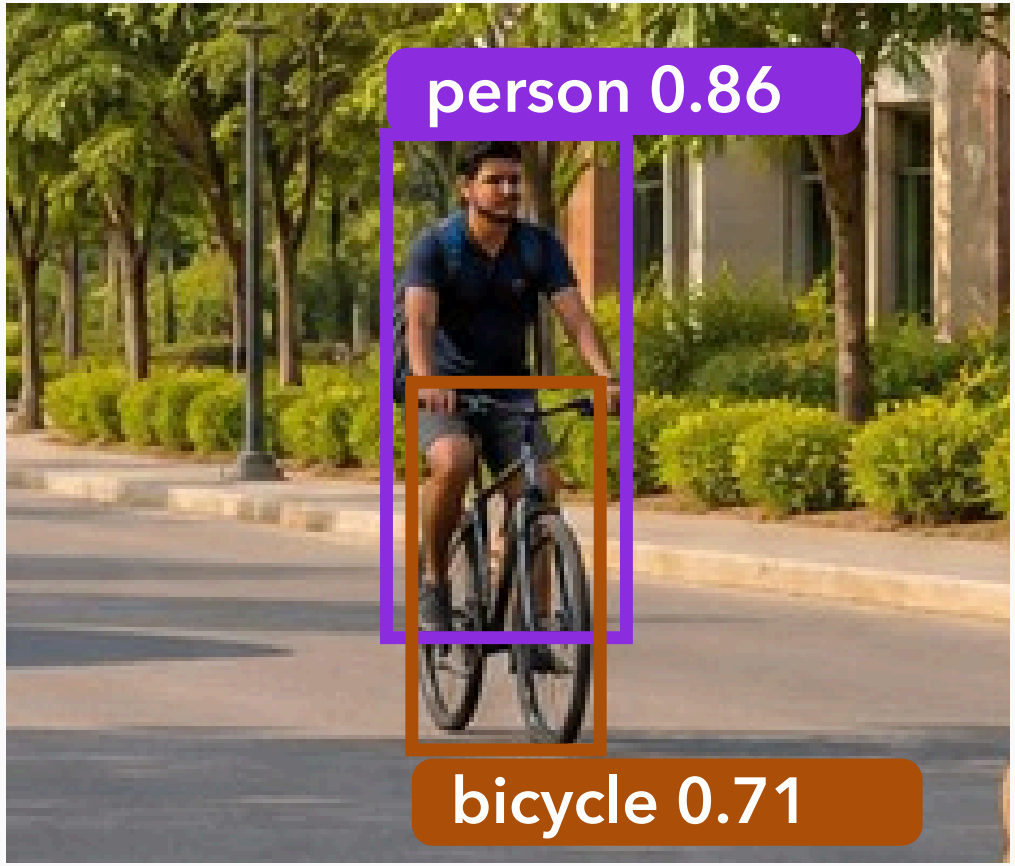
Classes ignored, threshold 0.3: the bicycle is deleted.

> If NMS ignored classes, the bicycle would overlap the higher-scoring person by more than 0.3 and be deleted.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Boxes of different classes do not suppress each other



$$\mathrm{IoU}(\text{person}, \text{bicycle}) = 0.339$$

Classes ignored, threshold 0.3: the bicycle is deleted.

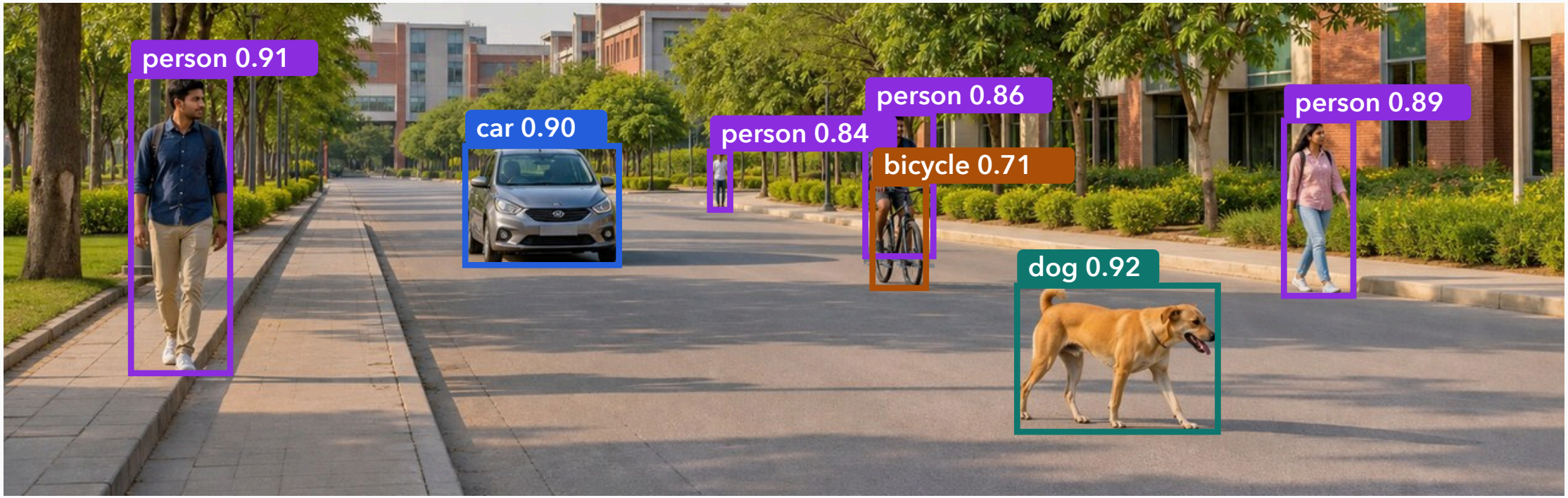
Class-aware NMS, the usual choice (torchvision's batched_nms): both survive.

> Run NMS separately for each class and both are kept. That is why step 3 of the recipe says "of the same class".

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

Try it: thresholds on the street photo



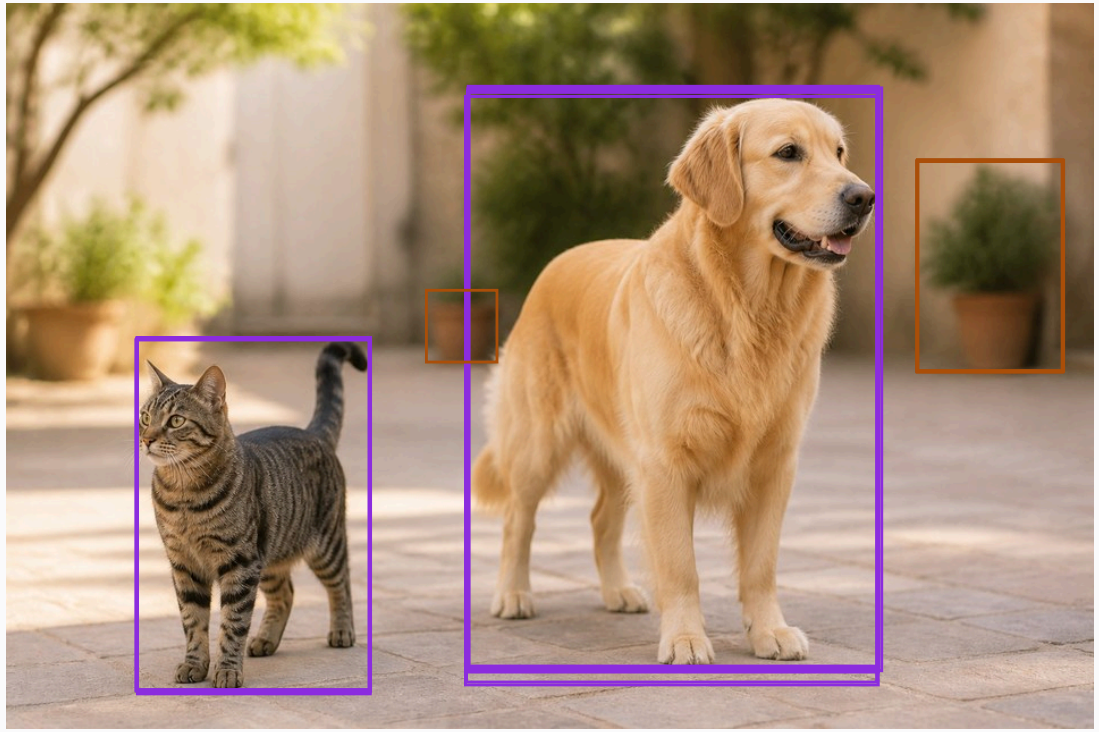
> Move the score threshold, then the IoU threshold; try “all classes together” and watch the bicycle; Step runs one round of the loop.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```
def nms(boxes, scores, thr=0.7):
```



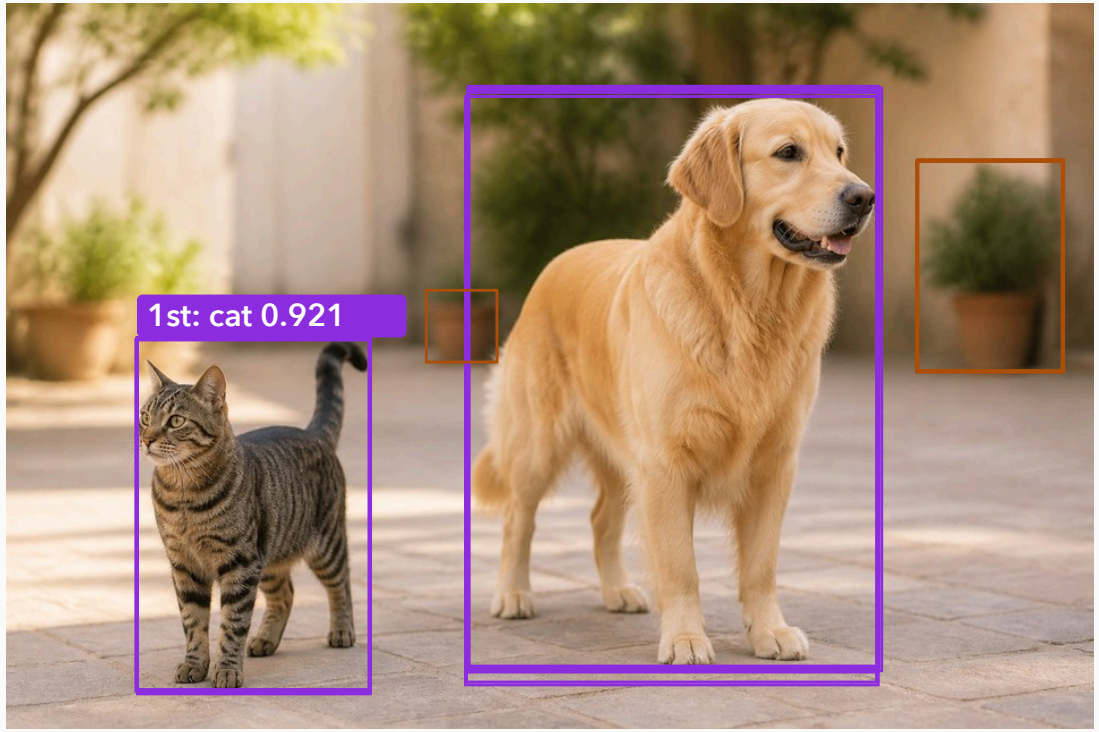
The greedy loop of 6.1 as a function. Its input: the 31 boxes that passed the score filter, with their scores.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
```



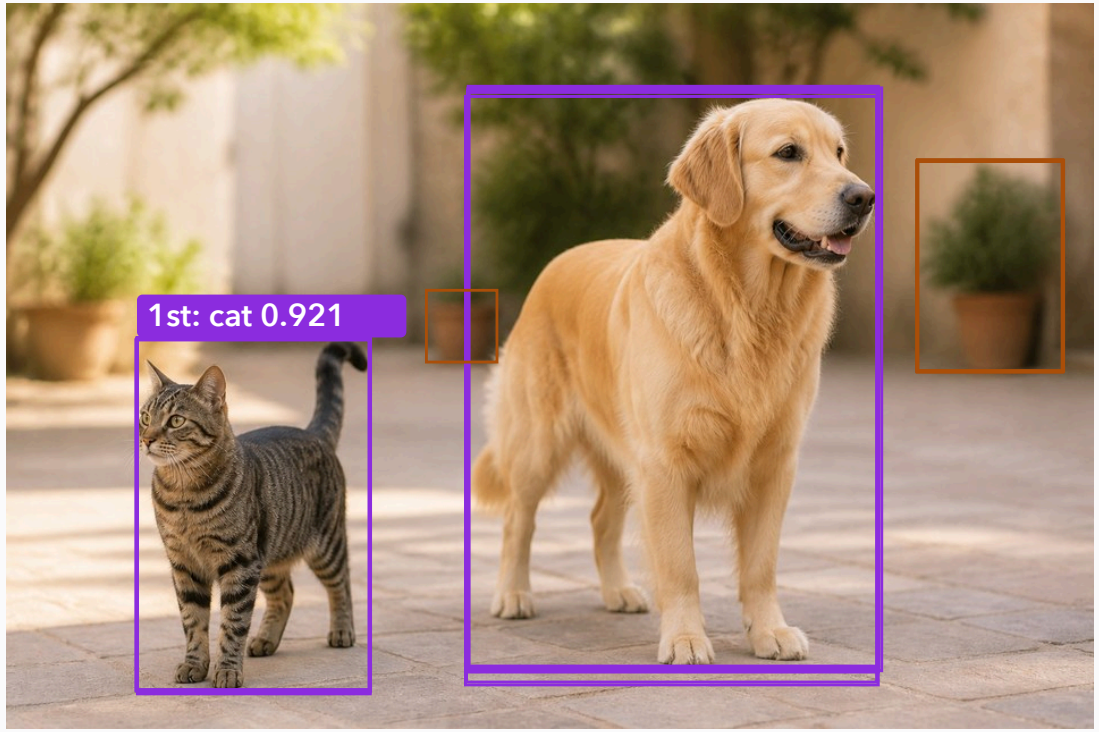
> Sort by score, best first. The best of all is a cat box, 0.921.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
    keep = []
    while order.numel() > 0:
```



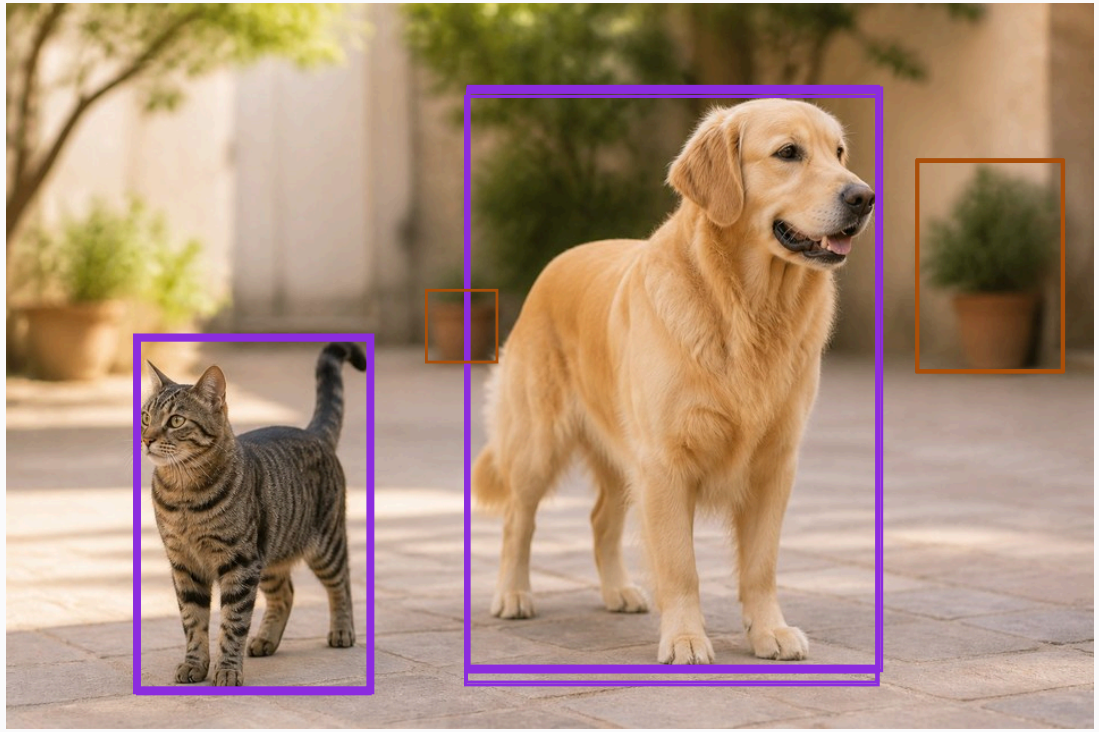
keep starts empty. The loop runs while any box is left in order.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
    keep = []
    while order.numel() > 0:
        i, rest = order[0], order[1:]
        keep.append(i)
```



> Take the first box in order and keep it. Everything after it is rest.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
    keep = []
    while order.numel() > 0:
        i, rest = order[0], order[1:]
        keep.append(i)
        ious = box_iou(boxes[[i]], boxes[rest])[0]
```



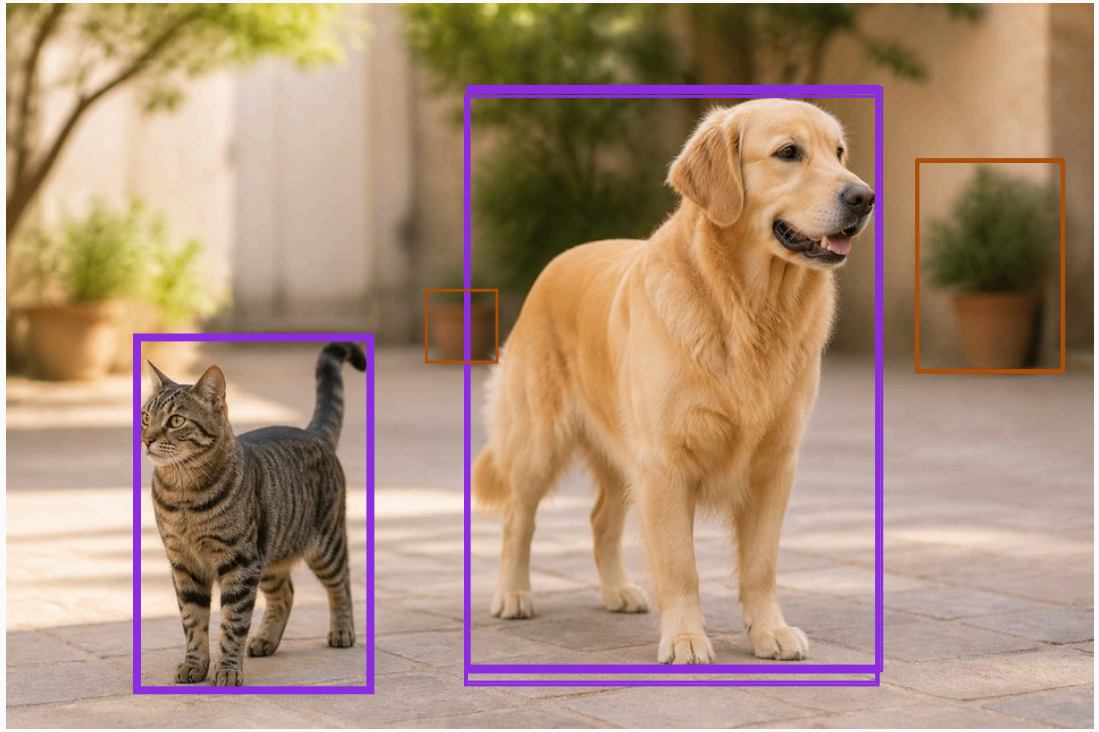
> IoU of the kept box with every box in rest, all at once: its 9 twins score 0.98 or more, and every other box about 0.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
    keep = []
    while order.numel() > 0:
        i, rest = order[0], order[1:]
        keep.append(i)
        ious = box_iou(boxes[[i]], boxes[rest])[0]
        order = rest[ious <= thr]
```



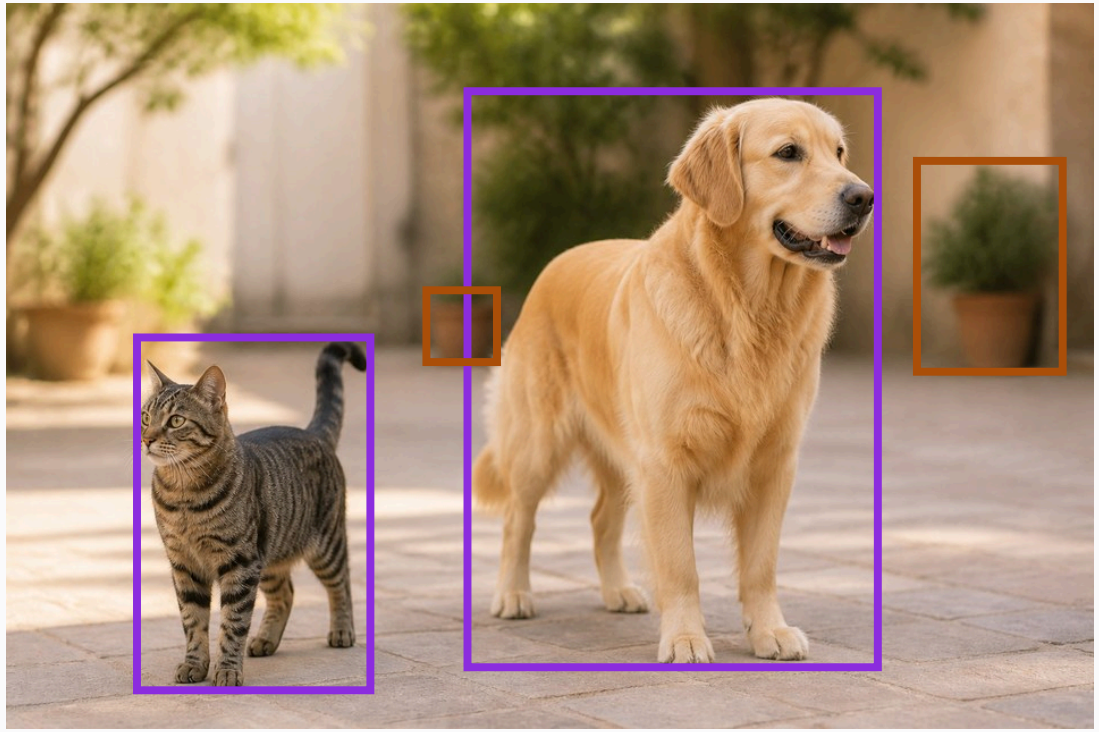
> Keep only the boxes with IoU ≤ 0.7: the cat's 9 twins go, and nothing else is touched.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
    keep = []
    while order.numel() > 0:
        i, rest = order[0], order[1:]
        keep.append(i)
        ious = box_iou(boxes[[i]], boxes[rest])[0]
        order = rest[ious <= thr]
```



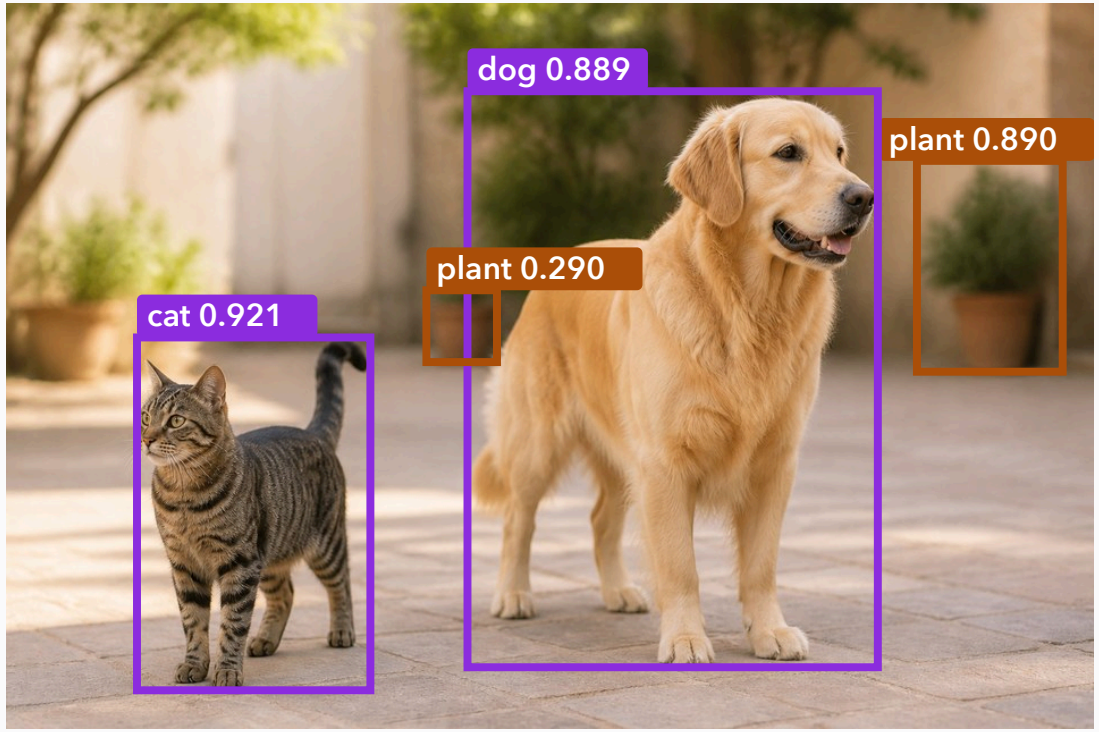
> The loop repeats: a plant box drops 9 twins, the dog box 9, the small plant none. order is now empty.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

NMS in code, one line at a time

```python
def nms(boxes, scores, thr=0.7):
    order = scores.argsort(descending=True)
    keep = []
    while order.numel() > 0:
        i, rest = order[0], order[1:]
        keep.append(i)
        ious = box_iou(boxes[[i]], boxes[rest])[0]
        order = rest[ious <= thr]
    return torch.stack(keep)
```



> Return the 4 kept boxes, one per object. torchvision.ops.nms returns the same four.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The dense answer, end to end

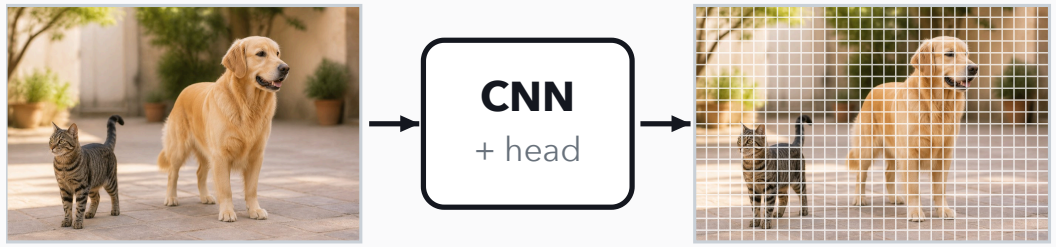


photo
960 × 640

one pass
learned

12,600 records
same shape for any photo

> Photo in, one pass of the CNN and its head, and out come 12,600 records, the same number for every photo this size.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The dense answer, end to end

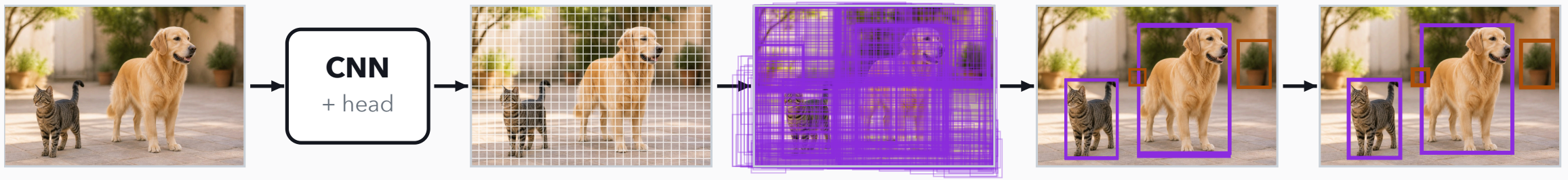


photo
960 × 640

one pass
learned

12,600 records
same shape for any photo

a box per cell
decode

31 boxes
score ≥ 0.25

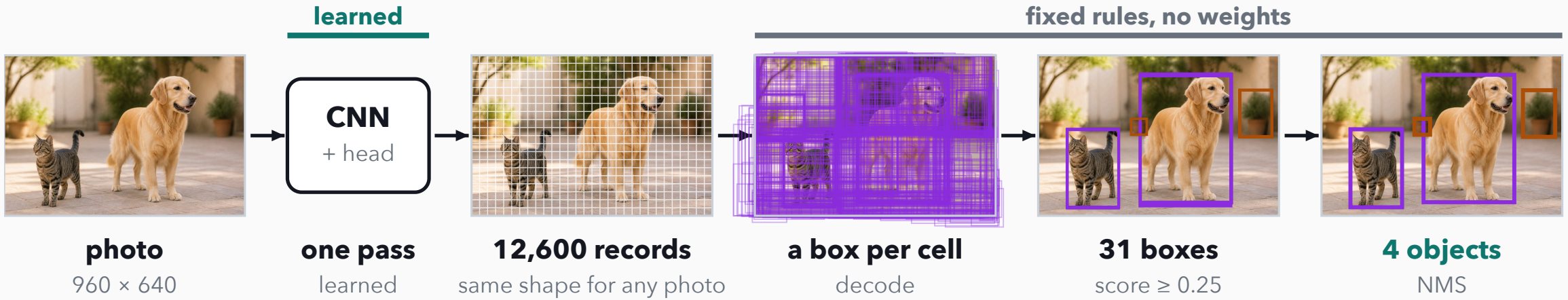
4 objects
NMS

> Fixed rules turn them into detections: decode every record into a box, drop the low scores, and remove duplicates with NMS.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The dense answer, end to end



The output never changes shape. The score filter and NMS decide how many objects come out: here 4, for the crowd 11, for an empty wall 0.

> Only the CNN and head have learned weights. This is answer A to the lecture's question: outputs tied to places, then one box per object.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```
def detect(x, net):
```



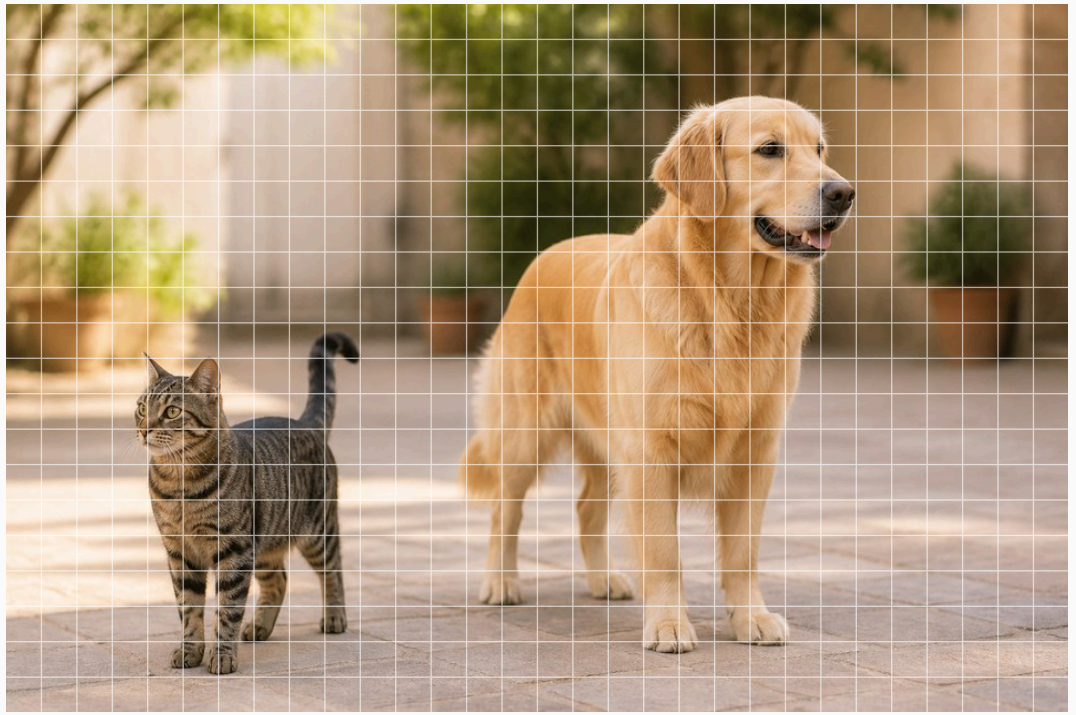
Parts 5 and 6 in one function. x is the photo as a tensor, and net is the trained YOLOv8n.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```python
def detect(x, net):
    y = net(x)[0][0]                # (84, 12600)
```



One pass of the network: 84 numbers for each of the 12,600 cells, whatever the photo shows.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```
def detect(x, net):
    y = net(x)[0][0]              # (84, 12600)
    box = xyxy(y[:4].T)           # (12600, 4)
```



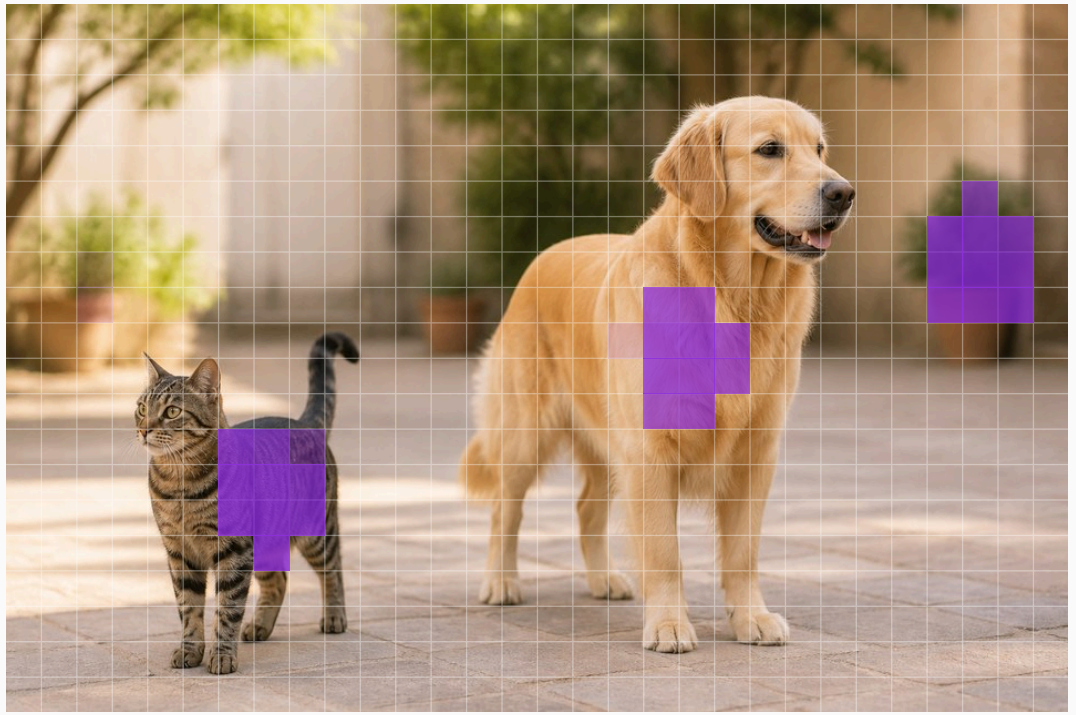
Turn each cell's centre and size into corners: 12,600 boxes, one per cell. Our grid's 600 are drawn.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```
def detect(x, net):
    y = net(x)[0][0]                # (84, 12600)
    box = xyxy(y[:4].T)             # (12600, 4)
    score, label = y[4:].max(0)     # (12600,)
```



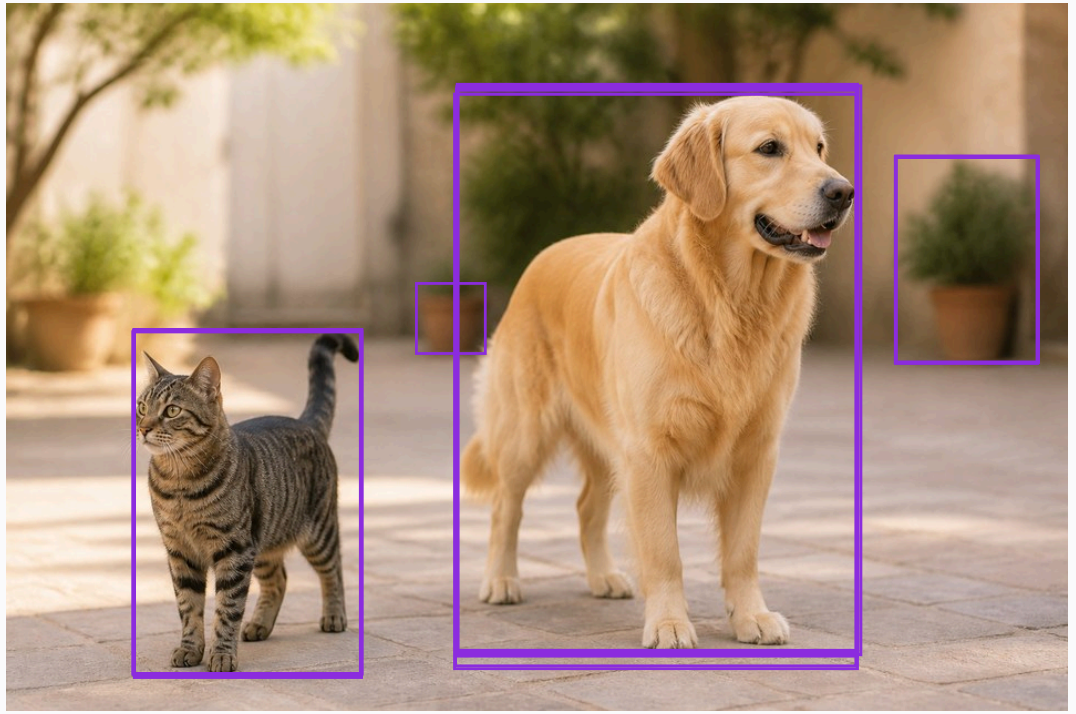
For each cell, its best class and that class's score. The scores are high only near the animals and the plants.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```
def detect(x, net):
    y = net(x)[0][0]                # (84, 12600)
    box = xyxy(y[:4].T)             # (12600, 4)
    score, label = y[4:].max(0)     # (12600,)
    k = score >= 0.25               # 31 True
```



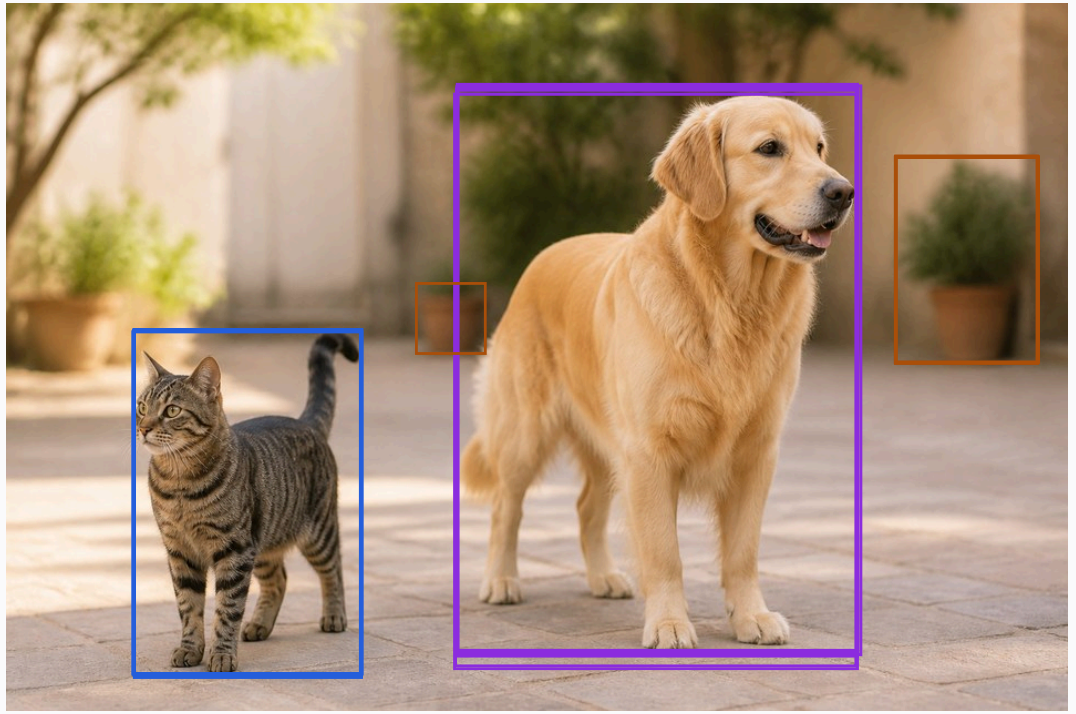
A mask of the cells whose best score is at least 0.25: 31 of 12,600.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```
def detect(x, net):
    y = net(x)[0][0]                # (84, 12600)
    box = xyxy(y[:4].T)             # (12600, 4)
    score, label = y[4:].max(0)     # (12600,)
    k = score >= 0.25               # 31 True
    box, score, label = box[k], score[k], label[k]
```



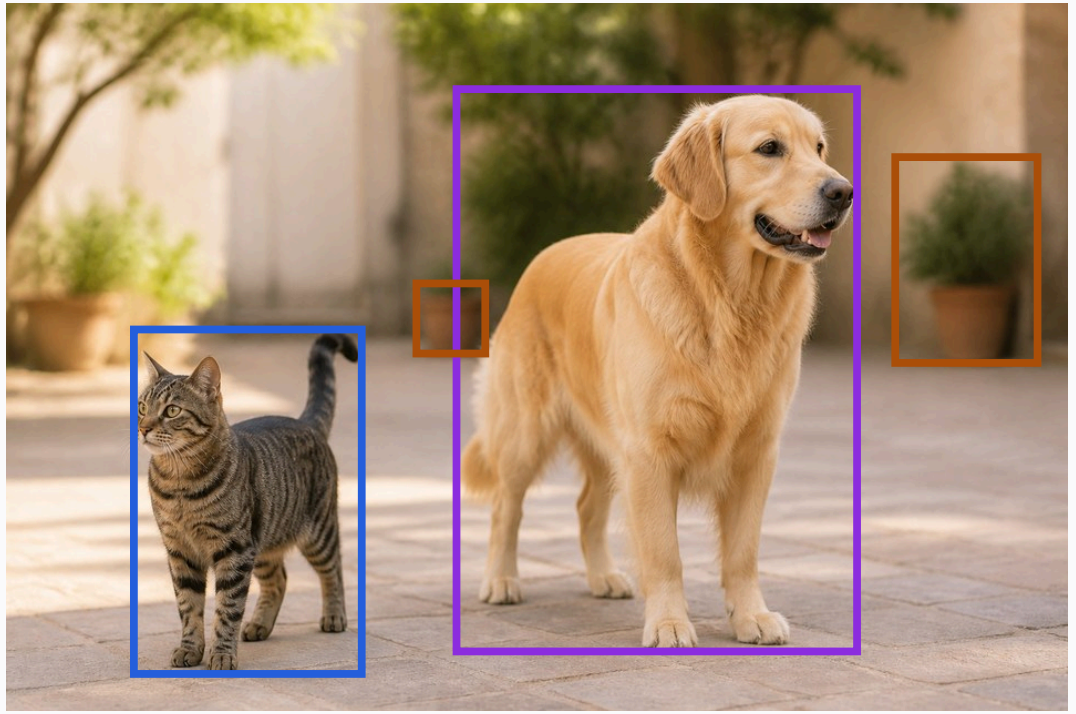
Keep those rows only: 31 boxes, each with its score and its class.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```
def detect(x, net):
    y = net(x)[0][0]                # (84, 12600)
    box = xyxy(y[:4].T)             # (12600, 4)
    score, label = y[4:].max(0)     # (12600,)
    k = score >= 0.25               # 31 True
    box, score, label = box[k], score[k], label[k]
    k = batched_nms(box, score, label, 0.7)  # 4
```



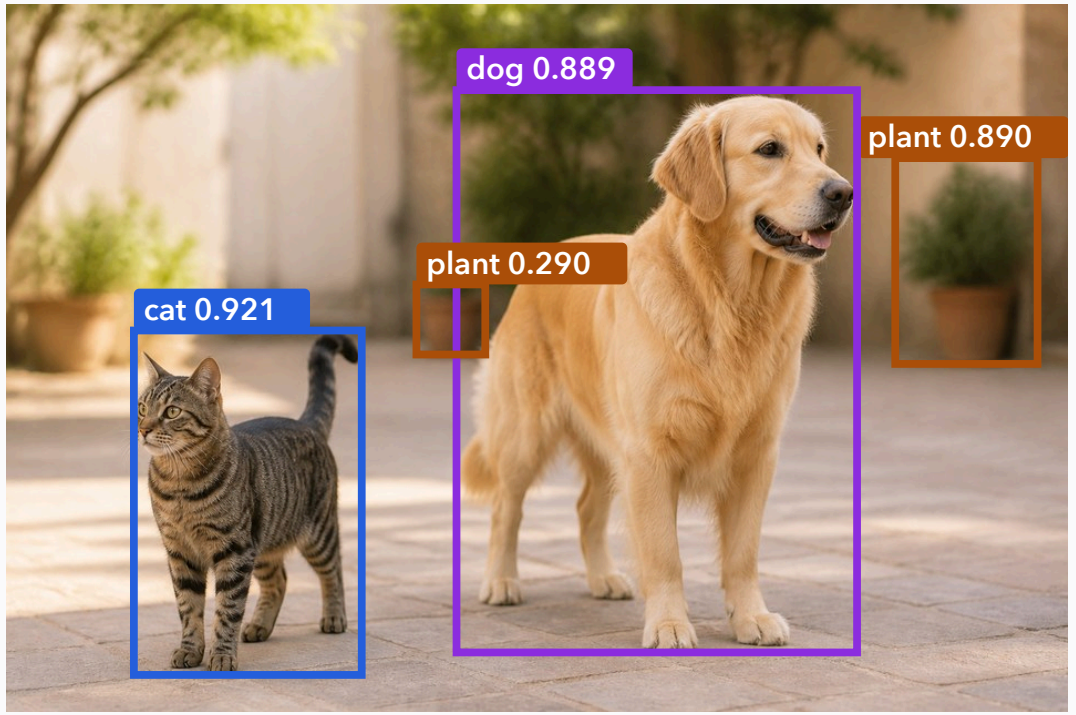
NMS within each class: 4 boxes remain, one per object.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 6

The whole detector at test time, one line at a time

```python
def detect(x, net):
    y = net(x)[0][0]                # (84, 12600)
    box = xyxy(y[:4].T)             # (12600, 4)
    score, label = y[4:].max(0)     # (12600,)
    k = score >= 0.25               # 31 True
    box, score, label = box[k], score[k], label[k]
    k = batched_nms(box, score, label, 0.7)  # 4
    return box[k], score[k], label[k]
```



> Return them: the four detections of the lecture's first slide. No line depends on how many objects there are.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 7

Part 7 · Small and large objects: three grids

A 32-pixel cell is coarse for a person 13 pixels wide. YOLOv8 predicts from three grids at once.

1. See the range of sizes in one street photo, now with the grid.
2. Count the cells of the three grids: where 12,600 came from.
3. Check which grid found each real detection.

> YOLOv8 uses three grids with different cell sizes, and every cell writes the same record.

▶ Notebook, part 7

Part 1's street again, with the 32-pixel grid



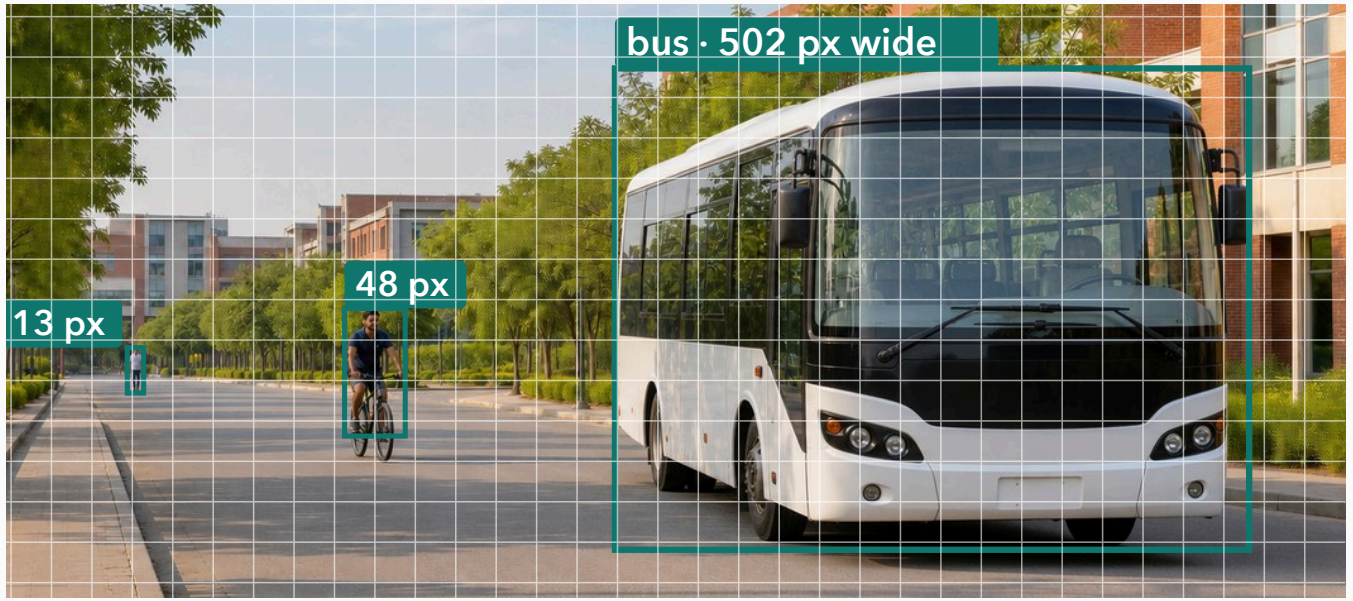
The bus is 502 px wide; the cyclist 48 px; the far pedestrian 13 px.

> Part 1's street again: a bus, a cyclist and a far pedestrian.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 7

Part 1's street again, with the 32-pixel grid



The bus is 502 px wide; the cyclist 48 px; the far pedestrian 13 px.

On a 32-px grid the pedestrian is under half a cell.

> Draw the 32-px grid. The bus spans about 16 cells; the pedestrian sits inside one. A coarse grid suits big objects; small ones need finer cells.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, part 7

Three grids: cells of 8, 16 and 32 pixels

grid (cell size)	courtyard, 960 × 640	street, 1280 × 704	built from
8 px	120 × 80 = 9,600	160 × 88 = 14,080	early layers: sharp, local
16 px	60 × 40 = 2,400	80 × 44 = 3,520	middle layers
32 px	30 × 20 = 600	40 × 22 = 880	deep layers: wide view
all			

> YOLOv8 reads three feature maps from different depths of the CNN: cells of 8, 16 and 32 pixels. Every cell writes the same 84-number record.

▶ Notebook, part 7

Three grids: cells of 8, 16 and 32 pixels

grid (cell size)	courtyard, 960 × 640	street, 1280 × 704	built from
8 px	120 × 80 = 9,600	160 × 88 = 14,080	early layers: sharp, local
16 px	60 × 40 = 2,400	80 × 44 = 3,520	middle layers
32 px	30 × 20 = 600	40 × 22 = 880	deep layers: wide view
all	**12,600**	**18,480**	

> Add them up: 12,600 for the courtyard, the number we decoded in Part 5. A bigger photo gets more cells: 18,480 for the street.

▶ Notebook, part 7

Which grid found each object? The real answer



object	grid
bus	32 px
person	16 px
person	8 px
bicycle	16 px

> After NMS, look up which grid each kept box came from: the bus from 32 px, the cyclist from 16 px, the far pedestrian from 8 px.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 7

Which grid found each object? The real answer



object	grid
bus	32 px
person	16 px
person	8 px
bicycle	16 px

Street photo: dog and near person from 32 px; car, cyclist and bicycle from 16 px; the far person from 8 px.

> The street photo agrees: large from coarse, small from fine. Training assigns each object to cells whose grid suits its size.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

Part 7 in one slide

1. One grid cannot suit a 502-px bus and a 13-px pedestrian.
2. YOLOv8 predicts from grids of 8, 16 and 32-px cells: 12,600 records.
3. Every cell writes the same 84-number record; NMS runs on all of them.
4. Small objects come from the fine grid, large ones from the coarse.

> **Next:** measure how good the detections are.

▶ Notebook, part 8

Part 8 · Measuring a detector

Compare detections with annotations: decide which are right, then summarise.

1. Decide true positive, false positive and miss, on the street photo's real detections.
2. Precision and recall at one threshold, then at all of them: walk down a ranked list, then AP in code and live.
3. AP at stricter IoU thresholds, then mAP over the classes.

> We judge a detector against annotations, one detection at a time, then summarise.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…

> A detection counts as correct if its class is right and it overlaps an annotated box with IoU of at least 0.5.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.

> Each annotated object can be claimed once. Detections go in score order; the first good one claims it.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.
3. **False positive (FP)**: everything else: a box where nothing is, a wrong class, or a second box on a claimed object.

> Every detection that claims nothing is a false positive, including a duplicate that NMS missed.

▶ Notebook, part 8

When is a detection correct?

1. **True positive (TP)**: the right class and IoU ≥ 0.5 with an annotated box…
2. …that no higher-scoring detection has already claimed. Go through detections by score.
3. **False positive (FP)**: everything else: a box where nothing is, a wrong class, or a second box on a claimed object.
4. **False negative (FN)**: an annotated object that no detection claimed: a miss.

> Every annotated object left unclaimed is a false negative.

▶ Notebook, part 8

Real check: people in the street photo



Teal: the 4 annotated people. YOLOv8n's person detections after NMS, down to score 0.01: 12.

> We keep every person detection down to a score of 0.01, so that the wrong ones show up too.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Real check: people in the street photo



The 4 most confident (0.907, 0.893, 0.864, 0.845) each claim a different person, IoU 0.97, 0.97, 0.92, 0.91: **4 TP**.

> The four most confident boxes each land on a different person.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Real check: people in the street photo



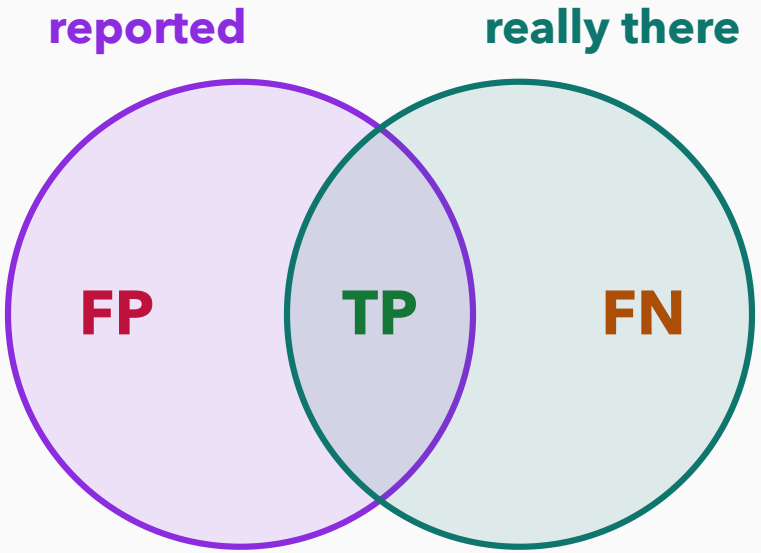
The other 8 (scores 0.042 down to 0.011) sit on the car’s windscreen, IoU 0 with every person: **8 FP**. No person is missed: 0 FN.

> The rest sit on the car’s windscreen, where nobody is.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Precision and recall: two questions



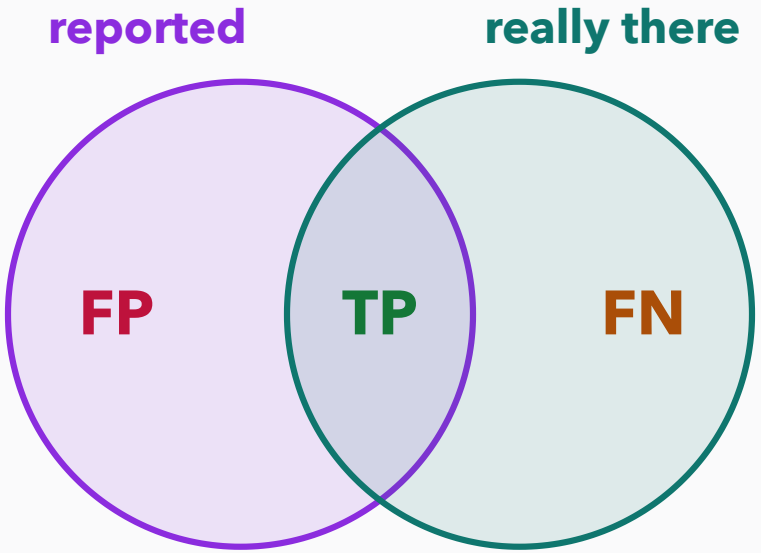
$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Of what we reported, how much was right?

Precision looks at the left circle, everything we reported: the fraction of it that was right.

▶ Notebook, part 8

Precision and recall: two questions



$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Of what we reported, how much was right?

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Of what was really there, how much did we find?

Recall looks at the right circle, everything that was really there: the fraction of it we found.

▶ Notebook, part 8

The street's people at two score cut-offs

keep scores ≥	TP	FP	FN	precision	recall
0.25	4	0	0	4/4 = 1	4/4 = 1

With the usual 0.25 score cut-off, the street photo is perfect: four people found, nothing extra. Precision 1, recall 1.

▶ Notebook, part 8

The street's people at two score cut-offs

keep scores ≥	TP	FP	FN	precision	recall
0.25	4	0	0	4/4 = 1	4/4 = 1
0.01	4	8	0	4/12 = 0.33	4/4 = 1

> Keep everything down to 0.01 and the 8 windscreen boxes join: recall stays 1, precision drops to 0.33. The cut-off trades one for the other.

▶ Notebook, part 8

Back to the bird: a lower cut-off finds it



keep scores ≥	found	TP	FP	FN	precision	recall
0.25	cat, dog	2	0	1	1.00	2/3

> At the usual 0.25: the cat and the dog are found, the bird (0.098) is missed. Precision 1, recall 2/3.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Back to the bird: a lower cut-off finds it



keep scores ≥	found	TP	FP	FN	precision	recall
0.25	cat, dog	2	0	1	1.00	2/3
0.05	bird, cat, dog	3	0	0	1.00	3/3

> At 0.05 the bird's box gets through: recall 3/3, and nothing false comes with it. On the street, going lower brought eight false people.

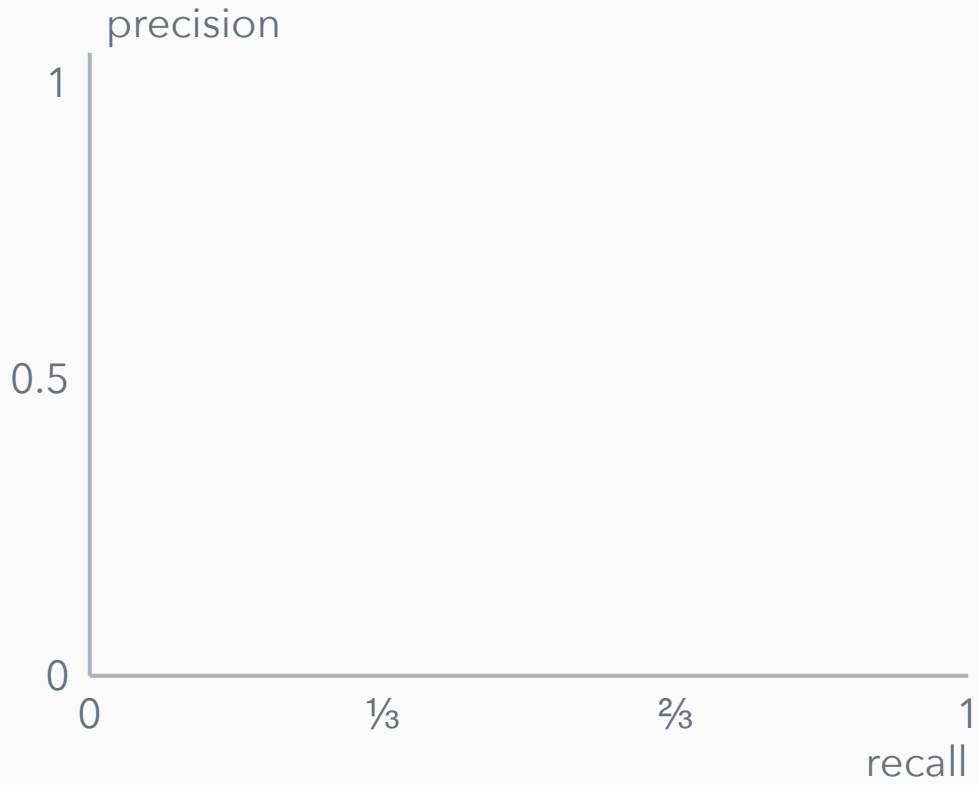
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95				
#2	0.90				
#3	0.80				
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



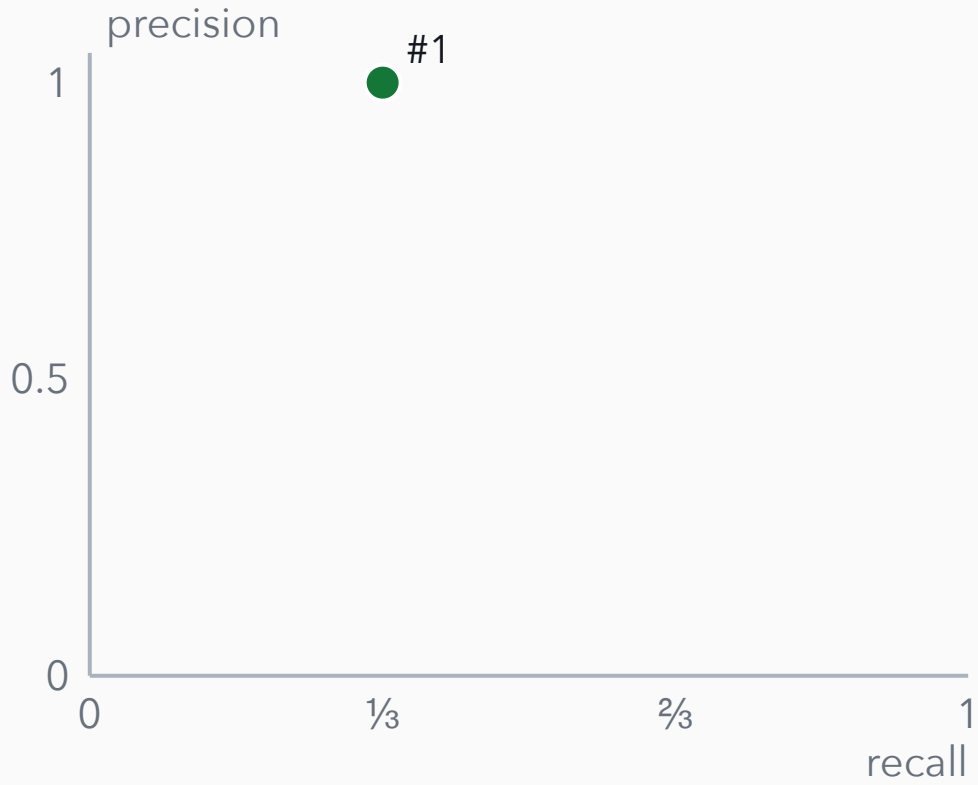
> Six detections sorted by score; three objects in the photos. Lower the threshold one detection at a time and track precision and recall.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90				
#3	0.80				
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



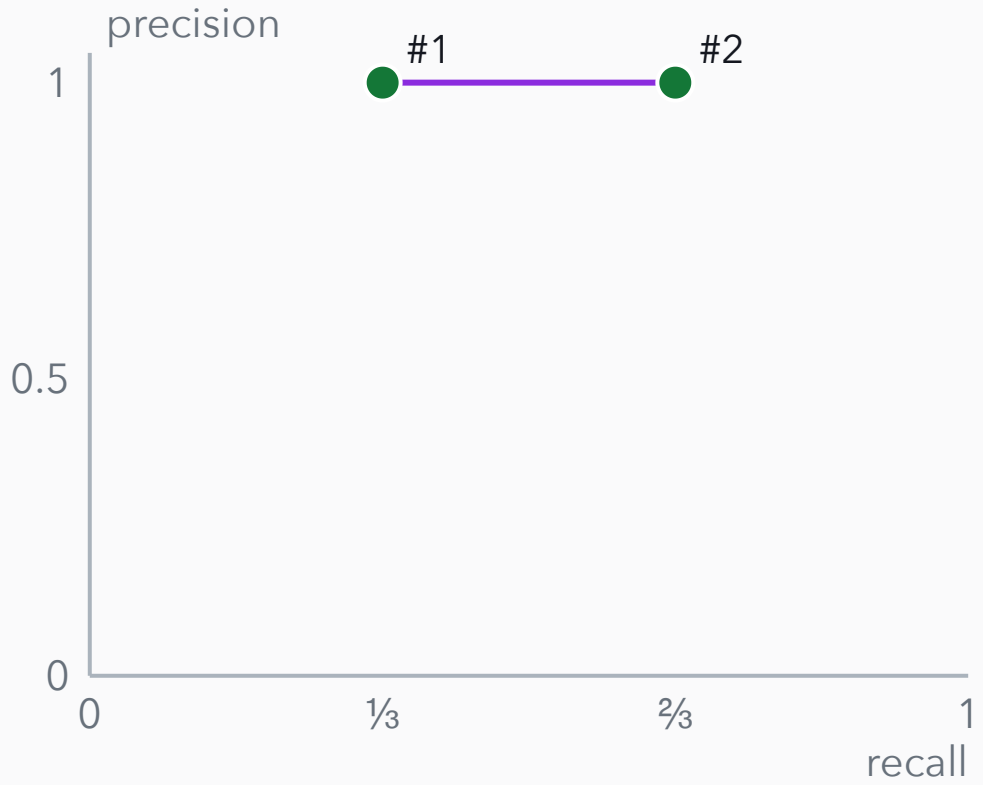
> #1 (0.95) claims object A: TP. Precision 1/1, recall 1/3.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80				
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



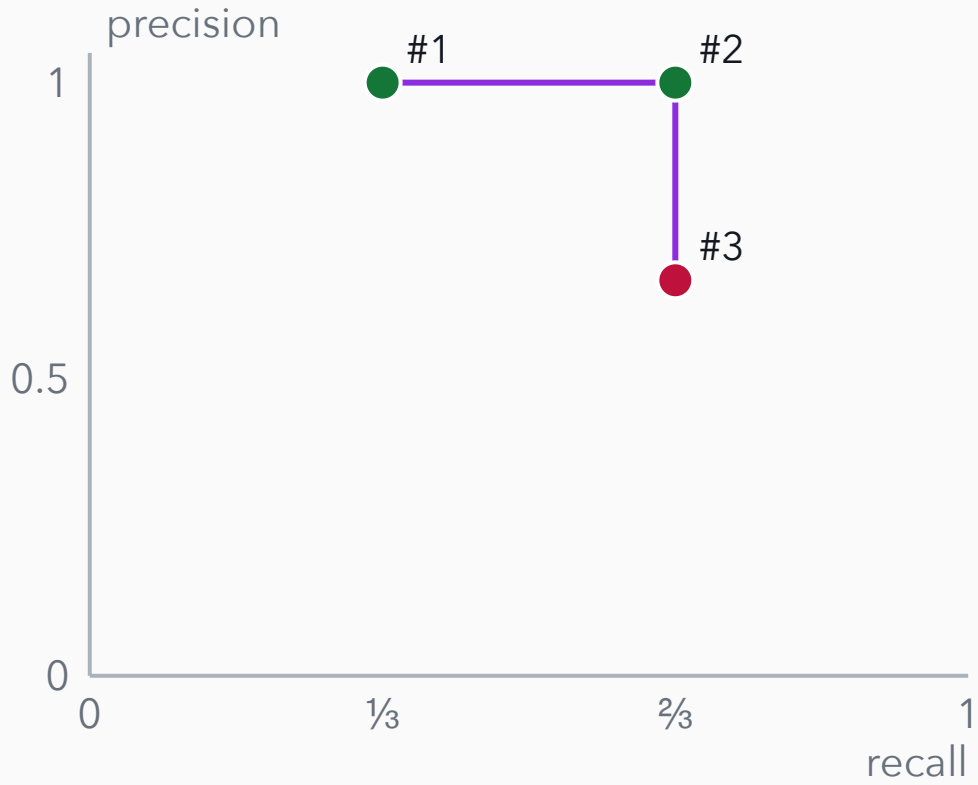
> #2 (0.90) claims B: TP. 2/2, 2/3.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70				
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



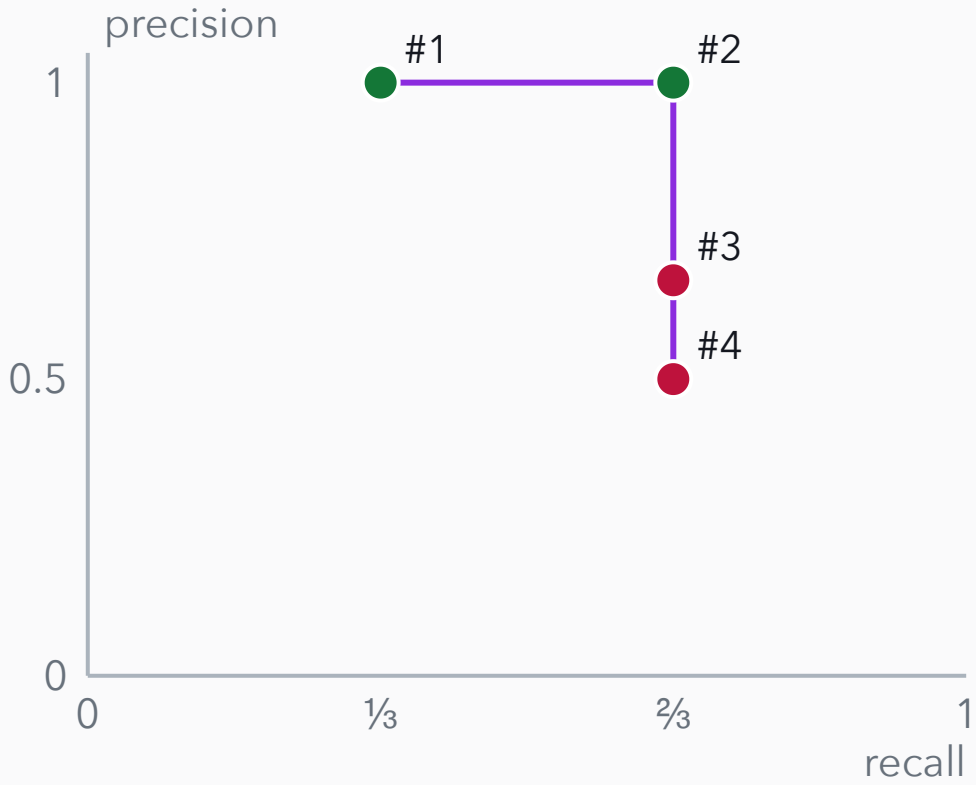
> #3 (0.80) lands on A again, already claimed: a duplicate, FP. Precision falls to 2/3; recall stays.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70	**FP**	no truth here	2/4	2/3
#5	0.60				
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



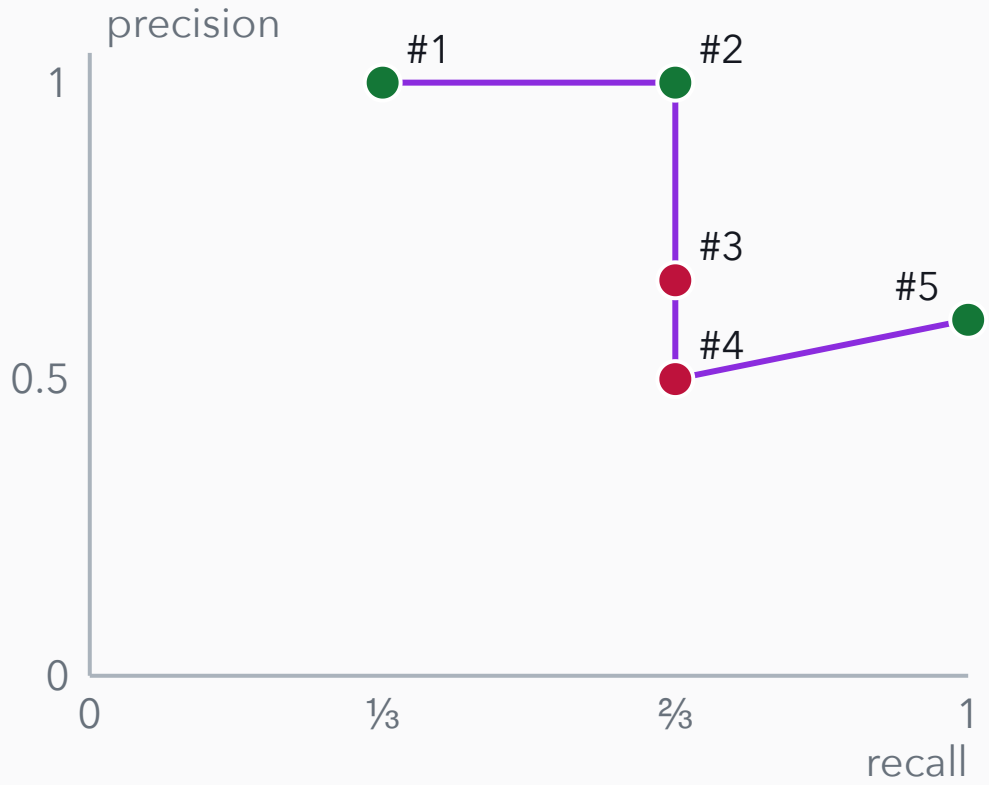
> #4 (0.70) where nothing is: FP. 2/4, 2/3.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70	**FP**	no truth here	2/4	2/3
#5	0.60	**TP**	match	3/5	3/3
#6	0.40				

A small invented list (3 objects, 6 detections) so that hits and misses interleave.



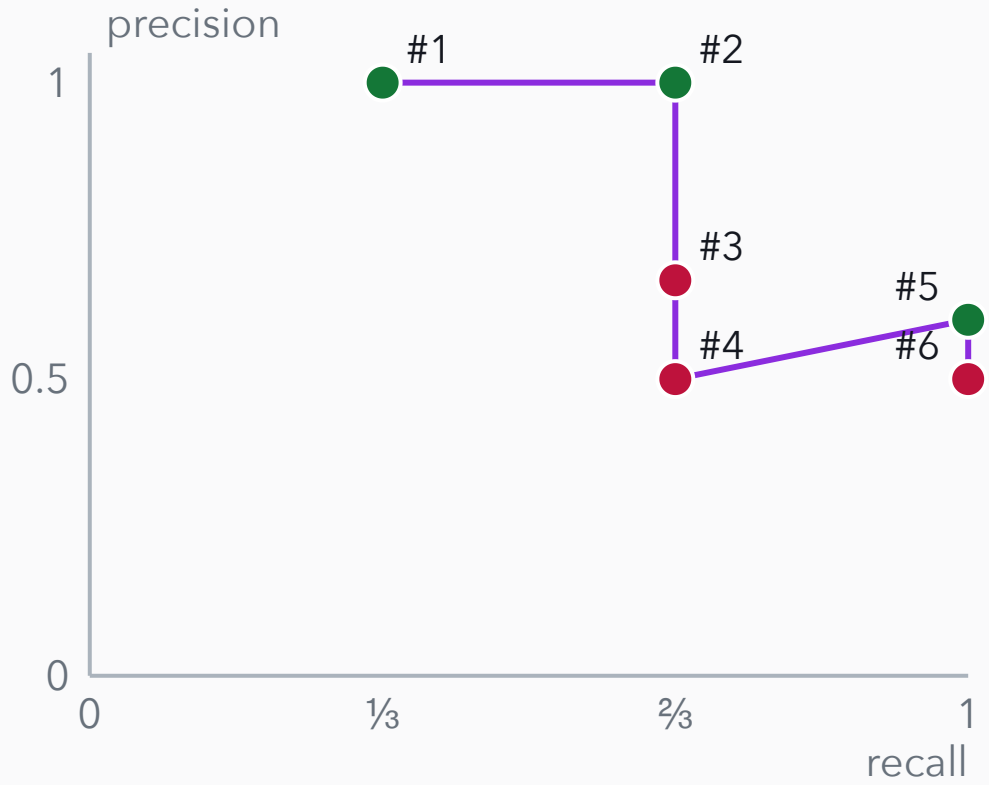
> #5 (0.60) claims C: TP. Recall reaches 1; precision 3/5.

▶ Notebook, part 8

Walk down the ranked list, one detection at a time

rank	score	outcome	why	precision	recall
#1	0.95	**TP**	match	1/1	1/3
#2	0.90	**TP**	match	2/2	2/3
#3	0.80	**FP**	duplicate	2/3	2/3
#4	0.70	**FP**	no truth here	2/4	2/3
#5	0.60	**TP**	match	3/5	3/3
#6	0.40	**FP**	no truth here	3/6	3/3

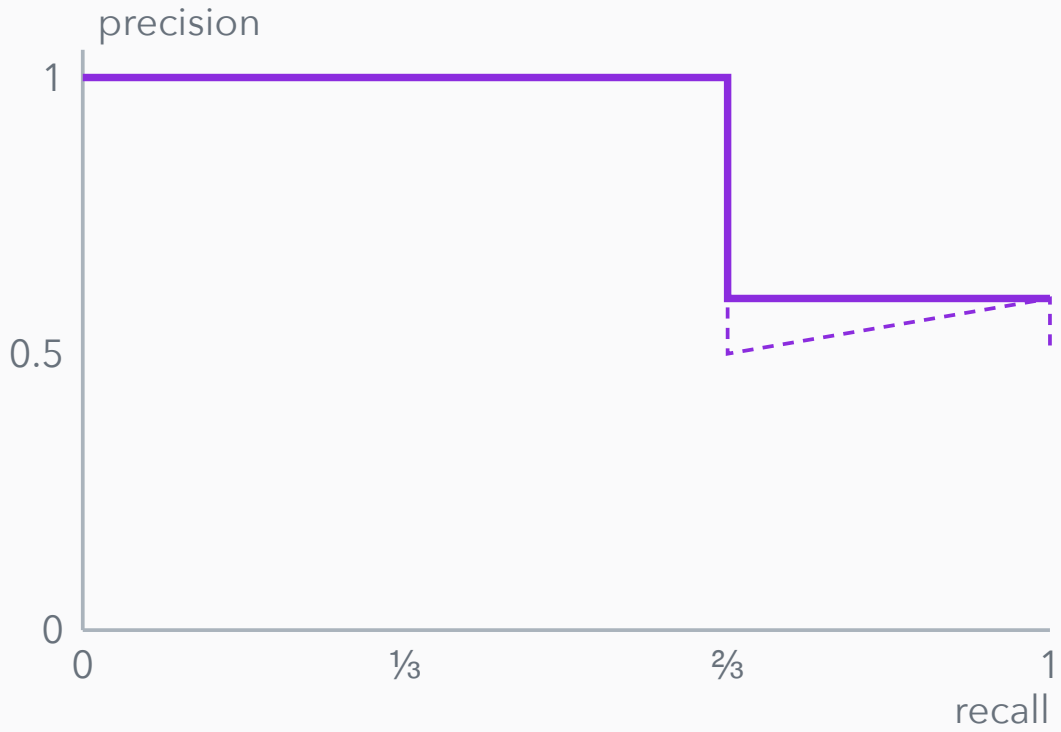
A small invented list (3 objects, 6 detections) so that hits and misses interleave.



> #6 (0.40): FP. 3/6, 3/3. Every point is one possible threshold.

▶ Notebook, part 8

Average precision: the area under the curve

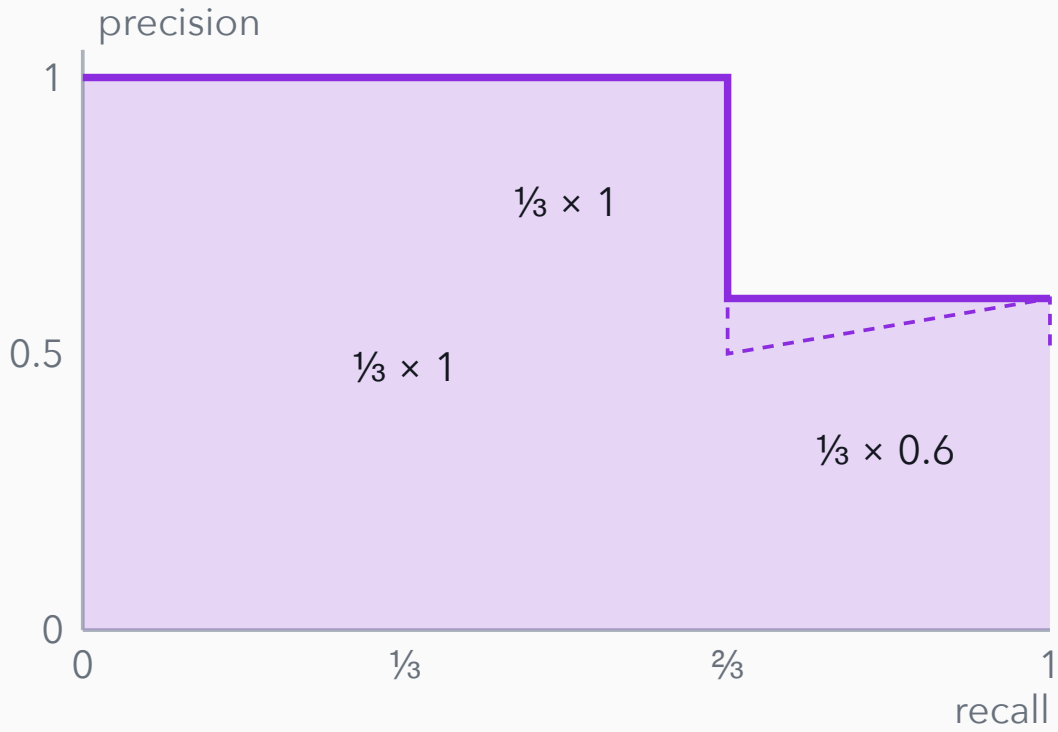


Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

> Smooth the zig-zag: at each recall, take the best precision reachable by lowering the threshold further. That is the envelope.

▶ Notebook, part 8

Average precision: the area under the curve



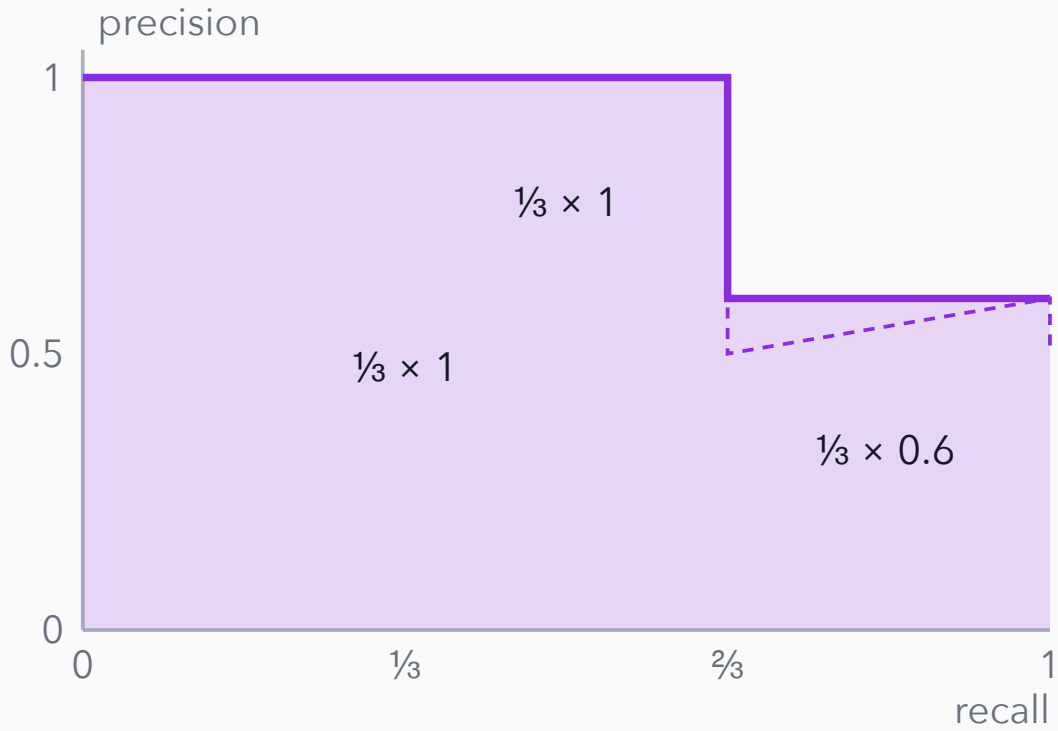
Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

$$\text{AP} = \tfrac{1}{3}(1) + \tfrac{1}{3}(1) + \tfrac{1}{3}(0.6) = 0.867$$

Its area: a third of the recall at precision 1, another third at 1, the last third at 0.6. AP = 0.867.

▶ Notebook, part 8

Average precision: the area under the curve



Replace each precision by the best precision at that recall or beyond. The curve now only steps down.

$$\text{AP} = \tfrac{1}{3}(1) + \tfrac{1}{3}(1) + \tfrac{1}{3}(0.6) = 0.867$$

AP = 1 only if every object is found before the first false positive, as for the street's people.

> AP rewards ranking: correct detections should come before wrong ones. The street people scored AP = 1.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
```

rank	hit
#1	1
#2	1
#3	0
#4	0
#5	1
#6	0

> AP in code, on the six ranked detections of the walk: write each one as a hit (1) or a miss (0), in score order.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
tp = hits.cumsum(0)  # true positives so far
```

rank	hit	tp
#1	1	1
#2	1	2
#3	0	2
#4	0	2
#5	1	3
#6	0	3

> Count the true positives so far, rank by rank.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
tp = hits.cumsum(0)  # true positives so far
fp = (~hits).cumsum(0)  # false positives so far
```

rank	hit	tp	fp
#1	1	1	0
#2	1	2	0
#3	0	2	1
#4	0	2	2
#5	1	3	2
#6	0	3	3

> And the false positives so far.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
tp = hits.cumsum(0)  # true positives so far
fp = (~hits).cumsum(0)  # false positives so far
p = tp / (tp + fp)  # precision
```

rank	hit	tp	fp	p
#1	1	1	0	1
#2	1	2	0	1
#3	0	2	1	0.67
#4	0	2	2	0.5
#5	1	3	2	0.6
#6	0	3	3	0.5

> Precision at each rank: the true positives over everything reported up to that rank.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
tp = hits.cumsum(0)  # true positives so far
fp = (~hits).cumsum(0)  # false positives so far
p = tp / (tp + fp)  # precision
r = tp / 3  # recall: 3 objects
```

rank	hit	tp	fp	p	r
#1	1	1	0	1	0.33
#2	1	2	0	1	0.67
#3	0	2	1	0.67	0.67
#4	0	2	2	0.5	0.67
#5	1	3	2	0.6	1
#6	0	3	3	0.5	1

Recall at each rank: the true positives over the 3 objects.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
tp = hits.cumsum(0)  # true positives so far
fp = (~hits).cumsum(0)  # false positives so far
p = tp / (tp + fp)  # precision
r = tp / 3  # recall: 3 objects
for i in range(len(p) - 2, -1, -1):
    p[i] = max(p[i], p[i + 1])  # envelope
```

rank	hit	tp	fp	p	r	envelope
#1	1	1	0	1	0.33	1
#2	1	2	0	1	0.67	1
#3	0	2	1	0.67	0.67	0.67
#4	0	2	2	0.5	0.67	0.6
#5	1	3	2	0.6	1	0.6
#6	0	3	3	0.5	1	0.5

> The envelope: walk up from the bottom and keep the best precision seen so far, so the curve only steps down.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
tp = hits.cumsum(0)  # true positives so far
fp = (~hits).cumsum(0)  # false positives so far
p = tp / (tp + fp)  # precision
r = tp / 3  # recall: 3 objects
for i in range(len(p) - 2, -1, -1):
    p[i] = max(p[i], p[i + 1])  # envelope
w = r.diff(prepend=torch.zeros(1))  # widths
```

rank	hit	tp	fp	p	r	envelope	w
#1	1	1	0	1	0.33	1	0.33
#2	1	2	0	1	0.67	1	0.33
#3	0	2	1	0.67	0.67	0.67	0
#4	0	2	2	0.5	0.67	0.6	0
#5	1	3	2	0.6	1	0.6	0.33
#6	0	3	3	0.5	1	0.5	0

> The recall each rank adds: the width of its step. Only the three hits add any; the rows with width 0 do not count.

▶ Notebook, part 8

AP in code, one line at a time

```
hits = torch.tensor([1, 1, 0, 0, 1, 0]).bool()
tp = hits.cumsum(0)  # true positives so far
fp = (~hits).cumsum(0)  # false positives so far
p = tp / (tp + fp)  # precision
r = tp / 3  # recall: 3 objects
for i in range(len(p) - 2, -1, -1):
    p[i] = max(p[i], p[i + 1])  # envelope
w = r.diff(prepend=torch.zeros(1))  # widths
ap = (w * p).sum()  # 0.867
```

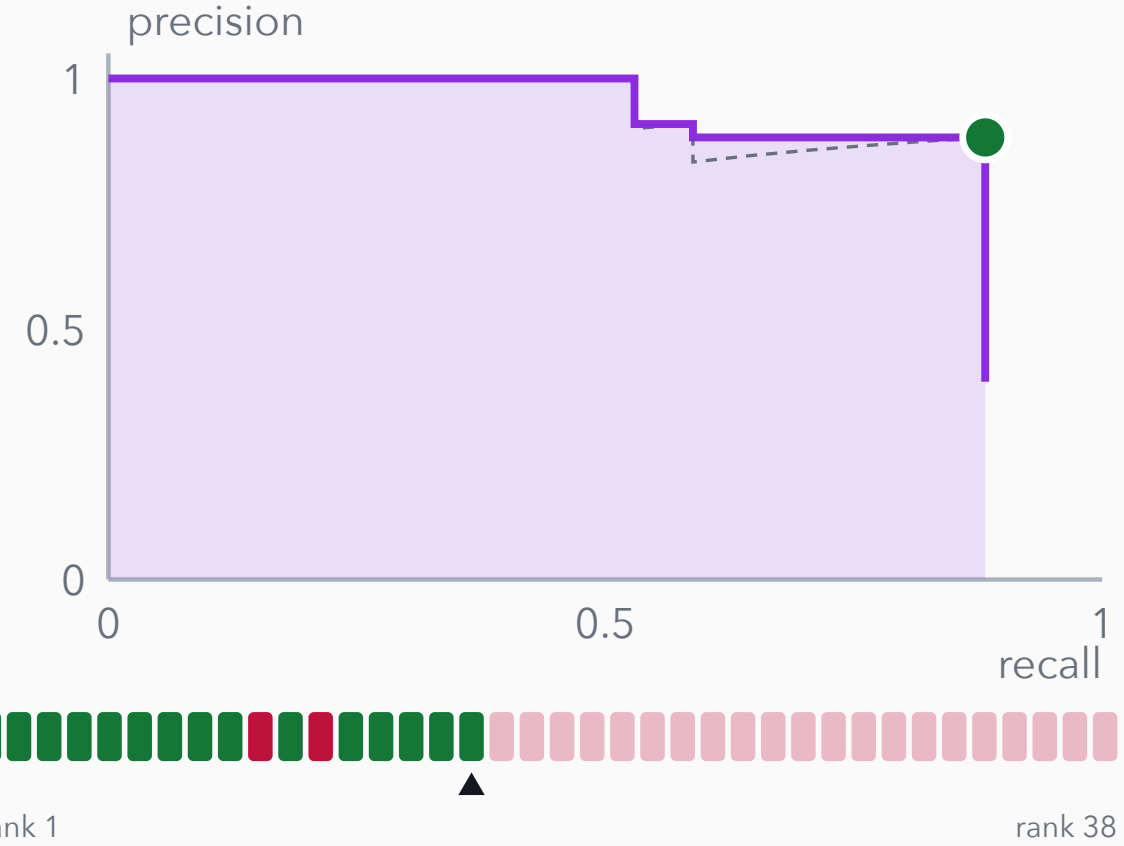
rank	hit	tp	fp	p	r	envelope	w
#1	1	1	0	1	0.33	1	0.33
#2	1	2	0	1	0.67	1	0.33
#3	0	2	1	0.67	0.67	0.67	0
#4	0	2	2	0.5	0.67	0.6	0
#5	1	3	2	0.6	1	0.6	0.33
#6	0	3	3	0.5	1	0.5	0

$$\text{AP} = \tfrac{1}{3}(1) + \tfrac{1}{3}(1) + \tfrac{1}{3}(0.6) = 0.867$$

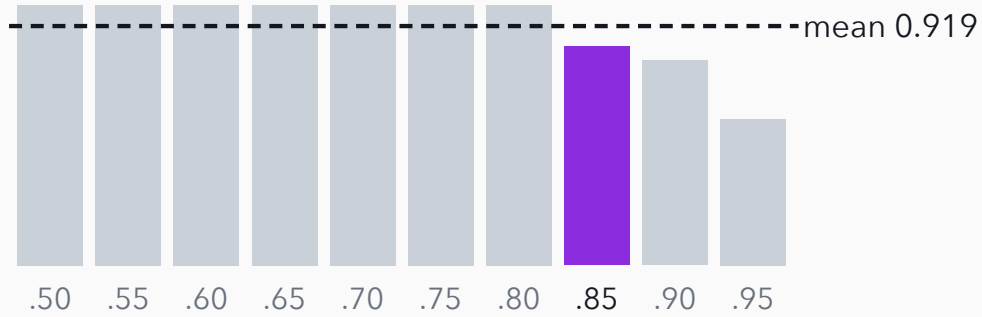
> Multiply each width by its height and add: ⅓ × 1 + ⅓ × 1 + ⅓ × 0.6 = 0.867, the area of the last slide.

▶ Notebook, part 8

Try it: AP on real detections



top 17	TP 15, FP 2
precision	0.88
recall	0.88
AP at IoU 0.85	**0.84**

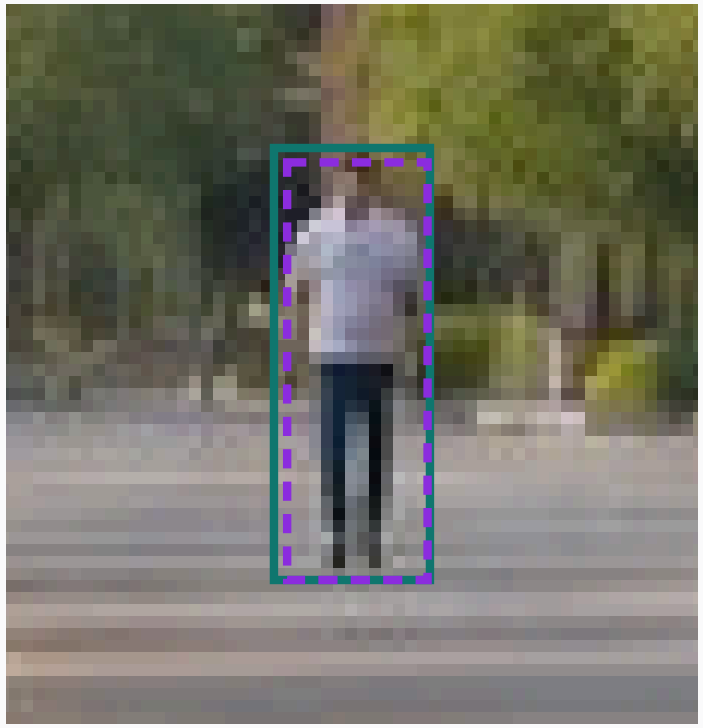


YOLOv8n's 38 person detections on our eight photos, ranked by score, against 17 people. Raise the IoU threshold: from 0.85, loose boxes turn into false positives.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

COCO's AP: the same, at ten IoU thresholds



$$\mathrm{IoU}(\mathrm{YOLOv8n}, \mathrm{ours}) \approx 0.87$$

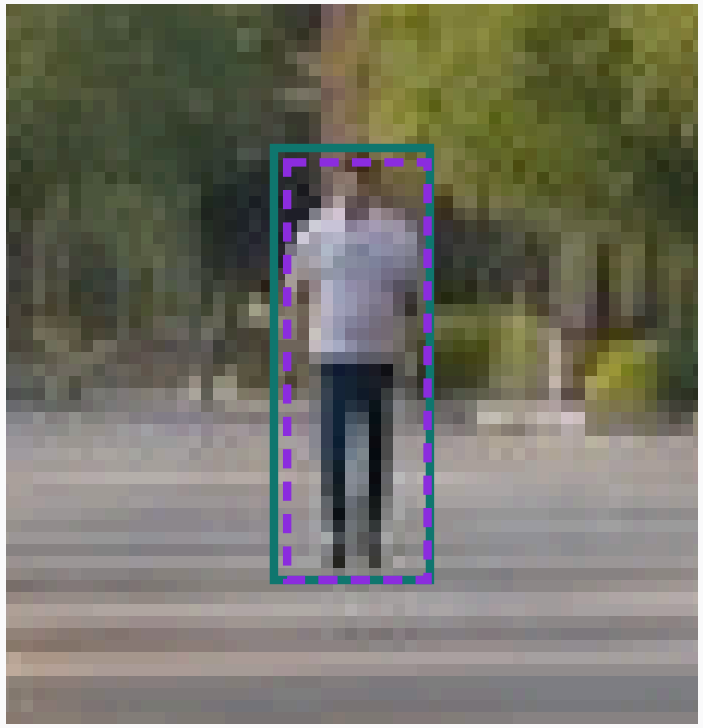
.50	.55	.60	.65	.70	.75	.80	.85	.90	.95
TP	TP	TP	TP	TP	TP	TP	TP	FP	FP

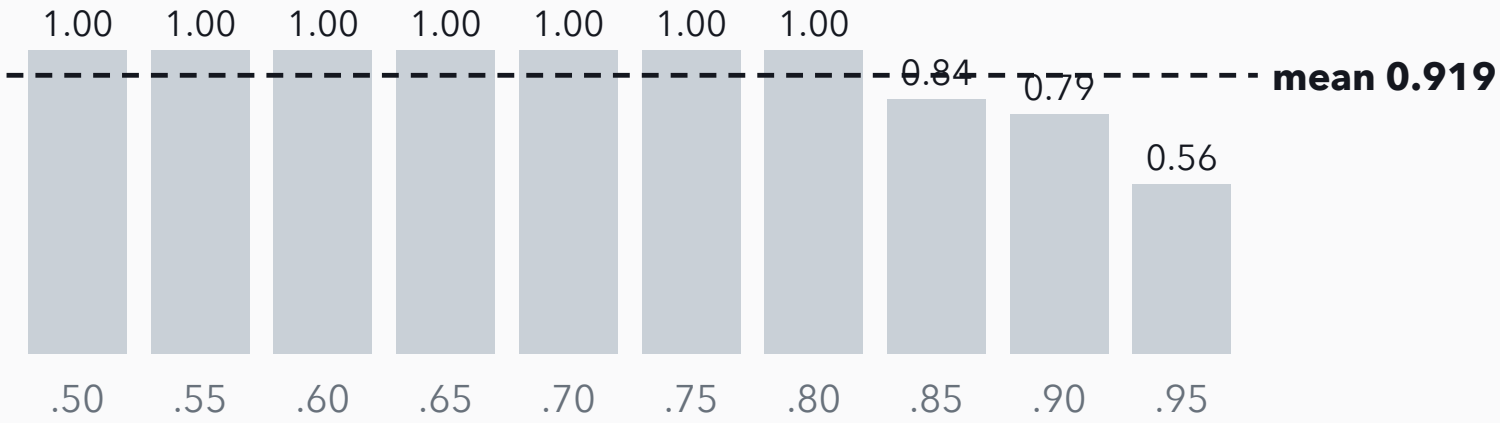
> Part 1's 13-pixel pedestrian: YOLOv8n's box is a pixel off, IoU 0.87. A true positive up to a threshold of 0.85; a false positive at 0.90 and 0.95.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

COCO's AP: the same, at ten IoU thresholds



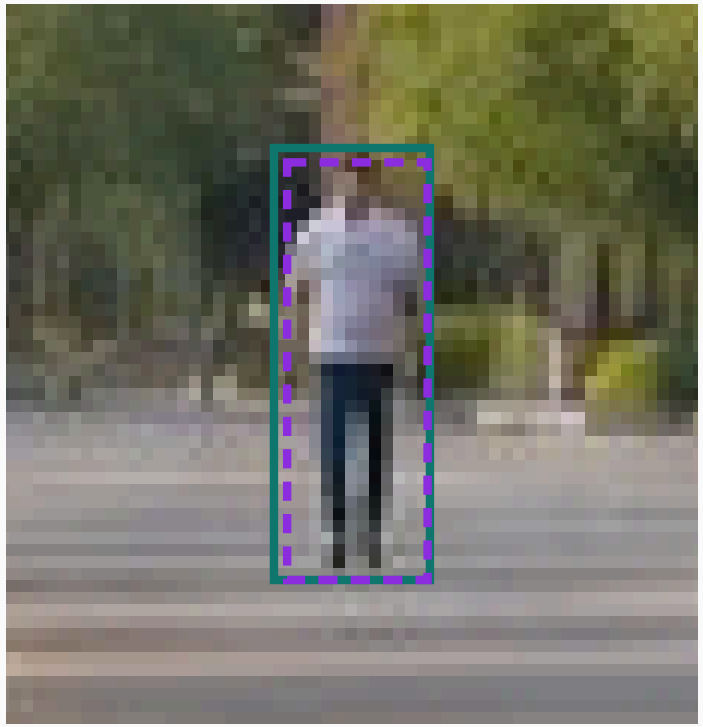


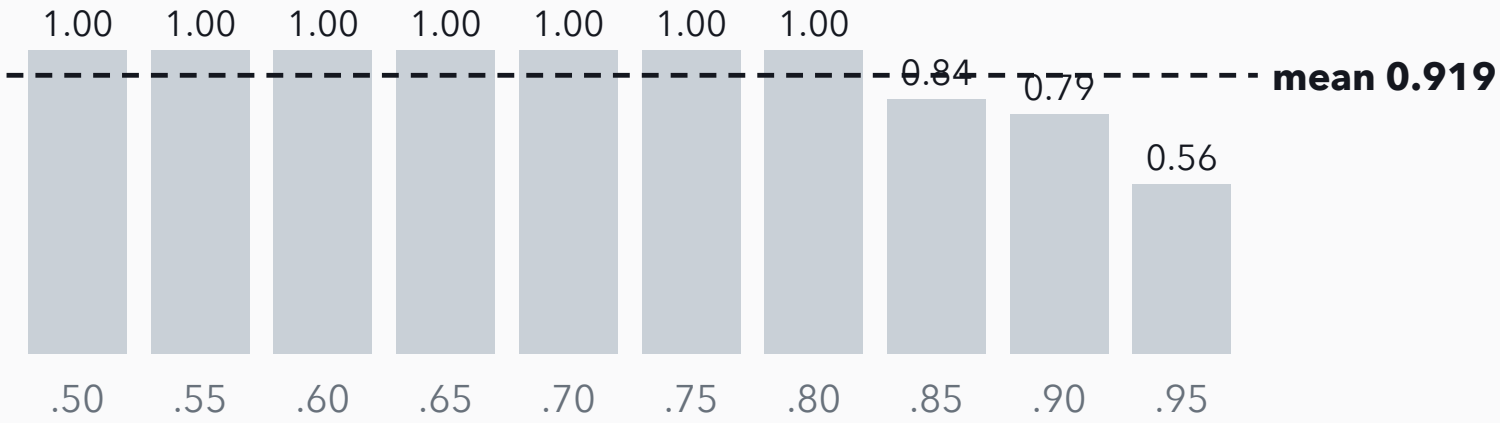
Person AP at each of COCO's ten thresholds: 1 up to 0.80, then 0.84, 0.79, 0.56, as boxes that are slightly loose stop counting.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

COCO's AP: the same, at ten IoU thresholds





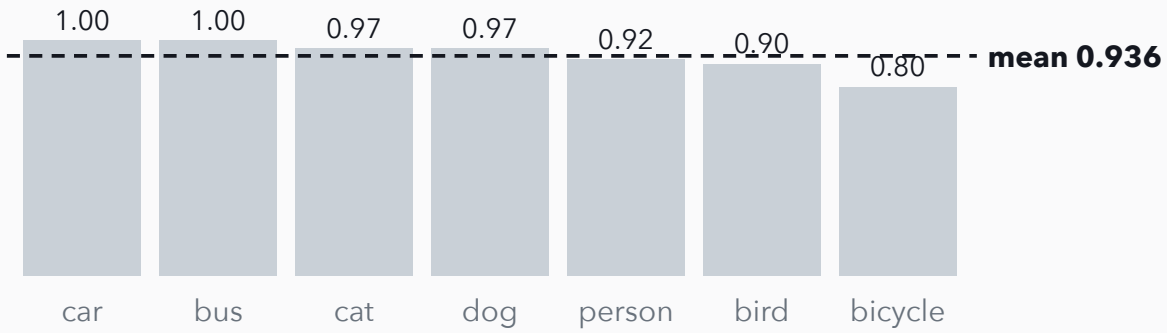
$$\mathrm{AP}_{[.5:.95]} = \frac{1}{10}\sum_{t}\mathrm{AP}_t = 0.919$$

> COCO's headline AP averages the ten: 0.919. It rewards boxes that are tight, not just roughly right.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

▶ Notebook, part 8

mAP: average the AP over the classes



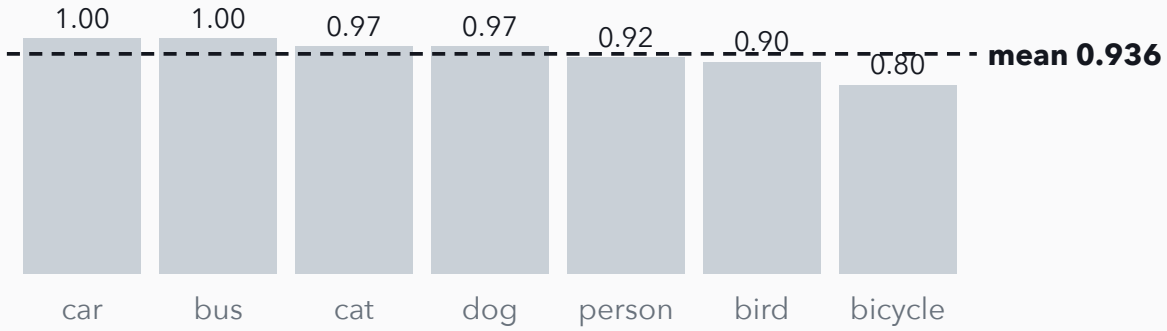
$$\text{mAP} = \frac{1}{C}\sum_{c=1}^{C} \text{AP}_c = 0.936$$

> Do the same for every class and average: mAP. YOLOv8n on our eight photos ranges from 0.80 for bicycles to 1 for buses and cars: mAP 0.936.

Our evaluator on data/annotations.json; COCO number: Ultralytics YOLOv8 documentation.

▶ Notebook, part 8

mAP: average the AP over the classes



IoU 0.5 only: every class 1, $\mathrm{mAP}_{50} = 1.000$

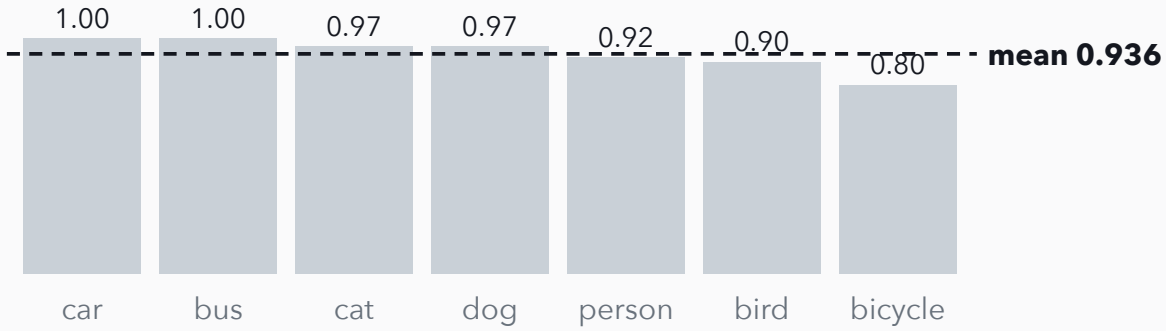
$$\mathrm{mAP} = \frac{1}{C}\sum_{c=1}^{C} \mathrm{AP}_c = 0.936$$

> At IoU 0.5 alone every class scores 1, so mAP@0.5 is 1.000. On clean photos only the strict average tells detectors apart.

Our evaluator on data/annotations.json; COCO number: Ultralytics YOLOv8 documentation.

▶ Notebook, part 8

mAP: average the AP over the classes



$$\text{mAP} = \frac{1}{C}\sum_{c=1}^{C} \text{AP}_c = 0.936$$

$$\text{IoU } 0.5 \text{ only: every class } 1, \quad \text{mAP}_{50} = 1.000$$

YOLOv8n	mAP@0.5	AP@[.5:.95]
our 8 photos, 7 classes, 31 objects	1.000	0.936
COCO validation, 5,000 photos, 80 classes (published)	not given	0.373

> COCO's 5,000 cluttered photos and 80 classes give YOLOv8n 0.373. Our handful of clean photos is a sanity check, not a benchmark.

Our evaluator on data/annotations.json; COCO number: Ultralytics YOLOv8 documentation.

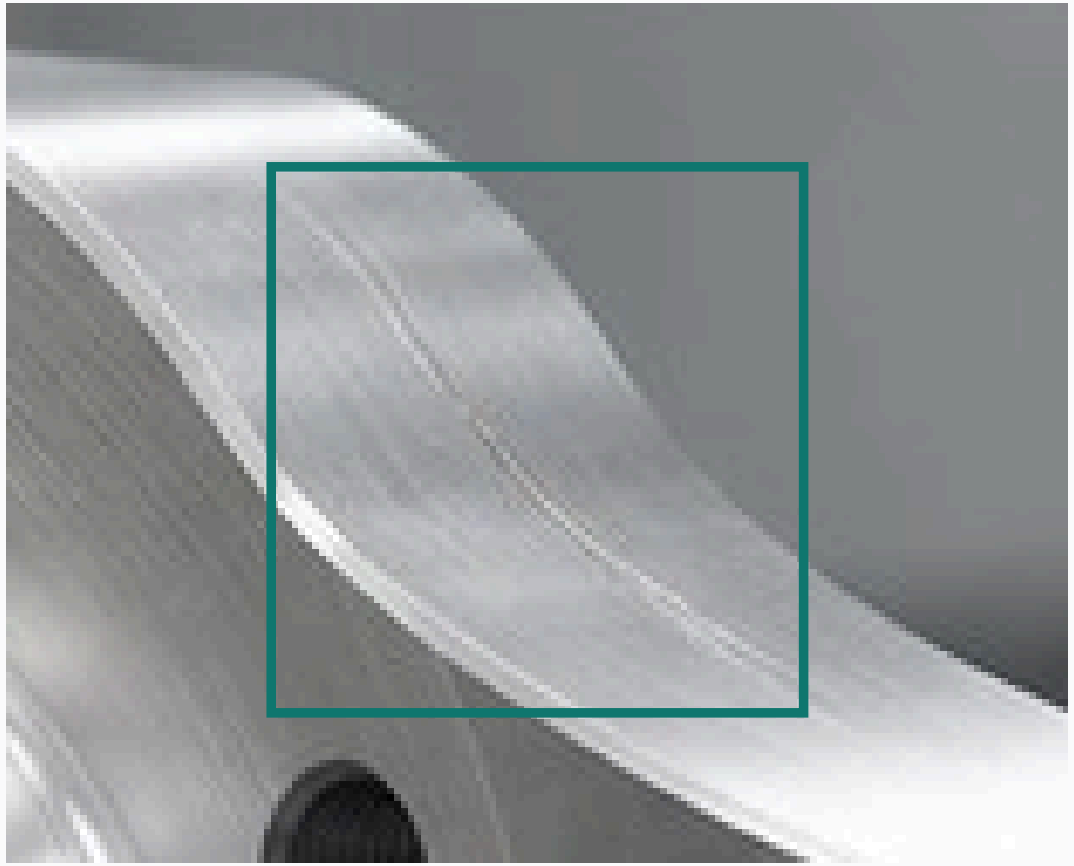
▶ Notebook, part 8

Evaluation in code

```
from torchmetrics.detection import MeanAveragePrecision
metric = MeanAveragePrecision(iou_type="bbox")
metric.update(preds=[{"boxes": boxes, "scores": scores, "labels": labels}],
              target=[{"boxes": gt_boxes, "labels": gt_labels}])
res = metric.compute()          # res["map"] = AP@[.5:.95], res["map_50"] = AP@0.5
```

> Libraries do the matching, ranking and averaging. The notebook also has a twenty-line version that reproduces the walk-through.

Boxes that turn: axis-aligned or oriented?

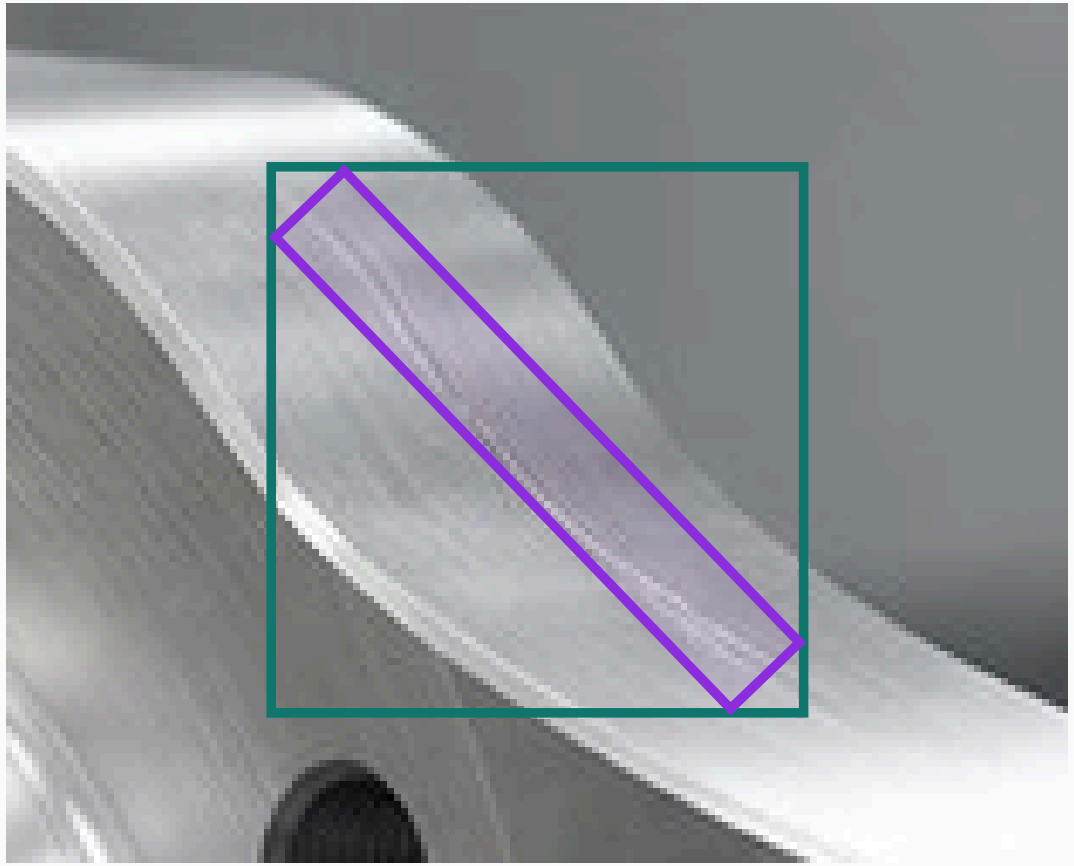


axis-aligned: $(x_1, y_1, x_2, y_2) = (579, 142, 657, 222)$

> Every box so far had its edges parallel to the photo's sides: an axis-aligned box, four numbers. Around this thin, slanted crack it is mostly metal.

Our oriented box on the course-owned photo (see SOURCES.md).

Boxes that turn: axis-aligned or oriented?



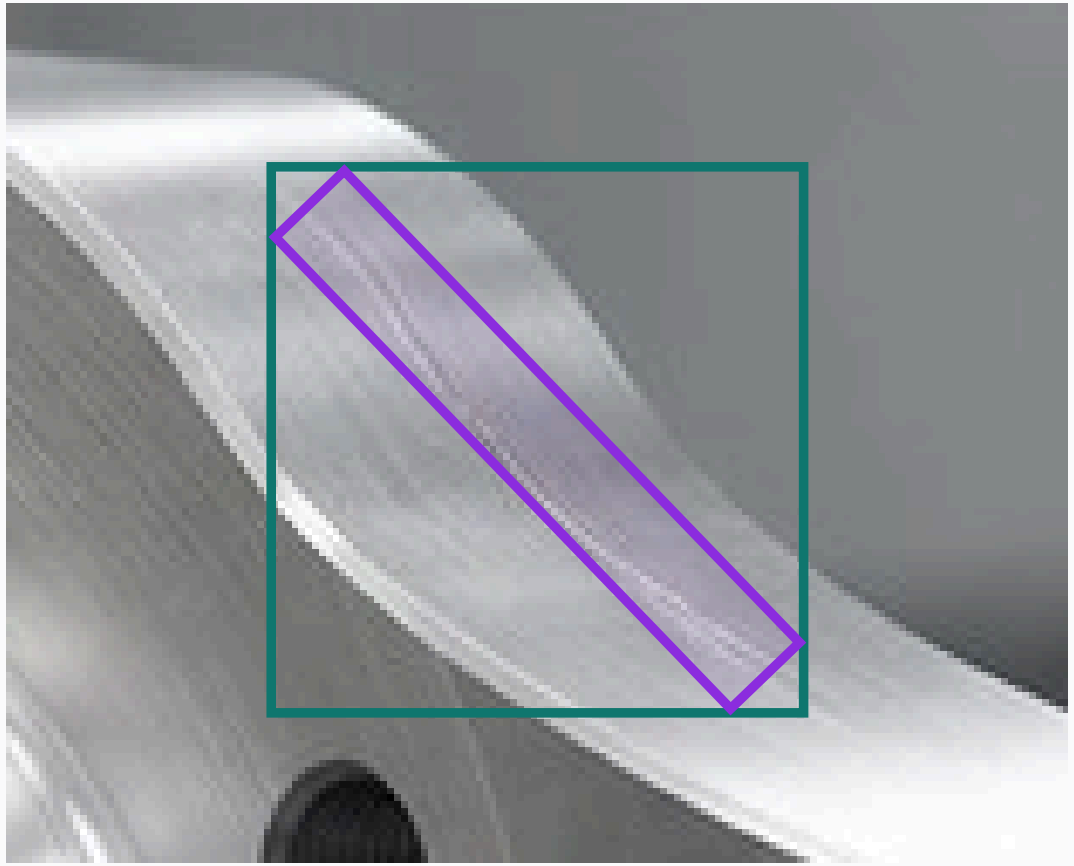
$$\text{axis-aligned: } (x_1, y_1, x_2, y_2) = (579, 142, 657, 222)$$

$$\text{oriented: } (c_x, c_y, w, h, \theta) = (618, 182, 96, 14, 46^\circ)$$

> An oriented box adds one number, the angle θ: (c_x, c_y, w, h, θ). It turns with the crack and hugs it.

Our oriented box on the course-owned photo (see SOURCES.md).

Boxes that turn: axis-aligned or oriented?



$$\text{axis-aligned: } (x_1, y_1, x_2, y_2) = (579, 142, 657, 222)$$

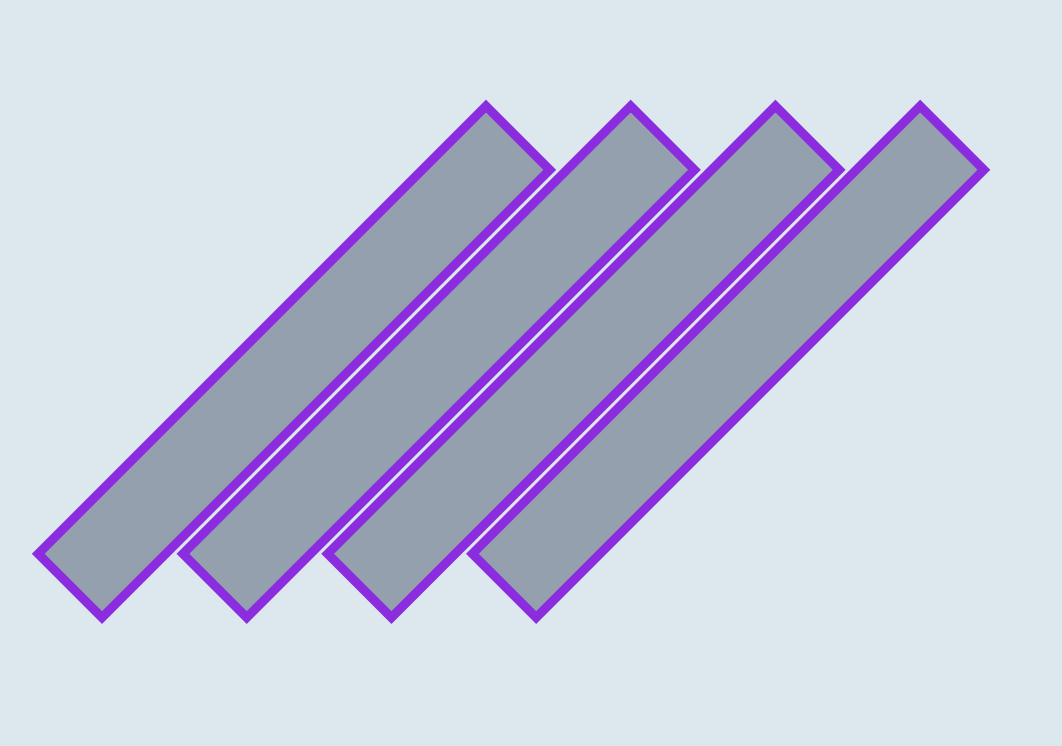
$$\text{oriented: } (c_x, c_y, w, h, \theta) = (618, 182, 96, 14, 46^\circ)$$

$$\frac{78 \times 80}{96 \times 14} \approx 4.6, \qquad \text{IoU} \approx 0.22$$

> The axis-aligned box is 4.6 times bigger: about 78% of it is not crack. The two boxes' IoU is only 0.22.

Our oriented box on the course-owned photo (see SOURCES.md).

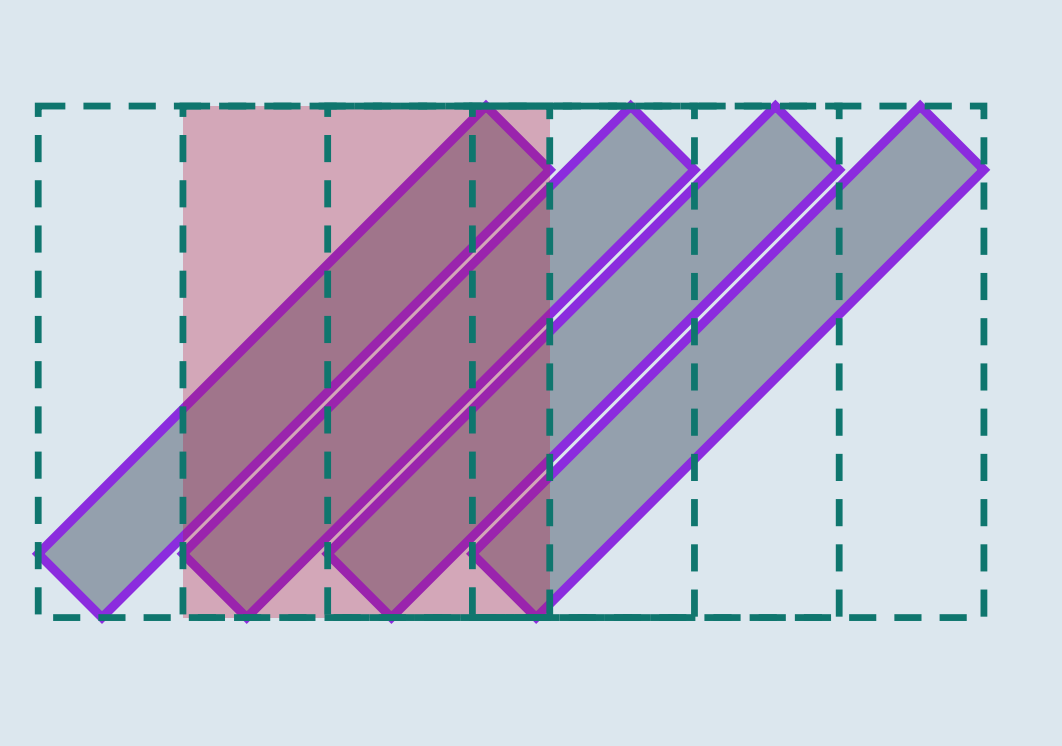
Packed, slanted objects need the angle



$$\mathrm{IoU}(\text{oriented boxes}) = 0$$

Seen from above, four ships moored side by side at 45°. Their oriented boxes do not touch: IoU 0 between neighbours.

Packed, slanted objects need the angle

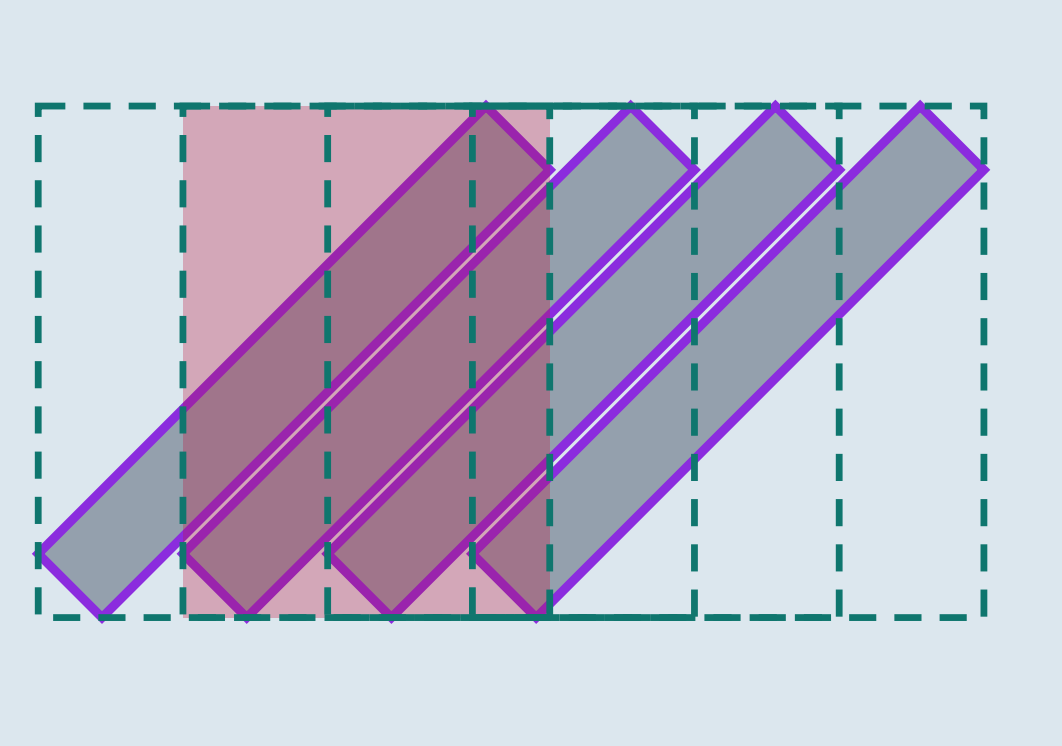


$$\text{IoU}(\text{oriented boxes}) = 0$$

$$\text{IoU}(\text{axis-aligned boxes}) \approx 0.56$$

Their axis-aligned boxes overlap a lot: IoU 0.56 between neighbours. NMS at 0.5 would keep one ship and delete the next as a duplicate.

Packed, slanted objects need the angle



$$\text{IoU}(\text{oriented boxes}) = 0$$

$$\text{IoU}(\text{axis-aligned boxes}) \approx 0.56$$

$$\text{YOLOv8-OBB, per cell: } (c_x, c_y, w, h, \theta)$$

So detectors for aerial photos, scene text and documents predict the angle too: YOLOv8-OBB writes (c_x, c_y, w, h, θ) for every cell.

One question, one answer

Question: a network's output has a fixed shape; the answer is a list of any length, in no order.

> The lecture's question: how can a network with a fixed output return a variable, unordered set?

One question, one answer

Question: a network's output has a fixed shape; the answer is a list of any length, in no order.

1. A candidate at every place: each cell of three grids writes a box and 80 class scores.
2. In training, the cells near each object learn it; every other cell learns "nothing here".
3. At test time, keep the confident cells, then NMS keeps one box per object.

> In the dense answer, the score filter and NMS decide the list's length, after the network.

Three uses of IoU, easy to confuse

IoU used for	pairs	when
assignment	cells ↔ annotated objects	training

> Training: pair cells with annotated objects. YOLOv8 ranks the cells by how well their boxes fit, measured with IoU.

Three uses of IoU, easy to confuse

IoU used for	pairs	when
assignment	cells ↔ annotated objects	training
NMS	predictions ↔ predictions	test time

> Test time: NMS compares predictions with each other. Overlap above 0.7 means a duplicate.

Three uses of IoU, easy to confuse

IoU used for	pairs	when
assignment	cells ↔ annotated objects	training
NMS	predictions ↔ predictions	test time
evaluation	predictions ↔ annotated objects	measuring

> Measuring: a detection is right if its IoU with an annotation is at least 0.5. All three pairings use the same measure.

Back to the six jobs: the three everyday ones

job	what is hard	what to do
road	far pedestrians are tiny	
crowd	people stand close and overlap	
camera trap	small animals get unsure scores, like the bird	

> A pretrained COCO detector already knows these classes. What is hard in each job?

Back to the six jobs: the three everyday ones

job	what is hard	what to do
road	far pedestrians are tiny	the fine grid (Part 7); follow boxes across frames
crowd	people stand close and overlap	check the NMS threshold (Part 6)
camera trap	small animals get unsure scores, like the bird	a lower cut-off (Part 8); more photos of small animals

> What remains is tuning: the grids, the NMS threshold, the score cut-off.

Back to the six jobs: the three new ones

job	new classes	what is hard	what to do
microscope	cell types	cells touch	
warehouse	parcel	many sizes	
factory	crack, scratch, dent	thin, small defects	

> COCO has none of these classes, so no pretrained detector finds them as it is.

Back to the six jobs: the three new ones

job	new classes	what is hard	what to do
microscope	cell types	cells touch	label cells, fine-tune; NMS must keep touching cells
warehouse	parcel	many sizes	label parcels, fine-tune the same detector
factory	crack, scratch, dent	thin, small defects	higher resolution, the fine grid; measure recall

> Label a few hundred photos with boxes, fine-tune the same detector, and measure precision and recall on photos it has not seen.

Next: beyond boxes



A box is a rough answer. Next lecture: which pixels belong to each object (masks), keypoints, depth, and boxes found from words: “the dog on the right”.

Beyond Boxes · Language-grounded vision

Practice: the notebook runs every number in this lecture. Appendices: A, DETR · B, more depth.

> Boxes are the coarsest description of where an object is. Next: masks, keypoints, and finding objects named in words.

Masks: YOLOv8n-seg run on the photo (scripts/run_models.py).

▶ Notebook, appendix A

Appendix A · DETR: a fixed set of slots, matched to objects

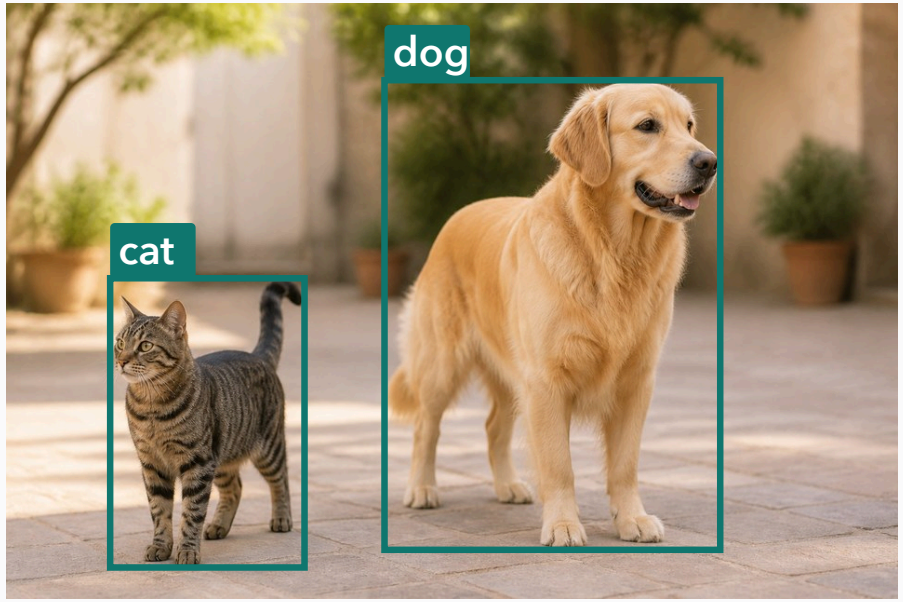
Another way to output a list: a fixed set of slots, matched to objects before the loss, so no NMS is needed.

1. Why slots are hard: too few for a crowd, and no order.
2. Follow a photo through DETR to 100 predictions, and see where two queries looked.
3. Watch duplicates vanish inside the decoder.
4. Build the matching cost one cell at a time, solve the matching, and compute the loss.

> The other answer to the lecture's question: DETR, 100 slots that read the image with attention, matched to objects instead of cleaned up by NMS.

▶ Notebook, appendix A

Give the head K slots



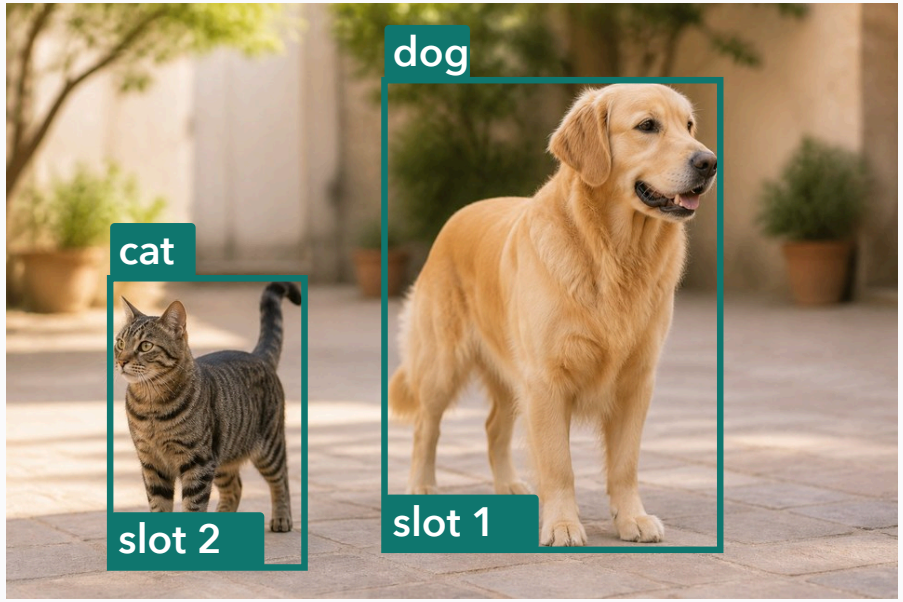
	class	c_x	c_y	w	h
slot 1					
slot 2					
slot 3					
slot 4					
slot 5					

Give the head K slots instead of one. Each slot holds a record like the one-object target: a class and four box numbers. Here K = 5.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Give the head K slots



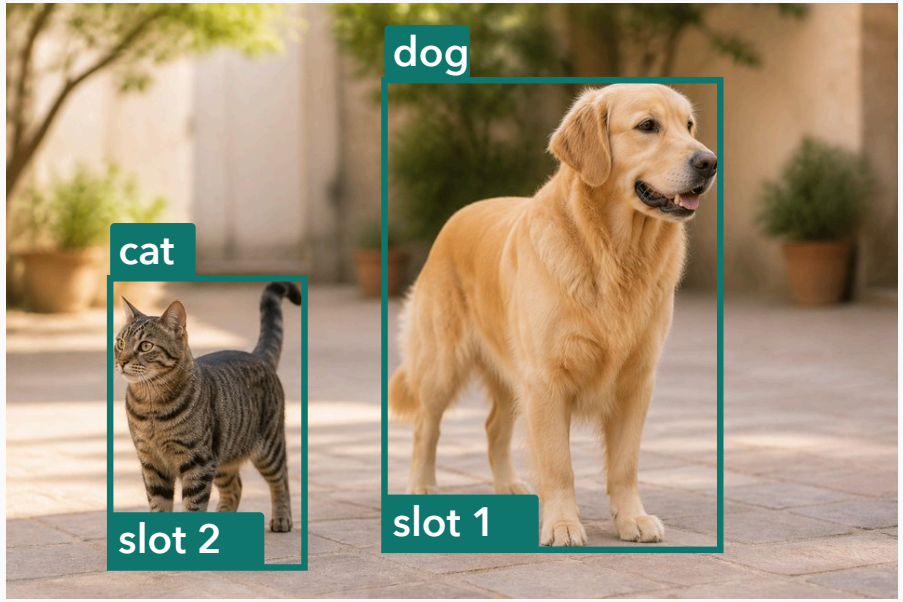
	class	c_x	c_y	w	h
slot 1	dog	0.612	0.524	0.376	0.789
slot 2	cat	0.226	0.705	0.218	0.486
slot 3					
slot 4					
slot 5					

The target for our photo puts the dog in one slot and the cat in another, with their boxes in normalised units.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Give the head K slots



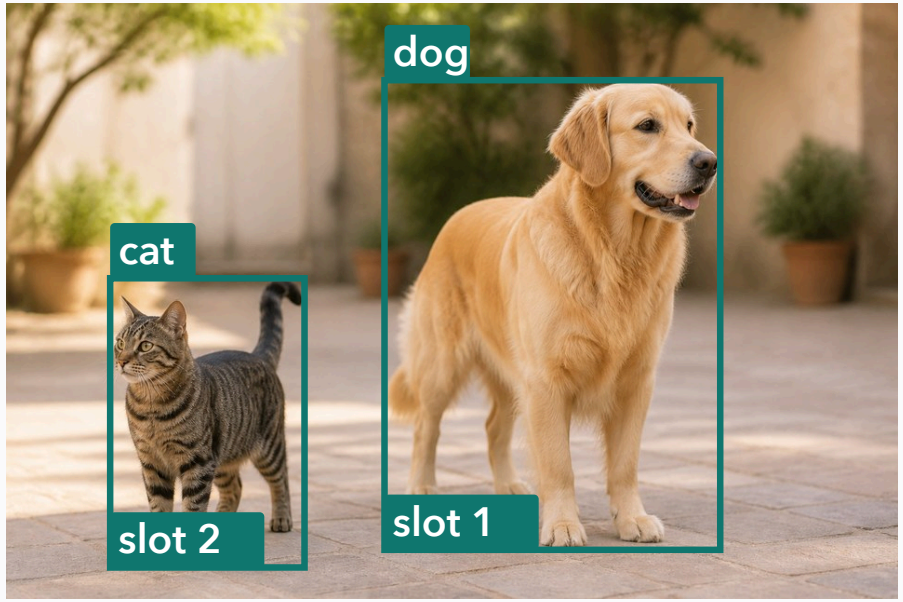
	class	c_x	c_y	w	h
slot 1	dog	0.612	0.524	0.376	0.789
slot 2	cat	0.226	0.705	0.218	0.486
slot 3	no object	?	?	?	?
slot 4	no object	?	?	?	?
slot 5	no object	?	?	?	?

> The three spare slots need a way to say “nothing here”: one more class, no object. Their boxes have no right answer, so they are “?”.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Give the head K slots



	class	c_x	c_y	w	h
slot 1	dog	0.612	0.524	0.376	0.789
slot 2	cat	0.226	0.705	0.218	0.486
slot 3	no object	?	?	?	?
slot 4	no object	?	?	?	?
slot 5	no object	?	?	?	?

$$K \times (C + 1 + 4) = 5 \times (80 + 1 + 4) = 425$$

> However many objects a photo has, the output keeps one size: K slots, each with C class scores, a no-object score and four box numbers.

Our annotation of the courtyard photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Problem 1: more objects than slots



11

people in the photo

5

slots, if K = 5

> Problem 1: the crowd photo has 11 people. A head with K = 5 slots can report at most 5 of them, however well it is trained.

Our annotation of the crowd photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Problem 1: more objects than slots



100

slots, if K = 100

89

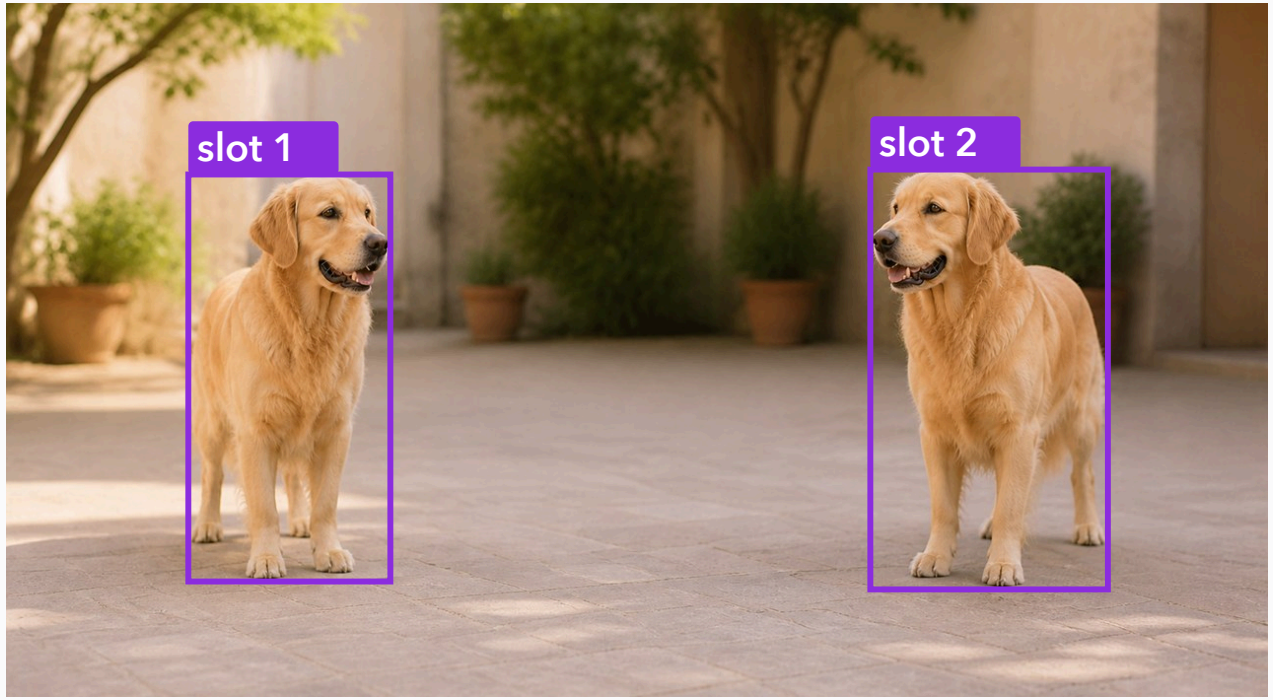
of them say “no object” here

> So make K larger than any photo needs, say 100. Here 11 slots would hold a person, and the other 89 would say “no object”.

Our annotation of the crowd photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Problem 2: which dog belongs in slot 1?



> Suppose the head is perfect: slot 1 boxes the left dog exactly, slot 2 the right dog. Nothing could be more right.

Our annotation of the two-dog photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Problem 2: which dog belongs in slot 1?



$$\text{by position:}\quad 0.603 + 0.603 = 1.206$$

> But the annotator happened to list the right dog first. Compare slot k with entry k: each slot is a whole dog away from its entry, 0.603 in normalised units.

Our annotation of the two-dog photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Problem 2: which dog belongs in slot 1?



$$\text{by position: } 0.603 + 0.603 = 1.206$$

$$\text{by object: } 0 + 0 = 0$$

> Compare each slot with the dog it actually boxed: both differences are 0.

Our annotation of the two-dog photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Problem 2: which dog belongs in slot 1?



$$\text{by position: } 0.603 + 0.603 = 1.206$$

$$\text{by object: } 0 + 0 = 0$$

Same perfect answer, two losses. The loss must not depend on the list's order.

> Nothing about the prediction changed, only the order of the list. A loss that pairs outputs with targets by position punishes a perfect answer.

Our annotation of the two-dog photo (data/annotations.json). The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Two ways to decide which output answers for which object

A · by place

- Each output owns a place in the image
- Outputs at the dog's place answer for the dog

Parts 5 to 7: YOLO

> Fix A gives every output a fixed place in the image, and whatever is there is its job, so order cannot matter.

▶ Notebook, appendix A

Two ways to decide which output answers for which object

A · by place

- Each output owns a place in the image
- Outputs at the dog's place answer for the dog

Parts 5 to 7: YOLO

B · by matching

- Keep free slots
- Before the loss, match slots to objects one to one, at the lowest total cost

the rest of this appendix: DETR

> Fix B keeps free slots and, before each loss, matches them to objects at the lowest total cost, so order cannot matter either.

▶ Notebook, appendix A

Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.

> Problem 1 was capacity: use K = 100 slots. Unused slots predict a 92nd class, “no object”.

▶ Notebook, appendix A

Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.
2. **Each slot reads the whole image** with cross-attention, as the VLM lectures did.

> A slot needs a way to look at the photo: give it a learned query vector that attends to the image features.

▶ Notebook, appendix A

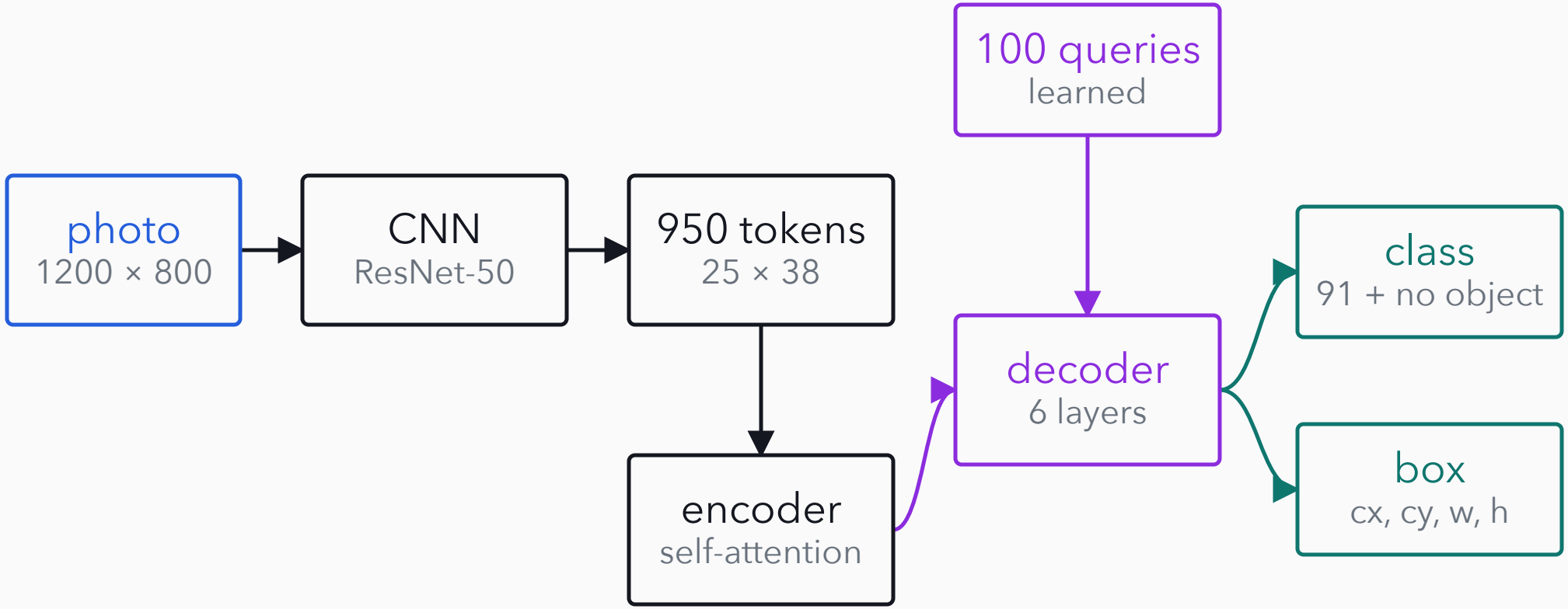
Back to the slots, with three changes

1. **100 slots**, more than almost any photo has objects; each may say “no object”.
2. **Each slot reads the whole image** with cross-attention, as the VLM lectures did.
3. **Match before the loss**: pair slots with objects so that the total cost is lowest.

> Problem 2 was order: before computing the loss, find the cheapest one-to-one pairing of slots and objects. Order in the annotation list stops mattering.

▶ Notebook, appendix A

DETR, from photo to 100 predictions

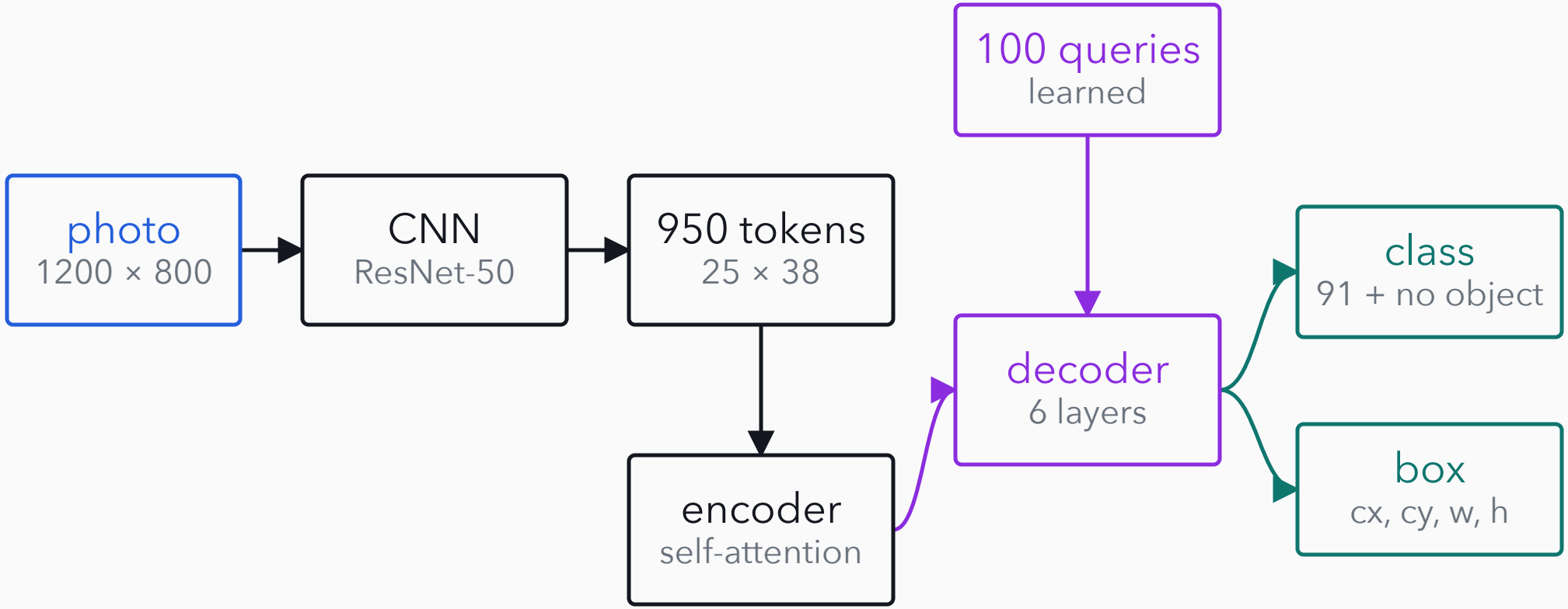


A ResNet-50 turns the photo, resized to 1200 × 800, into a 25 × 38 grid; each position becomes a token of 256 numbers. 950 tokens.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR, from photo to 100 predictions

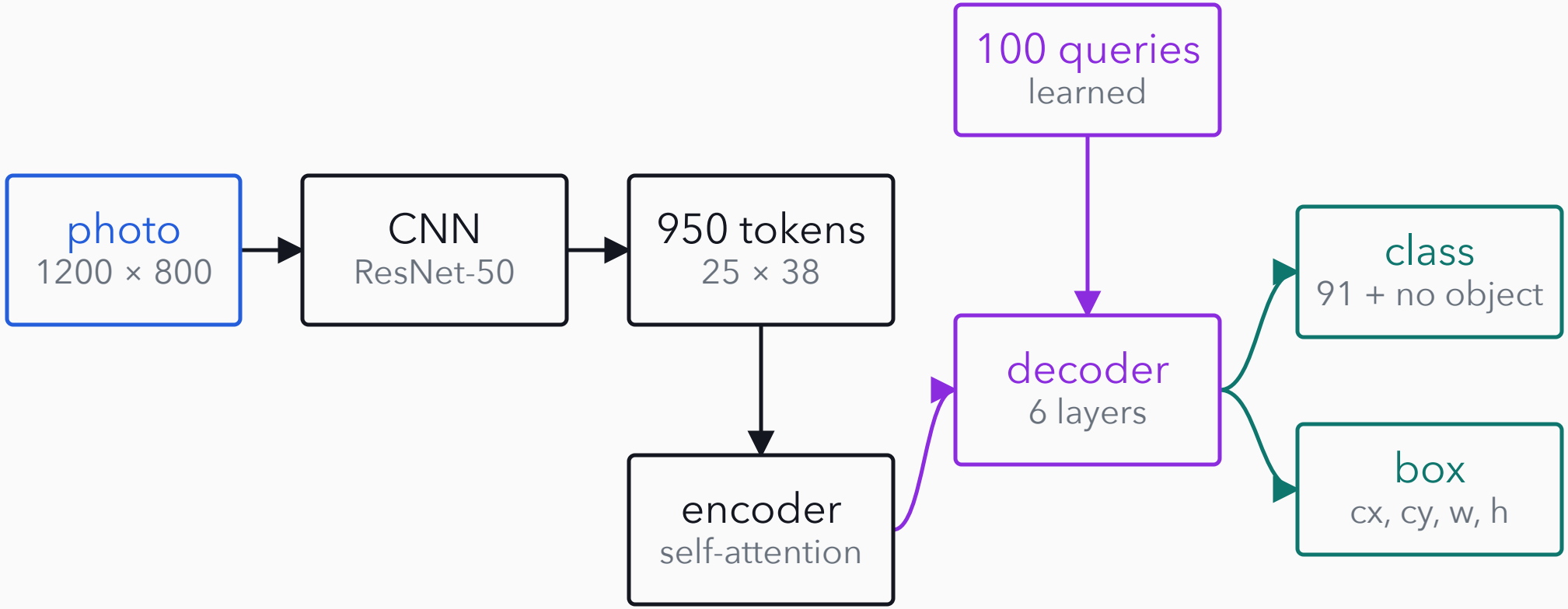


A transformer encoder lets the 950 tokens attend to each other, so each token knows about the rest of the photo.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR, from photo to 100 predictions

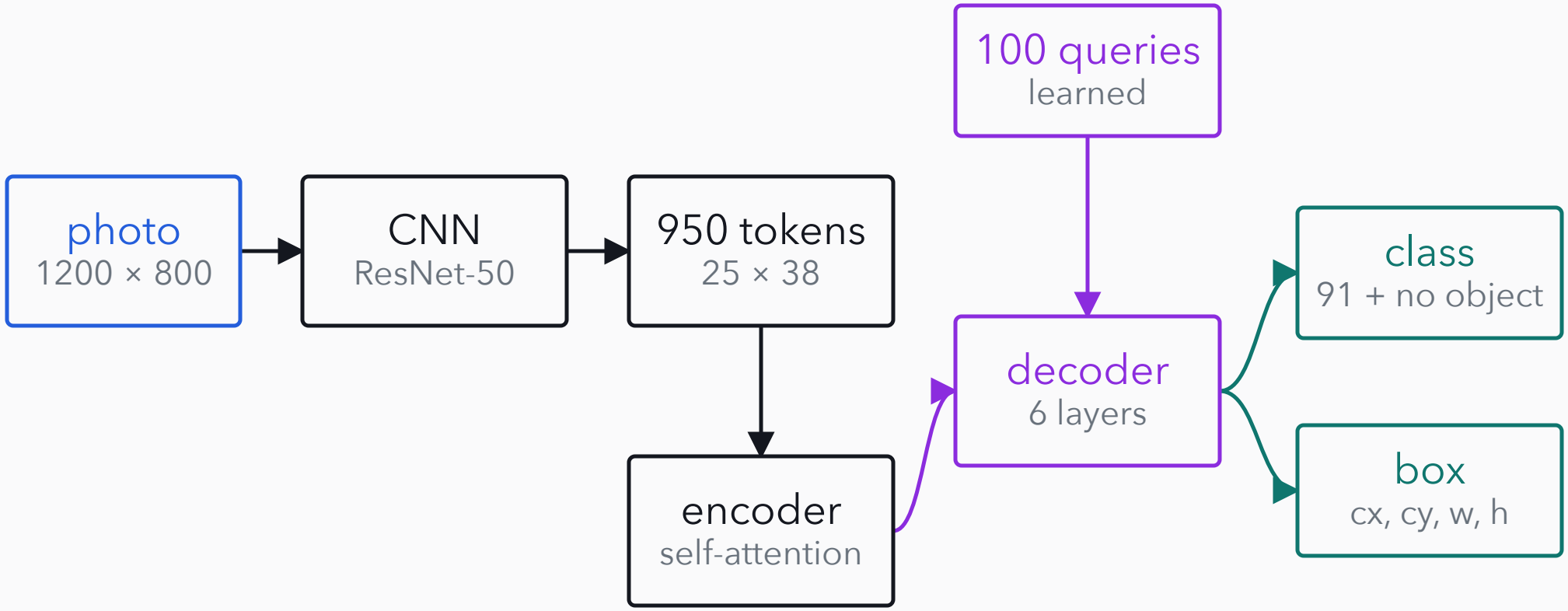


The decoder starts from 100 learned query vectors. In each of 6 layers, the queries attend to each other, then to the 950 image tokens (cross-attention).

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR, from photo to 100 predictions



> Two small heads read each of the 100 outputs: a class among 91 COCO ids plus “no object”, and a box as normalised (c_x, c_y, w, h).

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

A query is learned once and reused for every photo

Stored: 100 query vectors, 256 numbers each. Parameters, trained like weights; the same 100 for every photo.

> The 100 queries are stored parameters: 100 × 256 numbers learned in training, identical for every photo.

▶ Notebook, appendix A

A query is learned once and reused for every photo

Stored: 100 query vectors, 256 numbers each. Parameters, trained like weights; the same 100 for every photo.

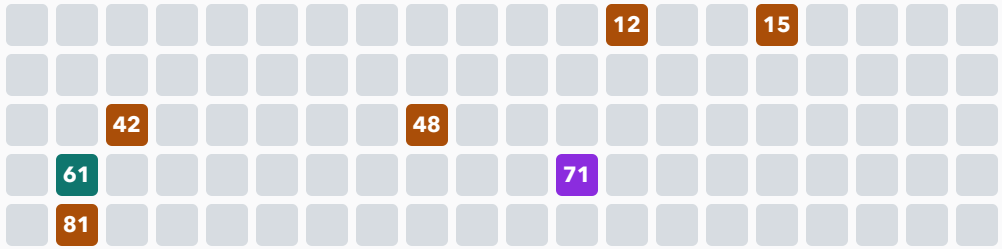
Computed: what each query writes depends on the photo, because it reads the photo through cross-attention.

As in the VLM lectures: stored query slots, computed per-image summaries.

> Their outputs are computed: the same query writes a dog box for one photo and “no object” for another, depending on what it read.

▶ Notebook, appendix A

DETR on the courtyard: 100 slots, most of them empty



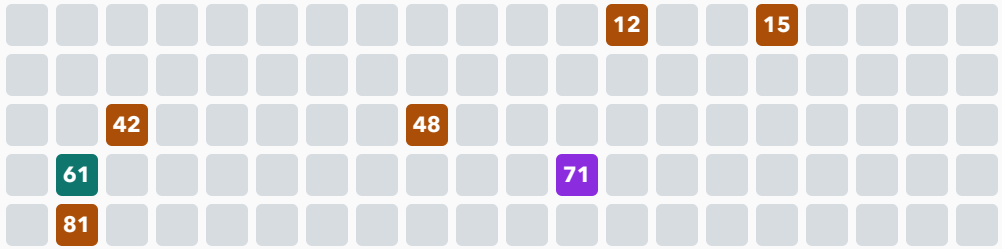
Grey: “no object”. 88 of the 100 slots give it more than 0.9.

> One square per query, numbered 0 to 99. Coloured if the query names an object; grey if it says “no object”.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

DETR on the courtyard: 100 slots, most of them empty



Grey: “no object”. 88 of the 100 slots give it more than 0.9.

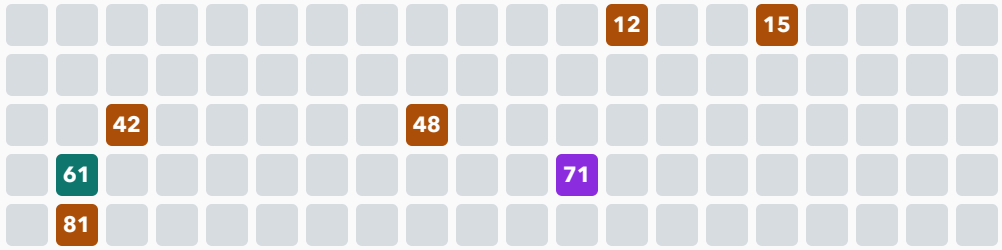
query	class	p	p(no object)
71	dog	0.999	0.001
61	cat	0.999	0.001
12	potted plant	0.998	0.002
81	potted plant	0.989	0.011
48	potted plant	0.973	0.027

> The busy ones: query 71 says dog (0.999), query 61 cat (0.999), others plants. Their numbers are arbitrary: training decided which slot does what.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

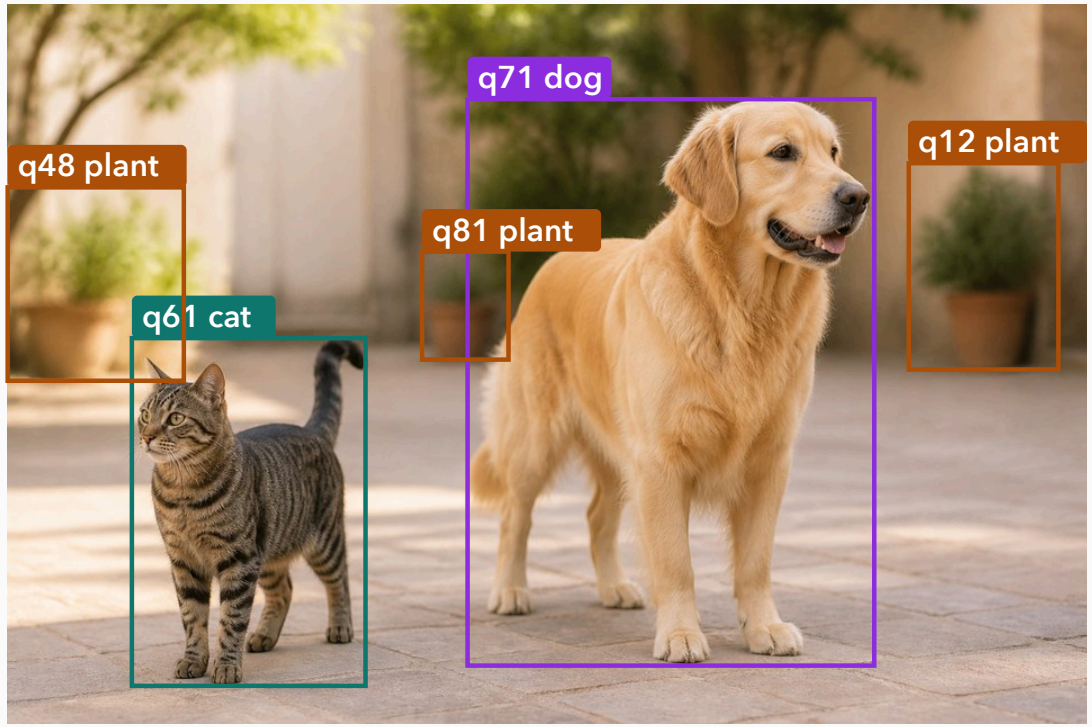
▶ Notebook, appendix A

DETR on the courtyard: 100 slots, most of them empty



Grey: “no object”. 88 of the 100 slots give it more than 0.9.

query	class	p	p(no object)
71	dog	0.999	0.001
61	cat	0.999	0.001
12	potted plant	0.998	0.002
81	potted plant	0.989	0.011
48	potted plant	0.973	0.027



> Their boxes: one per object. No duplicates to remove, and no NMS.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Where two queries looked



query 71 → dog

> Query 71's cross-attention in the last decoder layer, averaged over its 8 heads: where its weights over the 950 tokens went. Bright = much attention.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Where two queries looked



query 71 → dog



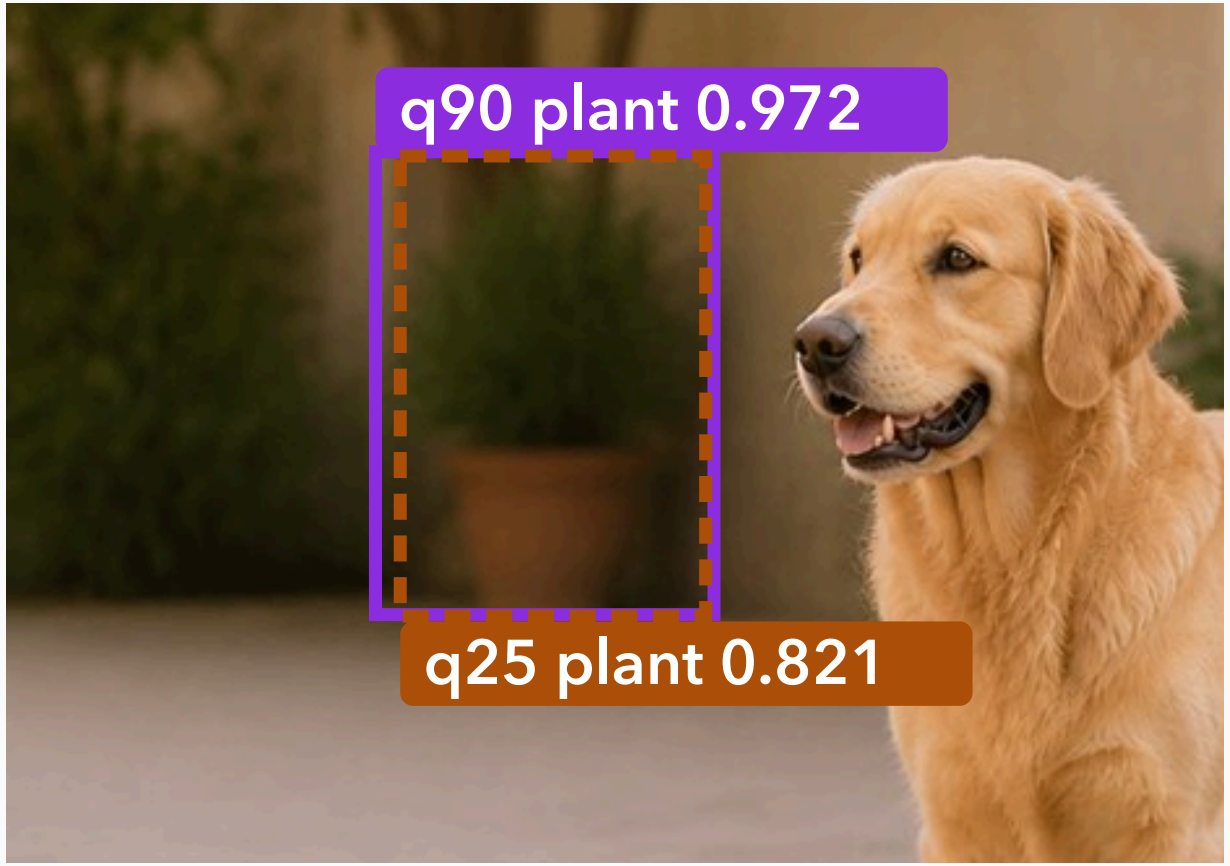
query 61 → cat

> Query 61, the cat: its attention sits on the cat. Each query attends mostly to the parts of the photo that fix its box: the extremities more than the middle.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Duplicates disappear inside the decoder



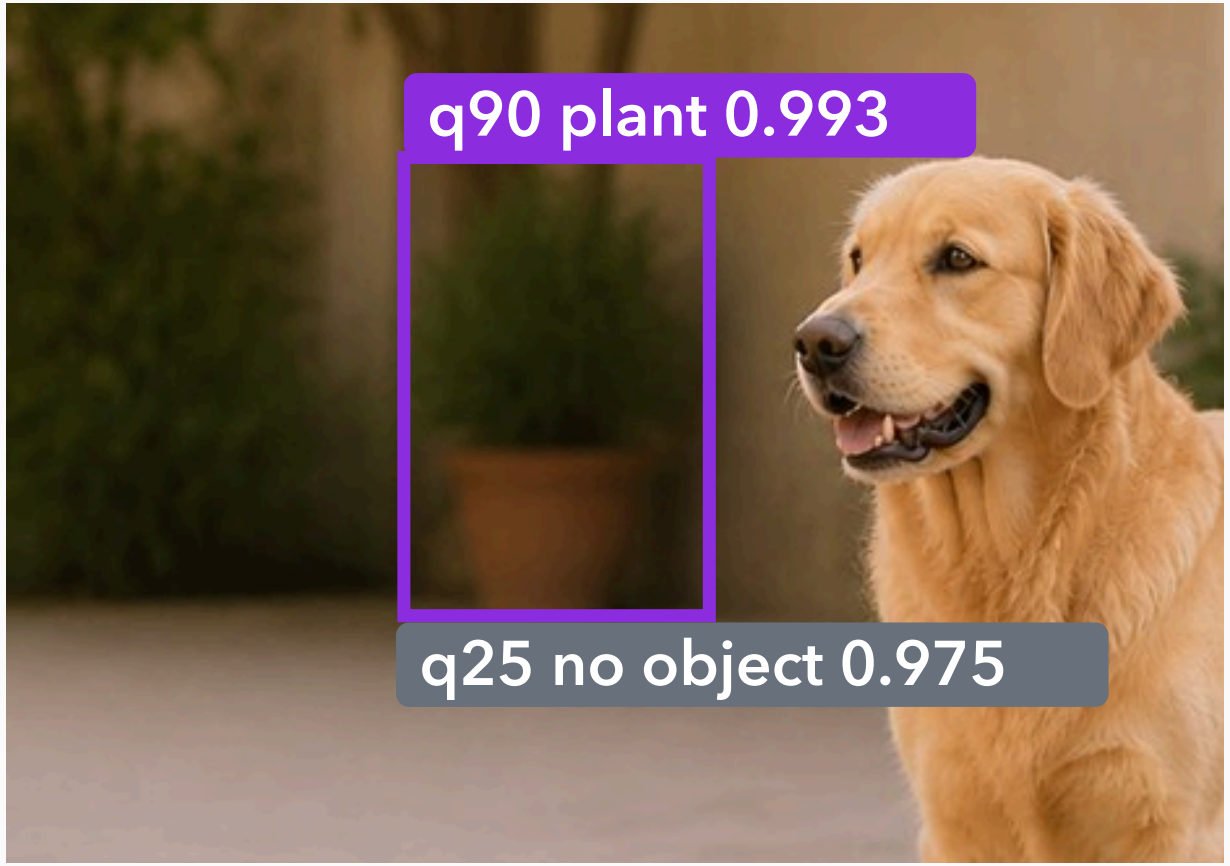
After layer 1: queries 90 and 25 both say “potted plant” for the same pot (IoU 0.891).

DETR’s heads can read the queries after any decoder layer. After layer 1, two queries describe the same pot in the two-dogs photo.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Duplicates disappear inside the decoder



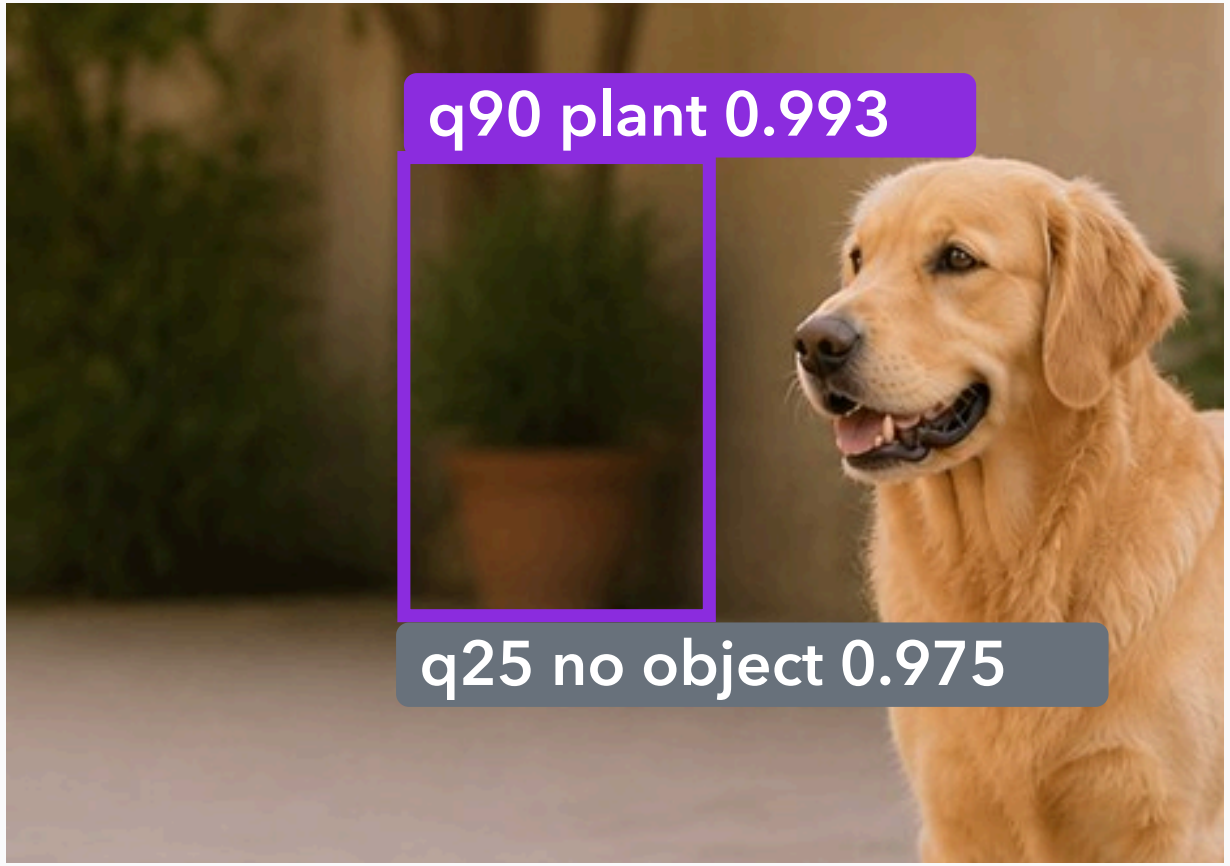
After layer 2: query 25 says “no object” (0.975); query 90 keeps the plant.

After layer 2, query 25 has dropped out: “no object” at 0.975. The duplicate was removed by the network, not by a post-processing rule.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Duplicates disappear inside the decoder



After layer 2: query 25 says “no object” (0.975); query 90 keeps the plant.

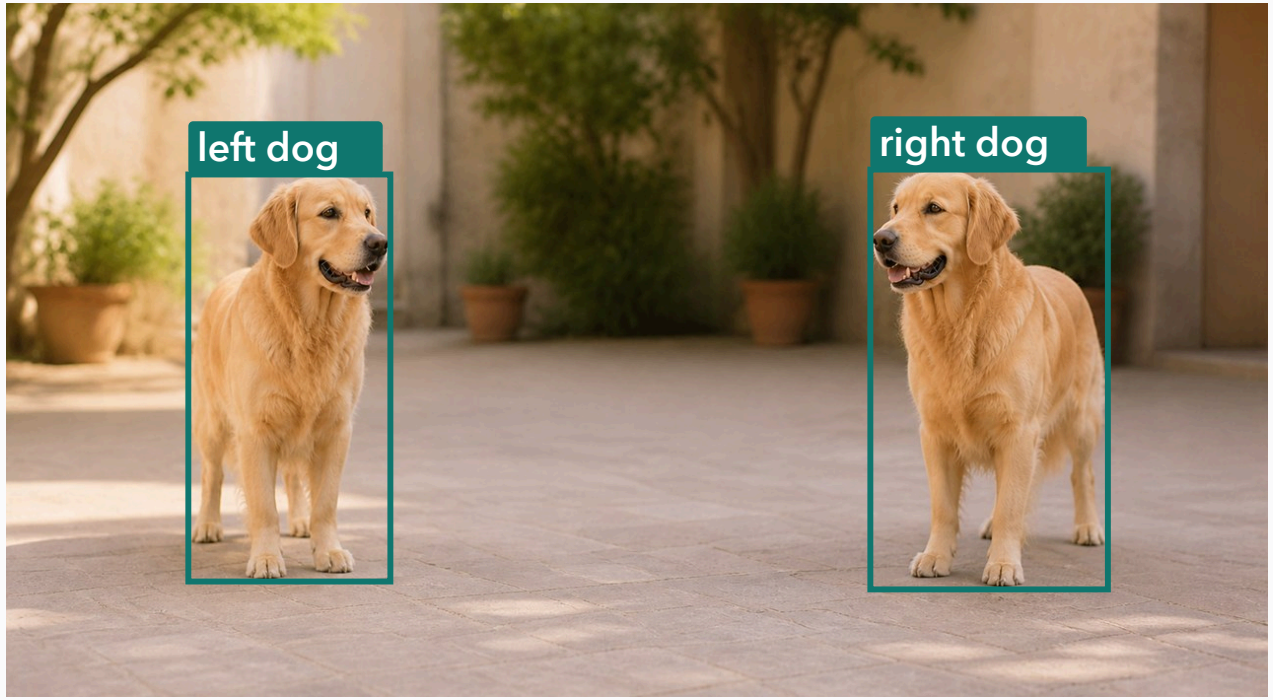
Queries attend to each other in every decoder layer, so one can see that another already has the pot. Training rewards exactly one of them per object (next slides).

> This works because training never rewards two queries for one object. That comes from the matching.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Training: which prediction should learn which dog?



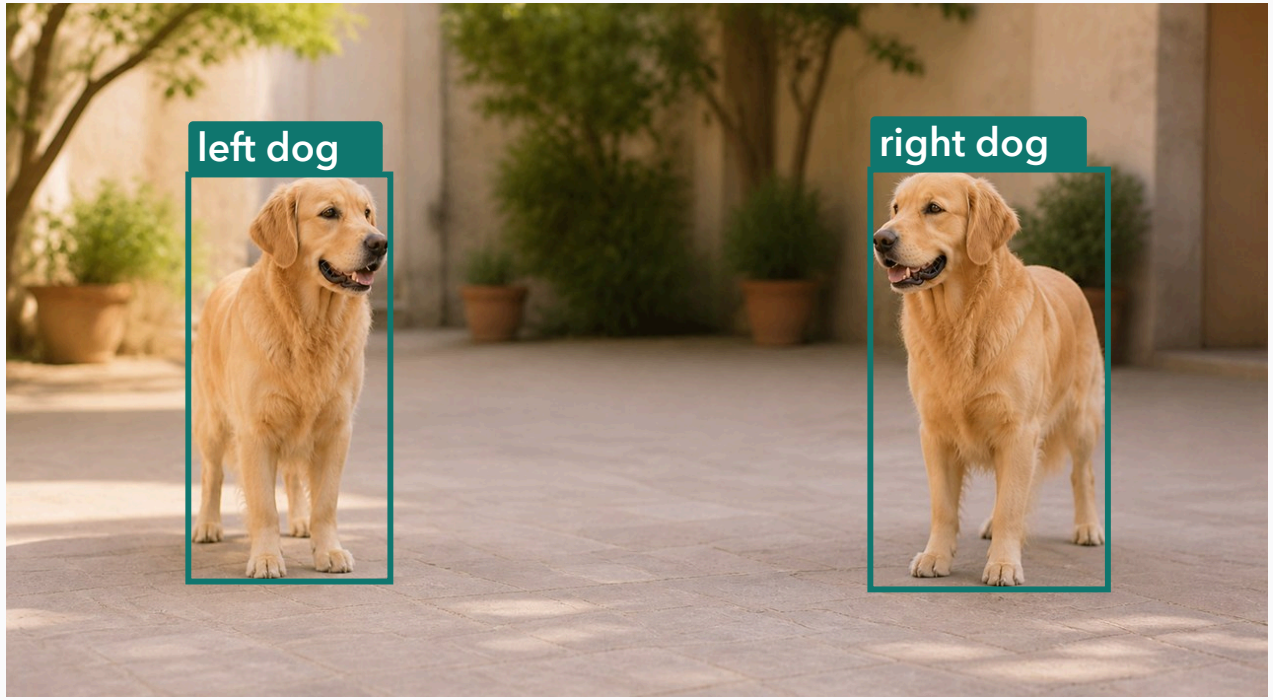
Two targets. 100 predictions. In Part 4, comparing slot k with list entry k punished a correct answer.

> Training photo: two annotated dogs. DETR made 100 predictions. Which prediction is compared with which dog?

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

Training: which prediction should learn which dog?



Two targets. 100 predictions. In Part 4, comparing slot k with list entry k punished a correct answer.

Instead: give every (prediction, target) pair a **cost**, then choose the one-to-one pairing with the lowest total.

> Score every pairing with a cost: low when the prediction has the right class and a box close to the target's. Then pick, among all one-to-one pairings, the cheapest.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

▶ Notebook, appendix A

One more measure: generalised IoU



Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

> IoU cannot tell a near miss from a far miss: both are 0. A loss built on IoU gives no direction to move.

▶ Notebook, appendix A

One more measure: generalised IoU



Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

$$\mathrm{GIoU} = \mathrm{IoU} - \frac{\mathrm{area}(C) - \mathrm{area}(A \cup B)}{\mathrm{area}(C)}$$

C: the smallest box enclosing both. The emptier C is, the lower the score; GIoU runs from −1 to 1.

> Generalised IoU subtracts the empty fraction of the smallest box C that encloses both. Far apart, C is mostly empty and GIoU approaches −1.

▶ Notebook, appendix A

One more measure: generalised IoU



Any two boxes that do not touch have IoU 0: a near miss scores the same as a far miss.

$$\mathrm{GIoU} = \mathrm{IoU} - \frac{\mathrm{area}(C) - \mathrm{area}(A \cup B)}{\mathrm{area}(C)}$$

C: the smallest box enclosing both. The emptier C is, the lower the score; GIoU runs from −1 to 1.

DETR's right-dog query against the left dog: IoU 0, GIoU −0.529.

> Real example: query 98 predicts the right dog; against the left dog its IoU is 0 but its GIoU is −0.529.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

DETR's matching cost has three parts: minus the probability of the target's class, the L1 distance between boxes, and minus the GIoU. Weights 1, 5 and 2.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

Query 61 gives “dog” probability 0.999: the class part is −0.999.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

$$\text{box, L1: } |0.224 - 0.225| + \cdots = 0.004, \quad \times 5 = 0.020$$

$$\text{box, GIoU: } 0.984, \quad \times(-2) = -1.968$$

> Its normalised box differs from the left dog's by 0.004 in total (L1), and GIoU is 0.984.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

The cost of one pairing: query 61 and the left dog

$$C = -\,p(\text{class}) \;+\; 5\,\left\|\hat{b} - b\right\|_1 \;-\; 2\,\text{GIoU}(\hat{b}, b)$$

$$\text{class:}\;\; -\,p(\text{dog}) = -0.999$$

$$\text{box, L1: } |0.224 - 0.225| + \cdots = 0.004, \quad \times 5 = 0.020$$

$$\text{box, GIoU: } 0.984, \quad \times(-2) = -1.968$$

$$C = -0.999 + 0.020 - 1.968 = -2.95$$

Total −2.95. Low cost = good pairing; negative is fine, only comparisons matter.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Fill the table: the cost of every pairing

	q61 (dog)	q98 (dog)	q90 (plant)	q2 (no object)
left dog	−2.95	3.09	6.07	−1.71
right dog	?	?	?	?

Row 1: the left dog against four of the queries. q61 is cheapest; q2 is second: its box is on the left dog too.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Fill the table: the cost of every pairing

	q61 (dog)	q98 (dog)	q90 (plant)	q2 (no object)
left dog	−2.95	3.09	6.07	−1.71
right dog	3.08	−2.90	4.47	4.26

> Row 2: the right dog. q98 is cheapest.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Fill the table: the cost of every pairing

	q61 (dog)	q98 (dog)	q90 (plant)	q2 (no object)
left dog	**−2.95**	3.09	6.07	−1.71
right dog	3.08	**−2.90**	4.47	4.26

Cheapest pairing: left dog ↔ q61, right dog ↔ q98, total −5.85. Every other query is matched to nothing and learns “no object”.

> Choose one query per dog with the lowest total: q61 and q98. Here each dog’s cheapest query is different, so the choice is easy. It is not always.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Why not let each object grab its cheapest slot?

	q1	q2
left dog	**1**	2
right dog	2	**9**

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

> A made-up two-by-two case. Greedy: the left dog grabs q1, its cheapest at 1; the right dog is stuck with q2 at 9. Total 10.

▶ Notebook, appendix A

Why not let each object grab its cheapest slot?

	q1	q2
left dog	1	**2**
right dog	**2**	9

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

Better: left dog → q2 (2), right dog → q1 (2). Total **4**.

> The other pairing costs 2 + 2 = 4. Grabbing the cheapest first was wrong for the total.

▶ Notebook, appendix A

Why not let each object grab its cheapest slot?

	q1	q2
left dog	1	**2**
right dog	**2**	9

Greedy: the left dog takes its cheapest, q1 (1). The right dog must take q2 (9). Total **10**.

Better: left dog → q2 (2), right dog → q1 (2). Total **4**.

Trying every pairing is hopeless with 100 queries. The **Hungarian algorithm** (Kuhn, 1955) finds the cheapest one-to-one pairing in polynomial time: `scipy.optimize.linear_sum_assignment`.

> The best pairing must be found globally. The Hungarian algorithm does it efficiently; scipy has it.

▶ Notebook, appendix A

After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

A matched query learns its object: cross-entropy for the class, plus L1 and 1 − GIoU for the box. q61 is already almost perfect: tiny loss.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

$$\text{q2} \to \text{no object:} \;\; 0.1 \times (-\log 0.997) = 0.0003$$

> An unmatched query learns “no object”, its cross-entropy weighted by 0.1 because most queries are empty. q2 already says no object: almost no loss.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

After matching: the loss

$$\text{q61} \to \text{left dog:} \quad -\log 0.999 + 5 \times 0.004 + 2 \times (1 - 0.984)$$

$$= 0.001 + 0.020 + 0.032 = 0.053$$

$$\text{q2} \to \text{no object:} \;\; 0.1 \times (-\log 0.997) = 0.0003$$

$$\text{q90 (a plant)} \to \text{no object:} \;\; 0.1 \times (-\log 0.003) = 0.581$$

> q90 named a real plant, but plants are not in this target list, so it pays 0.58 for being right.

DETR (facebook/detr-resnet-50, Hugging Face) run on this photo by scripts/run_models.py.

▶ Notebook, appendix A

Why DETR needs no NMS

DETR · one-to-one

- Each object is matched to exactly one query
- A second query on the same object is trained to say “no object”
- So the trained network outputs one box per object

> Matching is one-to-one: in every training step, a duplicate query is told “no object”. The network learns not to duplicate, using self-attention among queries.

▶ Notebook, appendix A

Why DETR needs no NMS

DETR · one-to-one

- Each object is matched to exactly one query
- A second query on the same object is trained to say “no object”
- So the trained network outputs one box per object

YOLO · one-to-many

- About ten cells per object are trained as positives
- All ten learn to fire
- So NMS must delete nine of them

> The dense detector trained ten cells per object on purpose (more positives, more signal) and pays for it with NMS.

▶ Notebook, appendix A

DETR in code: matching, then inference

```
from scipy.optimize import linear_sum_assignment
cost = (-prob[:, gt_labels]                                  # (100, N)
        + 5 * torch.cdist(pred_boxes, gt_boxes, p=1)
        - 2 * generalized_box_iou(to_xyxy(pred_boxes), to_xyxy(gt_boxes)))
q_idx, gt_idx = linear_sum_assignment(cost.detach())         # one query per object
```

> The cost matrix is three lines: class, L1 and GIoU for every (query, object) pair. One call solves the matching. The loss then uses the matched pairs.

▶ Notebook, appendix A

DETR in code: matching, then inference

```
from scipy.optimize import linear_sum_assignment
cost = (-prob[:, gt_labels]                                  # (100, N)
        + 5 * torch.cdist(pred_boxes, gt_boxes, p=1)
        - 2 * generalized_box_iou(to_xyxy(pred_boxes), to_xyxy(gt_boxes)))
q_idx, gt_idx = linear_sum_assignment(cost.detach())         # one query per object
```

```
model = DetrForObjectDetection.from_pretrained("facebook/detr-resnet-50")
out = model(**processor(images=img, return_tensors="pt"))
dets = processor.post_process_object_detection(out, threshold=0.9,
                                               target_sizes=[img.size[::-1]])
```

> At test time there is nothing to match: run the model, keep the queries whose best class beats a threshold. There is no NMS step.

▶ Notebook, appendix A

Two answers to one question

	dense (YOLOv8)	set (DETR)
outputs	one record per cell: 12,600	100 learned slots
who answers for an object	cells near it, by place	the matched query
training targets	one-to-many: ~10 cells	one-to-one: Hungarian matching
at test time	score filter + NMS	threshold only
on our courtyard	4 detections	5 objects at p ≥ 0.9

> The main difference is how outputs are assigned to objects in training: by place, several per object, or by matching, one per object.

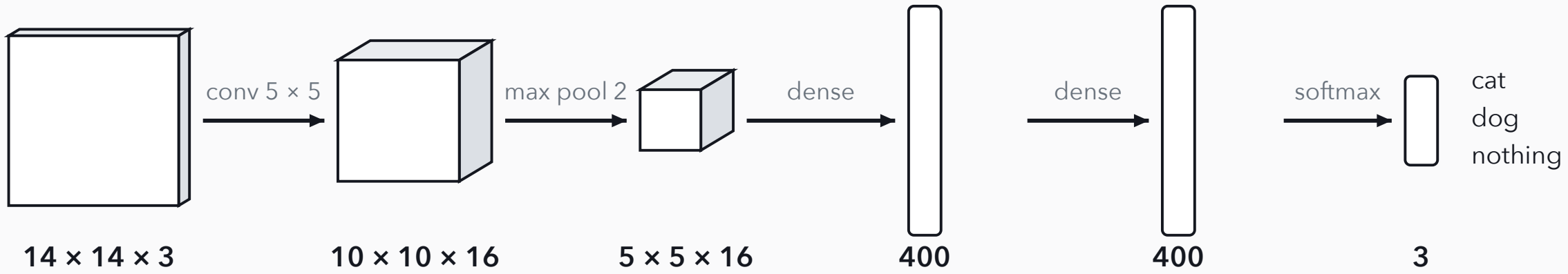
▶ Notebook, appendix A

Two answers to one question

	dense (YOLOv8)	set (DETR)
outputs	one record per cell: 12,600	100 learned slots
who answers for an object	cells near it, by place	the matched query
training targets	one-to-many: ~10 cells	one-to-one: Hungarian matching
at test time	score filter + NMS	threshold only
on our courtyard	4 detections	5 objects at $p \geq 0.9$
our 8 photos, AP@[.5:.95]	0.936	0.882
COCO validation, AP (published)	0.373	0.420

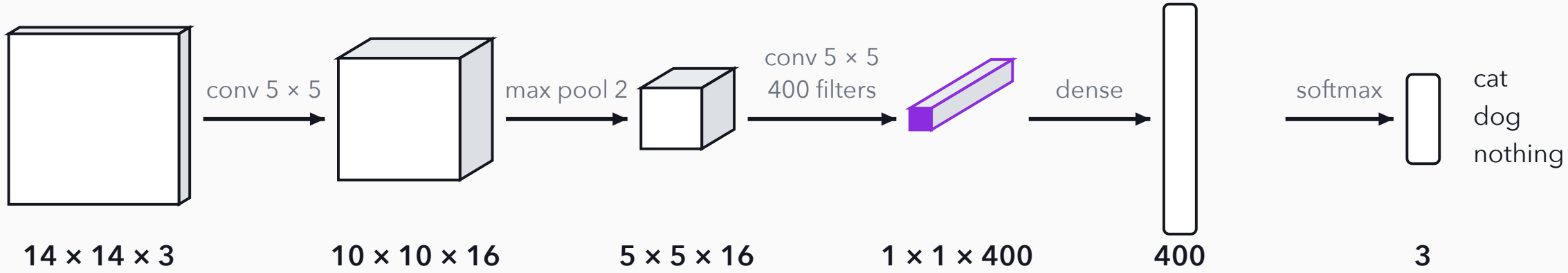
> Both solve the lecture's question with a fixed tensor plus a way to say "nothing here", and both score well on our photos.

Ng's trick, step 1: write the dense layers as convolutions



Ng's toy classifier for 14 × 14 crops, drawn as blocks of numbers: a convolution, a max pool, two dense layers of 400 units, then 3 answers.

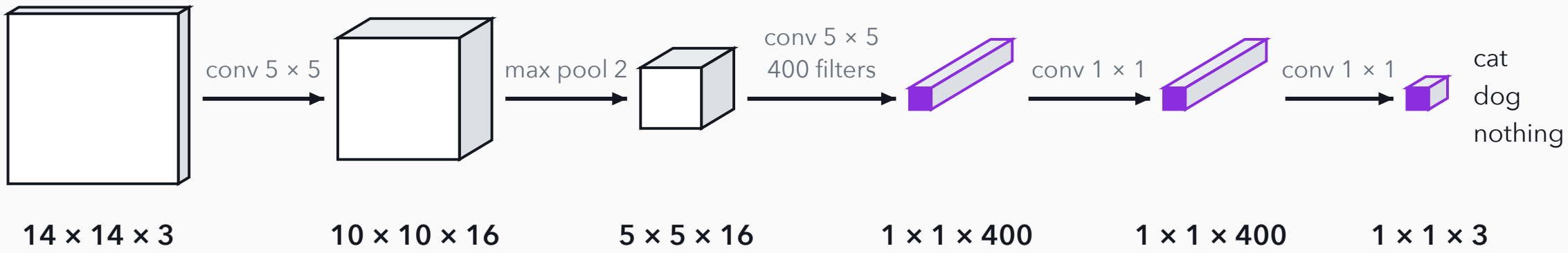
Ng's trick, step 1: write the dense layers as convolutions



$$\text{one filter: } 5 \times 5 \times 16 = 400 \text{ weights, as for one dense unit}$$

> Write the first dense layer as 400 filters of 5 × 5 × 16. Each one covers the whole block, so it sees all 400 inputs, like one dense unit.

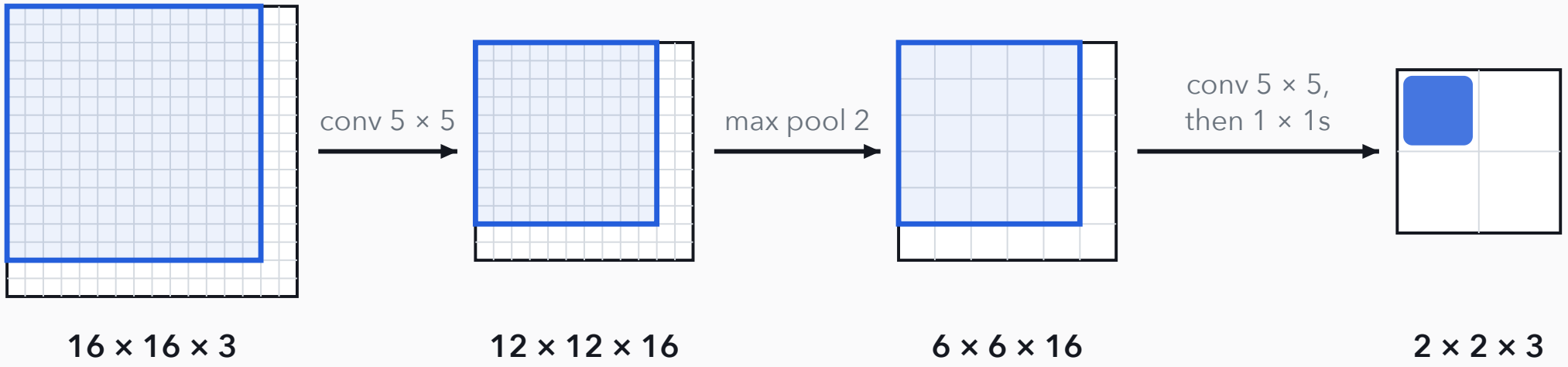
Ng's trick, step 1: write the dense layers as convolutions



every layer is a convolution

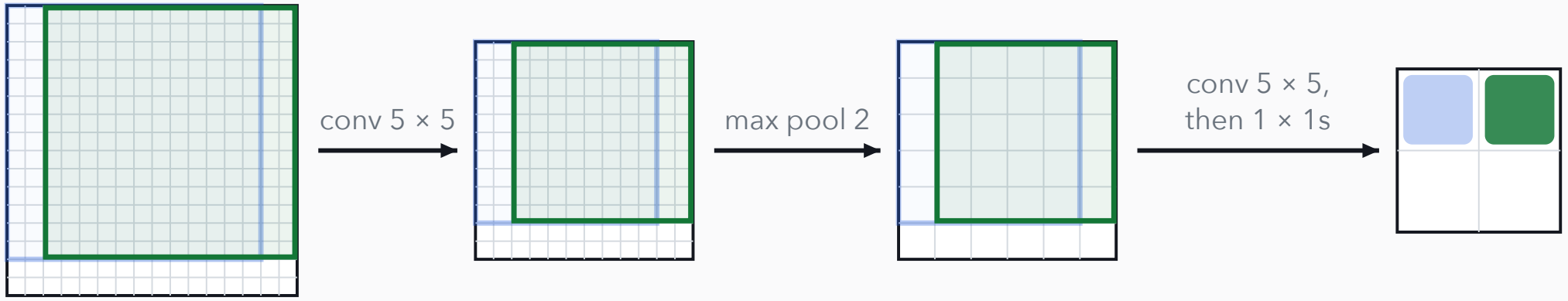
> The other two dense layers become 1 × 1 convolutions. The weights and the answers do not change, but a network of convolutions accepts a bigger image.

Ng's trick, step 2: a 16 × 16 image gives four answers at once



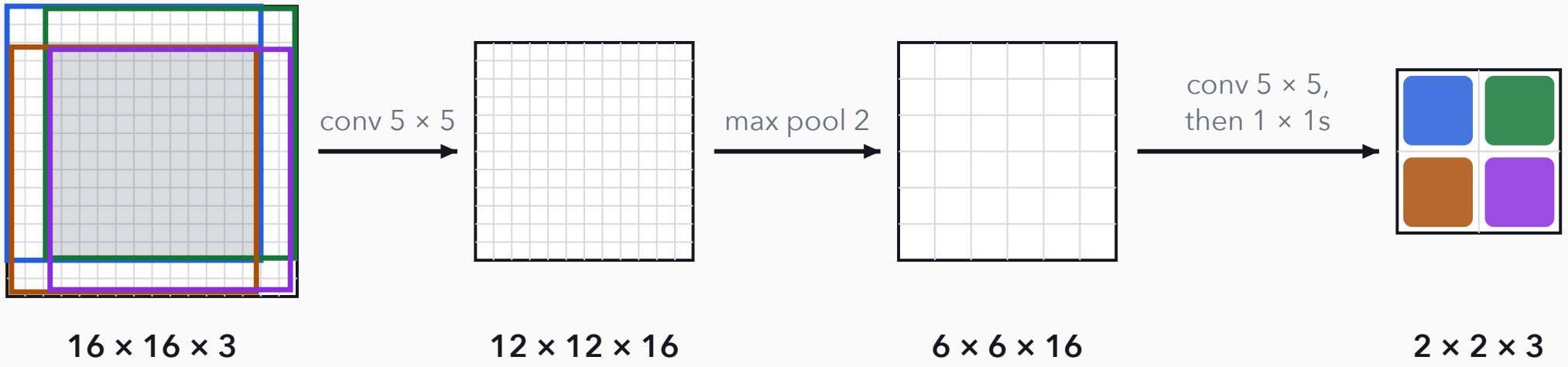
Give the same network a 16 × 16 image. The top-left 14 × 14 window flows through exactly as before and ends in the top-left output.

Ng's trick, step 2: a 16 × 16 image gives four answers at once



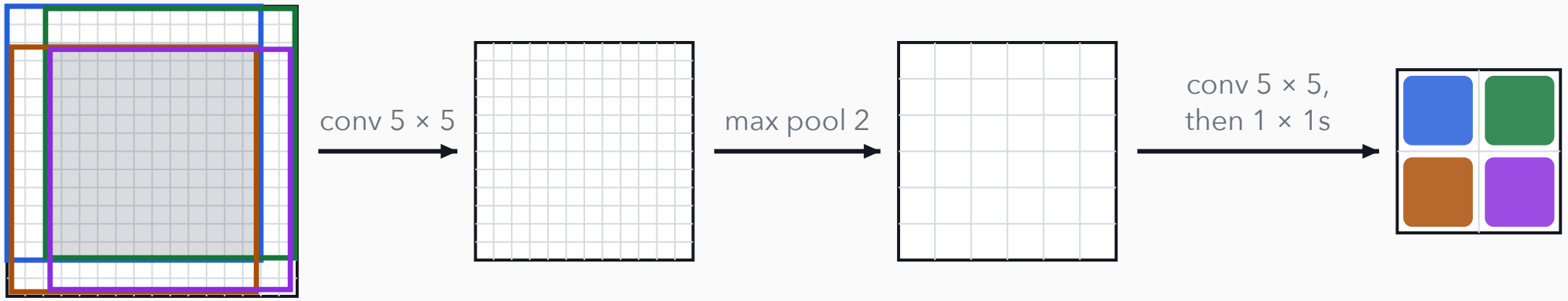
The window 2 pixels to the right moves 2 cells in the first layer and 1 after pooling. Its answer is the top-right output.

Ng's trick, step 2: a 16 × 16 image gives four answers at once



Four windows, four answers, from one pass. Where the windows overlap, each convolution is computed once instead of four times.

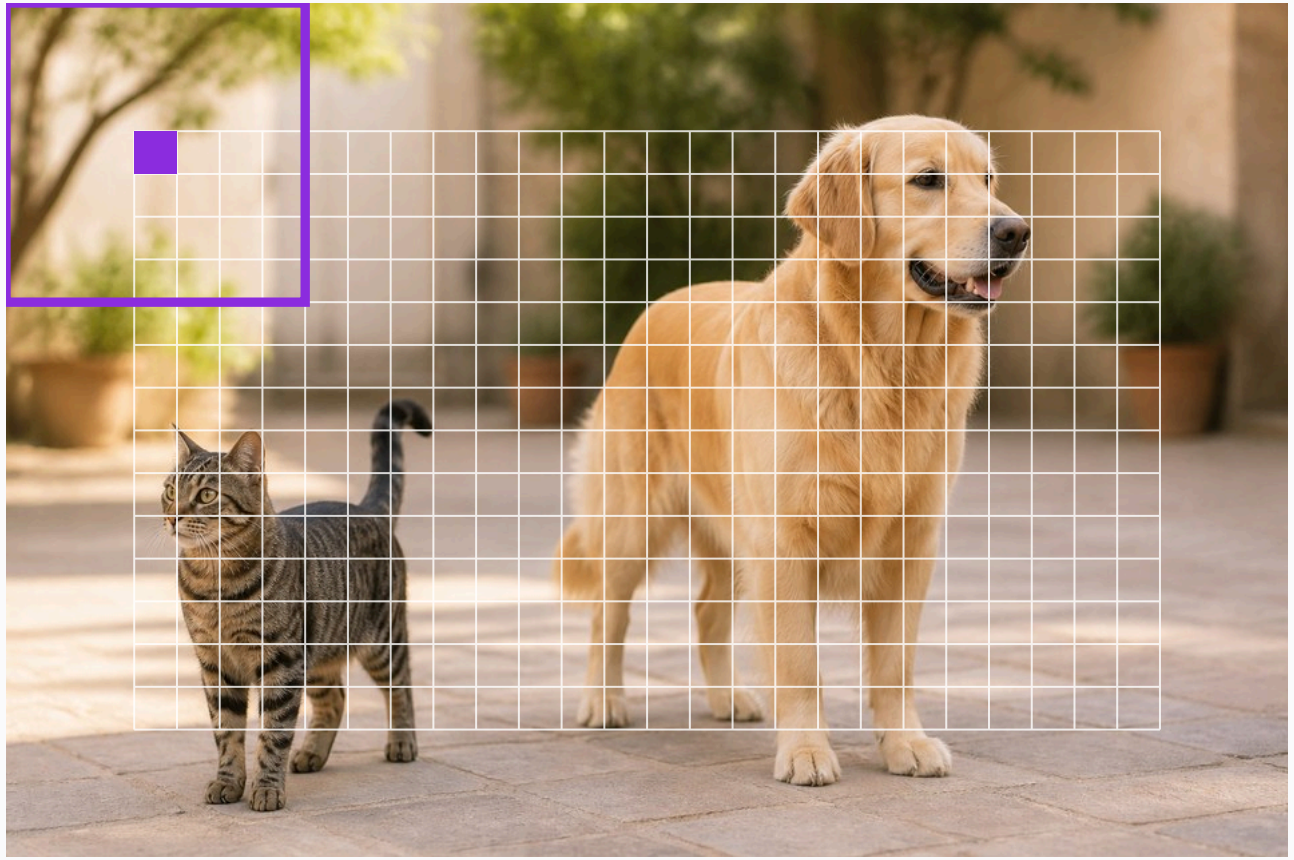
Ng’s trick, step 2: a 16 × 16 image gives four answers at once



$$28 \xrightarrow{\text{conv } 5\times5} 24 \xrightarrow{\text{pool } 2} 12 \xrightarrow{\text{conv } 5\times5} 8: \quad 8 \times 8 = 64 \text{ windows}$$

> Ng’s next example: a 28 × 28 image gives 8 × 8 answers, 64 windows 2 pixels apart. The step is 2 because of the max pool.

The same trick on our photo: all 336 windows in one pass



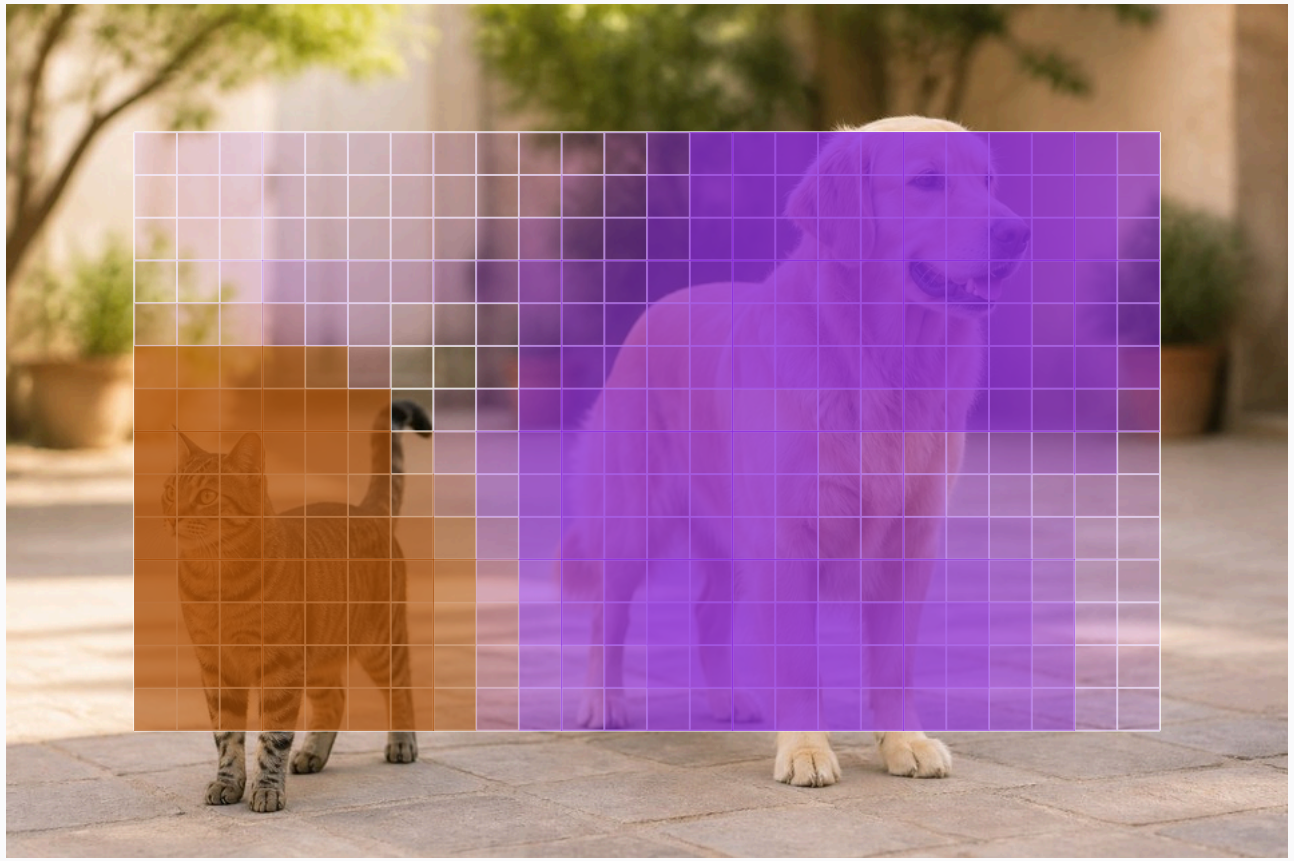
24 × 14

answers from one pass of VGG16

> On our photo: VGG16, an ImageNet classifier for 224 × 224 crops, with its dense layers written as convolutions. One pass gives 24 × 14 answers.

VGG16 (torchvision, ImageNet) run on this photo by scripts/run_models.py; timed on a laptop CPU. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

The same trick on our photo: all 336 windows in one pass



24 × 14

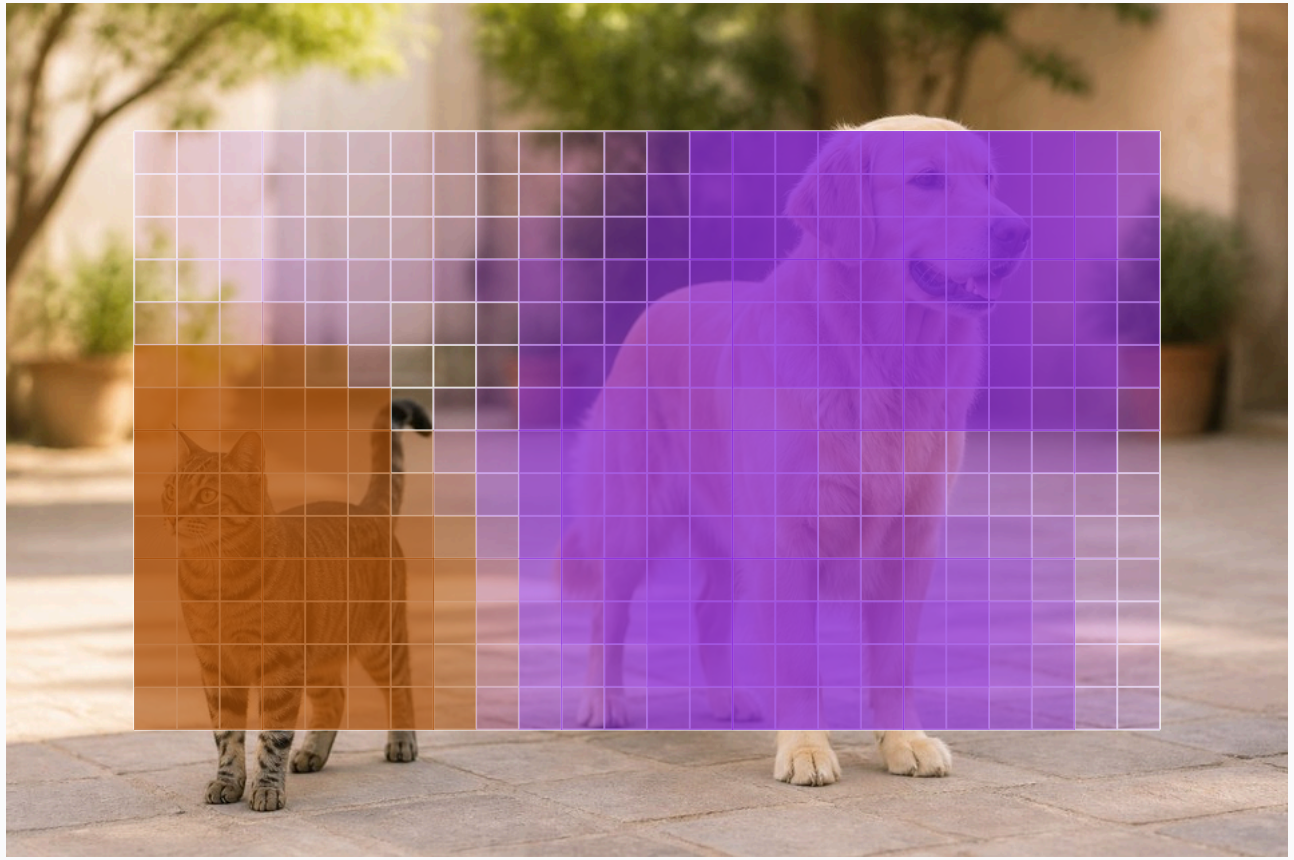
answers from one pass of VGG16

dog score cat score

> Each answer belongs to one window and is drawn at the window's centre. Windows over the dog say dog; windows over the cat say cat.

VGG16 (torchvision, ImageNet) run on this photo by scripts/run_models.py; timed on a laptop CPU. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

The same trick on our photo: all 336 windows in one pass



24 × 14

answers from one pass of VGG16

dog score cat score

one pass	0.7 s
336 crops	28 s
same answer	317 of 336

> On this laptop one pass takes 0.7 s; the 336 separate crops take 28 s. The two agree on 317 of the 336 windows.

VGG16 (torchvision, ImageNet) run on this photo by scripts/run_models.py; timed on a laptop CPU. The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv1 (2016): one responsible cell per object

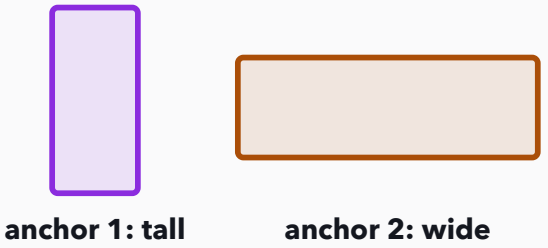
$$7 \times 7 \text{ cells} \times (2 \times 5 + 20) = 7 \times 7 \times 30$$

Each cell: two boxes, each (x, y, w, h, confidence), and one set of 20 class probabilities (PASCAL VOC). Only the cell containing an object's centre is responsible for it; x, y are offsets inside that cell, w, h fractions of the image.

Confidence was trained toward the IoU between the box and the truth. A cell could hold at most one object's class.

> Why did YOLOv1 struggle with flocks of birds?

Anchor boxes: two records per cell, one tall and one wide

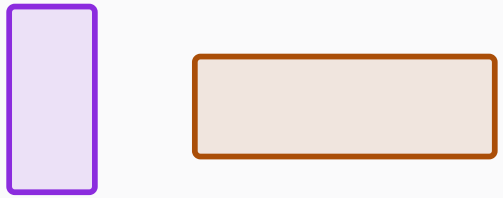


$$2 \times 7 = 14 \text{ numbers per cell}$$

An object goes to the cell that holds its centre, and to the anchor whose shape overlaps its box most: a person to the tall one, a car to the wide one.

> Ng's second fix: give each cell two records, one for a tall shape and one for a wide one, so that one cell can report a person and a car.

Anchor boxes: two records per cell, one tall and one wide



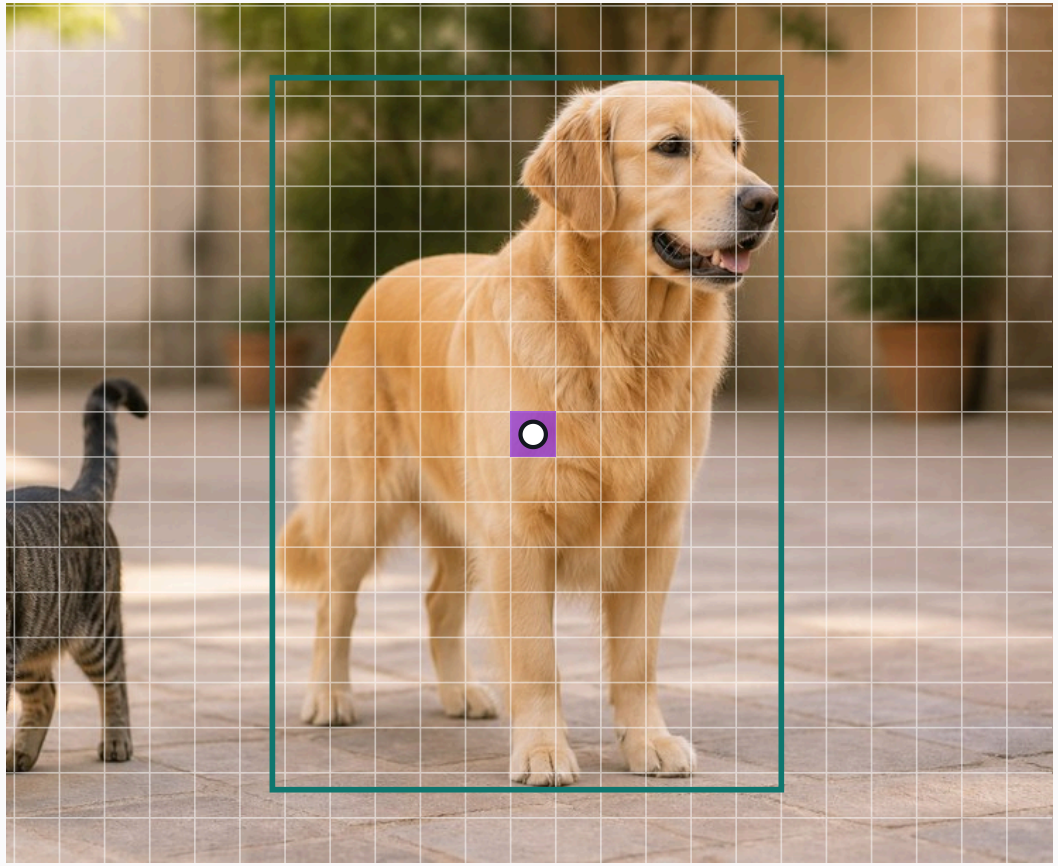
$2 \times 7 = 14$ numbers per cell

An object goes to the cell that holds its centre, and to the anchor whose shape overlaps its box most: a person to the tall one, a car to the wide one.

$$b_x = \sigma(t_x) + c_x, \;\; b_y = \sigma(t_y) + c_y, \;\; b_w = p_w \, e^{t_w}, \;\; b_h = p_h \, e^{t_h}$$

> YOLOv2 to v5 predict small corrections to each anchor's place and size; the anchor shapes come from k-means on the training boxes. YOLOv8 needs no anchors.

YOLOv8's box: four distances from the cell centre

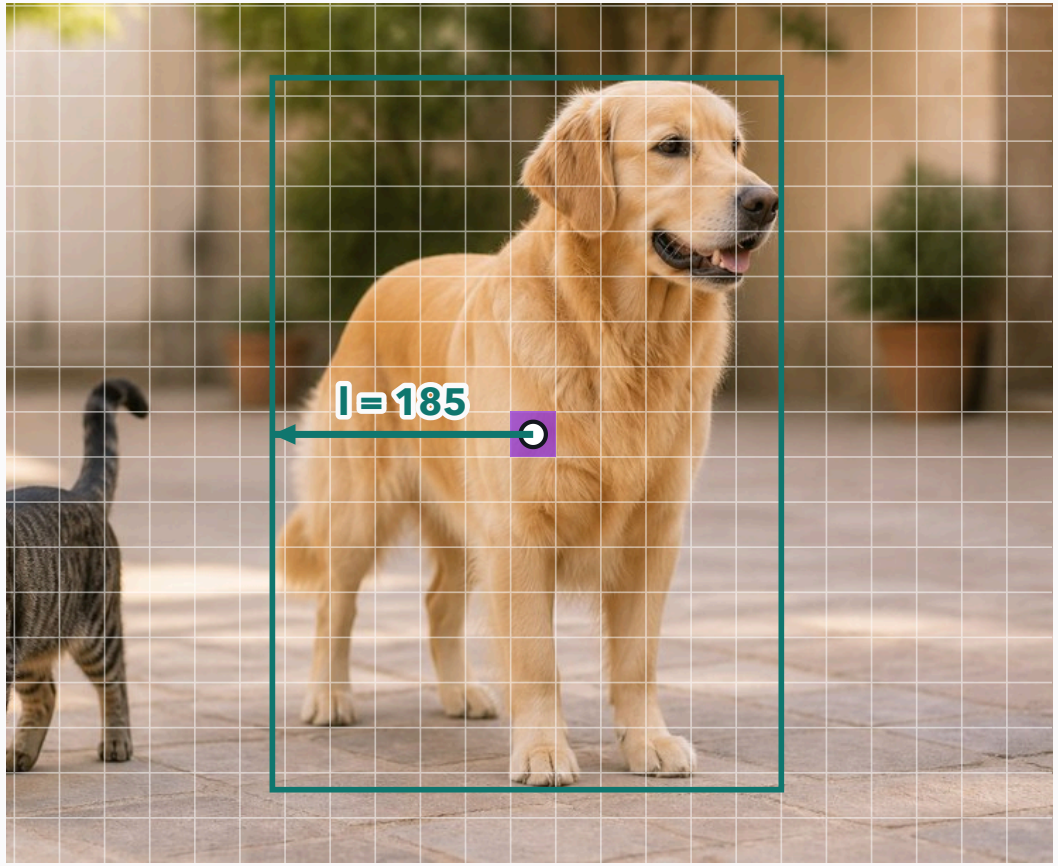


$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

> Stand at the centre of cell (10, 18): column 18 and row 10, plus a half, times 32 pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv8's box: four distances from the cell centre



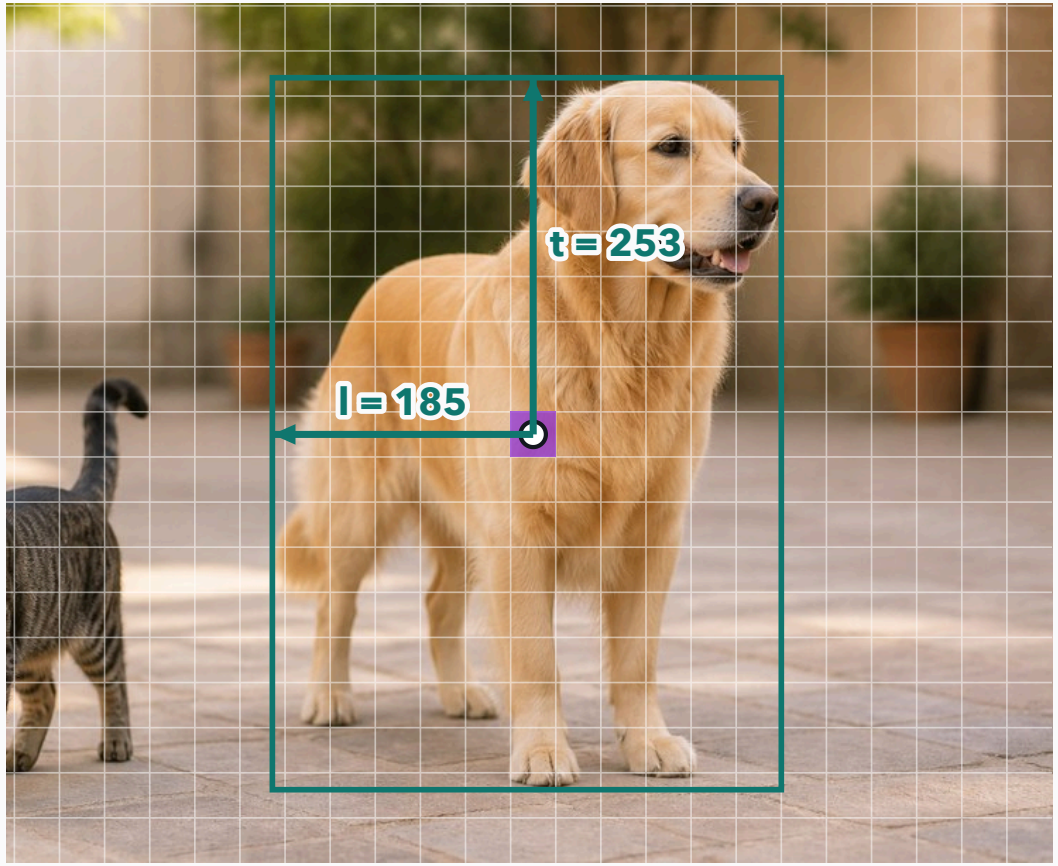
$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

Left: how far to the box's left edge. 592 − 407 = 185 pixels.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv8's box: four distances from the cell centre



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

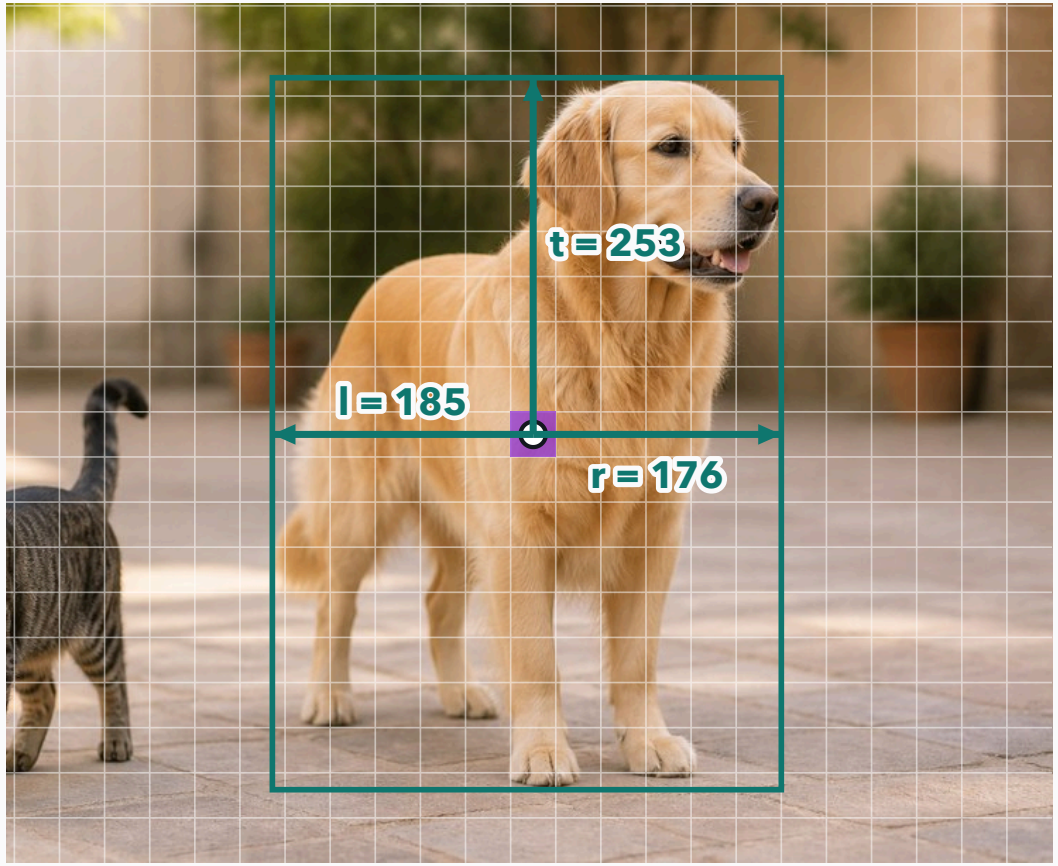
$$l = 592 - 407 = 185$$

$$t = 336 - 83 = 253$$

> Top: 336 − 83 = 253.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv8's box: four distances from the cell centre



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

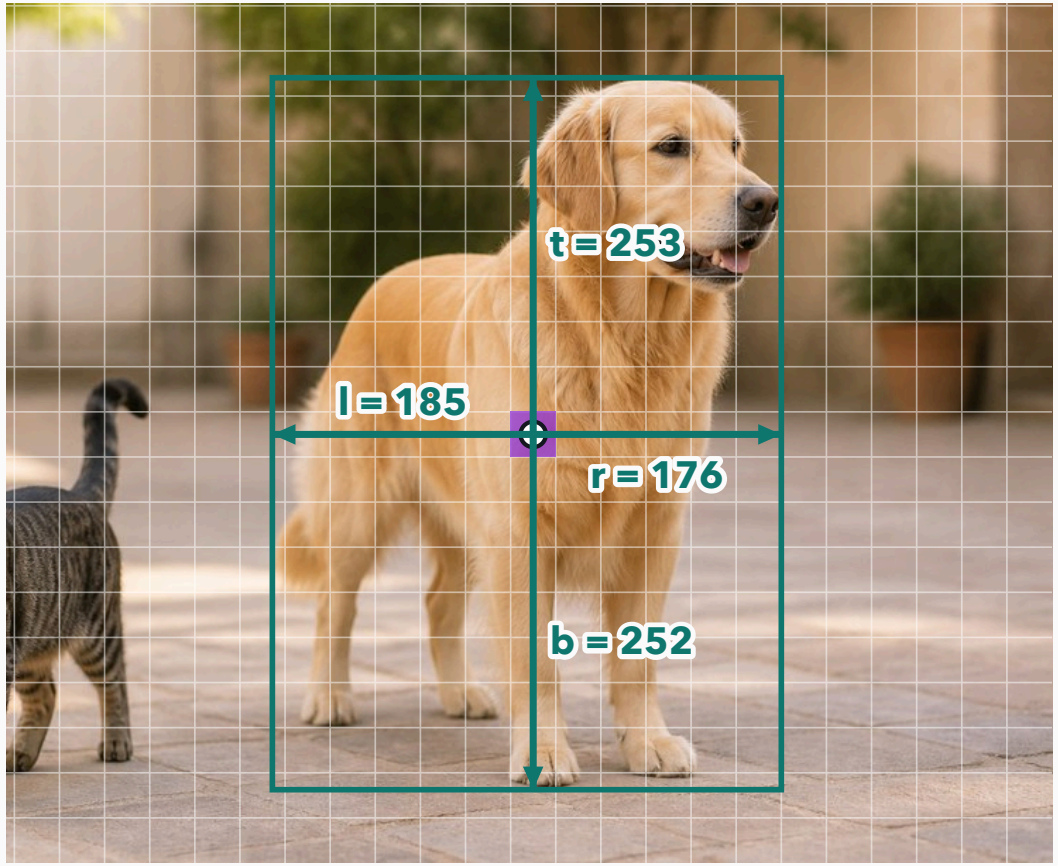
$$t = 336 - 83 = 253$$

$$r = 768 - 592 = 176$$

> Right: 768 − 592 = 176.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv8's box: four distances from the cell centre



$$\text{centre} = (18.5 \times 32,\ 10.5 \times 32) = (592,\ 336)$$

$$l = 592 - 407 = 185$$

$$t = 336 - 83 = 253$$

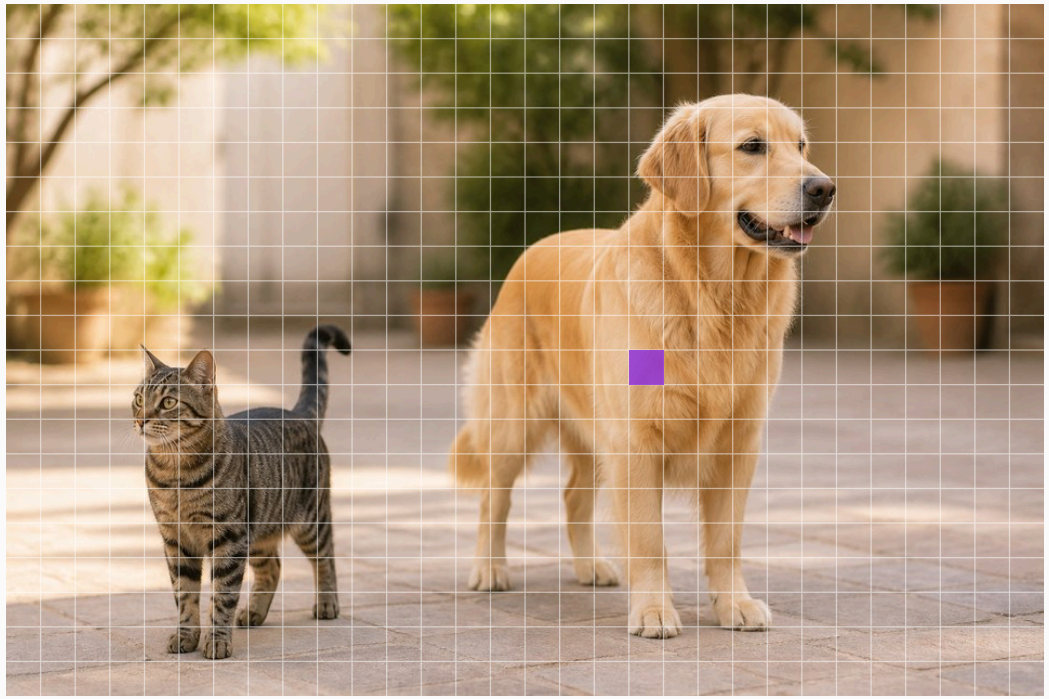
$$r = 768 - 592 = 176$$

$$b = 588 - 336 = 252$$

> Bottom: 588 − 336 = 252. Like standing in a room and measuring the distance to each wall: four numbers fix the box.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv8n's record: 80 classes, and no "object?"



4 box numbers

b_x b_y b_w b_h

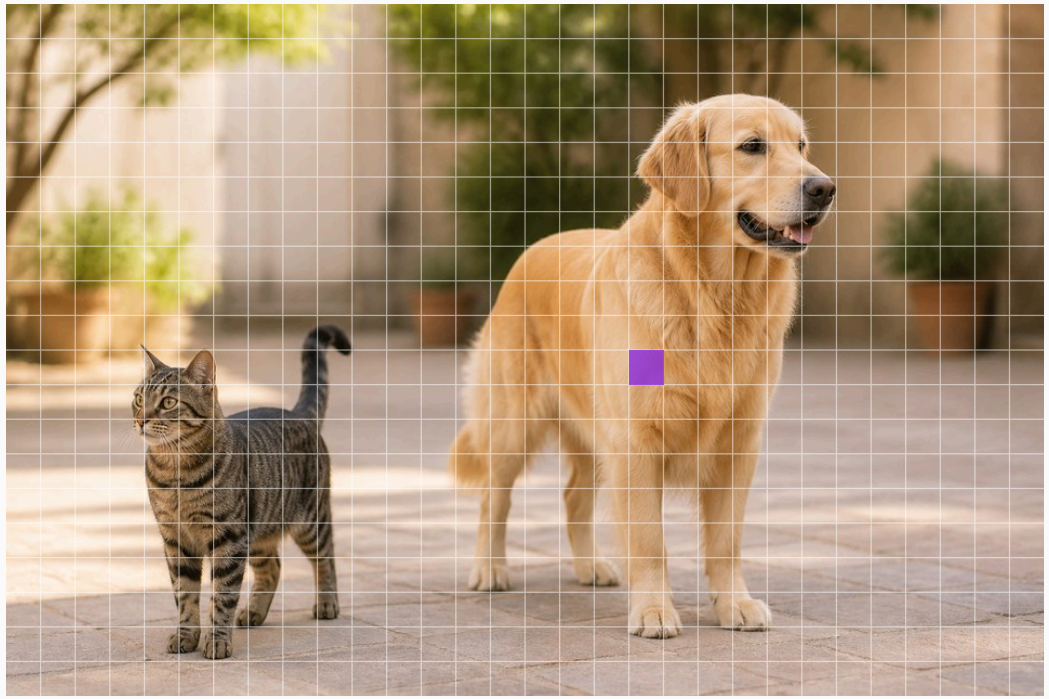
80 class scores, one per COCO class

person toothbrush

> YOLOv8n knows the 80 COCO classes, so its record has the 4 box numbers and 80 class scores: 84 numbers per cell.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv8n's record: 80 classes, and no "object?"



4 box numbers

b_x b_y b_w b_h

80 class scores, one per COCO class

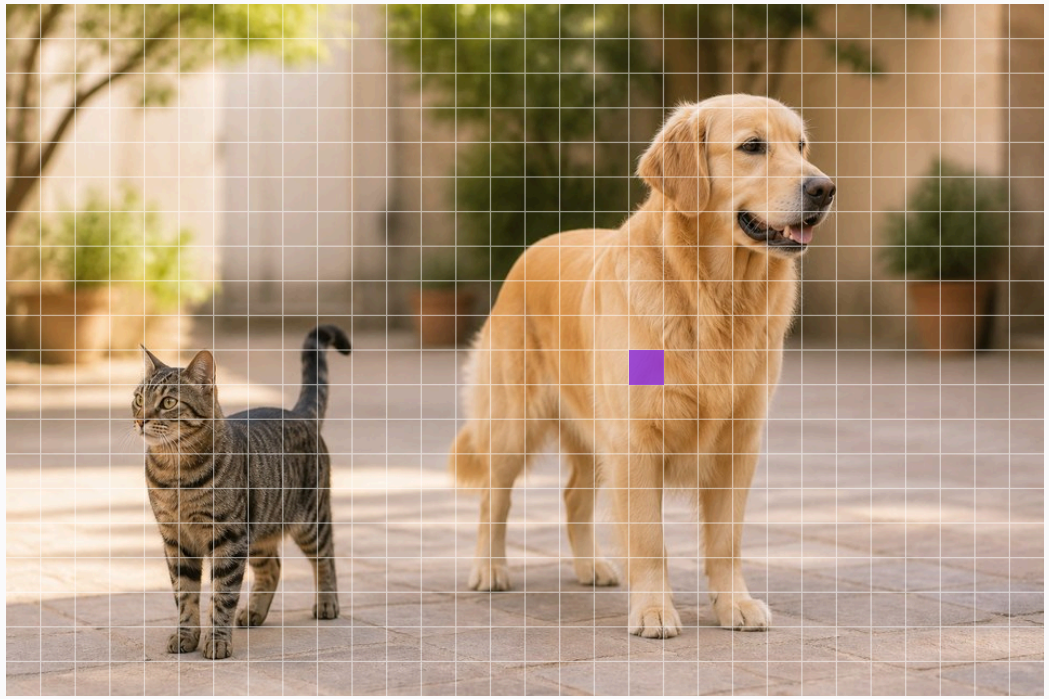
person toothbrush

No "object?" number: a cell with nothing in it gives every class a low score.

> It drops "object?": a cell says "nothing here" by giving every class a low score.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

YOLOv8n's record: 80 classes, and no "object?"



4 box numbers

b_x b_y b_w b_h

80 class scores, one per COCO class

person toothbrush

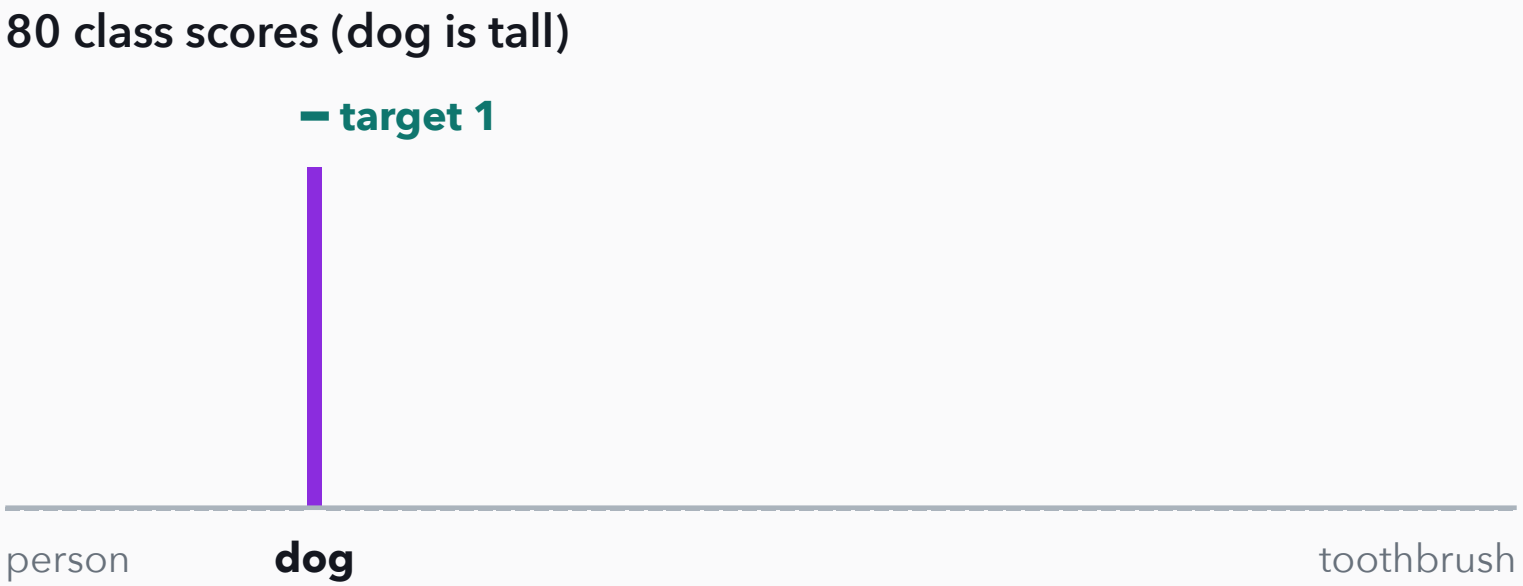
No "object?" number: a cell with nothing in it gives every class a low score.

$$30 \times 20 \times (4 + 80) = 50{,}400 \text{ numbers}$$

> The whole output, for any photo this size: 30 × 20 × 84 numbers, written in one pass.

The photographs are course-owned generated images (see SOURCES.md); every box and score drawn on them is computed.

What the trained network wrote at cell (10, 18)



Dog: 0.871 (target 1). Cat 0.0001, skateboard 0.00005; the other 77 are smaller still (targets 0).

> The real network's scores at this cell: dog a little below its target of 1, every other class near its target of 0.

YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

What the trained network wrote at cell (10, 18)

target

b_x	b_y	b_w	b_h
0.359	0.484	11.281	15.781

prediction

b_x	b_y	b_w	b_h
0.388	0.371	11.323	16.015



Box: close to the target. The predicted centre sits a little higher, and the box is a little taller.

> Its box, in cell units, is close to the target too. The loss turns all these differences into one number to reduce.

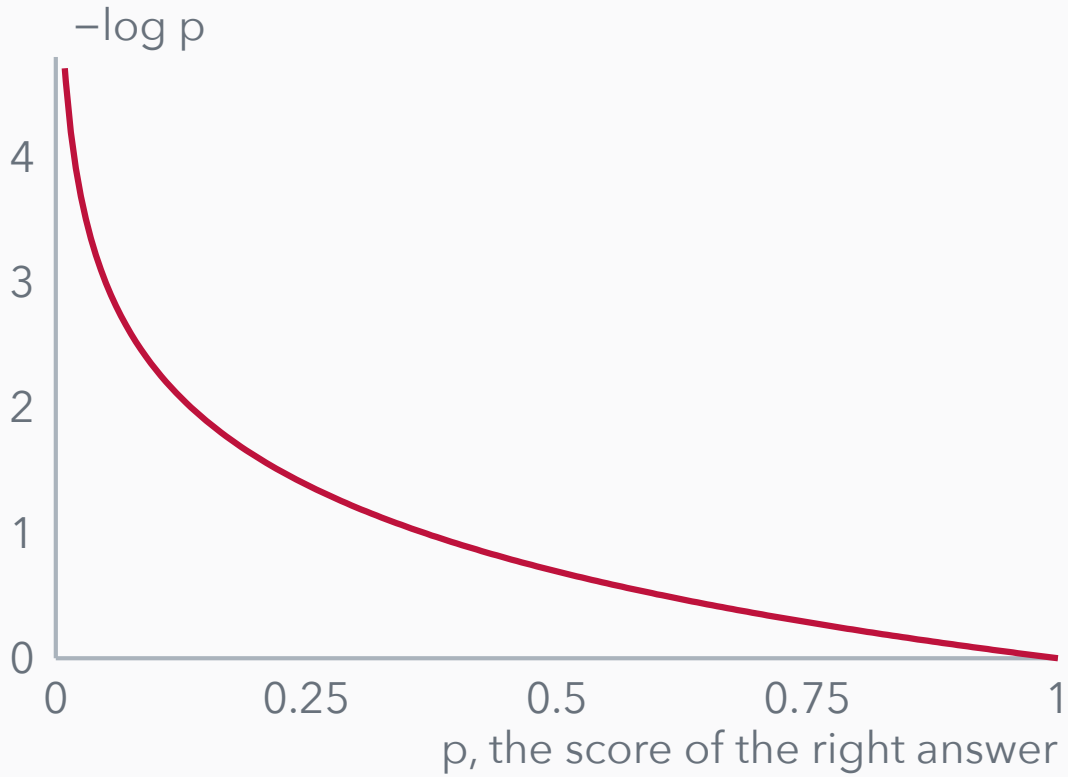
YOLOv8n (Ultralytics, COCO-trained) run on this photo by scripts/run_models.py; every value from data/science.json.

Scoring one yes/no answer: binary cross-entropy

$$\text{yes } (y = 1): \quad -\log p$$

$$\text{no } (y = 0): \quad -\log(1 - p)$$

p: the score the network gave this class; y: the target, 1 or 0. Natural logarithms.



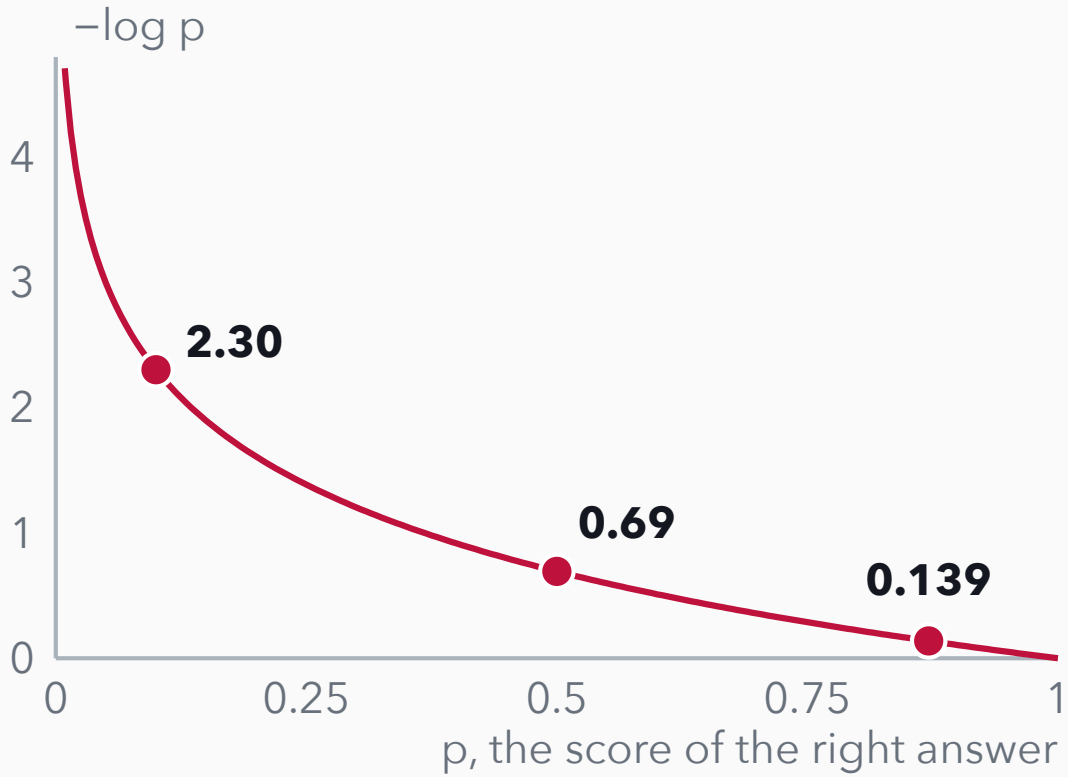
A class score answers one yes/no question. If the answer is yes, the loss is −log p; if no, −log(1 − p), the same curve mirrored.

Scoring one yes/no answer: binary cross-entropy

$$\text{yes } (y = 1): \quad -\log p$$

$$\text{no } (y = 0): \quad -\log(1 - p)$$

p: the score the network gave this class; y: the target, 1 or 0. Natural logarithms.



For a yes: sure and right costs almost nothing; unsure (0.5) costs 0.69; sure and wrong (0.1) costs 2.30. Our dog cell, 0.871: 0.139.

The loss at two cells: the dog's, and an empty one

$$\text{dog's cell, classes: } -\log 0.871 + \sum_{79} -\log(1 - p_c) = 0.139 + 0.0004 = 0.139$$

> At the dog's cell, the class part: dog is a yes, so −log 0.871. The other 79 are near-certain noes and add almost nothing.

The loss at two cells: the dog's, and an empty one

$$\text{dog's cell, classes: } -\log 0.871 + \sum_{79} -\log(1-p_c) = 0.139 + 0.0004 = 0.139$$

$$\text{dog's cell, box: } 0.029 + 0.113 + 0.042 + 0.234 = 0.418 \text{ cells}$$

> The box part: the L1 distance between the predicted and the target box numbers, in cells, as in Part 4.

The loss at two cells: the dog's, and an empty one

$$\text{dog's cell, classes: } -\log 0.871 + \sum_{79} -\log(1-p_c) = 0.139 + 0.0004 = 0.139$$

$$\text{dog's cell, box: } 0.029 + 0.113 + 0.042 + 0.234 = 0.418 \text{ cells}$$

$$\text{empty cell } (3,3)\text{: } \sum_{80} -\log(1-p_c) = 0.0038, \quad \text{no box}$$

> At an empty cell there is no box to compare, only the class part, and it is tiny: every class there is already near 0.

The loss for the whole photo, on the real grid

$$\mathcal{L} = \underbrace{\sum_{\text{all 600 cells}} \sum_{c=1}^{80} \mathrm{BCE}(p_c, y_c)}_{\text{class: every cell}} + \underbrace{\sum_{\text{object cells}} \mathcal{L}_{\text{box}}}_{\text{box: 2 cells here}}$$

The loss for the photo: class terms from every cell, box terms only from the cells that hold an object.

The loss for the whole photo, on the real grid

$$\mathcal{L} = \underbrace{\sum_{\text{all 600 cells}} \sum_{c=1}^{80} \mathrm{BCE}(p_c, y_c)}_{\text{class: every cell}} + \underbrace{\sum_{\text{object cells}} \mathcal{L}_{\text{box}}}_{\text{box: 2 cells here}}$$

$$600 \times 80 = 48{,}000 \text{ class terms}, \qquad 2 \text{ box terms}$$

Count them for our photo. The 598 empty cells still matter: they learn to say nothing.

The loss for the whole photo, on the real grid

$$\mathcal{L} = \underbrace{\sum_{\text{all 600 cells}} \sum_{c=1}^{80} \mathrm{BCE}(p_c, y_c)}_{\text{class: every cell}} + \underbrace{\sum_{\text{object cells}} \mathcal{L}_{\text{box}}}_{\text{box: 2 cells here}}$$

$$600 \times 80 = 48{,}000 \text{ class terms}, \qquad 2 \text{ box terms}$$

One number for the photo; gradient descent lowers it.

> Every cell uses the same weights, so every cell learns from every object in every photo.

Two-stage detectors: propose, then classify

Stage 1, a region proposal network: about 2,000 boxes that might contain something.
Stage 2: crop features for each proposal (RoI pooling), classify it and refine its box.

R-CNN (2014) ran a CNN per proposal; Fast R-CNN (2015) shared one CNN pass; Faster R-CNN (2015) learned the proposals too. Mask R-CNN adds a mask per proposal: Lecture 23.

torchvision's fasterrcnn_resnet50_fpn returns post-processed detections: its NMS is already inside.

> What does the first stage output?

Focal loss: quiet the easy background

$$\mathrm{FL}(p_t) = -(1-p_t)^{\gamma}\log p_t, \qquad p_t = \begin{cases} p & y = 1 \\ 1-p & y = 0 \end{cases}$$

A background cell with p = 0.001: $p_t = 0.999$, so with $\gamma = 2$ its loss is multiplied by $(1 - p_t)^2 \approx 10^{-6}$. Easy examples nearly vanish; hard ones keep their weight.

RetinaNet (Lin et al., 2017) made one-stage detectors as accurate as two-stage ones this way.

> Which examples does focal loss down-weight?

Box formats in practice

where	format	the dog
torchvision	corners, pixels	(407, 83, 768, 588)
COCO json	(x, y, w, h), top-left, pixels	(407, 83, 361, 505)
YOLO label file	normalised (c_x, c_y, w, h)	(0.612, 0.524, 0.376, 0.789)
DETR output	normalised (c_x, c_y, w, h)	same as YOLO

> Why do format bugs often go unnoticed?

Resizing and padding move the boxes too

Detectors often resize a photo to a fixed size and pad the rest (letterboxing). The boxes must get the same treatment: scale every coordinate by the resize factor, then add the padding offset.

$$960 \times 640 \;\to\; 640 \times 427 \;(r = \tfrac{2}{3}),\ \text{pad } 106 \text{ px above}: \qquad y' = \tfrac{2}{3}\, y + 106$$

Our photos are multiples of 32 on both sides, so this lecture ran the models without padding.

> What happens to the dog's $y_1 = 83$ after this letterbox?

After DETR

- **Deformable DETR** (2021): each query attends to a few sampled points per feature level; trains about 10× faster.
- **DINO, RT-DETR** (2022 and 2023): better queries and matching; real-time DETRs.
- **YOLOv10** (2024): one-to-many and one-to-one heads; NMS-free YOLO.
- **Open vocabulary** (OWL-ViT, Grounding DINO): classes given as text, matched with CLIP-like embeddings (the language-grounded vision lecture).

> What did DETR's successors mainly improve?